Java Örnekleriyle Nesne Yönelimli Programlama

Esnek Tasarım, Temiz Kod ve Mimari Standartlar

Elektronik Yük. Müh. İlhan ÖZKAN

Kapsamlı Uygulama Rehberi ve Ders Kitabı

2026 Baskısı

Java Örnekleriyle Nesne Yönelimli Programlama

Yazar: Elektronik Yük. Müh. İlhan ÖZKAN

© Eylül 2026, İlhan ÖZKAN. Tüm hakları saklıdır.

Bu kitap kâr amacı güdülmeden hazırlanmıştır ve **referans gösterilerek** her yerde kullanılabilir.

Bu kitabın bir kısmı veya tamamı, **yazarın önceden yazılı izni alınarak** elektronik, mekanik, fotokopi veya herhangi bir kayıt sistemi ile çoğaltılabilir, yayımlanabilir veya depolanabilir.

ISBN: ***-***-*****-*-*

Baskı Tarihi: Aralık 2026
Baskı Yeri: Ankara, Türkiye

Önsöz

Yazılım geliştirme, yalnızca çalışan kod yazmaktan ibaret değildir; gerçek anlamda nitelikli yazılım; anlaşılabilir, değiştirilebilir, sürdürülebilir ve genişletilebilir yapılar kurabilme becerisidir. Bugünün karmaşık yazılım dünyasında bir programcının yalnızca bir programlama dilinin söz dizimini bilmesi yeterli değildir; algoritmik düşünmeyi, veri ile davranış arasındaki ilişkiyi, nesne yönelimli yaklaşımı, hata yönetimini, sınıflar arası ilişkileri ve mimari tasarım ilkelerini de derinlemesine kavraması gerekir.

Bu kitap, yazılım geliştirme yolculuğunda sağlam bir temel oluşturmak isteyen öğrenciler, yeni başlayan geliştiriciler ve bilgilerini sistematik biçimde derinleştirmek isteyen uygulayıcılar için kapsamlı bir başvuru kaynağı ve ders kitabı olarak hazırlanmıştır. Eserin temel amacı; okuyucuyu düşük düzey programlama temellerinden başlayarak yapısal programlama, nesne yönelimli programlama (OOP), UML modelleme, SOLID ilkeleri, jenerik yapılar, tasarım kalıpları ve temiz kod yaklaşımına kadar adım adım ilerleyen bütüncül bir düşünce yapısına ulaştırmaktır.

Kitap boyunca yalnızca "nasıl kod yazılır?" sorusuna değil, aynı zamanda "neden böyle tasarlanmalıdır?" sorusuna da cevap aranmıştır. Çünkü yazılımda kalıcı başarı, ezberlenmiş kod parçacıklarından çok; doğru soyutlama yapabilme, sorumlulukları etkin biçimde ayırabilme ve değişime açık ama dağınıklıktan uzak mimariler kurabilme becerisine dayanır.

Bu eser, birbirini tamamlayan altı ana eksen üzerine kurulmuştur:

- ◇ Temel Yazılım Kavramları ve OOP'ye Giriş: Düşük düzey programlamadan genel amaçlı yüksek düzey dillere geçiş, değişkenler, fonksiyonlar, yapısal programlama paradigması ve nesne yönelimli yaklaşıma duyulan ihtiyaç.
- ◇ Nesne Odaklı Zihniyet: Sınıf ve nesne kavramları, kapsülleme, soyutlama, kalıtım, çok biçimlilik, arayüzler, yapıcılar, this göstericisi ve UML sınıf diyagramları ile modelleme.
- ◇ İstisnalar, Esnek Tasarım ve SOLID İlkeleri: Yapısal ve nesne yönelimli hata/istisna yönetimi, kaynak yönetimi (try-with-resources) ile yazılım mimarisinin temel yapı taşları olan SOLID ilkelerinin Java 17+ standartlarına uyarlanmış halleri.
- ◇ Şablonlar ve Jenerik Programlama: Jenerik yöntem ve sınıflar, Optional yapısı, otomatik bellek yönetimi (Çöp Toplayıcı), Java Koleksiyonlar Çerçevesi (List, Set, Map), Stream API, algoritmalar ve fonksiyonel arayüzler ile lambda ifadeleri.
- ◇ Tasarım Kalıpları (Design Patterns): Nesne imalatı ile ilgili yaratıcı kalıplar (Soyut Fabrika, Fabrika Yöntemi, Tekil Nesne/Singleton, Kurucu/Builder, Prototip), yapısal kalıplar (Önyüz/Facade, Adaptör, Bileşik/Composite, Süsleyici/Decorator, Sinek Sıklet/Flyweight, Vekil/Proxy, Köprü/Bridge) ve davranışsal kalıplar (Sorumluluk Zinciri, Komut, Yineleyici, Gözlemci, Strateji, Şablon Yöntem, Ziyaretçi, Hatıra, Ara Bulucu, Durum, Yorumlayıcı ve MVC).
- ◇ Temiz Kod ve Mimari Standartlar: Anlamlı isimlendirme, küçük fonksiyonlar ve sınıflar, DRY ilkesi, hata yönetimi, katmanlı mimari temelleri, bağımlılık enjeksiyonu (DI), test edilebilir kod yazma (Mock/Fake), yeniden düzenleme (Refactoring), güvenli kodlama standartları ve tüm bu ilkelerin uygulandığı kapsamlı bir Kütüphane Ödünç Verme Sistemi bölüm sonu uygulaması.

Kitabın ayırt edici yönlerinden biri, kavramları yalnızca teorik tanımlarla sınırlı bırakmaması; her konuyu sade bir dille, günlük hayattan benzetmelerle ve zengin Java kod örnekleriyle desteklemesidir. Böylece okuyucunun yalnızca kodu görmesi değil, kodun arkasındaki mühendislik ve tasarım kararını da içselleştirmesi hedeflenmiştir.

Nesne yönelimli programlama ve modern yazılım mimarileri ilk bakışta zorlayıcı görünebilir;

ancak bu kavramlar doğru sırayla ve pratik örneklerle ele alındığında, yazılım geliştirmenin yalnızca teknik bir uğraş değil, aynı zamanda disiplinli bir düşünme biçimi olduğu açıkça görülecektir. Bilginin paylaşıldıkça çoğaldığına olan inançla hazırlanan bu çalışmanın; öğrenciler, akademisyenler, eğitmenler ve profesyonel yazılım geliştiriciler için faydalı olmasını dilerim.

Kitabın güncel sürümüne ve tüm kaynak kodlarına GitHub deposu üzerinden erişilebilir. Katkı, düzeltme ve geri bildirimleriniz kitabın gelecekteki sürümleri için son derece değerlidir. Başarılar ve iyi çalışmalar dilerim.

Elektronik Yüksek Mühendisi, İlhan ÖZKAN
ilhanozkan[at]outlook.com
linkedin.com/in/ilhanozkan

İçindekiler

Önsöz **i**

1 Temel Yazılım Kavramları ve OOP'ye Giriş **1**
1.1 Düşük Düzey Programlama . . . 1
1.2 Genel Amaçlı Yüksek Düzey Diller . . . 2
1.3 Arapsaçı Kod . . . 3
1.4 Değişken . . . 4
1.5 Fonksiyon . . . 6
1.6 Yapısal Programlama . . . 7
1.7 Yazılım Geliştirme Süreci . . . 10
1.8 Yapısal Programlama Paradigması . . . 10
1.9 Nesne Yönelimli Programlama Paradigması . . . 11

2 Nesne Odaklı Zihniyet **13**
2.1 Sınıf ve Nesne Kavramı . . . 13
2.2 Nesnenin Durumu, Özellikleri ve Davranışları . . . 14
2.3 Sınıf ve Nesnelerin UML Sınıf Diyagramları ile Gösterilmesi . . . 14
2.4 Bilgi ve Veri Gizleme Nedir? . . . 15
2.5 Soyutlama ve Kapsülleme . . . 16
2.6 Nesnelerin İmal Edilmesi . . . 17
2.7 this Göstericisi . . . 18
2.8 Yapıcı . . . 18
2.8.1 Parametreli Yapıcı . . . 19
2.8.2 Kopyalama Yapıcısı . . . 20
2.9 Nesnelerin Yok Edilmesi ve Çöp Toplayıcı (Garbage Collector) . . . 20
2.10 Yöntem Çok Yükleme . . . 20
2.11 İşleç Çok Yükleme . . . 21
2.12 Kalıtım Kavramı . . . 21
2.12.1 Ata ve Çocuk Sınıflar . . . 21
2.12.2 UML Sınıf Diyagramlarında Kalıtımın Gösterilmesi . . . 22
2.12.3 Kalıtımda Ata Sınıfın Yapıcısının Çağrılması . . . 22
2.12.4 Çoklu Kalıtım (Multiple Inheritance) Durumu . . . 22
2.12.5 Arkadaş Fonksiyon ve Sınıflar . . . 23
2.13 Statik Sınıf Üyeleri . . . 23
2.14 const ve final Sınıf Üyeleri . . . 23
2.15 Geçersiz Kılma (Overriding) . . . 23
2.16 Soyut ve Somut Sınıf . . . 24
2.17 Arayüzler (Interfaces) . . . 24
2.18 Çok Biçimlilik (Polymorphism) . . . 24

3 İstisnalar, Esnek Tasarım ve SOLID İlkeleri **26**
3.1 Hatalar ve İstisnalar . . . 26
3.1.1 Yapısal Programlamada Hata Yönetimi . . . 26
3.1.2 Nesne Yönelimli Programlamada İstisna Yönetimi . . . 27
3.1.3 Standart İstisna Tipleri . . . 28
3.1.4 İstisna Nesnesi ve Hata Mesajı: getMessage() . . . 31
3.1.5 Kullanıcı Tanımlı İstisna Tipleri . . . 31

3.1.6 İstisnayı Yeniden Fırlatma ve Zincirleme . . . 32
3.1.7 Kaynak Yönetimi ve finally / try-with-resources . . . 32
3.2 Unified Modeling Language ve Sınıflar Arası İlişkiler . . . 33
3.2.1 Bileşim (Composition) Örneği (Java) . . . 33
3.3 Sınıf Tasarlama (SOLID) İlkeleri . . . 33

4 Şablonlar ve Jenerik Programlama 36
4.1 Şablon Nedir? . . . 36
4.2 Jenerik Yöntemler . . . 36
4.3 Değişken Sayıda Argüman (Varargs) . . . 37
4.4 Sınıf Şablonları (Jenerik Sınıflar) . . . 37
4.5 Faydalı Yapılar ve Optional . . . 38
4.5.1 std::optional Karşılığı: java.util.Optional . . . 38
4.6 Bellek Yönetimi ve Çöp Toplayıcı (Garbage Collector) . . . 39
4.6.1 Otomatik Bellek Yönetimi ve RAII Yaklaşımı . . . 39
4.7 Java Koleksiyonlar Çerçevesi (Collections Framework) . . . 39
4.7.1 ArrayList Örneği . . . 39
4.7.2 HashMap Örneği . . . 40
4.7.3 HashSet Örneği . . . 40
4.8 Stream API ve Algoritmalar . . . 40
4.9 Fonksiyonel Arayüzler ve Lambda İfadeleri . . . 41

5 Tasarım Kalıpları 42
5.1 Kalıp Nedir? . . . 42
5.2 Nesne İmalatı ile İlgili Kalıplar . . . 43
5.2.1 Soyut Fabrika . . . 43
5.2.2 Fabrika Yöntemi . . . 48
5.2.3 Tekil Nesne . . . 52
5.2.4 Kurucu . . . 57
5.2.5 Prototip . . . 63
5.3 Yapısal Kalıplar . . . 70
5.3.1 Önyüz . . . 70
5.3.2 Adaptör . . . 76
5.3.3 Bileşik . . . 80
5.3.4 Süsleyici . . . 88
5.3.5 Sinek Sıklet . . . 97
5.3.6 Vekil . . . 104
5.3.7 Köprü . . . 111
5.4 Davranışla İlgili Kalıplar . . . 119
5.4.1 Sorumluluk Zinciri . . . 119
5.4.2 Komut . . . 125
5.4.3 Yineleyici . . . 132
5.4.4 Gözlemci . . . 139
5.4.5 Strateji . . . 145
5.4.6 Şablon Yöntem . . . 153
5.4.7 Ziyaretçi . . . 159
5.4.8 Hatıra . . . 167
5.4.9 Ara Bulucu . . . 172
5.4.10 Durum . . . 177
5.4.11 Yorumlayıcı . . . 182
5.4.12 MVC Mimari Kalıbı . . . 186

6 Temiz Kod ve Mimari Standartlar 190
6.1 Temiz Kod Nedir? . . . 190

6.1.1 Kodun Okunabilir Olması . . . 190
6.1.2 Kodun Niyetini Açık Etmesi . . . 191
6.1.3 Yorum Satırları Ne Zaman Kullanılmalı? . . . 191
6.2 Anlamlı İsimlendirme . . . 191
6.2.1 Değişken İsimlendirme . . . 191
6.2.2 Fonksiyon ve Yöntem İsimlendirme . . . 192
6.2.3 Sınıf İsimlendirme . . . 192
6.2.4 Sabit, Enum ve Arayüz İsimlendirme . . . 192
6.3 Küçük Fonksiyonlar ve Yöntemler . . . 193
6.3.1 Fonksiyonların Tek İş Yapması . . . 193
6.3.2 Parametre Sayısını Azaltma . . . 193
6.3.3 Yan Etkilerden Kaçınma . . . 193
6.3.4 Erken Çıkış ve Guard Clause Kullanımı . . . 194
6.4 Küçük Sınıflar ve Sorumluluk Ayrımı . . . 194
6.4.1 Sınıfın Boyutu Ne ile Ölçülür? . . . 194
6.4.2 Tanrı Sınıf Problemi . . . 195
6.4.3 Sınıfların Birlikte Değişen Davranışlara Göre Ayrılması . . . 195
6.4.4 Bağlaşım ve Uyum . . . 195
6.5 Kod Tekrarı ve Yeniden Kullanılabilirlik . . . 195
6.5.1 DRY İlkesi . . . 195
6.5.2 Gereksiz Soyutlama Tehlikesi . . . 196
6.5.3 Kalıtım Yerine Bileşim . . . 196
6.6 Hata Yönetimi ve İstisna Mimarisi . . . 196
6.6.1 Hata Kodları Yerine İstisnalar . . . 196
6.6.2 Anlamlı İstisna Sınıfları . . . 197
6.6.3 İstisnayı Yutmamak . . . 197
6.6.4 Kaynak Yönetimi . . . 197
6.6.5 Katmanlara Göre Hata Yönetimi . . . 197
6.7 Boş Değerler, Optional ve Geçersiz Durum Yönetimi . . . 198
6.7.1 Null Referans Problemi . . . 198
6.7.2 Optional Kullanımı . . . 198
6.7.3 Null Object Pattern . . . 198
6.7.4 Geçersiz Nesne Oluşturmamak . . . 198
6.8 Kapsülleme ve Değişmez Nesneler . . . 198
6.8.1 Alanları Doğrudan Açmamak . . . 199
6.8.2 Değişmez Nesneler . . . 199
6.8.3 Nesne Durumunun Tutarlılığı . . . 200
6.9 Katmanlı Mimari Temelleri . . . 201
6.9.1 Katmanlı Mimari Nedir? . . . 201
6.9.2 Katmanlar Arası Bağımlılık . . . 201
6.9.3 DTO, Entity ve Model Ayrımı . . . 202
6.9.4 Repository ve Service Kavramları . . . 202
6.10 Bağımlılık Yönetimi ve Bağımlılık Enjeksiyonu . . . 202
6.10.1 Bağımlılık Enjeksiyonu (Dependency Injection) . . . 202
6.10.2 Denetimin Tersine Çevrilmesi (IoC) Kavramı . . . 203
6.11 Test Edilebilir Kod Yazmak . . . 203
6.11.1 Birim Test Mantığı . . . 203
6.11.2 Mock ve Fake Nesneler . . . 204
6.12 Kod Biçimlendirme Standartları . . . 204
6.13 Yeniden Düzenleme (Refactoring) ve Kod Kokuları . . . 204
6.14 Performans ve Temiz Kod Dengesi . . . 204
6.15 Güvenli Kodlama Standartları . . . 204
6.16 Dokümantasyon ve API Tasarımı . . . 205

6.17 Sürüm Kontrolü ve Kod Kalitesi Süreci 205
6.18 Temiz Mimari İlkelerine Giriş 205
6.18.1 İş Kurallarını Merkeze Almak 205
6.19 Bölüm Sonu Uygulaması: Kütüphane Ödünç Verme Sistemi 206

1. Bölüm
Temel Yazılım Kavramları ve OOP'ye Giriş

1.1 Düşük Düzey Programlama

1642 yılında *Blaise Pascal* tarafından icat edilen Pascaline Hesap Makinesi, eldeli toplama, ödünç almalı çıkarma yapıyordu.

1671 yılında *Gottfried Wilhelm Leibniz* tarafından icat edilen Leibniz Çarkı, toplama, ödünç almalı çıkarmanın yanında bölme, çarpma ve karekök işlemlerini yapıyordu.

1801 yılında *Joseph Maria Jacquard* dokuma tezgahlarında desenleri oluşturmak için **delikli kartlar** (*punch card*) kullanmayı icat etmiştir. Sonradan bilgisayara veri girdisi sağlamak ve sonuçların çıktısını göstermek için kullanılacaktır.

1830 yılında *Charles Babbage* fark makinesi ve analitik makineyi icat etmiştir. Analitik makine, buhar gücü kullanan bu makinenin mantıksal işlem birimi, veri depolama birimi, giriş çıkış üniteleri bulunuyordu. *Augusta Ada Lovelace*, Babbage ile beraber çalışmıştır. Ada Lovelace, analitik makinenin bir dizi matematiksel işlemi gerçekleştirebildiğini fark etti ve bu makineyi Bernoulli sayılarını üretmek için kullandı. Bu algoritmanın o dönemdeki "Analitik Makine" için devrimsel bir gelişmedir. Çünkü şartlı dallanma ve döngü mantığı içerir. Bu sayıların hesaplanması için yazdığı algoritma tarihteki ilk bilgisayar programı olarak kabul edilir.

İlk Programcı

Augusta Ada Lovelace, tarihteki **ilk programcı** olarak kabul edilir.

1854 yılında *George Boole*, ikili sayı sistemine temel olan ve günümüz bilgisayarlarının kullandığı bool cebiri üzerine çalışmalar yapmıştır.

1890 yılında *Herman Hollerith* tarafından delikli kartlarla bilgilerin yüklenebildiği ve bu bilgiler üzerinde toplama işlemlerinin yapılabildiği bir elektromekanik araç geliştirdi. ABD'nin 1890 nüfus sayımında başarılı biçimde kullanıldı. Hollerith başarılı olunca bir şirket kurdu ve daha sonra üç firma birleşerek 1924 yılında adını **IBM** olarak değiştirdi.

1941 yılında *Konrad Zuze* Z3 isimli elektrik motorları ile çalıştırılan mekanik bir bilgisayar yaptı. Bu (Z1, Z2, Z3 ve Z4 serisi) program kontrollü ilk bilgisayardır.

1944 yılında *Howard Aiken*, IBM ile iş birliği yaparak **MARK-I**'i yaptı. Bilgiler, delikli kartlarla veriliyor ve sonuçlar yine delikli kartlarla alınıyordu. Saniyede 5 işlem yapabiliyordu. Boyutları, 15,5 m uzunluğunda ve 2,5 m yüksekliğinde idi. İnsan müdahalesi olmadan sürekli olarak, hazırlanan programı yürüten ilk bilgisayar idi ve tam olarak elektronik bir bilgisayar değildi.

1946 yılında Pensilvanya Üniversitesi Moore Elektrik Mühendisliği Okulu'nda *J. Presper Eckert* ve *John Mauchly* önderliğinde **ENIAC** (*Electronic Numerical Integrator and Computer*, Elektronik Sayısal Entegratör ve Hesaplayıcı) isimli sayısal elektronik bilgisayar kamuoyuna tanıtıldı (yapımı 1945'te tamamlanmıştı). Askeri amaçla üretildi ve top mermilerinin menzil tablolarını hesaplamak için kullanıldı. Yaklaşık 18000 elektronik tüp kullanan ENIAC; 150 kW gücünde idi ve yaklaşık 30 ton

ağırlığıyla 167 m^2 yer kaplıyordu. Saniyede 5 bin toplama işlemi yapabiliyordu ve MARK-I bilgisayarından yaklaşık bin kat daha hızlıydı. ENIAC'ın programı kablo ve anahtarlarla değiştiriliyordu; yani program bellekte saklanmıyordu.

1945 yılında *John von Neumann*, **EDVAC** (*Electronic Discrete Variable Automatic Computer*) için yazdığı raporda, programın da veri gibi bellekte saklandığı mimariyi tanımladı. **Von Neumann mimarisi** (*stored-program computer*) olarak bilinen bu mimari, günümüzdeki bilgisayarlarda hala kullanılmaktadır. EDVAC ise 1949'da teslim edildi ve 1951 yılında tam olarak çalışır hale geldi.

Yukarıda anlatılan bilgisayarlar, **emir kodu** (*instruction code*) veya emir kodunu işaret eden **hatırlatıcı isimlerle** (*mnemonic*) programlar yazılmaktadır. Emir kodu ve hatırlatıcı isimlerle yapılan ve bilgisayar mimarisini bilmeyi gerektiren programlamaya **düşük düzey programlama** (*low-level programming*) denir. Daha çok elektronik ve bilgisayar mühendisleri ya da sistemi tasarlayan mühendisler bu programlama türünü kullanır.

Düşük Düzey Programlama Dili

Düşük düzey programlamanın yapılabilmesi için hem işlemcinin hem de hafıza ve işlemciye bağlanan diğer bileşenlerin nasıl çalıştığı ve tasarlandığını çok iyi bilinmesi gerekir.

1.2 Genel Amaçlı Yüksek Düzey Diller

Hatırlatıcı isimler veya emir kodu yerine bildiğimiz konuşma diline yakın **talimatlarla** (*statement*) metin olarak yazılan programlama dilleri, **yüksek düzey programlama dilleri** (*high-level programming language*) olarak adlandırılır.

Talimat

Yüksek seviyeli bir dilde talimatlar (*statement*), yapılması gerekeni kısa ve net olarak belirten **mantıksal satırlardır** (*logical sequence*).

1954 yılında **FORTRAN** (*FORmula TRANslation*), *John Backus* liderliğinde IBM 704 için geliştirilirdi. FORTRAN, `READ`, `WRITE`, üç yollu `IF` gibi 32 farklı talimata sahiptir. İlk yaygın kullanılan, yüksek düzey bir programlama dilidir.

Aşağıda üç kenarı teyp ünitesinden alınan bir üçgenin alanını hesaplayan FORTRAN-II programı verilmiştir [1]. Görüldüğü üzere talimatlar kısa öz ve anlaşılırdır.

```
C AREA OF A TRIANGLE - HERONS FORMULA
C INPUT - CARD READER UNIT 5, INTEGER INPUT
C OUTPUT -
C INTEGER VARIABLES START WITH I,J,K,L,M OR N
READ(5,501) IA,IB,IC
501 FORMAT(3I5)
IF (IA) 701, 777, 701
701 IF (IB) 702, 777, 702
702 IF (IC) 703, 777, 703
777 STOP 1
703 S = (IA + IB + IC) / 2.0
AREA = SQRT( S * (S - IA) * (S - IB) * (S - IC) )
WRITE(6,801) IA,IB,IC,AREA
801 FORMAT(4H A= ,I5,5H  B= ,I5,5H  C= ,I5,8H  AREA= ,F10.2,
$13H SQUARE UNITS)
STOP
END
```

[1] https://github.com/scivision/fortran-II-examples/blob/main/herons_formula.f

Yüksek Düzey Programlama Dili

Emir kodlarını (*instruction code*) ve sembolik isimleri (*mnemonic*) içermeyen programlama dilleri, **yüksek düzey programlama dili** (*high-level programming language*) olarak adlandırılır. Yüksek düzey dillerde **talimatlarla** (*statement*) yazılan programlar yine kendine has derleyiciler tarafından makine diline yani emir kodlarına çevrilir. Her talimat, genellikle birden fazla emirden (*instruction*) oluşur.

1.3 Arapsaçı Kod

1955-1959 yılları arasında **FLOW-MATIC B0** (*Business Language Version 0*) Dili, *Grace Hopper* tarafından UNIVAC-I için geliştirildi. İngilizceye benzeyen ilk yüksek düzey programlama dilidir. Aşağıda B0 programlama dilinde örnek bir program verilmiştir [2].

```
(0)  INPUT INVENTORY FILE-A PRICE FILE-B ; OUTPUT PRICED-INV FILE-C UNPRICED-INV
FILE-D ; HSP D .
(1)  COMPARE PRODUCT-NO (A) WITH PRODUCT-NO (B) ; IF GREATER GO TO OPERATION 10 ;
IF EQUAL GO TO OPERATION 5 ; OTHERWISE GO TO OPERATION 2 .
(2)  TRANSFER A TO D .
(3)  WRITE-ITEM D .
(4)  JUMP TO OPERATION 8 .
(5)  TRANSFER A TO C .
(6)  MOVE UNIT-PRICE (B) TO UNIT-PRICE (C) .
(7)  WRITE-ITEM C .
(8)  READ-ITEM A ; IF END OF DATA GO TO OPERATION 14 .
(9)  JUMP TO OPERATION 1 .
(10)  READ-ITEM B ; IF END OF DATA GO TO OPERATION 12 .
(11)  JUMP TO OPERATION 1 .
(12)  SET OPERATION 9 TO GO TO OPERATION 2 .
(13)  JUMP TO OPERATION 2 .
(14)  TEST PRODUCT-NO (B) AGAINST ZZZZZZZZZZZZ ; IF EQUAL GO TO OPERATION 16 ;
OTHERWISE GO TO OPERATION 15 .
(15)  REWIND B .
(16)  CLOSE-OUT FILES C ; D .
(17)  STOP . (END)
```

Görüldüğü üzere bu zamanlarda yazılan programlarda bir sürü `GO TO` ya da `JUMP TO` talimatı (*statement*) bulunmaktadır. Bu tür kodlarda hangi talimatın hangi sırada icra edileceğini yani kontrol akışını (*control flow*) tespit etmek oldukça zordur. Bu da programın **okunaklığını oldukça azaltır**. İşte okunaklılığın az olduğu bu program kodlarına **arapsaçı kod** (*spaghetti code*) adı verilir. Günümüzde ne yaptığı kolay anlaşılamayan kodlar için bu kavram çokça kullanılmaktadır.

Tüm Kaynak Kodlar Gözle Okunur

İşin doğası gereği **makine diline çevrilecek tüm kaynak kodlar gözle okunmalıdır!**

1958 yılında **LISP** programlama dili, *John McCarthy* önderliğinde *Steve Russell* tarafından geliştirilmiş yüksek düzey programlama dilidir;

```
(format t "Adinizi Giriniz: ")
(setq name (read))
(princ "Merhaba ")
(write name)
```

1959 yılında **COBOL** (*Common Business-Oriented Language*) programlama dili, CODASYL komitesi tarafından İngilizceye benzer yüksek düzey bir dil olarak geliştirilmiştir;

```
IDENTIFICATION DIVISION.
PROGRAM-ID. GREET.

DATA DIVISION.
```

[2] https://gist.github.com/marcgg/f9f2da31a973ba319809

```
WORKING-STORAGE SECTION.
01 USER-NAME PIC A(30).

PROCEDURE DIVISION.
DISPLAY "Enter your name: ".
ACCEPT USER-NAME.
DISPLAY "Hello, " USER-NAME "!".
STOP RUN.
```

1958 yılında daha sonra **ALGOL** (*ALGOritmic Language*) adını alan **IAL** (*International Algebraic Language*) programlama dili geliştirilmiştir. Bu dil daha sonra ortaya çıkan birçok yapısal dile önderlik etmiştir. Bu dil ile birlikte **kod blokları** (*code block*), iç içe fonksiyon (*nested function*), değişken kapsamı (*scope*) ve dizi (*array*) kavramları programcının hayatına girmiştir. 1958–1969 arası `PROCEDURE` talimatı bu dile eklendi. ALGOL dili, barındırdığı özellikler sebebiyle, kendisinden sonra ortaya çıkan bütün programlama dilleri için bir esin kaynağı olmuştur;

```
BEGIN
  FILE F (KIND=REMOTE);
  EBCDIC ARRAY E [0:11];
  REPLACE E BY "HELLO WORLD!";
  WRITE (F, *, E);
END.
```

1958 yılındaki FORTRAN diline çıktı üretmeyen fonksiyonlar için `SUBROUTINE`, değer üreten fonksiyonlar için `FUNCTION`, `CALL` ve `RETURN` özellikleri eklenmiş ve **FORTRAN-II** adını almıştır. Benzer şekilde 1958 ile 1969 yılları arasında ALGOL diline çıktı üretmeyen fonksiyonlar için `PROCEDURE` ve `BEGIN-END` arasında kod bloğu özellikleri eklenmiştir.

1966 yılındaki FORTRAN-IV diline `DOUBLE PRECISION`, `COMPLEX` ve `LOGICAL` **veri tipleri** (*data type*) ile Mantıksal `IF` talimatları (*statement*) eklenmiştir.

1964 yılında Dartmouth College'da *John G. Kemeny* ve *Thomas E. Kurtz* tarafından geliştirilen **BASIC** (*Beginner's All-purpose Symbolic Instruction Code*) öğrenilmesi ve kullanılması kolay olacak şekilde tasarlanmış bir programlama dilidir. Bilimsel olmayan alanlardaki öğrencilerin ve yeni başlayanların bilgisayar kullanabilmesini sağlamak amacıyla İngilizce benzeri komutlarla tasarlanmıştır. Karmaşık donanım detaylarını gizleyerek programcının mantığa odaklanmasına izin verir. Kodlar genellikle satır satır gerçek zamanlı yorumlanarak çalıştırılır, bu da hataların anında bulunmasını kolaylaştırır. 1980'li yılların başında kadar ev bilgisayarlarının (Apple II, Commodore 64, IBM PC gibi) varsayılan dili haline gelmiştir;

```
10 REM A simple program to greet the user
20 CLS ' Clears the screen (command might vary by implementation)
30 PRINT "What is your name? ";
40 INPUT NAME$ ' The '$' indicates a string variable
50 PRINT "Hello, "; NAME$; "!"
60 GOTO 30 ' Jumps back to line 30 to run the program again
70 END ' Terminates the program
```

1.4 Değişken

Değişkenler (*variable*) aslında cebirde kullandığımız değişkenlerdir. Cebir (*algebra*) işlemlerinde doğrudan problemde verilen sayıları değil, onları temsil eden değişkenleri kullanırız.

$$x \in \mathbb{R},\ 3x^2 + 2x + 10 \tag{1.1}$$

$$c^2 = a^2 + b^2 \tag{1.2}$$

$$c = 2\pi r \tag{1.3}$$

Yukarıdaki cebirsel ifadelerin ilkinde bir polinom, ikincisinde ise bir dik üçgenin kenar uzunlukları arasındaki ilişki verilmiştir. Bu ifadelerde x, a, b ve r **bağımsız değişken**, c ise **bağımlı değiş-**

kendir. Örneklerdeki 3,2,10,2 ve π ise **sabit** (*constant*) sayılardır.

Bilgisayarda Değişkenler

Değişkenler;
- ◇ Bellekte belirli bir yer kaplayan ve içinde veri saklayan yapılardır.
- ◇ Saklanan verinin bellekte kapladığı alan ve temsil edilme biçimi değişkenin **veri tipine** (*data type*) bağlıdır. Örneğin tam sayı ve kayan noktalı sayı farklı veri tipleridir.

Tam sayılar veri tiplerinden en çok kullanılanıdır ve bellekte **ikili** (*binary*) tabanda tutulur. Tam sayıyı ifade eden ikili sayının en anlamlı biti, işaret (*sign*) biti olarak adlandırılır ve bu bitin değeri `1` ise sayı negatif, `0` ise sayı pozitiftir. Tablo 1.1'de en çok kullanılan tam sayılara ilişkin bellek miktarı ve sayı sınırları verilmiştir.

Bit Sayısı	Bellek Miktarı	Sayı Sınırları
8 Bit	1 bayt	-128 ile +127 arasında veya 0 ile 255 arasında
16 bit	2 bayt	-32.768 ile +32.767 arasında veya 0 ile 65.535 arasında
32 bit	4 bayt	-2.147.483.648 ile +2.147.483.647 arasında veya 0 ile 4.294.967.295 arasında
64 bit	8 bayt	-9.223.372.036.854.775.808 ile 9.223.372.036.854.775.807 arasında veya 0 ile 18.446.744.073.709.551.615 arasında

Tablo 1.1: Tam Sayılar ve Sayı Sınırları

İşaretli ve İşaretsiz Tam Sayılar

n bitlik bir tamsayı işaretli (*signed*) olarak kullanılacaksa değer aralığı -2^{n-1} ile $2^{n-1}-1$ arasında olacaktır. Aynı bit sayısına sahip sayı, işaretsiz (*unsigned*) olarak kullanılacak ise 0 ile 2^n-1 aralığında değer alabilecektir.

Gerçek sayılar da en çok kullanılan veri tiplerindendir. Bu sayılar, **kayan noktalı sayı** (*floating point number*) olarak biçimlendirilir. Kayan noktalı sayı kavramı ilk olarak 1914 yılında Leonardo *Torres Quevedo* tarafından Charles Babage'ın analitik makinesine kullanılmak üzere yayınlanmıştır.

Türkçemizde gerçek sayılarda ondalık ayracı olarak virgül kullanılır (Örnek: 50,25). Aslında Türkçe olarak kayan virgüllü sayı dememiz gerekirdi. Ancak **kodu İngilizce yazacağımızdan ondalık ayracı olarak nokta kullanılmalıdır** (Örnek: 50.25). Bu nedenle bundan sonraki örneklerde ondalık ayracı olarak nokta kullanılacaktır.

$$12345 = 1234.5 \times 10^1 = 123.45 \times 10^2 = 12.345 \times 10^3 = 1.2345 \times 10^4 = 0.12345 \times 10^5 \tag{1.4}$$

Yukarıda ondalık tabanda verilen gerçek bir sayıya ilişkin kesir noktasının kaydırılması ile aynı sayı ifade edildiği bir örnek verilmiştir. Kesir noktası sola kaydırıldığında sayı, 10'a bölünmüş, sağa kaydırıldığında ise sayı 10 ile çarpılmış olur. Kayan noktalı sayının üs kısmı dışında kalan kısmı **kesir** (*fraction*) olarak adlandırılır.

Kayan Noktalı Sayılar Da İkili Tabanda Tutulur

Tam sayılarda olduğu gibi **kayan noktalı sayılar da bilgisayarlarda ikili (*binary*) sayı tabanında tutulur**. Örneğin ondalık tabanda 50.25 sayısı ikili tabanda 1.1001001×2^5 olarak ifade edilir.

1938 yılında *Konrad Zuse*, ilk ikili (*binary*) programlanabilir mekanik bilgisayar olan Z1'i yapmıştı. Bu bilgisayar, 7 bitlik işaretli üs, 17 bitlik anlamlı değer ve bir işaret biti içeren 24 bitlik ikili tabanda kayan noktalı sayı kullanmıştır. Günümüzde standartlaşmış ve çok kullanılan kayan noktalı sayılara ilişkin bellek miktarı ve sayı sınırları verilmiştir[3] (Tablo 1.2).

Bit Sayısı	Bellek Miktarı	Sayı Sınırları
32 bit	4 bayt	1.2×10^{-38} ile $3.4 \times 10^{+38}$ arasında **tek hassasiyetli** gerçek sayılar; en soldaki 1 bit işaret biti, sonraki 8 bit üs ve geri kalan 23 bit ise kesir olarak kullanılan ikilik tabanda sayılardır.
64 bit	8 bayt	2.2×10^{-308} ila $1.8 \times 10^{+308}$ arasında **çift hassasiyetli** gerçek sayılar; en soldaki 1 bit işaret biti, sonraki 11 bit üs ve geri kalan 52 bit ise kesir olarak kullanılan ikilik tabanda sayılardır.

Tablo 1.2: Kayan Noktalı Sayılar ve Sınırları

Program yazılırken tanımlanacak değişkenlere ilişkin bellek ihtiyacı programcı tarafından belirlenir. Programcı değişkene tahsis edilecek bellek miktarına ve tutacağı içeriğe göre **veri tipi** (*data type*) belirler. Değişkenlerin veri tipine kısaca **değişken tipi** (*variable type*) adı da verilir. Veri tiplerine verilen adlar dilden dile değişir; Örneğin PASCAL dilinde 8 bitlik tam sayıya `bayt` denilirken, C, C++ ve C# dilinde `char` adı verilir. Programcı tarafından kullanılacağı amaca göre belirleyeceği veri tipinde istediği kadar değişken tanımlayabilir.

1.5 Fonksiyon

Matematikte **fonksiyon** (*function*), A kümesinin her elemanını B kümesinin tam bir elemanına eşleyen **ilişkidir** (*relationship*).

Girdi	İlişki	Çıktı
1	x2	2
2	x2	4
3	x2	6
4	x2	8
...	...	...

Tablo 1.3: İki İle Çarpma Fonksiyonu

Girdi olan bağımsız değişkenler ile çıktı olan bağımlı değişken arasındaki bu ilişki, matematikte aşağıdaki şekilde **ifade** (*expression*) edilir.

$$f(x) = 2x \tag{1.5}$$

Burada f, fonksiyon anlamında x ise girdi anlamında olup **parametre** olarak adlandırılır. Böylece tabloda verilen fonksiyon tablosuna her girdiyi yazmak zorunda kalmayız. Bazen fonksiyona aşağıdaki örneği verilen şekilde bir ad verilmez;

$$y = 2x \tag{1.6}$$

Bazı durumlarda ise girdi daha detaylı tanımlanır;

$$x \in \mathbb{R}, f(x) = 2x + 3 \tag{1.7}$$

[3] IEEE Computer Society (2019-07-22). IEEE Standard for Floating-Point Arithmetic. IEEE STD 754-2019. IEEE. pp. 1–84.

Bu fonksiyon, reel sayı kümesindeki her x elemanını, hesaplanan f(x) değerine eşleyen bir ifadedir. Yani x=1.0 argümanını f(1.0)=5.0 reel sayısına eşler. Kısaca fonksiyon 5.0 reel sayısını hesaplayıp **döndürür** (*return*).

$$a \in \mathbb{R}, \quad b \in \mathbb{Z}, \quad g(a,b) = 2a + b \tag{1.8}$$

Bu fonksiyon ise reel sayı kümesinden her a elemanı ve tam sayılar kümesinden her b elemanını, hesaplanan g(a,b) değerine eşleyen bir ifadedir. Yani a=1.0 ve b=1.0 argümanlarını alarak yine reel sayı kümesinden 3.0 reel sayısına eşler. Kısaca bu fonksiyon, 3.0 reel sayısını hesaplayıp döndürür (*return*).

Örneklerdeki x, a ve b parametre olarak adlandırılır. f(1.0) ifadesindeki 1.0 ise parametrenin o an andığı değeri belirtir ve **gerçek parametre** (*actual parameter*) ya da **argüman** (*argument*) olarak adlandırılır.

$$f(3.0) + g(3.0, 2.0) + f(2.0) + g(1.0, 3.0) \tag{1.9}$$

Fonksiyonlar ifade olarak tanımlandıktan sonra yukarıdaki gibi tekrar tekrar **çağrılabilirler** (*call*).

1.6 Yapısal Programlama

1970'li yıllara kadar her ne kadar çıktı üretmeyen fonksiyonlar için `SUBROUTINE`, birden fazla çağrı noktası olabilen `COROUTINE` ve `FUNCTION` kullanılıyor olsa da hala `GOTO` ve `JUMPTO` ifadeleri kullanılmakta ve kafa karışıklığına devam etmekteydi.

GOTO İfadelerinin Zararlı Kanul Edilmesi

1968 yılında *Edsger W. Dijkstra* tarafından, kontrol akışını anlamakta zorlandığımızdan `GOTO` ve `JUMPTO` ifadeleri **zararlı olarak dile getirilmiştir**.

1970 yılında *Niklaus Wirth* tarafından **Pascal** dili geliştirildi. Bu dil yapısal programlamayı öğretmek için geliştirilmiştir. **Yapısal programlamada** (*structured programming*) verileri taşıyan veri yapıları (değişkenler ve diziler) ile bunu işleyen kontrol yapıları (`if`, `for`, `do`, `repeat`) birbirinden ayrılmıştır.

Yapısal Programlama

Yapısal programlama, fonksiyonların birbirini çağırmasıyla yapılır. Yapısal programlamada veri ile veriyi işleyen yapılar birbirinden ayrıldığından, arapsaçı programlamanın aksine kontrol akışını (*control flow*) anlamak oldukça kolaydır. Bu nedenle **yapısal programlamada okunaklık olukça yüksektir**.

Bunların yanında Pascal, kendi kendini çağıran **özyinelemeli** (*recursive*) fonksiyonlar ile değişkenlerin adreslerini tutan **göstericileri** (*pointer*) desteklemektedir[4]. Yapısal programlamanın en çok uygulandığı Pascal dilinde ana fonksiyon noktalı `BEGIN-END.` bloğudur. Aşağıda bir sayısı konsoldan alan ve faktöriyelini hesaplayıp sonucu yazdıran bir program örneği verilmiştir;

```
1  PROGRAM FaktoriyelHesaplama;
2
3  FUNCTION Faktoriyel(n: integer): longint;
4  VAR {1.Yerel Degiskenler}
5    i: integer;
6    sonuc: longint;
7  BEGIN {2. Yerel Fonksiyon Algoritmasi}
8    sonuc := 1;
9    FOR i := 1 TO n DO
10     sonuc := sonuc * i;
11   Faktoriyel := sonuc;  {3. sonuc degeri geri dondurluyor}
```

[4] Wirth, Niklaus (1976). Algorithms + Data Structures = Programs. Prentice-Hall. p. 126

```
12 END;
13
14 VAR {1. Ana Fonksiyon Verileri}
15   sayi: integer;
16 BEGIN
17   {2. Ana Fonksiyon Algoritmasi}
18   WRITELN('Faktoriyeli hesaplanacak sayiyi giriniz: ');
19   READLN(sayi);
20   IF sayi < 0 THEN
21     WRITELN('Negatif sayilarin faktoriyeli tanimsizdir.')
22   ELSE
23     WRITELN(sayi, '! = ', Faktoriyel(sayi));
24   {3.Ana Fonksiyondan Cikiliyor}
25 END.
```

Kod 1.1: Faktöriyel Hesaplayan Pascal Programı

Geri değer döndürmeyen ve sadece belli bir mantığı gerçekleştiren fonksiyonlar, Pascal dilinde `PROCEDURE` olarak adlandırılır. Aslında bu terim ALGOL dilinden gelmektedir.

Yapısal Programlamanın Genel Çerçevesi

1. Programın başladığı yeri belirten bir **ana fonksiyon** (*main function*) tanımlanır. Kodlaması yapılacak işi birbirini çağıran fonksiyonlar olarak bu ana fonksiyonda yazarız.
2. Her fonksiyonda ilk önce **veri yapıları** (*data structure*) tanımlanır. Bunlar;
   (a) En basit veri yapısı `char`, `integer`, `float` ve `double` gibi **ilkel veri tiplerinden** (*primitive data type*) olabilen **değişken** (*variable*),
   (b) İlkel veri tiplerinden meydana gelen **dizi** (*array*),
   (c) İlkel veri tiplerinin birkaçından veya dizilerinden bir araya gelmesiyle oluşan **yapılar** (*structure*),
   (d) Ya da dinamik olarak yani çalışma anında (*run-time*) birbirine bağlanmış verilerden oluşan **bağlı liste** (*linked list*) olabilir.
3. Her fonksiyonda tanımlanan veri yapılarının ardından bu yapıları işleyecek **kontrol yapıları** (*control structure*) tanımlanır. Kontrol yapıları bir veya daha fazla veya iç içe `if`, `if-else`, `switch`, `while`, `do-while`, `for`, `continue`, `break`, `goto` ve `return` **talimatlarından** (*statement*) oluşur.

C Programlama Dili,*Dennis M. Ritchie* tarafından Bell Laboratuvarlarında UNIX işletim sistemini geliştirmek için geliştirilen genel amaçlı, üst düzey bir dildir. C, ilk olarak 1972'de DEC PDP-11 bilgisayarında çalıştırılmıştır. 1978 yılında *Brian Kernighan ve Dennis Ritchie*, günümüzde K&R standardı olarak bilinen C'nin ilk kamuya açık kılavuzunu hazırlamışlardır.

C programlama dili; **yapısal programlama dilidir**, hızlı ve verimli programlar üretir, hatırlatıcı isimlerle (*mnemonic*) yazılan montaj (*assembly*) dili desteği ile **düşük düzey programlamayı** (*low-level programming*) da destekler ve standart başlık dosyaları sayesinde çeşitli bilgisayar platformlarında derlenebilir. Bu özellikleri nedeniyle neredeyse tüm işletim sistemlerinin çekirdeği C dilinde yazılmıştır ve sistem dili olarak da adlandırılmaktadır. Yukarıda verilen Pascal yapısal programının C dilinde kodlanmış hali aşağıda verilmiştir;

```
1  #include <stdio.h>
2
3  long faktoriyel(int n) {
4    //1.yerel degiskenler
5    int i;
6    long sonuc = 1;
7    //2. yerel fonksiyon algoritmasi
8    for (i = 1; i <= n; i++) {
9      sonuc = sonuc * i;
10   }
11   //3. sonuc degeri geri donduruluyor
12   return sonuc;
```

```c
}

int main() { // Ana fonksiyon
  //1.yerel degiskenler
  int sayi;
  //2. Ana fonksiyon algoritması
  printf("Faktoriyeli hesaplanacak sayiyi giriniz: ");
  scanf("%d", &sayi);
  if (sayi < 0) {
    printf("Negatif sayilarin faktoriyeli tanimsizdir.\n");
  } else {
    printf("%d! = %ld\n", sayi, faktoriyel(sayi));
  }
  //3.Ana fonksiyondan cikiliyor
  return 0;
}
```

Kod 1.2: Faktöriyel Hesaplayan C Programı

Yapısal programlamanın en çok uygulandığı bir başka dil olan C dilinde ana fonksiyon, `main()` fonksiyonudur. Pascal dilinde `PROCEDURE` olarak tanımlanan geri değer döndürmeyen fonksiyonlar, C dilinde `void` olarak tanımlanır.

Programın icrası çoğunlukla fonksiyonun ilk talimatından başlar ve sırasıyla icra edilir. Yani kontrol akışı (*control flow*) ilk talimattan başlar ve sırasıyla düzgün devam eder. İşte **programın icrasını değiştirmeyen** bu tür talimatlara **ardışık işlem** (*sequential operation*) adı verilir.

Ardışık İşlemler

1. Değişkenlerin kimliklendirildiği **değişken tanımlamaları** (*identifier definition*):
   (a) `int yariCap=3;`
   (b) `const float PI=3.14;`
   (c) `float daireninAlani,daireninCevresi;`
2. **İfadelerden** (*expression*): (Sabit, değişken ve operatör içeren söz dizimlerine **ifade** adı verilir)
   (a) `daireninAlani=PI*yariCap*yariCap;`
   (b) `float daireninCevresi=2.0*PI*yariCap;`
3. Klavyeden veri okuma ve konsola veri yazma gibi **giriş-çıkış işlemleri** (*input-output operation*):
   (a) `printf("Girdiginiz sayilarin toplami: %d\n", toplam);`
   (b) `scanf("%d", &sayi2);`

Kontrol işlemleri (*control operation*) ise programın icra sırasını yani kontrol akışını (*control flow*) değiştiren kontrol yapılarıdır.

Kontrol İşlemleri

1. **Duruma göre seçimler** (*conditional choice*): `if`, `if-else` ve `switch` talimatları.
2. **İlişkisel döngü** (*relational loop*): `while`, `do-while` ve `for` talimatları.
3. **Dallanmalar** (*jump*): `continue`, `break`, `goto` ve `return` talimatları.

Fiziksel ve Mantıksal Satır

Kontrol işlemlerinde; kodlanan **mantıksal satırlar** (*logical sequence*) ile **fiziksel olarak icra edilen satırlar** (*physical sequence*) birbirinden farklıdır.

Yapısal programlamada, yazılımı yapılacak sürece ilişkin nelerin yapılacağını değil, işin nasıl yapılacağı üzerinde durulur. Seksenli yıllara kadar yazılım ihtiyaçları yapısal programlama ile çözülmüştür.

1.7 Yazılım Geliştirme Süreci

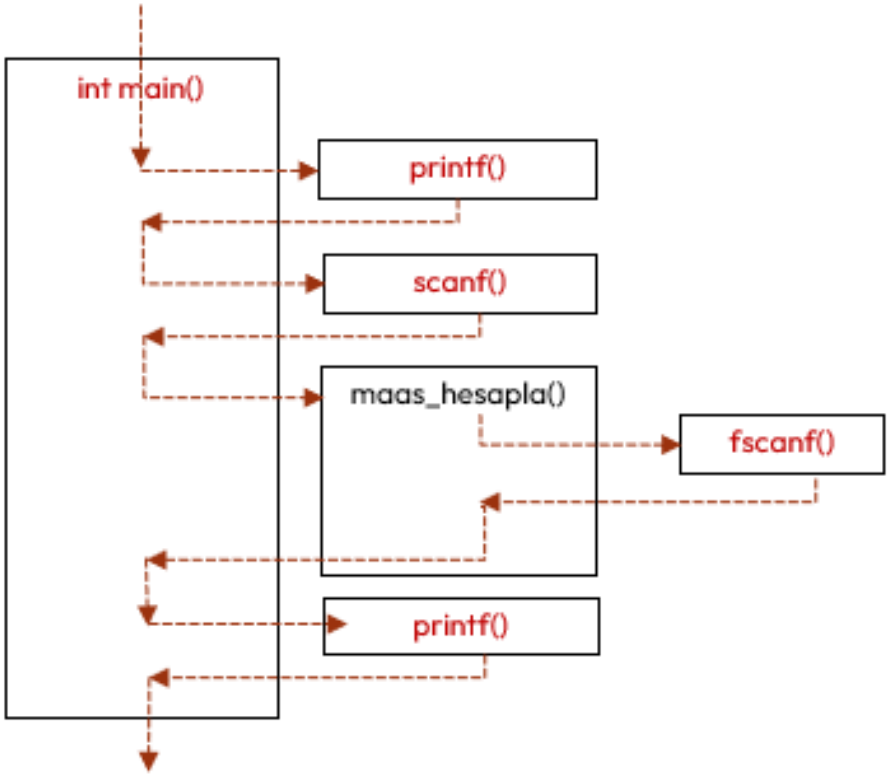
Bilgisayar programlanırken sadece karmaşık bir matematik formülünün hesaplanması problemini ele almaz. Bilgisayarların yaygınlaşması ve hayatımıza girmesiyle birlikte yaşantımızın bir parçası olan çoğu problemi çözer hale gelmiştir. Gümünüzde bilgisayarla çözülebilecek problemlerin karmaşıklığı o kadar artmıştır ki problemlerin tanımından yazılımın tamamlanmasına kadar olan sürecin yönetimi çok önemli hale gelmiştir. Bu süreç, **yazılım geliştirme süreci** (*software development process*) olup şu aşamalardan oluşur:

- **Gereksinim analizi** (*requirements analysis*),
- **Şartname** (*specification*),
- **Tasarım** (*design*),
- **Gerçekleştirim** (*implementation*),
- **Doğrulama ve geçerlilik** (*verification and validation*) ve
- **Bakım** (*maintenance*).

Programlama dili özellikle yazılım geliştirme sürecinin "gerçekleştirim aşamasında" (*implementation stage*) kullanılır; ancak dilin özellikleri tasarım kararlarını da etkileyebilir.

1.8 Yapısal Programlama Paradigması

Yapısal programlama paradigmasında yazılım ihtiyaçları, **birbirini çağıran fonksiyonlar** yazılarak çözülür (Şekil 1.1). Bu durum bir önceki başlıkta verilen Kod 1.1 ve Kod 1.2 örneklerinden de anlaşılabilir. Fonksiyonların algoritmasını oluşturan **talimatlar** (*statement*) art arda izlenerek programların neler yaptığı, bu talimatlar izlenerek **kontrol akışı** (*control flow*) anlaşılabilir. Bu paradigma yani fonksiyonların birbirini çağırmasıyla yapılan programlama yaklaşımına sahip diller, yazılımı yapılacak sürece ilişkin nelerin yapılacağını değil, işin nasıl yapılacağını belirtirler.



Şekil 1.1: Yapısal Programlama Paradigması

Yapısal programlamayla yazılım geliştirmede en sık karşılaşılan karmaşıklık, veri ile veriyi işleyen fonksiyonların birbirinden kopuk kurgulanmasıdır. Bu fonksiyon yaklaşımı küçük ölçekli projelerde hızlı sonuç verse de, sistem büyüdükçe durum yönetimi kontrolden çıkar:

- ◇ Kod ne kadar büyürse, modüllere ayırma imkânı olmasına rağmen, fonksiyonlar arasındaki bağımlılık da o kadar artar. Bir fonksiyonun parametrelerindeki değişiklik, onun kullanıldığı tüm yerlerde değişiklik gerektirir. **Değişim yönetimi** (*change management*) çok zordur ve uzun zaman alır.
- ◇ Hata durumunda yapılacak işlemlere ilişkin kod ile iş sürecini gerçekleştiren kod iç içedir. Aynı fonksiyonun bazı durumlarda hata, bazı durumlarda ise değer döndürmesi mümkündür. Çoğu durumda **hataların izini sürmek** (*error handling*) işi zorlaşır.
- ◇ Yapısal programlamada, **gösterici** (*pointer*) kullanımında erişilmesi istenmeyen bellek bölgelerine erişilmesi halinde istenmeyen program davranışları ortaya çıkar. Kodun **güvenli** (*safe code*) olarak çalıştığının incelenmesi çok zordur.
- ◇ Sürekli olarak benzer projelerde aynı kodları **tekrar yazmak** (*duplicate code*) gereklidir.
- ◇ Emreden paradigmanın burada belirtilen sebepler başta olmak üzere çeşitli problemleri bulunmaktadır. Bu nedenle 1980'li yıllarda birçok yazılım projesi **başarısız** (*fail*) olmuştur.

Yazılım Ne Zaman Başarısız Olur?

Bir yazılım yalnızca çalışmadığında değil; kullanıcıya beklenen değeri sunmadığında, maliyet hedeflerini aşınca veya zamanında teslim edilemediğinde de başarısız kabul edilir. Bu nedenle bir yazılım projesi özellikle şu durumlarda başarısız sayılır:
- ◇ Kullanıcı ihtiyaçlarını ve beklentilerini karşılayamıyorsa,
- ◇ Öngörülen bütçenin üzerine çıkıyorsa,
- ◇ Planlanan teslim tarihine yetiştirilemiyorsa.

Yapısal programlamadaki bu sıkıntıları gören yazılımcıların imdadına Simula ve Smalltalk programlama dillindeki yaklaşım yetişmiştir. Bu diller oldukça yavaş olmasına rağmen getirdiği çözümler oldukça yenilikçiydi. Smalltalk, 1972 yılında Xerox Park şirketinde *Alan Kay* önderliğinde *Dan Ingalls*, *Adele Goldberg*, *Ted Kaehler*, *Diana Merry*, *Scott Wallace* ve *Peter Deutsch* tarafından geliştirilmiş bir dildir. Smalltalk dilinde **nesneler** (*object*) temel yapı taşlarıdır.

Yapısal programlamada geliştirmede en sık karşılaşılan karmaşıklık, veri ile veriyi işleyen fonksiyonların birbirinden kopuk kurgulanmasıdır. Bu yaklaşım küçük ölçekli projelerde hızlı sonuç verse de, sistem büyüdükçe durum yönetimi kontrolden çıkar.

1.9 Nesne Yönelimli Programlama Paradigması

Nesne yönelimli programlama yani **OOP** (*Object-Oriented Programming*), durumu ve davranışı tek bir birim yani **nesne** (*object*) altında toplayarak bir önceki başlıktaki sorunların çoğunu çözer. Nesneler, **duruma** (*state*) sahiptir ve nesnenin o anki verisidir. Nesneler ayrıca textbfdavranışa (*behavior*) sahiptir ve bu davranışlar ile nesne verisini işler.

Nesne Yönelimli Programlama Paradigması

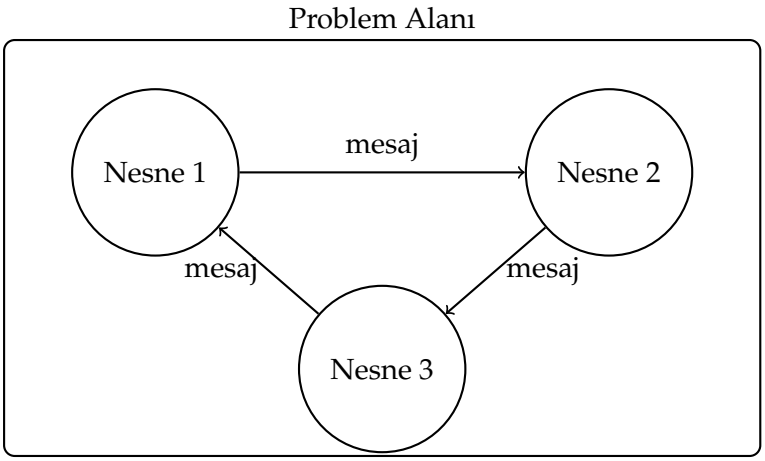
Yapısal programlamanın aksine **nesne yönelimli programlama** paradigmasında, nesneler programın temel yapı taşlarıdır. **Nesneler; durum ve davranışlara sahiptir**. Programlama, nesnelerin birbirlerine ileti göndermesi (*message-passing*) ile yapılır.

Buna; Üye, Kitap ve Kütüphane Görevlisi içeren bir Kütüphane Sistemi örnek verilebilir: Bu nesneler birbirleriyle mesajlaşarak/etkileşerek problemi modeller (Şekil 1.3).

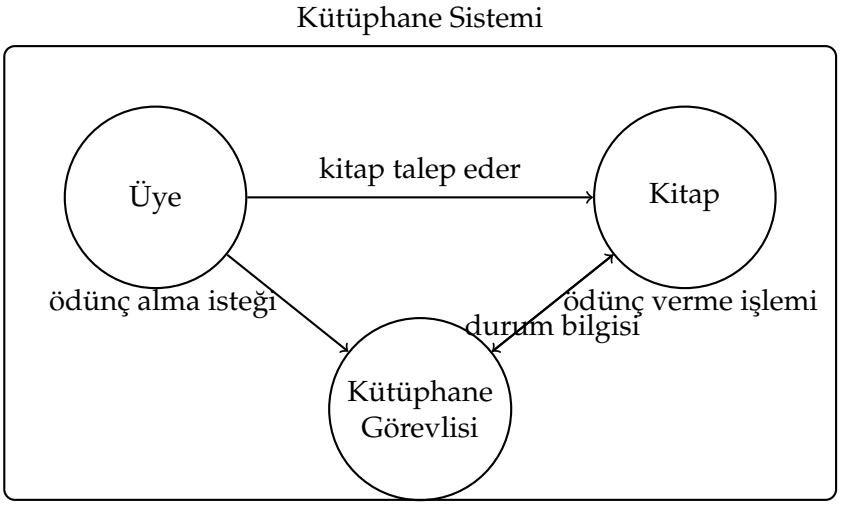
OOP ile yazılım geliştirme sürecinde **UML** (*Unified Modeling Language*) gibi grafiksel gösterimler tasarım aşamasında çoğunlukla kullanılır.

OOP dört temel kavram üzerine kuruludur:

- ◇ **Soyutlama (Abstraction):** Karmaşık iç çalışma mekanizmalarını dış dünyadan gizleyerek yalnızca gerekli arayüzleri sunmak.
- ◇ **Kapsülleme (Encapsulation):** Sınıf değişkenlerini korumaya almak (private) ve erişimi onaylanmış metotlar üzerinden sağlamak.



Şekil 1.2: Nesne Yönelimli Geliştirme Paradigması



Şekil 1.3: Kütüphane sisteminin nesneler ve aralarındaki etkileşimlerle modellenmesi

- ◇ **Kalıtım (Inheritance):** "Bir türüdür" ilişkisi kurarak ortak kod gruplarını üst sınıflara taşımak.
- ◇ **Çok Biçimlilik (Polymorphism):** Aynı mesaja farklı nesne tiplerinin kendi yapısına uygun özel yanıtlar verebilmesi.

OOP paradigmasında, yukarıdaki kavramları bir anda özümsemek gerektiğinden **öğrenme eğrisi diktir**.

2. Bölüm
Nesne Odaklı Zihniyet

2.1 Sınıf ve Nesne Kavramı

Nesne yönelimli programlamanın temelinde iki ana kavram bulunur: **sınıf** ve **nesne**.

Sınıf (*class*), nesnelerin hangi özelliklere ve davranışlara sahip olacağını tanımlayan bir şablondur. Bir sınıf doğrudan çalışan bir varlık değildir; nesne üretmek için kullanılan bir plandır.

Nesne (*object*) ise bu sınıftan üretilmiş somut varlıktır. Başka bir ifadeyle sınıf plan, nesne ise bu plandan üretilmiş gerçek örnektir.

Nesne-Sınıf İlişkisi

Nasıl ki bir mimari plandan birçok ev inşa edilebiliyorsa, bir sınıftan da birçok nesne üretilebilir. Mimari plan sınıfa, bu plandan yapılan evler ise nesnelere karşılık gelir.

Örneğin bir `Araba` sınıfı tanımlanabilir. Bu sınıftan `araba1`, `araba2` ve `araba3` gibi farklı nesneler üretilebilir. Her nesne aynı sınıftan türemiştir ancak her biri kendi verilerine yani durumuna sahiptir.

```
class Araba {
  public String marka;
  public int hiz;

  public void hizlan(int miktar) {
    this.hiz = this.hiz + miktar;
  }
}

// Kullanım:
public class Main {
  public static void main(String[] args) {
    Araba araba1 = new Araba();
    araba1.marka = "Toyota";
    araba1.hiz = 0;

    Araba araba2 = new Araba();
    araba2.marka = "BMW";
    araba2.hiz = 50;

    araba1.hizlan(20);
    araba2.hizlan(30);
  }
}
```

Bu örnekte `Araba` bir sınıftır. `araba1` ve `araba2` ise bu sınıftan üretilmiş iki farklı nesnedir.

2.2 Nesnenin Durumu, Özellikleri ve Davranışları

Bir nesnenin sahip olduğu veriler özellik (*property*) olarak adlandırılır. Nesne özellikleri, sınıflarda **alan** (*field*) olarak tanımlanır. Örnekte, `marka` ile `hiz`, `Araba` sınıfının alanlarıdır. Bu alanların belirli bir andaki değerleri ise nesnenin **durumunu** (*state*) oluşturur.

Özellik ve Alan İlişkisi

Genel olarak sınıflarda tanımlanan alanlar, kontrollü olarak dış dünyaya açıldığında **özellik** (*property*) olarak adlandırılır. Her alan bir özelliğe dönüşmeyebilir. İleride anlatılacaktır.

Bir nesnenin yapabildiği işlemlere ise **davranış** denir. OOP dillerinde davranışlar **yöntem** (*method*) olarak adlandırılır. Yapısal programlamadaki fonksiyonlara (*function*) benzer ve onların gibi kodlanırlar. Fonksiyonlardan farklı olarak yöntemler bir nesneye ait davranışlardır (*behavior*).

```
class BankaHesabi {
  public String hesapNo;
  public double bakiye;

  public void paraYatir(double miktar) {
    if (miktar > 0) {
      this.bakiye += miktar;
    }
  }

  public void paraCek(double miktar) {
    if (miktar > 0 && miktar <= this.bakiye) {
      this.bakiye -= miktar;
    }
  }

  public double bakiyeOgren() {
    return this.bakiye;
  }
}

// Kullanım:
public class Main {
  public static void main(String[] args) {
    BankaHesabi hesap = new BankaHesabi();
    hesap.bakiye = 0.0;
    hesap.paraYatir(1000.0);
  }
}
```

Kod 2.1: Banka Hesabı Sınıfı

Bu örnekte bir banka hesabı nesnesi için:

- **Özellikler**: hesap numarası, bakiye, hesap sahibi
- **Durum**: bakiyenin o andaki değeri, hesabın aktif olup olmaması
- **Davranışlar**: para yatırmak, para çekmek, bakiye öğrenmek

Burada `hesapNo` ve `bakiye` nesnenin durumlarıdır. `paraYatir()`, `paraCek()` ve `bakiyeOgren()` ise nesnenin davranışlarıdır. Bu davranışlar, nesnenin durumlarından etkilenirler. Dolayısıyla her nesnenin durumu aynı olmayacağı için davranışları da farklı olur.

2.3 Sınıf ve Nesnelerin UML Sınıf Diyagramları ile Gösterilmesi

UML (*Unified Modeling Language*), yazılımları görsel olarak modellemek için kullanılan standart bir gösterim dilidir. Bir sonraki bölümde detaylı anlatılacaktır. Sınıf diyagramları, sınıfların özelliklerini, davranışlarını ve aralarındaki ilişkileri göstermek için kullanılır. Bir UML sınıf diyagramı genellikle Sınıf adı, Özellikler ve Yöntemleri içeren üç bölümlü bir dikdörtgenden oluşur.



Şekil 2.1: Sınıfların UML Diyagramında Gösterilmesi

UML gösteriminde; genellikle şu sembollerle ifade edilir:

- Erişim belirleyiciler (*access modifier*): `+`: genel (*public*), `-`: özel (*private*),`#`: korumalı (*protected*)
- Soyut Sınıf ve Soyut Yöntemler: italik yazılır. İleride anlatılacaktır.
- `static` üyeler: Altı çizili yazılır. İleride anlatılacaktır.

2.4 Bilgi ve Veri Gizleme Nedir?

Bilgi gizleme (*information hiding*), bir sınıfın iç işleyişine ait ayrıntıların sınıf dışından saklanmasıdır. Başka bir ifadeyle, bir nesnenin içinde hangi verilerin tutulduğu, bu verilerin nasıl değiştirildiği veya bir işlemin içeride hangi adımlarla yapıldığı dış dünyaya açılmaz.

Kullanıcıya yalnızca gerekli olan dış arayüz sunulur. Bilgi gizleme kavramının sınıf içindeki verilerin tutulduğu alanlara (*field*) uygulanmış hâline çoğu zaman **veri gizleme** (*data hiding*) denir. Veri gizlemede amaç, sınıfın içindeki alan değişkenlerine doğrudan erişimi engellemektir. Çünkü nesnenin durumunu temsil eden veriler doğrudan değiştirilirse nesne geçersiz veya tutarsız bir duruma düşebilir.

Kod 2.1 örneğinde kullanım sırasında bir banka hesabının bakiyesi doğrudan dışarıdan değiştirilebilirse şu durum oluşabilir:

```
public class Main {
  public static void main(String[] args) {
    BankaHesabi hesap = new BankaHesabi();
    hesap.bakiye = -5000; //!!
    hesap.paraYatir(1000.0);
  }
}
```

Bu işlem teknik olarak mümkün olursa banka hesabı nesnesi geçersiz bir duruma düşer. Çünkü normal şartlarda bakiye, yalnızca belirli kurallar çerçevesinde değişmelidir. Bu nedenle `bakiye` gibi kritik verileri tutan alanlar `private` yapılır ve dışarıdan doğrudan erişime kapatılarak veri gizleme yapılır.

Veri gizleme (*data hiding*) ile sınıfın iç verilerini dış müdahaleye kapatarak nesnenin tutarsız duruma düşmesi engellenmesi işlemidir. Bu amaçla **erişim belirleyiciler** (*access modifier*) kullanılır. Nesne yönelimli programlamada yaygın olarak üç temel erişim belirleyici bulunur:

- **Özel** (`private`): Sadece sınıf içinden erişilebilir.
- **Genel** (`public`): Sınıf dışından da erişilebilir.
- **Korumalı** (*protected*): Sınıf içinden, aynı paketteki sınıflardan ve türetilmiş sınıflardan erişilebilir.

Aşağıda veri gizlemeye ilişkin bir örnek verilmiştir:

```
class Ogrenci {
  private String ad;
  private double notOrtalamasi;

  public void notOrtalamasiAyarla(double deger) {
    if (deger >= 0 && deger <= 100) {
      this.notOrtalamasi = deger;
    } else {
```

```
      System.out.println("Not 0 ile 100 arasinda olmalidir!");
    }
  }

  public double notOrtalamasiOgren() {
    return this.notOrtalamasi;
  }
}
```

Bu örnekte `notOrtalamasi` alanının erişim belirleyicisi `private` yapılarak dışarıdan doğrudan değiştirilmesi engellenmiştir. `public` yöntemler aracılığıyla `notOrtalamasi` alanına güvenli erişim sağlanır. `notOrtalamasiAyarla()` yöntemi, değerin 0 ile 100 arasında olup olmadığını kontrol eder. Böylece negatif veya 100'den büyük geçersiz değerlerin atanması engellenir.

2.5 Soyutlama ve Kapsülleme

Soyutlama (*abstraction*), karmaşık bir yapının sadece gerekli yönlerini öne çıkarıp gereksiz ayrıntılarını gizleme işlemidir. Programlamada soyutlama, bir nesnenin dışarıdan nasıl kullanılacağını gösterirken içeride nasıl çalıştığını saklamaya yardımcı olur.

Kod 2.1 örneği verilen bir `BankaHesabi` nesnesi (kullanıcısı) para yatırabilir, para çekebilir ve bakiyesini öğrenebilir. Ancak bu işlemlerin sistem içinde hangi veritabanı işlemleriyle yapıldığını bilmek zorunda değildir.

```
class BankaHesabi {
  private String hesapNo;
  private double bakiye;

  private void bakiyeKaydet() {
    // Veritabani islemleri
  }

  public String getHesapNo() {
    return this.hesapNo;
  }

  public void setHesapNo(String yeniHesapNo) {
    if (yeniHesapNo != null && !yeniHesapNo.isEmpty()) {
      this.hesapNo = yeniHesapNo;
    }
  }

  public void paraYatir(double miktar) {
    if (miktar > 0) {
      this.bakiye += miktar;
    }
    bakiyeKaydet();
  }

  public void paraCek(double miktar) {
    if (miktar > 0 && miktar <= this.bakiye) {
      this.bakiye -= miktar;
    }
    bakiyeKaydet();
  }

  public double bakiyeOgren() {
    return this.bakiye;
  }
}
```

Bu örnekte kullanıcı sadece `paraYatir()`, `paraCek()` ve `bakiyeOgren()` yöntemlerini kullanır. `bakiye` bilgisinin nasıl işlendiği sınıfın iç ayrıntısıdır.

Veri soyutlaması (*data abstraction*), bir nesnenin verisiyle ne yaptığına odaklanıp, bu verileri nasıl sakladığına dair ayrıntıları gizleme ilkesidir. Soyutlama sayesinde kullanıcı nesnenin sunduğu veriyi bilir; fakat bu verinin arka planda nasıl saklandığını bilmek zorunda değildir. Bunu sağlamak için `hesapNo` ve `bakiye` alanları dışarıya kapatılarak (`private`) veri gizliliği (*data hiding*) yapılır. Ardından `hesapNo` alanına ait `getHesapNo()` ve `setHesapNo()` yöntemleri eklenerek bu alan üzerinde veri soyutlaması yapılır.

Özellik

Nesnenin verisini **getiren** (*getter*) ve nesnenin verilerini **kaydeden** (*setter*) yöntemlerin ilgili sınıf **alanına** (*field*) uygulanmasıyla veri soyutlaması yapılır. Böylece nesnenin kendi verilerine nasıl erişileceği üzerine hakları (*property*) olmuş olur. Dışarıdan istediğimiz gibi nesne verilerini değiştiremeyiz. İşte bu nedenle soyutlama yapılmış nesne durumlarına, **özellik** (*property*) adı verilir.

Bu sınıflardan oluşturulmuş bir banka hesabı nesnesi dışarıya şu işlemleri sunabilir:

```
hesap.paraYatir(100.0);
hesap.paraCek(50.0);
hesap.bakiyeOgren();
```

Kullanıcı bu işlemleri çağırarak hesabı kullanır. Ancak bakiyenin bellekte nasıl tutulduğunu, işlem geçmişinin nasıl kaydedildiğini veya verilerin hangi güvenlik kontrollerinden geçtiğini bilmesine gerek yoktur. Bunun için dışarıya kapalı `bakiyeKaydet()` yöntemi yazılarak **kontrol soyutlaması** (*control abstraction*) yapılmıştır.

Kapsülleme

Bir sınıf üzerinde hem veri soyutlaması ve hem de kontrol soyutlaması yapılması işlemine **kapsülleme** (*encapsulation*) adı verilir. Kapsülleme kısaca veri ile bu veriyi işleyen davranışların aynı sınıf içinde bir araya getirilmesidir.

2.6 Nesnelerin İmal Edilmesi

Bir sınıftan nesne imal etme işlemine **nesne oluşturma** (*instantiation*) denir. Java'da nesneler dinamik olarak **öbek** (*heap*) bellekte, **`new` işleci** (*new operator*) kullanılarak imal edilir. Nesne imal edildiği anda bellekte o nesne için yer ayrılır ve sınıfın **yapıcısı** (*constructor*) oluşan nesnenin durumlarına (verilerine) ilk değer vermek için çalıştırılır.

C++'taki gibi doğrudan **yığın** (*stack*) bellekte statik nesne imalatı Java dilinde bulunmaz; tüm nesne örnekleri referanslar aracılığıyla yönetilir ve bellek yönetimi Java'nın **Çöp Toplayıcısı** (*Garbage Collector*) tarafından otomatik olarak gerçekleştirilir. Bu sayede C++'taki manuel `delete` işlemlerine ve **bellek sızıntısı** (*memory leak*) risklerine Java'da gerek kalmaz.

Aşağıda `BankaHesabi` sınıfından nesne imal eden örnek verilmiştir:

```
public class Main {
  public static void main(String[] args) {
    BankaHesabi hesap = new BankaHesabi(); // hesap nesnesi öbekte (heap) imal edildi

    hesap.setHesapNo("TR123456789");
    hesap.paraYatir(1000);
    hesap.paraCek(250);

    System.out.println("Hesap No: " + hesap.getHesapNo());
    System.out.println("Bakiye: " + hesap.bakiyeOgren());
  }
}
```

Yukarıdaki örnekte bir adet hesap nesnesi imal edilmiştir. İstersek aynı sınıftan sınırsız sayıda nesne imal edebiliriz:

```
public class Main {
  public static void main(String[] args) {
    BankaHesabi hesap1 = new BankaHesabi();
    System.out.println("hesap1 No: " + hesap1.getHesapNo()); // null
    System.out.println("hesap1 Bakiye: " + hesap1.bakiyeOgren()); // 0.0

    BankaHesabi hesap2 = new BankaHesabi();
    System.out.println("hesap2 No: " + hesap2.getHesapNo()); // null
    System.out.println("hesap2 Bakiye: " + hesap2.bakiyeOgren()); // 0.0

    BankaHesabi hesap3 = new BankaHesabi();
    System.out.println("hesap3 No: " + hesap3.getHesapNo()); // null
    System.out.println("hesap3 Bakiye: " + hesap3.bakiyeOgren()); // 0.0
  }
}
```

Burada şöyle bir sorun ortaya çıkar: İmal edilecek nesnelerin hepsinin `hesapNo` durumları `null` ve `bakiye` durumları `0.0` olur. İmal edilecek her nesnede bu durumların farklı farklı olması istenir. Yani nesneler farklı verilerle imal edilebilmelidir. Bu durumda nesne yapıcısı devreye girer.

2.7 this Göstericisi

Java'da her nesne, `this` adı verilen önemli bir anahtar sözcük aracılığıyla kendi referansına erişebilir. `this`, o an üzerinde işlem yapılan nesneyi temsil eder ve üye değişkenlerle yöntem parametrelerini ayırt etmek için sıklıkla kullanılır. Aşağıda karmaşık sayılar örneği verilmiştir;

```
class Karmasik {
  private float gercek;
  private float sanal;

  public Karmasik(float gercek, float sanal) {
    this.gercek = gercek;
    this.sanal = sanal;
  }

  public void yaz() {
    System.out.println(this.gercek + "+" + this.sanal + "i");
  }
}

public class Main {
  public static void main(String[] args) {
    Karmasik c1 = new Karmasik(1.0f, 2.0f);
    c1.yaz();
    Karmasik c2 = new Karmasik(3.0f, 5.0f);
    c2.yaz();
  }
}
```

2.8 Yapıcı

Yapıcı (*constructor*), nesnenin imal edildiği anda otomatik olarak çalışan özel bir yöntemdir. Yapıcının temel amacı nesneyi geçerli bir başlangıç durumuna getirmektir. Bu nedenle bir değer döndürmezler, sadece nesne verisine başlangıç değeri vermek için kullanılırlar. Yapıcılar sınıf ismiyle aynı kimliğe sahip, geri dönüş tipi olmayan yöntemlerdir.

```
class BankaHesabi {
  private String hesapNo;
  private double bakiye;

  private void bakiyeKaydet() {
    // Veritabani islemleri
  }
```

```
  public BankaHesabi() { // Yapıcı
    this.hesapNo = "TR123456789";
    this.bakiye = 1000.0;
  }

  public String getHesapNo() {
    return this.hesapNo;
  }

  public void setHesapNo(String yeniHesapNo) {
    if (yeniHesapNo != null && !yeniHesapNo.isEmpty()) {
      this.hesapNo = yeniHesapNo;
    }
  }

  public void paraYatir(double miktar) {
    if (miktar > 0) {
      this.bakiye = this.bakiye + miktar;
    }
    bakiyeKaydet();
  }

  public void paraCek(double miktar) {
    if (miktar > 0 && miktar <= this.bakiye) {
      this.bakiye = this.bakiye - miktar;
    }
    bakiyeKaydet();
  }

  public double bakiyeOgren() {
    return this.bakiye;
  }
}
```

Buradaki örnekte imal edilecek her nesnenin verisi, yine hep aynı başlangıç değeriyle (yani `hesapNo` verisi `"TR123456789"` ile ve `bakiye` verisi `1000.0`) olacaktır. Her nesneyi aynı verilerle imal etmek yerine `hesapNo` ve `bakiye` değerlerini parametre olarak alan **parametreli yapıcı** (*parameterized constructor*) tanımlanır.

§2.8.1 Parametreli Yapıcı

Parametreli yapıcı (*parameterized constructor*) ile imal edilecek her nesnenin farklı başlangıç verisine (duruma) sahip olarak imal edilmesi sağlanır.

```
class BankaHesabi {
  private String hesapNo;
  private double bakiye;

  public BankaHesabi(String hesapNo, double bakiye) { // Parametreli Yapıcı
    this.hesapNo = hesapNo;
    this.bakiye = bakiye;
  }
  // Diğer metotlar...
}

public class Main {
  public static void main(String[] args) {
    BankaHesabi hesap1 = new BankaHesabi("TR111111111", 4000);
    System.out.println("hesap1 No: " + hesap1.getHesapNo()); // TR111111111
    System.out.println("hesap1 Bakiye: " + hesap1.bakiyeOgren()); // 4000.0

    BankaHesabi hesap2 = new BankaHesabi("TR222222222", 8000);
    System.out.println("hesap2 No: " + hesap2.getHesapNo()); // TR222222222
    System.out.println("hesap2 Bakiye: " + hesap2.bakiyeOgren()); // 8000.0
```

```
  }
}
```

§2.8.2 Kopyalama Yapıcısı

C++'taki açık kopyalama yapıcısı (*copy constructor*) kavramı Java'da standart bir dil özelliği olarak yer almaz; bunun yerine nesneleri kopyalamak için geleneksel olarak kopyalama yapıcı yöntemler veya `clone()` mekanizmaları kullanılır. Var olan bir nesneden yeni bir nesne oluşturmak için Java'da şu yaklaşım izlenir:

```
class BankaHesabi {
  private String hesapNo;
  private double bakiye;

  public BankaHesabi(String hesapNo, double bakiye) {
    this.hesapNo = hesapNo;
    this.bakiye = bakiye;
  }

  // Kopyalama Yapıcı Yöntem (Copy Constructor Benzeri)
  public BankaHesabi(BankaHesabi digerHesap) {
    this.hesapNo = digerHesap.hesapNo;
    this.bakiye = digerHesap.bakiye;
  }
}

public class Main {
  public static void main(String[] args) {
    BankaHesabi hesap1 = new BankaHesabi("TR123456789", 1300);
    BankaHesabi hesap2 = new BankaHesabi(hesap1); // Kopyalama yapıcısı çağrılıyor

    System.out.println("Hesap 1 Bakiye: " + hesap1.bakiyeOgren());
    System.out.println("Hesap 2 Bakiye: " + hesap2.bakiyeOgren());
  }
}
```

2.9 Nesnelerin Yok Edilmesi ve Çöp Toplayıcı (Garbage Collector)

C++'ın aksine Java'da geliştiricinin nesneleri manuel olarak silmesi (delete) gerekm vitesses yoktur. Java çalışma ortamında arka planda çalışan **Çöp Toplayıcı** (*Garbage Collector*), hiçbir referansı kalmayan nesneleri otomatik olarak tespit eder ve bellekten temizler.

C++'taki yıkıcılara (*destructor*) karşılık Java'da nesne bellekten atılmadan hemen önce çağrılması beklenen `finalize()` metodu bulunmaktaydı ancak modern Java sürümlerinde `finalize()` yöntemi kullanım dışı bırakılmıştır (*deprecated*). Java'da nesnelerin bellekten ne zaman silineceğine çöp toplayıcı karar verdiğinden Java'da nesne yok edilirken kesin olarak çağrılan bir yıkıcı mekanizması bulunmaz. Java'da dosya, veritabanı bağlantısı, ağ bağlantısı gibi kaynakları kapatmak için genellikle aşağıdaki `AutoCloseable` arayüzü kullanılır.

```
class Sinif implements AutoCloseable {
  @Override
  public void close() {
    // kaynak temizleme
  }
}
```

Arayüzler ileride anlatılacaktır.

2.10 Yöntem Çok Yükleme

Yöntem çok yükleme (*method overloading*), aynı kimliğe sahip birden fazla yöntemin farklı parametre listeleriyle tanımlanmasıdır. Derleyici, çağrılan yöntemin parametrelerine bakarak hangi sürümün çalıştırılacağını belirler. Çok yüklemede; kimlik aynı, parametre listesi farklı olmalıdır;

yalnızca dönüş tipiyle ayrım yapılamaz.

```
class Hesaplayici {
  public int topla(int a, int b) {
    return a + b;
  }

  public int topla(int a, int b, int c) {
    return a + b + c;
  }

  public String topla(String a, String b) {
    return a + b;
  }
}

public class Main {
  public static void main(String[] args) {
    Hesaplayici h = new Hesaplayici();

    System.out.println("2 + 3 = " + h.topla(2, 3));
    System.out.println("2 + 3 + 4 = " + h.topla(2, 3, 4));
    System.out.println("Metin toplama: " + h.topla("Merhaba ", "Dunya"));
  }
}
/* Program Çıktısı:
2 + 3 = 5
2 + 3 + 4 = 9
Metin toplama: Merhaba Dunya
*/
```

Bu örnekte `topla()` yöntemi adı üç farklı biçimde kullanılmıştır. Parametre sayısı veya türü değiştiği için bu geçerli bir çok yüklemedir.

2.11 İşleç Çok Yükleme

C++ dilinin aksine Java dili **işleç çok yüklemeyi** (*operator overloading*) desteklemez. Java'da yalnızca `+` işleçleri metinler (*String*) için birleştirme amacıyla dahili olarak aşırı yüklenmiştir; ancak geliştiricilerin kendi sınıfları için yeni operatörler (+, -, *, / vb.) tanımlamasına izin verilmez. Bu tür işlemler için özel adlandırılmış yöntemler (*topla*, `cikar`, *carp*, vb.) kullanılır.

2.12 Kalıtım Kavramı

Kalıtım (*inheritance*), bir sınıfın başka bir sınıfın özelliklerini ve davranışlarını devralmasıdır. Sınıflar arasında **bir türüdür** (*is a*) ilişkisi kurar ve kod tekrarını önler. **Değişim yönetimini** kolayca yapma üstünlüğünü verir.

Kalıtımın İki Önemli Özelliği

1. **Paylaşılan bir ortak kod** (*shared code*) vardır.
2. Paylaşılan **kodda yapılan değişiklik anında alt sınıflara yansır** (*snap change*).

§2.12.1 Ata ve Çocuk Sınıflar

Öğrenci, öğretmen ve müdürün olduğu bir okul sistemini ele alalım. Ortak özellik ve davranışların bir sınıfta toplanarak bu sınıftan miras alınmasına **kalıtım** (*inheritance*) adı verilir. Java'da kalıtım `extends` anahtar kelimesi ile gerçekleştirilir.

```
class Kisi {
  protected int yas;
  protected char cinsiyet;

  public Kisi() {
```

```
    this.yas = 18;
    this.cinsiyet = 'E';
  }

  public void setYas(int yeniYas) {
    if (yeniYas < 18)
    System.out.println("Yaş 18 den Küçük Olamaz.");
    else
    this.yas = yeniYas;
  }

  public int getYas() {
    return this.yas;
  }

  public void yasiniSoyle() {
    System.out.println("Yaşım: " + this.yas);
  }
}

class Ogrenci extends Kisi { // Kisi sınıfından türemiş Ogrenci sınıfı
  private int ogrenciNo;

  public Ogrenci() {
    super();  // ata sınıfın yapıcısını çağır!
    this.ogrenciNo = 0;
  }

  public void dersCalis() {
    System.out.println("Ders Çalışıyorum...");
  }
}

class Ogretmen extends Kisi { // Kisi sınıfından türemiş Ogretmen sınıfı
  private int verdigiDersKodu;

  public Ogretmen() {
    super(); // ata sınıfın yapıcısını çağır!
    this.verdigiDersKodu = 0;
  }

  public void dersVer() {
    System.out.println("Ders Veriyorum...");
  }
}
```

Kalıtımda özellikleri devralınan sınıf **taban sınıf** (*base class*) veya **ata sınıf** (*parent class*) olarak adlandırılır. Özellikleri devralan sınıflar ise **türemiş sınıf** (*derived class*) veya **çocuk sınıf** (*child class*) olarak adlandırılır.

§2.12.2 UML Sınıf Diyagramlarında Kalıtımın Gösterilmesi

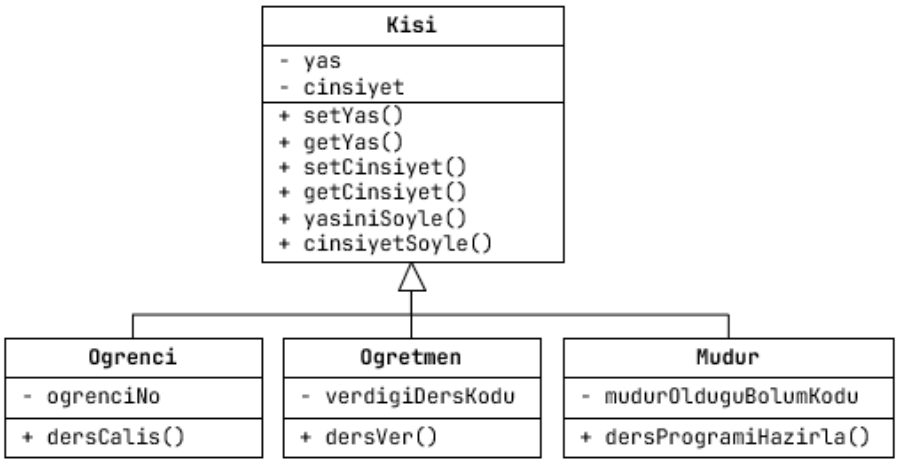
UML sınıf diyagramlarında kalıtım, çocuk sınıftan ata sınıfa doğru uzanan boş üçgen uçlu bir ok ile gösterilir.

§2.12.3 Kalıtımda Ata Sınıfın Yapıcısının Çağrılması

Çocuk sınıftan bir nesne üretildiğinde önce ata sınıfın yapıcısı (`super()`), sonra çocuk sınıfın yapıcısı çalışır.

§2.12.4 Çoklu Kalıtım (Multiple Inheritance) Durumu

Modern nesne yönelimli diller olan Java ve C# dillerinde sınıflar arasında çoklu kalıtım (*multiple inheritance*) desteklenmez! Bir sınıf yalnızca tek bir sınıftan (`extends`) miras alabilir. Belirsizlikleri (elmas problemi vb.) önlemek amacıyla bu kısıtlama getirilmiştir. Ancak birden fazla rolün



Şekil 2.2: Kalıtım Uygulanmış Okul Projesi Sınıfları

kazandırılması istentiğinde arayüzler (*interfaces*) kullanılır.

§2.12.5 Arkadaş Fonksiyon ve Sınıflar

C++ dilinde bulunan arkadaşlık (*friend*) mekanizması Java dilinde bulunmaz. Java, kapsülleme ilkelerini korumak adına başka sınıfların `private` üyelerine doğrudan arkadaşlık yoluyla erişilmesine izin vermez; bunun yerine paket içi görünürlük (*package-private*) veya getiren (*getter*) ve ayarlayan (*setter*) yöntemleri kullanılır.

2.13 Statik Sınıf Üyeleri

Java'da **statik yöntemler** (*static method*) ve **statik alanlar** (*static field*), herhangi bir nesne örneği oluşturulmadan doğrudan sınıf ismi üzerinden çağrılabilir ve paylaşılır.

```
class YardimciSinif {
  public static int genelSayac = 100;

  public static void programciAdi() {
    System.out.println("Programcı Adı: İlhan ÖZKAN");
    System.out.println(genelSayac);
  }
}

public class Main {
  public static void main(String[] args) {
    YardimciSinif.programciAdi();
    YardimciSinif.genelSayac--;
    YardimciSinif.programciAdi();
  }
}
```

2.14 const ve final Sınıf Üyeleri

C++'taki `const` anahtar kelimesinin karşılığı Java'da değişkenler ve metotlar için **`final`** anahtar kelimesidir. `final` olarak tanımlanan bir değişkenin değeri sonradan değiştirilemez (sabit olur).

2.15 Geçersiz Kılma (Overriding)

Geçersiz kılma (*overriding*), ata sınıfta tanımlanmış bir metodun çocuk sınıfta aynı imza ile yeniden yazılmasıdır. Java'da bu durum `@Override` anatasyonu ile açıkça belirtilir ve çalışma zamanı çok biçimliliğinin temelini oluşturur.

2.16 Soyut ve Somut Sınıf

Soyut sınıf (*abstract class*), `abstract` anahtar kelimesi ile tanımlanan ve doğrudan `new` ile nesnesi üretilemeyen sınıflardır. İçerisinde gövdesiz soyut metotlar barındırabilir. **Somut sınıf** (*concrete class*) ise soyut sınıflardan türeyen ve tüm soyut yöntemleri geçersiz kılan (*override*), nesnesi imal edilebilir sınıflardır.

```
abstract class SoyutKisi {
  private String adi;

  public SoyutKisi(String adi) {
    this.adi = adi;
  }

  public abstract void raporYaz(); // Soyut metot

  public String getAdi() {
    return this.adi;
  }
}

class SomutKisi extends SoyutKisi {
  public SomutKisi(String adi) {
    super(adi);
  }

  @Override
  public void raporYaz() {
    System.out.println(getAdi() + ": Rapor Yazıyor...");
  }
}
```

2.17 Arayüzler (Interfaces)

Java'da sınıflar tek bir sınıftan miras alabilirken, birden fazla arayüzü `implements` anahtar kelimesiyle uygulayabilirler. Bu sayede çoklu kalıtımın sağladığı esneklik arayüzler aracılığıyla güvenli bir şekilde sağlanır. Java'da arayüzler `interface` anahtar kelimesi ile tanımlanır.

```
interface IBaba {
  void cocugaBak();
}

interface IMuzisyen {
  void gitarCal();
}

class Kisi implements IBaba, IMuzisyen {
  @Override
  public void cocugaBak() {
    System.out.println("Çocuğa Bakıyorum...");
  }

  @Override
  public void gitarCal() {
    System.out.println("Gitar Çalıyorum...");
  }
}
```

2.18 Çok Biçimlilik (Polymorphism)

Çok biçimlilik (*polymorphism*), ata-çocuk ilişkisi olan birden çok nesnenin olduğu bir ortamda, verilen aynı iletiye karşı nesnelerin farklı davranışlar göstermesidir (*same message-different behavior*). Java'da tüm nesne metotları varsayılan olarak sanal (virtual) davranır.

```java
abstract class SoyutKisi {
  protected String adi;

  public SoyutKisi(String adi) {
    this.adi = adi;
  }

  public abstract void raporYaz();
}

class Ogrenci extends SoyutKisi {
  public Ogrenci(String adi) { super(adi); }

  @Override
  public void raporYaz() {
    System.out.println(this.adi + ": Öğrenci Raporu Yazıyor...");
  }
}

class Ogretmen extends SoyutKisi {
  public Ogretmen(String adi) { super(adi); }

  @Override
  public void raporYaz() {
    System.out.println(this.adi + ": Öğretmen Raporu Yazıyor...");
  }
}

public class Main {
  public static void main(String[] args) {
    SoyutKisi[] kisiler = new SoyutKisi[2];
    kisiler[0] = new Ogrenci("İlhan ÖZKAN");
    kisiler[1] = new Ogretmen("Hasan YILMAZ");

    for (SoyutKisi kisi : kisiler) {
      kisi.raporYaz(); // Aynı mesaja karşı farklı davranış (Çok biçimlilik)
    }
  }
}
/* Program Çıktısı:
İlhan ÖZKAN: Öğrenci Raporu Yazıyor...
Hasan YILMAZ: Öğretmen Raporu Yazıyor...
*/
```

3. Bölüm
İstisnalar, Esnek Tasarım ve SOLID İlkeleri

3.1 Hatalar ve İstisnalar

Bu bölümde hata durumlarının yapısal programlamada ve nesne yönelimli dillerde nasıl yönetildiği, Java örnekleriyle anlatılmaktadır. İstisna yönetimi `try`, `catch`, `throw`, `throws` ve `finally` sözcükleriyle yapılır.

§3.1.1 Yapısal Programlamada Hata Yönetimi

Yapısal programlamada hata yönetimindeki asıl sorun, hata denetim kodunun işlevsel kodla iç içe geçmesidir. Hata genellikle fonksiyonun dönüş değeriyle (*hata kodu*) bildirilir ve çağıran kod her çağrıdan sonra bu değeri denetlemek zorundadır. Denetim unutulursa hata fark edilmeden ilerler; hatalı bir durumla karşılaşıldığında ise çoğu zaman programdan çıkılır. İstisna mekanizması bu sorunu büyük ölçüde azaltır: hata bildirimi dönüş değerinden ayrılır ve hata, onu işleyebilecek koda kadar otomatik olarak taşınır. Aşağıda geleneksel bir C/yapısal tarzdaki yaklaşımın Java karşılığı verilmiştir;

```
#include <stdio.h>

int faktoriyel(int sayi) {
  if (sayi < 0) return -1;  // Aslında sıfırdan küçük sayının faktöriyeli -1 değil, tan
      ımsızdır.
  if (sayi <= 1) return 1;
  return sayi * faktoriyel(sayi - 1);
}

int main() {
  int okunan, deger, hata = 0;
  do {
    printf("Bir Sayı Giriniz:");
    scanf("%d", &okunan);
    deger = faktoriyel(okunan);
    printf("%d nin faktöriyeli: %d\n", okunan, deger);
    if (deger == -1) {
      hata = 100;
      break; // exit(100);
    }
  } while (okunan != 0);
  return hata;
}
/* Program Çıktısı:
Bir Sayı Giriniz:-1
-1 nin faktöriyeli: -1
*/
```

Bu örnekte `faktoriyel` fonksiyonu hata durumunda `-1` değerini geri döndürmektedir. Bu yaklaşımın iki zayıflığı vardır:

1. Hata değeri ile gerçek sonuç aynı dönüş değeri içinde taşınır (*in-band signaling*). Burada `-1`

geçerli bir faktöriyel olamayacağı için çalışır; ancak her fonksiyon için hataya ayrılabilecek böyle bir değer bulmak mümkün değildir.
2. Denetim çağıranın sorumluluğundadır. Örnekte `System.out.println()` satırı denetimden önce çalıştığı için hatalı sonuç ekrana yazdırılmıştır. Denetim hiç yazılmasaydı `-1` bir sonraki hesaplamada sessizce kullanılacaktı.

§3.1.2 Nesne Yönelimli Programlamada İstisna Yönetimi

Nesne yönelimli programlamada **istisna** (*exception*), programın yürütülmesi sırasında normal akışı bozan beklenmeyen bir durumdur. İstisna oluştuğunda durumu tanımlayan bir **istisna nesnesi** (*exception object*) oluşturulur ve **fırlatılır**. Çalışma zamanı sistemi (*run-time*) bu nesneyi işleyebilecek uygun bir kod bloğu arar. İstisnalar iki ana kaynaktan doğar:

1. **Programcı kaynaklı hatalar**: Geçersiz argüman, boş (`null`) bir nesneye erişim, dizi sınırının aşılması gibi, kodun doğru yazılmasıyla önlenebilecek durumlar.
2. **Çevre kaynaklı durumlar**: Dosyanın bulunamaması, ağ bağlantısının kopması, diskin dolması, belleğin yetersiz kalması gibi, programın kontrolü dışındaki durumlar.

İstisnalar Eşzamanlıdır

Java'da istisnalar yalnızca programın kendi akışı içinde, bir `throw` ifadesiyle ortaya çıkar. Bu anlamda **eşzamanlıdır** (*synchronous*). Klavye kesmesi (*interrupt*) gibi eşzamansız (*asynchronous*) olaylar istisna olarak fırlatılmaz; işletim sistemi bunları sinyal veya olay dinleyicileri olarak ele alır. Bir sistem çağrısının başarısız olması (örneğin diskten okuma hatası) ise çağrıyı yapan kodda, istisna olarak bildirilir.

İstisnai durumları yönetmek için `throw`, `try` ve `catch` anahtar sözcükleri kullanılır. İstisnaları işlemek için genel yapı aşağıda verilmiştir;

```
try {
  /*
  ...
  İstisnai Durumun Ortaya Çıkabileceği Kod
  ...
  */
  throw new SomeExceptionType("İstisnai Durum Açıklaması");
  /* throw an exception: istisnai bir duruma düşme */
} catch (ExceptionName1 e1) {
  /* catch bloğu try bloğundaki istisnayı yakalamak ve gerekli işlemi yapmak için
      kullanılır. */
}
```

Bu kodlar şu şekilde açıklanabilir;

- ◇ **`try` talimatı**: İçerisinde istisnai bir duruma düşülebileceğini tahmin ettiğimiz bir kod bloğunu temsil eder. Bu bloğu bir veya daha fazla `catch` bloğu takip eder.
- ◇ **`catch` talimatı**: `try` bloğunda bir istisnai duruma düşüldüğünde yürütülecek bir kod bloğunu temsil eder. İstisnayı işlemek için kullanılan kod, `catch` bloğunun içine yazılır.
- ◇ **`throw` talimatı**: Bir istisna nesnesi oluşturup fırlatır (*throw an exception*).
- ◇ **İstisna sonrası süreç**: Program bir `throw` ifadesiyle karşılaştığında normal akış hemen kesilir. Çalışma zamanı sistemi önce içinde bulunulan `try` bloğunun `catch` bloklarında, bulunamazsa çağıran fonksiyonlarda sırayla yukarı doğru eşleşen bir `catch` bloğu arar. Bir `try` bloğu içinde birden fazla `throw` ifadesi bulunabilir; ancak ilk fırlatılan istisna işlendiğinde `try` bloğunun kalanı çalıştırılmaz.
- ◇ **`finally` bloğu**: İstisna fırlatılsa da fırlatılmasa da çalışan temizlik bloğudur. C++'ta `finally` yoktur; temizlik yıkıcılarla yapılır.

Bu yeni bakış açısıyla yapısal programlamadaki faktöriyel örneğini Java ile aşağıdaki gibi yazabiliriz;

```
public class FaktoriyelIstisnaOrnegi {
```

```
  // Faktöriyel fonksiyonu: Hata durumunda RuntimeException fırlatır
  public static int faktoriyel(int sayi) {
    if (sayi < 0) {
      throw new IllegalArgumentException("Negatif sayilarin faktoriyeli hesaplanamaz!")
          ;
    }
    if (sayi <= 1) return 1;
    return sayi * faktoriyel(sayi - 1);
  }

  public static void main(String[] args) {
    java.util.Scanner scanner = new java.util.Scanner(System.in);
    int okunan;

    do {
      System.out.print("Bir Sayi Giriniz (Cikis icin 0): ");
      okunan = scanner.nextInt();
      if (okunan == 0) break;

      try { // Hata riski olan kodu try bloğu içine alıyoruz!
        int sonuc = faktoriyel(okunan);
        System.out.println(okunan + " sayisinin faktoriyeli: " + sonuc);
      }
      catch (IllegalArgumentException e) { // İstisna nesnesini yakalıyoruz.
        System.err.println("HATA: " + e.getMessage());
      }
    } while (okunan != 0);

    scanner.close();
  }
}
/* Program Çıktısı:
Bir Sayi Giriniz (Cikis icin 0): -5
HATA: Negatif sayilarin faktoriyeli hesaplanamaz!
*/
```

İstisna işleme yapısal programdakinden farklıdır. Şöyle ki;

- ◇ Hata işleme kodu, fonksiyonel koddan yani işin gereği yazılan koddan ayrılır.
- ◇ Fonksiyonlar ve yöntemler yalnızca seçtikleri istisnaları işleyebilir. Bir yöntem veya fonksiyonda birden çok istisnai duruma düşülebilir; ancak bunlardan bazılarını `catch` bloğunda işlemeyi seçebiliriz. Geri kalanları yöntemi çağıran yere bırakabiliriz. Böylece istisnayı kimin işleyeceğini programcı seçebilir.
- ◇ Java'da istisnalar her zaman `Throwable` sınıfından türetilmiş birer nesnedir. Temel tipler (`int`, `char` vb.) doğrudan fırlatılamaz; nesne sarmalayıcıları (*wrapper*) veya istisna sınıfları kullanılır.

Taşma ve Sıfıra Bölme: Java'da işaretli tam sayı taşması (*integer overflow*) istisna fırlatmaz; alt sınırdan üste dönerek (*wrap around*) yanlış sonuç üretir:

```
Bir Sayi Giriniz (Cikis icin 0): 13
13 sayisinin faktoriyeli: 1932053504
```

Tam sayıyı sıfıra bölmek (`int / 0`) ise Java'da `ArithmeticException` türünde bir çalışma zamanı istisnası fırlatır. Kayan noktalı sayılarda ise (`double` veya `float`) sıfıra bölme işlemi istisna üretmez; `Infinity` veya `NaN` üretir.

§3.1.3 Standart İstisna Tipleri

Aşağıda sıfıra bölme hatasını işleyen bir Java program örneği verilmiştir. Örnekte standart istisna tiplerinden biri olan `ArithmeticException` veya genel `RuntimeException` ele alınmıştır.

```
public class SifiraBolmeOrnegi {
  public static void main(String[] args) {
```

```
    System.out.println("Burasi her zaman icra edilir.");
    try {
      int bolunen = 10;
      int bolen = 0;
      int sonuc;
      if (bolen == 0) {
        // Standart aritmetik istisna fırlatıldı.
        throw new ArithmeticException("Sifira Bolmeye Izin Verilmez!");
      }
      sonuc = bolunen / bolen;
      System.out.println("Bolme Sonucu: " + sonuc);
    } catch (ArithmeticException e) {
      // catch BLOĞU: Hata yakalandığında bu blok çalışır
      System.out.println("Istisna Yakalandi: " + e.getMessage());
    }
    System.out.println("Burasi da her zaman icra edilir.");
  }
}
/* Program Çıktısı:
Burasi her zaman icra edilir.
Istisna Yakalandi: Sifira Bolmeye Izin Verilmez!
Burasi da her zaman icra edilir.
*/
```

Java'daki standart istisna hiyerarşisi `Throwable` kök sınıfından başlar ve `Exception` ile `Error` olarak ikiye ayrılır. Uygulama geliştiricileri ilgilendiren ana yapı `RuntimeException` (unchecked) ve kontrol edilen istisnalardır (*checked exceptions*).

İstisna Sınıfı	Üst Kategori	Açıklama
`java.lang.IllegalArgumentException`	`RuntimeException`	Metoda geçersiz bir argüman gönderilmesi.
`java.lang.NullPointerException`	`RuntimeException`	Boş (`null`) referans üzerinden üye erişilmeye çalışılması.
`java.lang.IndexOutOfBoundsException`	`RuntimeException`	Dizi veya koleksiyon sınırlarının dışına çıkılması.
`java.lang.ArithmeticException`	`RuntimeException`	Aritmetik hata durumları (örn. sıfıra bölme).
`java.lang.IllegalStateException`	`RuntimeException`	Nesnenin iç durumunun yapılan işlem için uygun olmaması.
`java.io.IOException`	`Exception`	Girdi/çıktı işlemlerinde (dosya, ağ) oluşan hatalar (Checked).
`java.sql.SQLException`	`Exception`	Veritabanı erişim hataları (Checked).

Tablo 3.1: Java Standart İstisna Hiyerarşisi ve Kullanım Alanları

3.1.3.1 Belli Bir İstisnayı Yakalama

Bir `try` bloğunda farklı türde istisnalar fırlatılabilir. Java'da çoklu `catch` blokları veya tek blokta birleşik yakalama (*multi-catch*) mekanizması kullanılabilir. Aşağıda buna ilişkin Java örneği verilmiştir;

```
public class CokluCatchOrnegi {
  public static void main(String[] args) {
    try {
      // İlk fırlatılan istisna akışı ilgili catch bloğuna yönlendirir.
      boolean hataOlustur = true;
      if (hataOlustur) {
        throw new NumberFormatException("Sayisal donusum hatasi");
      }
    }
    catch (NumberFormatException e) {
      System.out.println("Sayisal Format Istisnasi Yakalandi: " + e.getMessage());
```

```
    }
    catch (IllegalArgumentException e) {
      System.out.println("Gecersiz Arguman Istisnasi Yakalandi: " + e.getMessage());
    }
    catch (Exception e) {
      // Genel catch: Yukarıdakilere uymayan her şey buraya düşer.
      System.out.println("Genel bir istisna yakalandi: " + e.getMessage());
    }
  }
}
/* Program Çıktısı:
Sayisal Format Istisnasi Yakalandi: Sayisal donusum hatasi
*/
```

catch Sırası

Java derleyicisi, `catch` blokları yukarıdan aşağıya yazılırken daha genel istisnaların (örneğin `Exception`) alt kısımlara yazılmasını zorunlu kılar. Eğer genel istisnayı en üste yazarsanız, altındaki özel catch blokları için "ulaşılamaz kod" (*unreachable code*) derleme hatası alırsınız.

3.1.3.2 Sınıf İçinde İstisna Kullanımı

Aşağıda `Kisi` sınıfında `yas` özelliğine ilişkin istisna içeren bir Java kod örneği verilmiştir. Geçersiz bir yaş değeri, çağıran koda `IllegalArgumentException` ile bildirilmektedir.

```
class Kisi {
  private int yas;

  public Kisi() {
    this.yas = 15;
  }

  public int getYas() {
    return yas;
  }

  public void setYas(int pYas) {
    if (pYas < 0) {
      throw new IllegalArgumentException("Gecersiz Yas Degeri: <0!");
    }
    if (pYas > 120) {
      throw new IllegalArgumentException("Cok Buyuk Yas Degeri: >120!");
    }
    this.yas = pYas;
  }
}

public class KisiTest {
  public static void main(String[] args) {
    Kisi ali = new Kisi();
    try {
      ali.setYas(12);
      System.out.println("Guncel Yas: " + ali.getYas());

      ali.setYas(-1); // Hata fırlatıldığı an try bloğu terk edilir

      System.out.println("Bu satira asla ulasilamayacak: " + ali.getYas());
    }
    catch (IllegalArgumentException istisna) {
      System.out.println("Istisnai Bir Durum Yakalandi!");
      System.out.println("Durum Aciklamasi: " + istisna.getMessage());
    }
  }
}
```

```
/* Program Çıktısı:
Guncel Yas: 12
Istisnai Bir Durum Yakalandi!
Durum Aciklamasi: Gecersiz Yas Degeri: <0!
*/
```

§3.1.4 İstisna Nesnesi ve Hata Mesajı: getMessage()

Java'da tüm istisna sınıfları `Throwable` sınıfından türetilmiştir ve hatanın açıklamasını almak için `getMessage()` veya `printStackTrace()` yöntemleri kullanılır.

```
import java.util.ArrayList;
import java.util.List;

public class ListeSinirOrnegi {
  public static void main(String[] args) {
    List sayilar = new ArrayList<>();
    sayilar.add(10);
    sayilar.add(20);
    sayilar.add(30);

    try {
      // Liste sınırları dışında bir elemana erişmeye çalışıyoruz (indis 5 yok)
      // Java'da get() metodu IndexOutOfBoundsException fırlatır.
      System.out.println(sayilar.get(5));
    }
    catch (IndexOutOfBoundsException e) {
      System.out.println("Hata Yakalandi: " + e.getMessage());
    }
  }
}
/* Program Çıktısı:
Hata Yakalandi: Index 5 out of bounds for length 3
*/
```

§3.1.5 Kullanıcı Tanımlı İstisna Tipleri

Java'da kendi hata durumlarımızı anlatan özel istisna sınıfları tanımlamak için `RuntimeException` (kontrol edilmeyen) veya `Exception` (kontrol edilen) sınıflarından türetme yapılır.

```
// Özel kontrol edilmeyen (unchecked) istisna sınıfı
class YasSinirException extends RuntimeException {
  public YasSinirException(String mesaj) {
    super(mesaj);
  }
}

class KisiOzel {
  private int yas = 15;

  public void setYas(int pYas) {
    if (pYas < 0) {
      throw new YasSinirException("Gecersiz Yas Degeri: <0!");
    } else if (pYas > 120) {
      throw new YasSinirException("Cok Buyuk Yas Degeri: >120!");
    }
    this.yas = pYas;
  }
}

public class OzelIstisnaTest {
  public static void main(String[] args) {
    KisiOzel ali = new KisiOzel();
    try {
      ali.setYas(130);
```

```
    } catch (YasSinirException e) {
      System.out.println("Ozel Istisna Yakalandi: " + e.getMessage());
    }
  }
}
/* Program Çıktısı:
Ozel Istisna Yakalandi: Cok Buyuk Yas Degeri: >120!
*/
```

§3.1.6 İstisnayı Yeniden Fırlatma ve Zincirleme

Java'da yakalanan bir istisnayı üst katmana iletmek için doğrudan `throw e;` ifadesi kullanılır. Ayrıca Java, istisnaların kök nedenini korumak için *exception chaining* mekanizmasını destekler.

```
public class IstisnaZinciriOrnegi {

  public static void dosyaOku() {
    // Bir alt hata simülasyonu
    throw new NullPointerException("Dosya akisi bos");
  }

  public static void kayitYukle() {
    try {
      dosyaOku();
    } catch (NullPointerException e) {
      System.out.println("[kayitYukle] Hata yakalandi, ust katmana aktariliyor.");
      // İstisnayı sararak (chaining) yeniden fırlatıyoruz:
      throw new RuntimeException("Kritik Yukleme Hatasi", e);
    }
  }

  public static void main(String[] args) {
    try {
      kayitYukle();
    } catch (RuntimeException e) {
      System.out.println("[main] Yakalandi: " + e.getMessage());
      System.out.println("[main] Kok neden: " + e.getCause().getMessage());
    }
  }
}
/* Program Çıktısı:
[kayitYukle] Hata yakalandi, ust katmana aktariliyor.
[main] Yakalandi: Kritik Yukleme Hatasi
[main] Kok neden: Dosya akisi bos
*/
```

§3.1.7 Kaynak Yönetimi ve finally / try-with-resources

Java'da dosya, soket veya veritabanı bağlantısı gibi dış kaynaklar açıldığında, hata oluşsa bile bu kaynakların güvenli bir şekilde kapatılması gerekir. Geleneksel olarak `finally` blokları kullanılırken, modern Java'da `try-with-resources` yapısı tercih edilir.

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class KaynakYonetimiOrnegi {

  public static void dosyaBilgisiOku(String dosyaYolu) {
    // try-with-resources: AutoCloseable uygulayan kaynaklar otomatik kapatılır
    try (BufferedReader reader = new BufferedReader(new FileReader(dosyaYolu))) {
      String satir = reader.readLine();
      System.out.println("İlk Satır: " + satir);
    } catch (IOException e) {
      System.out.println("G/Ç Hatası: " + e.getMessage());
```

```
    }
  }
}
```

3.2 Unified Modeling Language ve Sınıflar Arası İlişkiler

Java, nesne yönelimli bir dil olduğu için UML sınıf diyagramındaki tüm kavramlar (Ortaklık, Biriktirme, Bileşim, Bağımlılık, Genelleştirme, Gerçekleştirme) birebir Java dil yapısına (alanlar, referanslar, `extends`, `implements`) karşılık gelir.

§3.2.1 Bileşim (Composition) Örneği (Java)

Java'da bileşim ilişkisinde parça nesne, bütün nesnenin constructor içinde oluşturulur.

```
class Motor {
  private String marka;
  private int beygirGucu;

  public Motor(String marka, int beygirGucu) {
    this.marka = marka;
    this.beygirGucu = beygirGucu;
  }

  public String motorBilgisi() {
    return marka + " motor, " + beygirGucu + " HP";
  }
}

class Arac {
  private String marka;
  private String model;
  private Motor motor; // Bileşim (Composition)

  public Arac(String marka, String model) {
    this.marka = marka;
    this.model = model;
    // Motor, Araç ile birlikte üretiliyor
    this.motor = new Motor("D4D", 120);
  }

  public void motoruCalistir() {
    System.out.println(marka + " " + model + " çalıştırılıyor...");
    System.out.println("Kullanılan motor: " + motor.motorBilgisi());
  }
}
```

3.3 Sınıf Tasarlama (SOLID) İlkeleri

SOLID ilkeleri, Java projelerinde de sürdürülebilir, esnek ve test edilebilir mimariler kurmanın temelidir. Aşağıda SOLID ilkelerinin Java 17+ standartlarına uyarlanmış örnekleri sunulmuştur.

1. Tek Sorumluluk İlkesi (SRP) Bir sınıfın değişmesi için yalnızca bir nedeni olmalıdır.

```
// İŞ KURALI: Fatura hesaplama sorumluluğu
class Fatura {
  private String musteri;
  private int tutarTL;

  public Fatura(String musteri, int tutarTL) {
    this.musteri = musteri;
    this.tutarTL = tutarTL;
  }

  public String getMusteri() { return musteri; }
  public int kdvliToplam() { return tutarTL * 120 / 100; }
```

```
}

// VERİTABANI İŞLEMİ: Kalıcılık sorumluluğu
class FaturaDeposu {
  public void kaydet(Fatura fatura) {
    System.out.println("[Veritabani] " + fatura.getMusteri() + " kaydedildi.");
  }
}

// GÖRÜNÜM / ÇIKTI İŞLEMİ: Yazdırma sorumluluğu
class FaturaYazici {
  public void yazdir(Fatura fatura) {
    System.out.println("FATURA: " + fatura.getMusteri() + " - " + fatura.kdvliToplam()
        + " TL");
  }
}
```

2. Açık-Kapalı İlkesi (OCP) Sınıflar genişlemeye açık, değişikliğe kapalı olmalıdır. Çok biçimlilik (polimorfizm) ile sağlanır.

```
interface IndirimKurali {
  int hesapla(int tutarTL);
}

class OgrenciIndirimi implements IndirimKurali {
  public int hesapla(int tutarTL) { return tutarTL * 10 / 100; }
}

class EmekliIndirimi implements IndirimKurali {
  public int hesapla(int tutarTL) { return tutarTL * 15 / 100; }
}

class Kasa {
  public int odenecekTutar(int tutarTL, IndirimKurali kural) {
    return tutarTL - kural.hesapla(tutarTL);
  }
}
```

3. Liskov İkame İlkesi (LSP) Türemiş sınıfın nesnesi, taban sınıf nesnesinin kullanıldığı her yerde programın doğruluğunu bozmadan kullanılabilmelidir.

```
class Kus {
  public void ye() { System.out.println("Kus yem yiyor"); }
}

interface Ucan {
  void uc();
}

class Serce extends Kus implements Ucan {
  public void uc() { System.out.println("Serce ucuyor"); }
}

class Penguen extends Kus {
  public void yuz() { System.out.println("Penguen yuzuyor"); }
}
```

4. Arayüzleri Ayırma İlkesi (ISP) İstemciler, kullanmadıkları yöntemlere bağımlı olmaya zorlanmamalıdır. Küçük ve role özel arayüzler tanımlanır.

```
interface Yazdirabilir {
  void yazdir();
}

interface Tarayabilir {
```

```
  void tara();
}

class BasitYazici implements Yazdirabilir {
  public void yazdir() {
    System.out.println("Yazdırılıyor...");
  }
}
```

5. Bağımlılıkları Tersine Çevirme İlkesi (DIP) Üst düzey modüller alt düzey modüllere bağımlı olmamalıdır; ikisi de soyutlamalara (*interface*) bağımlı olmalıdır.

```
interface BildirimGonderici {
  void gonder(String mesaj);
}

class EpostaGonderici implements BildirimGonderici {
  public void gonder(String mesaj) {
    System.out.println("[E-POSTA] " + mesaj);
  }
}

class SiparisServisi {
  private final BildirimGonderici gonderici;

  // Bağımlılık Enjeksiyonu (Dependency Injection)
  public SiparisServisi(BildirimGonderici gonderici) {
    this.gonderici = gonderici;
  }

  public void siparisVer(String urun) {
    System.out.println(urun + " siparisi alindi.");
    gonderici.gonder(urun + " siparisiniz alindi.");
  }
}
```

4. Bölüm
Şablonlar ve Jenerik Programlama

4.1 Şablon Nedir?

Şablonlar (*templates* veya Java'daki adıyla **Jenerikler** - *Generics*), veri tiplerine bağlı kalmadan çalışan fonksiyonlar, sınıflar ve algoritmalar yazmak için oluşturulan yapılardır. C++'taki şablon mantığı, Java dünyasında **Jenerikler** (*Generics*) ve **Tür Silme** (*Type Erasure*) mekanizmalarıyla karşılık bulur. Java, nesne yönelimli yapısı ve otomatik **Çöp Toplayıcı** (Garbage Collector) mekanizması sayesinde bellek yönetimini otomatik yapar; ancak tip güvenliğini sağlamak için jenerik yapılara yoğun şekilde güvenir.Jenerik programlama, kodun belirli bir veri tipine (`int`, `double`, `String` vb.) bağlı kalmadan, veri tipinden bağımsız bir şekilde yazılmasına olanak tanıyan bir yazılım geliştirme yaklaşımıdır. En temel amacı, aynı mantığa sahip işlemleri farklı veri tipleri için tekrar tekrar yazmak yerine, bir kez hazırlayıp her tipte güvenle kullanabilmektir. Java'da jenerik programlamanın temel özellikleri şunlardır:

- Bir kez yazılan bir algoritma, farklı nesne türleri için çalışabilir. Böylece kodun yeniden kullanımı (*reusing*) sağlanır.
- Kod derleme zamanında (*compile-time*) tip denetiminden geçer. Hatalar program çalışmadan yakalanır, böylece tip güvenliği (*type safety*) sağlanmış olur.
- C++'ın aksine Java, çalışma zamanında (*run-time*) tür bilgilerini *Type Erasure* ile siler ancak genel mantıkta benzer performans ve güvenlik avantajları sunar.

Jenerik programlamada bir sınıf veya metot yazarken tip kısmına gerçek bir isim yerine bir yer tutucu (*placeholder*) konur. Genellikle bu yer tutucu için `T` harfi kullanılır.

4.2 Jenerik Yöntemler

Java'da tıpkı C++ fonksiyon şablonlarında olduğu gibi, farklı veri tipleriyle çalışabilen **jenerik metotlar** tanımlayabilirsiniz. Metot imzasında dönüş türünden önce köşe parantezler içinde tip parametresi belirtilir.Aşağıdaki örnekte, dizileri konsola yazdıran jenerik bir metot yapısını Java ile inceleyelim:

```
import java.util.Arrays;public class MainJenerikMetot {
  // Jenerik Metot Tanımı
  public static  void diziYazdir(T[] dizi) {
    System.out.println("Dizi Elemanlari: " + Arrays.toString(dizi));
  }

  public static void main(String[] args) {
    // 1. Tam Sayılarla Kullanım
    Integer[] sayilar = {10, 20, 30};
    diziYazdir(sayilar);

    // 2. Metinlerle Kullanım
    String[] metinler = {"Java", "C++", "Python"};
    diziYazdir(metinler);
  }
}
```

```
/*Program Çıktısı:
Dizi Elemanlari: [10, 20, 30]
Dizi Elemanlari: [Java, C++, Python]
*/
```

Kod 4.1: Jenerik Metot Örneği

4.3 Değişken Sayıda Argüman (Varargs)

Java, C++'taki değişken sayıda argüman alabilen şablonlara (*variadic templates*) benzer şekilde **Varargs** (*variable arguments*) özelliğini destekler. Üç nokta (`...`) sözdizimi kullanılarak bir metoda sınırsız sayıda aynı türden veya nesneden argüman gönderilebilir.

```
public class MainVarargs {
  // Sınırsız sayıda argüman alabilen metot
  public static void Yazdir(Object... args) {
    for (Object arg : args) {
      System.out.print(arg + " ");
    }
    System.out.println();
  }

  public static void main(String[] args) {
    Yazdir(1, 2.5, "Merhaba", 'A', 42);
  }
}
/*Program Çıktısı:
1 2.5 Merhaba A 42
*/
```

Kod 4.2: Değişken Sayıda Argüman Alan Metot Örneği

4.4 Sınıf Şablonları (Jenerik Sınıflar)

Jenerik sınıflar (*generic classes*), bir sınıfın içerisindeki alanların (*fields*) ve metotların türlerini dışarıdan parametre olarak almasını sağlar. C++ sınıf şablonlarının Java'daki doğrudan karşılığıdır.

```
// 1. Jenerik Sınıf Tanımı
public class SablonSinif {
  private T icerik; // T tipinde bir veri saklarpublic SablonSinif(T pIcerik) {
    this.icerik = pIcerik;
  }

  public T getIcerik() {
    return icerik;
  }

  public void setIcerik(T pYeniIcerik) {
    this.icerik = pYeniIcerik;
  }
}

// 2. İstemci (Client)
class MainSinifSablonu {
  public static void main(String[] args) {
    // T yerine 'Integer' geçiyoruz (İlkel türler yerine wrapper sınıflar kullanılır)
    SablonSinif tamSayiSinifi = new SablonSinif<>(42);
    // T yerine 'String' geçiyoruz
    SablonSinif metinSinifi = new SablonSinif<>("Merhaba Java");    System.out.println(
        tamSayiSinifi.getIcerik());
    System.out.println(metinSinifi.getIcerik());
  }
}
/*Program Çıktısı:
42
Merhaba Java
```

```
*/
```

Kod 4.3: Sınıf Şablonu Örneği

Birden fazla tür parametresi alan sınıflar da tanımlanabilir:

```
class Cift<T1, T2> {
  public T1 birincil;
  public T2 ikincil;public Cift(T1 a, T2 b) {
    this.birincil = a;
    this.ikincil = b;
  }

  public void ciftYaz() {
    System.out.println(birincil + ": " + ikincil);
  }
}

class MainCift {
  public static void main(String[] args) {
    Cift<String, Integer> yasVerisi = new Cift<>("Yasim", 45);
    yasVerisi.ciftYaz();
  }
}
/*Program Çıktısı:
Yasim: 45
*/
```

Kod 4.4: Sınıf Şablonunda Farklı Veri Tiplerini Kullanma

4.5 Faydalı Yapılar ve Optional

Faydalı yapılar Java standart kütüphanesinde yer alır.

§4.5.1 std::optional Karşılığı: java.util.Optional

C++17'deki `std::optional` yapısının Java'daki karşılığı `java.util.Optional` sınıfıdır. Bir değerin var olup olmadığını güvenli bir şekilde temsil eder.

```
import java.util.Optional;   public class MainOptional {
  public static Optional divide(float a, float b) {
    if (b != 0.0f) {
      return Optional.of(a / b);
    }
    return Optional.empty();
  }

  public static void main(String[] args) {
    Optional<Float> sonuc = divide(10.0f, 2.0f);
    if (sonuc.isPresent()) {
      System.out.println("Bolum: " + sonuc.get());
    } else {
      System.out.println("Sifira bolunme hatasi!");
    }
  }
}
/*Program Çıktısı:
Bolum: 5.0
*/
```

Kod 4.5: Optional Kullanımı

4.6 Bellek Yönetimi ve Çöp Toplayıcı (Garbage Collector)

§4.6.1 Otomatik Bellek Yönetimi ve RAII Yaklaşımı

C++ dilindeki **RAII** prensibinin (*Resource Acquisition Is Initialization*) ve akıllı göstericilerin aksine, Java otomatik bir **Çöp Toplayıcı GC** (*Garbage Collector*) mekanizmasına sahiptir. Java'da dinamik olarak `new` anahtar sözcüğüyle oluşturulan nesneler öbek (*heap*) bellekte saklanır; bu nesnelere artık hiçbir referans kalmadığında çöp toplayıcı belleği otomatik olarak geri kazanır. Bellek sızıntısı riski minimumdur. Bununla birlikte, dosya akışları veya veritabanı bağlantıları gibi dış kaynakları güvenle kapatmak için Java'da *Try-with-Resources* yapısı kullanılır:

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;public class MainKaynakYonetimi {
  public static void dosyaOku(String dosyaAdi) {
    // Kaynak kapsam bittiğinde otomatik olarak kapatılır
    try (BufferedReader br = new BufferedReader(new FileReader(dosyaAdi))) {
      System.out.println(br.readLine());
    } catch (IOException e) {
      System.out.println("Dosya okuma hatasi: " + e.getMessage());
    }
  }
}
```

Kod 4.6: Try-with-Resources ile Güvenli Kaynak Yönetimi

4.7 Java Koleksiyonlar Çerçevesi (Collections Framework)

C++ Standart Şablon Kütüphanesi'ndeki (*STL*) konteynerlerin Java dünyasındaki karşılığı **Java Collections Framework** yapısıdır. Koleksiyonlar temel olarak şu kategorilere ayrılır:

1. **Listeler** (*List*): Sıralı eleman koleksiyonlarıdır (Örn: `ArrayList`, `LinkedList`).
2. **Kümeler** (*Set*): Benzersiz elemanları tutan koleksiyonlar (Örn: `HashSet`, `TreeSet`).
3. **Haritalar** (*Map*): Anahtar-değer (*key-value*) çiftleri (Örn: `HashMap`, `TreeMap`).

§4.7.1 ArrayList Örneği

C++'taki `std::vector` yapısının Java'daki karşılığı dinamik olarak büyüyen `ArrayList` sınıfıdır.

```
import java.util.ArrayList;
import java.util.List;   public class MainArrayList {
  public static void main(String[] args) {
    List vec = new ArrayList<>();    for (int i = 0; i < 5; i++) {
      vec.add(i * 1.5);
    }

    System.out.println("Vektör Boyutu = " + vec.size());
    for (int i = 0; i < vec.size(); i++) {
      System.out.println("vec[" + i + "] = " + vec.get(i));
    }

    // Gelişmiş for döngüsü (For-each)
    for (Double val : vec) {
      System.out.print(val + ", ");
    }
    System.out.println();
  }
}
/*Program Çıktısı:
Vektör Boyutu = 5
vec[0] = 0.0
vec[1] = 1.5
vec[2] = 3.0
vec[3] = 4.5
vec[4] = 6.0
0.0, 1.5, 3.0, 4.5, 6.0,
```

```
*/
```

Kod 4.7: ArrayList Kullanımı

§4.7.2 HashMap Örneği

C++'taki `std::map` karşılığı Java'da `HashMap` veya sıralı versiyonu olan `TreeMap` sınıflarıdır.

```
import java.util.HashMap;
import java.util.Map;   public class MainHashMap {
  public static void main(String[] args) {
    Map<String, Integer> meyveSayisi = new HashMap<>();    meyveSayisi.put("uzum", 20);
    meyveSayisi.put("portakal", 30);
    meyveSayisi.put("muz", 40);

    System.out.println("Muz miktari: " + meyveSayisi.get("muz"));

    // Harita üzerinde gezinme
    for (Map.Entry<String, Integer> entry : meyveSayisi.entrySet()) {
      System.out.println(entry.getKey() + " : " + entry.getValue());
    }
  }
}
/*Program Çıktısı:
Muz miktari: 40
uzum : 20
portakal : 30
muz : 40
*/
```

Kod 4.8: HashMap Kullanımı

§4.7.3 HashSet Örneği

C++'taki `std::set` yapısının karşılığı Java'da `HashSet` sınıfıdır. Aynı eleman kümede iki kez saklanmaz.

```
import java.util.HashSet;
import java.util.Set;   public class MainHashSet {
  public static void main(String[] args) {
    Set isimler = new HashSet<>();
    isimler.add("Ali");
    isimler.add("Veli");
    isimler.add("Ali"); // Yinelenen eleman eklenmez    System.out.println("Kume eleman
        sayisi: " + isimler.size());
    for (String isim : isimler) {
      System.out.println(isim);
    }
  }
}
/*Program Çıktısı:
Kume eleman sayisi: 2
Veli
Ali
*/
```

Kod 4.9: HashSet Kullanımı

4.8 Stream API ve Algoritmalar

Java 8 ile gelen **Stream API**, koleksiyonlar üzerinde veri işleme, filtreleme, sıralama ve dönüştürme işlemlerini fonksiyonel bir yaklaşımla yapmanızı sağlar.

```
import java.util.Arrays;
import java.util.List;public class MainStreams {
  public static void main(String[] args) {
    List sayilar = Arrays.asList(50, 10, 40, 20, 30);    // Sıralama ve ekrana yazdırma
```

```
        (Stream API)
    sayilar.stream()
    .sorted()
    .forEach(n -> System.out.print(n + " "));
  }
}
/*Program Çıktısı:
10 20 30 40 50
*/
```

Kod 4.10: Java Streams Örneği

4.9 Fonksiyonel Arayüzler ve Lambda İfadeleri

C++'taki fonksiyon nesneleri (functor) ve `std::function` yapısı, Java dünyasında **Fonksiyonel Arayüzler** (*Functional Interfaces*) ve **Lambda İfadeleri** ile ifade edilir. Tek bir soyut metoda sahip arayüzler (`Function`, `Consumer`, `Predicate` vb.) lambda ifadeleriyle doğrudan atanabilir.

```
import java.util.function.Function;public class MainLambda {
  public static void main(String[] args) {
    // Lambda ifadesi ile fonksiyon tanımlama
    Function<Integer, Integer> artirim = x -> x + 1;    int sonuc = artirim.apply(5);
    System.out.println("Sonuc: " + sonuc);
  }
}
/*Program Çıktısı:
Sonuc: 6
*/
```

Kod 4.11: Lambda İfadeleri ve Fonksiyonel Arayüzler

5. Bölüm
Tasarım Kalıpları

5.1 Kalıp Nedir?

Nesne yönelimli programlamanın ortaya çıkışıyla beraber, daha önceden yazılmış hazır bileşenlerin (*component*) yeniden kullanımı (*reusing*) konusunda oldukça önemli ilerlemeler sağlamıştır. Bunun paralelinde tecrübelerin (*experience*) yeniden kullanımı konusunda da çeşitli çalışmalar yapılmış ve **Kalıp** (*pattern*) kavramı ortaya çıkmıştır. kalıpların aksine **Anti-Kalıp** (*anti-pattern*) ise **başlangıçta çekici görünen** ancak uygulama sürecinde **verimsiz olduğu kanıtlanmış yaygın hatalı çözümlerdir**.

Yazılım geliştirme öncesi, geliştirme aşması ve sonrasında daha önce yaşanmış çeşitli problemlere karşı birçok kimse tarafından çeşitli çözümler getirilmiştir. **kalıplar, daha önce geliştirme yapan programcıların yanlış yaparak doğru yapmayı öğrendikleri başarılı tecrübelerin anlatılması için bir araçtır**. Gerçekleştirilen bu çözümlerin, daha sonradan yaşanacak benzer nitelikli problemlerde kullanılması amacıyla kalıplar kullanılır. Kalıp kavramının temelleri Mimar *Christopher Alexander*'ın 1970 sonlarında başlattığı çalışmalara dayanmaktadır[1].

1987 yılında uluslararası "*Nesne Yönelimli Programlama, Sistemler, Diller ve Uygulamalar*" konferansına kadar kalıplarla ilgili bir çalışma ortaya çıkmamıştır[2]. Bu tarihten sonra ise başta *Grady Booch*, *Richard Helm*, *Erich Gamma* ve *Kent Beck* olmak üzere pek çok bilim adamı kalıplarla ilgili makale ve sunumlar yayınlamışlardır. 1994 yılında *Erich Gamma*, *Richard Helm*, *Ralph Johnson* ve *John Vlissides* tarafından yayınlanan "*Tasarım kalıpları: Tekrar kullanılabilir Nesne Yönelimli Yazılımın Temelleri*" kitabı, tasarım kalıpların yazılımda kullanılmasında dönüm noktası olmuştur[3]. Yazılımlarda kullanılan kalıplar yedi ana başlıkta toplanırlar. Bunlar;

1. Mimari kalıplar (*architectural pattern*)
2. Analiz kalıpları (*analysis pattern*)
3. Tasarım kalıpları (*design pattern*)
4. Kodlama kalıpları (*implementation pattern*)
5. Test kalıpları (*test pattern*)
6. Çözüm kalıpları (*solution pattern*)
7. Veri kalıpları (*data pattern*)

Bu kalıpların en önemlisi ve ilk gelişenlerden birisi, **tasarım kalıplarıdır** (*design pattern*). Tasarım kalıpları aslında, Nesne Yönelimli Tasarım Prensiplerini, yazılımlara uygulamanın bir başka ifadesidir. Bu nedenle çok önemlidir. Tasarım kalıpları ise üç ana başlıkta toplanmaktadırlar.

1. Nesne imali ile ilgili kalıplar (*creational patterns*)
2. Yapısal kalıplar (*structural patterns*)
3. Davranışlarla ilgili kalıplar (*behavioral patterns*)

kalıpların çok sık kullanıldığı programlama yöntemine günümüzde **kalıp yönelimli program-**

[1] http://gee.cs.oswego.edu/dl/ca/ca/ca.html
[2] OOPSLA, Object Oriented Programming, Systems, Languages, and Applications
[3] Design Patterns: Elements of Reusable Object-Oriented Software, ISBN 0-201-63361-2

lama (*pattern oriented programming*) adı verilmektedir[4]. İzleyen sayfalarda verilen kalıplara ait UML diyagramlarının birçoğu *Erich Gamma*'nın **Design Pattern CD** yayınından alınmıştır[5]. Bölümün devamında verilecek tasarım kalıplarının UML diyagramları visual-paradigm adresinden alınmıştır[6].

5.2 Nesne İmalatı ile İlgili Kalıplar

Nesne imalatı ile ilgili kalıplar (*creational patterns*), nesne (*object*) örnekleme ile ilgili ortaya çıkabilecek çeşitli problemlere çözüm getirmektedirler. Bu kalıplarda nesneler dinamik olarak imal edilirler, çünkü değişim yönetiminde statik nesneler kullanılmazlar.

§5.2.1 Soyut Fabrika

Soyut Fabrika Kalıbı (*Abstract Factory Pattern*), birbiriyle ilişkili veya birbirine bağımlı nesne ailelerini, bu nesnelerin somut sınıflarını istemci koda belirtmeden oluşturmayı sağlayan yaratıcı (*creational*) bir tasarım kalıbıdır.

Bu kalıbın temel amacı, nesnelerin oluşturulma sürecini nesnelerin kullanılma sürecinden ayırmaktır. Böylece istemci kod, hangi somut sınıfın oluşturulduğunu bilmek zorunda kalmaz. İstemci yalnızca soyut arayüzlerle çalışır. Somut ürün sınıfları ve somut fabrika sınıfları istemci koddan gizlenmiş olur.

Soyut fabrika kalıbı, çoğu zaman **fabrikaların fabrikası** olarak da adlandırılır. Çünkü bu kalıpta yalnızca tek bir nesne değil, birbiriyle uyumlu nesne aileleri oluşturulur.

Örneğin bir arayüz kütüphanesinde Windows ve Mac işletim sistemleri için farklı görünüme sahip bileşenler olduğunu düşünelim. Windows için oluşturulan buton, Windows penceresiyle uyumlu olmalıdır. Mac için oluşturulan buton ise Mac penceresiyle uyumlu olmalıdır. Bu durumda istemci kodun doğrudan `WindowsButton`, `MacButton`, `WindowsWindow` veya `MacWindow` sınıflarını bilmesi istenmez. Bunun yerine istemci, soyut fabrika üzerinden uyumlu nesneleri elde eder.

Soyut fabrika kalıbındaki temel roller şunlardır:

- **Soyut Fabrika** (*Abstract Factory*): İlgili ürün ailesindeki nesneleri oluşturacak metotların imzalarını tanımlar. Örneğin `butonOlustur()` ve `pencereOlustur()` gibi metotlar içerir.
- **Somut Fabrika** (*Concrete Factory*): Belirli bir ürün ailesine ait somut nesneleri oluşturur. Örneğin `WindowsArayuzFabrikasi`, Windows ailesine ait buton ve pencere nesnelerini üretir.
- **Soyut Ürün** (*Abstract Product*): Ürün ailesindeki her ürün türü için ortak arayüzü tanımlar. Örneğin tüm butonlar için `Button`, tüm pencereler için `Window` arayüzü tanımlanabilir.
- **Somut Ürün** (*Concrete Product*): Soyut ürün arayüzlerini gerçekleştiren gerçek sınıflardır. Örneğin `WindowsButton`, `MacButton`, `WindowsWindow` ve `MacWindow` sınıfları somut ürünlerdir.
- **İstemci** (*Client*): Somut sınıflarla değil, soyut fabrika ve soyut ürün arayüzleriyle çalışır.

Bu kalıp özellikle şu durumlarda kullanılır:

- Sistem birden fazla ürün ailesiyle çalışıyorsa.
- Aynı aileye ait nesnelerin birlikte ve uyumlu kullanılması gerekiyorsa.
- İstemci kodun somut sınıflardan bağımsız olması isteniyorsa.
- Ürün ailesi değiştirilirken istemci kodun değişmemesi hedefleniyorsa.

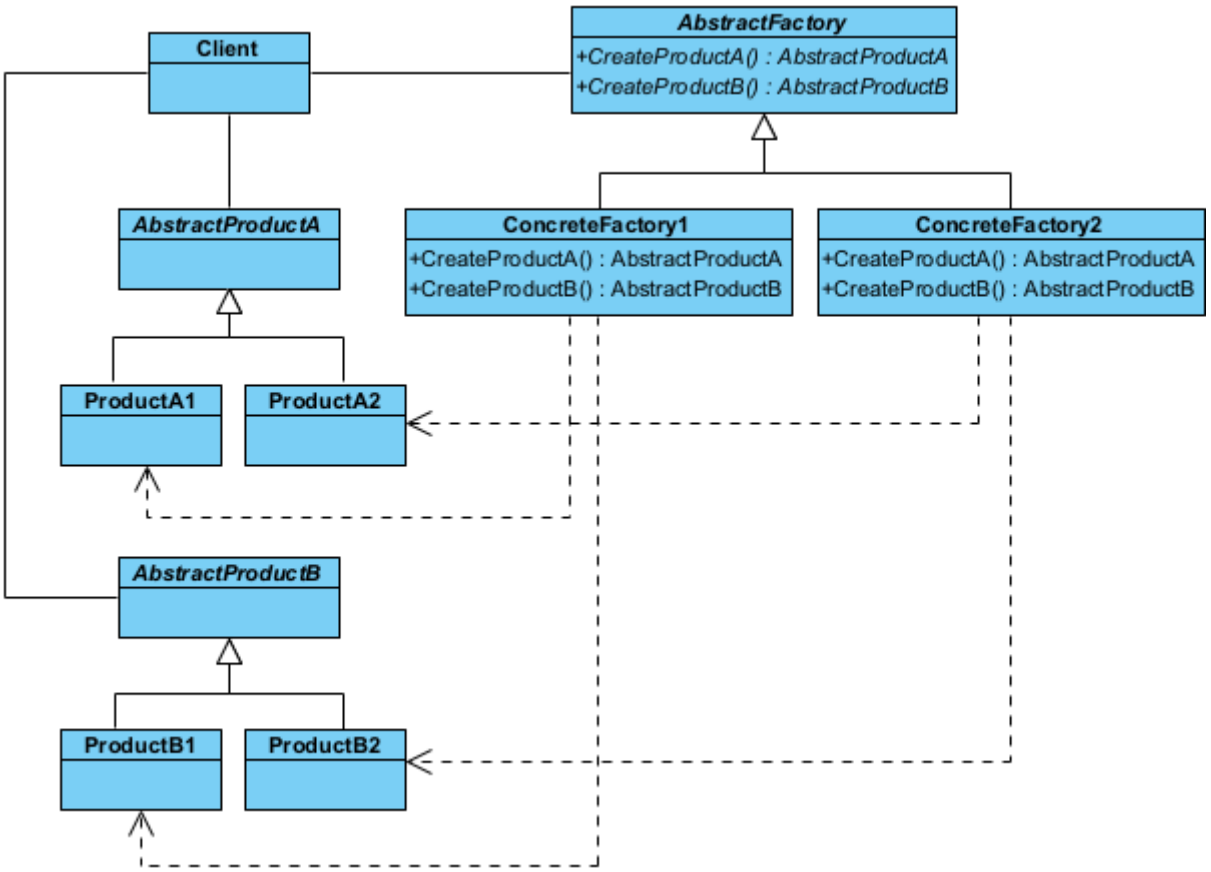
Java ile Soyut Fabrika Örneği

Aşağıdaki örnekte Windows ve Mac işletim sistemleri için farklı arayüz bileşenleri üreten bir soyut fabrika tasarlanmıştır. Her işletim sistemi için bir buton ve bir pencere oluşturulmaktadır.

Önce soyut ürün arayüzleri tanımlanır:

```
interface Button {
```

[4] www.mathcs.sjsu.edu/faculty/pearce/patterns.html
[5] Design Pattern CD, Addison-Wesley,1998
[6] https://circle.visual-paradigm.com/category/uml-diagrams/gof-design/



Şekil 5.1: Soyut Fabrika Kalıbı UML Diyagramı

```
  void ciz();
}

interface Window {
  void ac();
}
```

Kod 5.1: Java Dilinde Soyut Ürün Arayüzleri

Burada `Button` ve `Window`, ürün ailesindeki iki farklı ürün türünü temsil eder. Her buton çizilebilir, her pencere açılabilir.

Şimdi Windows ailesine ait somut ürünleri yazalım:

```
class WindowsButton implements Button {
  @Override
  public void ciz() {
    System.out.println("Windows butonu çizildi.");
  }
}

class WindowsWindow implements Window {
  @Override
  public void ac() {
    System.out.println("Windows penceresi açıldı.");
  }
}
```

Kod 5.2: Java Dilinde Windows Ürünleri

Mac ailesine ait somut ürünler ise şu şekildedir:

```
class MacButton implements Button {
```

```
    @Override
    public void ciz() {
      System.out.println("Mac butonu çizildi.");
    }
  }

  class MacWindow implements Window {
    @Override
    public void ac() {
      System.out.println("Mac penceresi açıldı.");
    }
  }
```

Kod 5.3: Java Dilinde Mac Ürünleri

Bu aşamada iki farklı ürün ailesi oluşmuştur:

- Windows ürün ailesi: `WindowsButton`, `WindowsWindow`
- Mac ürün ailesi: `MacButton`, `MacWindow`

Şimdi soyut fabrika arayüzü tanımlanır:

```
  interface ArayuzFabrikasi {
    Button butonOlustur();
    Window pencereOlustur();
  }
```

Kod 5.4: Java Dilinde Soyut Fabrika Arayüzü

Bu arayüz, bir arayüz ailesi oluşturmak için gerekli metotları tanımlar. Ancak hangi somut sınıfların oluşturulacağını belirtmez.

Windows ürün ailesini üreten somut fabrika:

```
  class WindowsArayuzFabrikasi implements ArayuzFabrikasi {
    @Override
    public Button butonOlustur() {
      return new WindowsButton();
    }

    @Override
    public Window pencereOlustur() {
      return new WindowsWindow();
    }
  }
```

Kod 5.5: Java Dilinde Windows Fabrikası

Mac ürün ailesini üreten somut fabrika:

```
  class MacArayuzFabrikasi implements ArayuzFabrikasi {
    @Override
    public Button butonOlustur() {
      return new MacButton();
    }

    @Override
    public Window pencereOlustur() {
      return new MacWindow();
    }
  }
```

Kod 5.6: Java Dilinde Mac Fabrikası

Artık istemci kod yalnızca `ArayuzFabrikasi`, `Button` ve `Window` arayüzlerini bilir. Somut sınıfları doğrudan kullanmaz.

```
  class Uygulama {
```

```
    private final Button button;
    private final Window window;

    public Uygulama(ArayuzFabrikasi fabrika) {
      this.button = fabrika.butonOlustur();
      this.window = fabrika.pencereOlustur();
    }

    public void calistir() {
      button.ciz();
      window.ac();
    }
  }
```

Kod 5.7: Java Dilinde İstemci Sınıf

Son olarak uygulama çalıştırılır:

```
  public class Main {
    public static void main(String[] args) {
      ArayuzFabrikasi fabrika;

      String isletimSistemi = "windows";

      if (isletimSistemi.equalsIgnoreCase("windows")) {
        fabrika = new WindowsArayuzFabrikasi();
      } else if (isletimSistemi.equalsIgnoreCase("mac")) {
        fabrika = new MacArayuzFabrikasi();
      } else {
        throw new IllegalArgumentException("Desteklenmeyen işletim sistemi.");
      }

      Uygulama uygulama = new Uygulama(fabrika);
      uygulama.calistir();
    }
  }
```

Kod 5.8: Java Dilinde Soyut Fabrika Kullanımı

Program çıktısı aşağıdaki gibi olur:

```
Windows butonu çizildi.
Windows penceresi açıldı.
```

Eğer `isletimSistemi` değişkeni `"mac"` yapılırsa, istemci kod değiştirilmeden Mac ailesine ait nesneler oluşturulur:

```
Mac butonu çizildi.
Mac penceresi açıldı.
```

Kalıbın Sağladığı Avantaj

Bu tasarımda `Uygulama` sınıfı doğrudan şu sınıflara bağımlı değildir:

- `WindowsButton`
- `MacButton`
- `WindowsWindow`
- `MacWindow`

Bunun yerine yalnızca şu soyutlamalara bağımlıdır:

- `ArayuzFabrikasi`
- `Button`
- `Window`

Bu durum, bağımlılıkları tersine çevirme ilkesiyle de uyumludur. Üst seviye istemci kod, alt

seviye somut sınıflara değil, soyutlamalara bağlıdır.

Yeni Bir Ürün Ailesi Eklemek

Soyut fabrika kalıbının güçlü yönlerinden biri, yeni bir ürün ailesi eklemeyi kolaylaştırmasıdır. Örneğin Linux arayüz ailesi eklemek istersek yeni ürün sınıfları ve yeni bir fabrika yazabiliriz.

```
class LinuxButton implements Button {
  @Override
  public void ciz() {
    System.out.println("Linux butonu çizildi.");
  }
}

class LinuxWindow implements Window {
  @Override
  public void ac() {
    System.out.println("Linux penceresi açıldı.");
  }
}

class LinuxArayuzFabrikasi implements ArayuzFabrikasi {
  @Override
  public Button butonOlustur() {
    return new LinuxButton();
  }

  @Override
  public Window pencereOlustur() {
    return new LinuxWindow();
  }
}
```

Kod 5.9: Java Dilinde Linux Ürün Ailesi

Bu ekleme sonucunda `Uygulama` sınıfını değiştirmeye gerek kalmaz. Yalnızca uygun fabrika nesnesi verilerek farklı ürün ailesiyle çalışılabilir.

Soyut Fabrika Kalıbının Avantajları

- İstemci kod somut sınıflardan bağımsız hale gelir.
- Birbiriyle uyumlu nesne aileleri birlikte oluşturulur.
- Ürün ailesi değiştirmek kolaylaşır.
- Bağımlılıkları tersine çevirme ilkesine uygun bir yapı sağlar.
- Kodun test edilebilirliği ve genişletilebilirliği artar.

Soyut Fabrika Kalıbının Dezavantajları

- Sınıf ve arayüz sayısını artırır.
- Küçük projelerde gereğinden fazla karmaşıklık oluşturabilir.
- Yeni bir ürün türü eklemek zordur. Örneğin `Menu` adında yeni bir ürün türü eklenirse, tüm fabrika arayüzlerinin ve somut fabrikaların güncellenmesi gerekir.

Ne Zaman Kullanılmalı?

Soyut fabrika kalıbı şu durumlarda tercih edilmelidir:

- Sistem farklı ürün aileleriyle çalışıyorsa.
- Aynı aileye ait nesnelerin birlikte kullanılması gerekiyorsa.
- İstemci kodun somut sınıfları bilmesi istenmiyorsa.
- Ürün ailesi çalışma zamanında veya yapılandırmaya göre değiştirilecekse.

Özet

Soyut fabrika kalıbı, ilişkili nesne ailelerini üretmek için kullanılan güçlü bir yaratıcı tasarım kalıbıdır. Bu kalıp sayesinde istemci kod, somut sınıflara bağımlı olmadan farklı ürün aileleriyle çalışabilir. Özellikle platforma, temaya, veritabanı sağlayıcısına veya dış sistem türüne göre farklı nesne gruplarının oluşturulması gereken durumlarda oldukça kullanışlıdır.

§5.2.2 Fabrika Yöntemi

Fabrika Yöntemi Kalıbı (*Factory Method Pattern*), oluşturulacak nesnenin somut sınıfını istemci kodda açıkça belirtmeden nesne üretmeyi sağlayan bir tasarım kalıbıdır.

Bu kalıpta nesne oluşturma sorumluluğu doğrudan istemciye verilmez. Bunun yerine nesne üretme işi bir fabrika yöntemine bırakılır. Hangi somut ürün sınıfının oluşturulacağına ise çoğunlukla alt sınıflar karar verir.

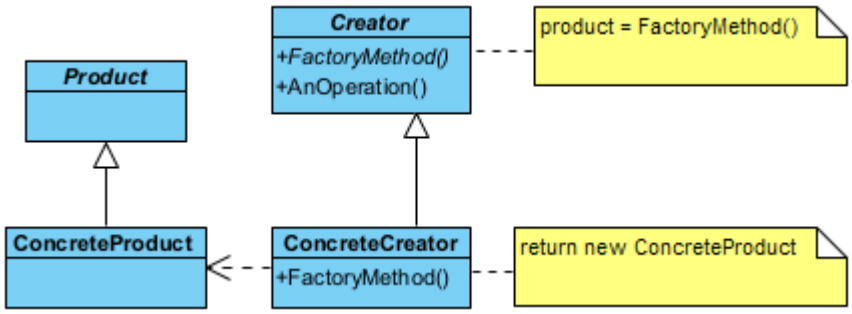
Başka bir ifadeyle, istemci kod şunu yapmak yerine:

```
Product urun = new ConcreteProduct();
```

şu şekilde bir soyutlama üzerinden çalışır:

```
Product urun = creator.factoryMethod();
```

Böylece istemci kod, somut ürün sınıfına doğrudan bağımlı olmaz.



Şekil 5.2: Fabrika Yöntemi Kalıbı UML Diyagramı

Fabrika yöntemi kalıbındaki temel roller şunlardır:

- **Ürün** (`Product`): Üretilecek nesnelerin ortak arayüzünü veya soyut sınıfını temsil eder.
- **Somut Ürün** (`ConcreteProduct`): Ürün arayüzünü gerçekleştiren gerçek nesne sınıfıdır.
- **Üretici** (`Creator`): Fabrika yöntemini tanımlayan soyut sınıf veya arayüzdür. Genellikle `factoryMethod()` gibi bir metot içerir.
- **Somut Üretici** (`ConcreteCreator`): Fabrika yöntemini ezerek hangi somut ürünün oluşturulacağını belirleyen sınıftır.

Bu kalıbın temel amacı, nesne oluşturma kodunu istemci koddan ayırmaktır. Böylece yeni ürün türleri eklendiğinde istemci kodun değişme ihtiyacı azalır.

Java ile Fabrika Yöntemi Örneği

Aşağıdaki örnekte farklı türde belgeler üreten bir yapı tasarlanacaktır. Sistemde PDF ve Word belgeleri üretilebilmektedir. İstemci kod, doğrudan `PdfBelge` veya `WordBelge` sınıflarını oluşturmak yerine, bir üretici sınıf üzerinden belge oluşturacaktır.

Önce ürün arayüzü tanımlanır:

```
interface Belge {
  String getBelgeAdi();
  void ac();
}
```

Kod 5.10: Java Dilinde Ürün Arayüzü

Bu arayüz, tüm belge türlerinin sahip olması gereken ortak davranışları tanımlar.

Şimdi somut ürün sınıfları yazılır:

```
class PdfBelge implements Belge {
  private final String belgeAdi;

  public PdfBelge(String belgeAdi) {
    this.belgeAdi = belgeAdi;
  }

  @Override
  public String getBelgeAdi() {
    return belgeAdi;
  }

  @Override
  public void ac() {
    System.out.println(belgeAdi + " adlı PDF belge açıldı.");
  }
}

class WordBelge implements Belge {
  private final String belgeAdi;

  public WordBelge(String belgeAdi) {
    this.belgeAdi = belgeAdi;
  }

  @Override
  public String getBelgeAdi() {
    return belgeAdi;
  }

  @Override
  public void ac() {
    System.out.println(belgeAdi + " adlı Word belge açıldı.");
  }
}
```

Kod 5.11: Java Dilinde Somut Ürün Sınıfları

Bu aşamada `PdfBelge` ve `WordBelge`, `Belge` arayüzünü gerçekleştiren somut ürünlerdir.

Şimdi üretici sınıf tanımlanır:

```
abstract class BelgeUretici {
  public abstract Belge belgeOlustur(String belgeAdi);

  public void belgeyiAc(String belgeAdi) {
    Belge belge = belgeOlustur(belgeAdi);

    System.out.println("Belge oluşturuldu: " + belge.getBelgeAdi());
    belge.ac();
  }
}
```

Kod 5.12: Java Dilinde Soyut Üretici Sınıf

Burada `belgeOlustur()` metodu fabrika yöntemidir. Hangi somut belgenin oluşturulacağını bu soyut sınıf bilmez. Bu karar alt sınıflara bırakılır.

`belgeyiAc()` metodu ise fabrika yöntemini kullanarak nesne oluşturur ve bu nesne üzerinde işlem yapar. Bu yapı, fabrika yönteminin yalnızca nesne üretmek için değil, daha büyük bir iş

akışının parçası olarak da kullanılabileceğini gösterir.

PDF belge üreten somut üretici:

```
class PdfBelgeUretici extends BelgeUretici {
  @Override
  public Belge belgeOlustur(String belgeAdi) {
    return new PdfBelge(belgeAdi);
  }
}
```

Kod 5.13: Java Dilinde PDF Belge Üreticisi

Word belge üreten somut üretici:

```
class WordBelgeUretici extends BelgeUretici {
  @Override
  public Belge belgeOlustur(String belgeAdi) {
    return new WordBelge(belgeAdi);
  }
}
```

Kod 5.14: Java Dilinde Word Belge Üreticisi

Son olarak istemci kod yazılır:

```
public class Main {
  public static void main(String[] args) {
    BelgeUretici uretici = new PdfBelgeUretici();

    uretici.belgeyiAc("TasarimKaliplari");

    uretici = new WordBelgeUretici();

    uretici.belgeyiAc("Rapor");
  }
}
```

Kod 5.15: Java Dilinde Fabrika Yöntemi Kullanımı

Program çıktısı aşağıdaki gibi olur:

```
Belge oluşturuldu: TasarimKaliplari
TasarimKaliplari adlı PDF belge açıldı.
Belge oluşturuldu: Rapor
Rapor adlı Word belge açıldı.
```

Bu örnekte istemci kod `Belge` ve `BelgeUretici` soyutlamalarıyla çalışır. Nesnenin gerçekte `PdfBelge` mi yoksa `WordBelge` mi olduğu, kullanılan somut üretici sınıfa bağlıdır.

Kalıbın Sağladığı Avantaj

Fabrika yöntemi kalıbı sayesinde istemci kod doğrudan somut ürün sınıflarını oluşturmaz. Örneğin istemci kodda şu kullanım azaltılır:

```
Belge belge = new PdfBelge("Rapor");
```

Bunun yerine nesne oluşturma işlemi üretici sınıfa bırakılır:

```
BelgeUretici uretici = new PdfBelgeUretici();
Belge belge = uretici.belgeOlustur("Rapor");
```

Bu yaklaşımın sonucu olarak:

- ◇ Nesne oluşturma kodu tek bir yerde toplanır.
- ◇ İstemci kod somut ürün sınıflarına daha az bağımlı hale gelir.
- ◇ Yeni ürün türleri eklemek kolaylaşır.
- ◇ Üretim süreci gerektiğinde alt sınıflar tarafından özelleştirilebilir.

Yeni Bir Ürün Türü Eklemek

Sisteme yeni bir belge türü eklemek istediğimizi düşünelim. Örneğin `ExcelBelge` adında yeni bir ürün sınıfı eklenebilir.

```
class ExcelBelge implements Belge {
  private final String belgeAdi;

  public ExcelBelge(String belgeAdi) {
    this.belgeAdi = belgeAdi;
  }

  @Override
  public String getBelgeAdi() {
    return belgeAdi;
  }

  @Override
  public void ac() {
    System.out.println(belgeAdi + " adlı Excel belge açıldı.");
  }
}

class ExcelBelgeUretici extends BelgeUretici {
  @Override
  public Belge belgeOlustur(String belgeAdi) {
    return new ExcelBelge(belgeAdi);
  }
}
```

Kod 5.16: Java Dilinde Yeni Ürün Türü Eklemek

Bu ekleme sonucunda mevcut `PdfBelgeUretici` veya `WordBelgeUretici` sınıflarının değişmesine gerek kalmaz. Yeni davranış, yeni sınıflar eklenerek sisteme dahil edilir.

Fabrika Yöntemi ve Basit Fabrika Arasındaki Fark

Fabrika yöntemi kalıbı, basit fabrika yaklaşımıyla karıştırılabilir. Basit fabrikada genellikle tek bir sınıf içinde koşul ifadeleriyle nesne oluşturulur:

```
class BasitBelgeFabrikasi {
  public Belge belgeOlustur(String tur, String belgeAdi) {
    if (tur.equalsIgnoreCase("pdf")) {
      return new PdfBelge(belgeAdi);
    }

    if (tur.equalsIgnoreCase("word")) {
      return new WordBelge(belgeAdi);
    }

    throw new IllegalArgumentException("Bilinmeyen belge türü.");
  }
}
```

Bu yaklaşım küçük projelerde kullanılabilir. Ancak yeni ürün türü eklendiğinde fabrika sınıfının içindeki koşullar değiştirilir. Fabrika yöntemi kalıbında ise üretim davranışı alt sınıflara dağıtılır.

Basit Fabrika	Fabrika Yöntemi
Genellikle tek fabrika sınıfı vardır.	Üretici sınıf hiyerarşisi vardır.
Koşul ifadeleriyle nesne seçilir.	Nesne seçimi alt sınıflara bırakılır.
Yeni ürün eklenince fabrika sınıfı değişebilir.	Yeni ürün için yeni üretici sınıf eklenir.
Resmi GoF kalıplarından biri değildir.	GoF yaratıcı tasarım kalıplarından biridir.

Fabrika Yöntemi Kalıbının Avantajları

- İstemci kodun somut sınıflara bağımlılığını azaltır.
- Nesne oluşturma sorumluluğunu alt sınıflara bırakır.
- Açık-kapalı ilkesine daha uygun bir yapı sağlar.
- Ürün oluşturma sürecini merkezi ve kontrollü hale getirir.
- Farklı ürün türleriyle çalışmayı kolaylaştırır.

Fabrika Yöntemi Kalıbının Dezavantajları

- Sınıf sayısını artırabilir.
- Küçük ve basit projelerde gereğinden fazla soyutlama oluşturabilir.
- Her yeni ürün için çoğu zaman yeni bir üretici sınıf yazmak gerekir.

Ne Zaman Kullanılmalı?

Fabrika yöntemi kalıbı şu durumlarda tercih edilebilir:

- Oluşturulacak nesnenin somut sınıfı önceden kesin olarak bilinmiyorsa.
- Nesne oluşturma sorumluluğunun alt sınıflara bırakılması isteniyorsa.
- İstemci kodun `new` anahtar sözcüğüyle somut sınıflara bağımlı olması istenmiyorsa.
- Yeni ürün türlerinin sisteme daha kontrollü eklenmesi gerekiyorsa.

Özet

Fabrika yöntemi kalıbı, nesne oluşturma işlemini istemci koddan ayırarak daha esnek ve genişletilebilir bir yapı kurmayı sağlar. İstemci kod doğrudan somut ürün sınıflarını oluşturmak yerine, soyut bir üretici üzerinden çalışır. Hangi ürünün oluşturulacağına somut üretici sınıflar karar verir.

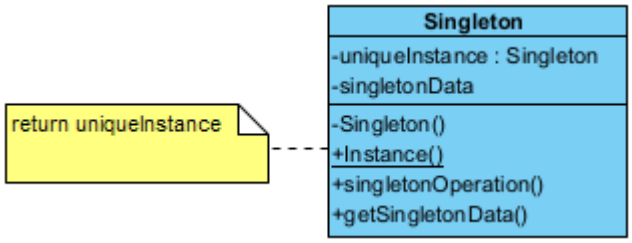
§5.2.3 Tekil Nesne

Tekil Nesne Kalıbı (*Singleton Pattern*), bir sınıftan uygulama boyunca yalnızca bir adet nesne oluşturulmasını garanti etmeyi amaçlayan yaratıcı (*creational*) bir tasarım kalıbıdır. Bu kalıpta sınıf, kendi örneğini kendisi yönetir ve dışarıdan rastgele nesne oluşturulmasını engeller.

Tekil nesne kalıbının temel amacı şudur:

- Bir sınıftan yalnızca bir nesne oluşturulmasını sağlamak.
- Bu tek nesneye uygulamanın farklı noktalarından erişim sunmak.
- Ortak kullanılan bir kaynağın merkezi olarak yönetilmesini sağlamak.

Örneğin uygulama ayarlarını tutan bir sınıf, günlükleme (*logging*) servisi veya bağlantı yöneticisi bazı durumlarda tekil nesne olarak tasarlanabilir. Ancak bu kalıp dikkatli kullanılmalıdır. Çünkü gereksiz kullanıldığında global değişken benzeri bir yapı oluşturabilir ve test edilebilirliği zorlaştırabilir.



Şekil 5.3: Tekil Nesne Kalıbı UML Diyagramı

Tekil nesne kalıbındaki temel bileşenler şunlardır:

- **Mahrem Yapıcı** (*private constructor*): Sınıf dışından `new` anahtar sözcüğü ile nesne oluşturulmasını engeller.
- **Statik Tekil Örnek** (*static unique instance*): Sınıfın tek nesnesini kendi içinde saklar.
- **Statik Erişim Yöntemi** (*static access method*): Tek nesneye erişim sağlayan metottur. Genel-

likle `getInstance()` adıyla kullanılır.

Java ile Tekil Nesne Örneği

Java'da tekil nesne kalıbını uygulamanın farklı yolları vardır. En yaygın yöntemlerden biri, sınıf içinde `private static final` bir nesne oluşturmak ve bu nesneye `getInstance()` metodu ile erişim sağlamaktır.

Aşağıdaki örnekte uygulama boyunca yalnızca bir adet `Singleton` nesnesi oluşturulur:

```
class Singleton {
  private static final Singleton INSTANCE = new Singleton();

  private int singletonData;

  private Singleton() {
    this.singletonData = 10;
    System.out.println("Nesne ilk kez oluşturuldu.");
  }

  public static Singleton getInstance() {
    return INSTANCE;
  }

  public void anOperation() {
    System.out.println("Tekil nesne görev başında...");
  }

  public int getSingletonData() {
    return singletonData;
  }
}

public class Main {
  public static void main(String[] args) {
    Singleton s1 = Singleton.getInstance();
    Singleton s2 = Singleton.getInstance();

    if (s1 == s2) {
      System.out.println("s1 ve s2 AYNI nesnedir.");
    }

    s1.anOperation();

    System.out.println("Veri: " + s1.getSingletonData());
  }
}
```

Kod 5.17: Java Dilinde Tekil Nesne Kalıbı

Program çıktısı şu şekilde olur:

```
Nesne ilk kez oluşturuldu.
s1 ve s2 AYNI nesnedir.
Tekil nesne görev başında...
Veri: 10
```

Bu örnekte dikkat edilmesi gereken noktalar şunlardır:

- ◇ `Singleton` sınıfının yapıcısı `private` olarak tanımlanmıştır.
- ◇ Bu nedenle sınıf dışından `new Singleton()` ile nesne oluşturulamaz.
- ◇ Sınıfın tek örneği `INSTANCE` adlı statik alanda tutulur.
- ◇ `getInstance()` metodu her çağrıldığında aynı nesneyi döndürür.
- ◇ `s1 == s2` karşılaştırması, iki değişkenin aynı nesneyi gösterdiğini kanıtlar.

Private Constructor Kullanımının Önemi

Tekil nesne kalıbının temel noktası, yapıcının dış dünyaya kapatılmasıdır. Eğer yapıcı metot `public` olsaydı, kullanıcılar istedikleri kadar nesne oluşturabilirdi:

```
Singleton s1 = new Singleton();
Singleton s2 = new Singleton();
```

Bu durumda sınıf artık tekil nesne özelliğini kaybederdi. Bu nedenle yapıcı metot `private` olarak tanımlanır:

```
private Singleton() {
  this.singletonData = 10;
}
```

Böylece sınıf dışından doğrudan nesne oluşturma engellenmiş olur.

Tembel Yükleme ile Tekil Nesne

Önceki örnekte tekil nesne, sınıf yüklendiği anda oluşturulur. Buna erken oluşturma (*eager initialization*) denir. Bazı durumlarda nesnenin yalnızca ihtiyaç duyulduğunda oluşturulması istenebilir. Buna tembel yükleme (*lazy initialization*) adı verilir.

Basit bir tembel yükleme örneği aşağıdaki gibidir:

```
class LazySingleton {
  private static LazySingleton instance;

  private LazySingleton() {
    System.out.println("LazySingleton nesnesi oluşturuldu.");
  }

  public static LazySingleton getInstance() {
    if (instance == null) {
      instance = new LazySingleton();
    }

    return instance;
  }

  public void calistir() {
    System.out.println("LazySingleton çalışıyor.");
  }
}
```

Kod 5.18: Java Dilinde Tembel Yükleme ile Singleton

Bu örnekte nesne, `getInstance()` metodu ilk kez çağrıldığında oluşturulur. Ancak bu sürüm çok iş parçacıklı (*multithreaded*) ortamlarda güvenli değildir. Aynı anda iki iş parçacığı `getInstance()` metodunu çağırırsa, birden fazla nesne oluşabilir.

Thread-Safe Singleton

Çok iş parçacıklı uygulamalarda tekil nesne oluşturma işlemi dikkatli yapılmalıdır. Basit bir çözüm, `getInstance()` metodunu `synchronized` olarak tanımlamaktır:

```
class ThreadSafeSingleton {
  private static ThreadSafeSingleton instance;

  private ThreadSafeSingleton() {
    System.out.println("ThreadSafeSingleton nesnesi oluşturuldu.");
  }

  public static synchronized ThreadSafeSingleton getInstance() {
    if (instance == null) {
      instance = new ThreadSafeSingleton();
    }
```

```
    return instance;
  }
}
```

Kod 5.19: Java Dilinde synchronized Singleton

Bu yaklaşım thread-safe bir yapı sağlar. Ancak `synchronized` anahtar sözcüğü her çağrıda kilitleme maliyeti oluşturabilir. Bu nedenle daha iyi bir yaklaşım olarak statik iç sınıf yöntemi kullanılabilir.

Statik İç Sınıf ile Singleton

Java'da sık tercih edilen güvenli ve verimli yöntemlerden biri, statik iç sınıf kullanımıdır. Bu yöntem hem tembel yükleme sağlar hem de thread-safe çalışır.

```
class AyarYoneticisi {
  private int temaKodu;

  private AyarYoneticisi() {
    this.temaKodu = 1;
    System.out.println("AyarYoneticisi oluşturuldu.");
  }

  private static class Holder {
    private static final AyarYoneticisi INSTANCE = new AyarYoneticisi();
  }

  public static AyarYoneticisi getInstance() {
    return Holder.INSTANCE;
  }

  public int getTemaKodu() {
    return temaKodu;
  }

  public void setTemaKodu(int temaKodu) {
    this.temaKodu = temaKodu;
  }
}

public class Main {
  public static void main(String[] args) {
    AyarYoneticisi ayar1 = AyarYoneticisi.getInstance();
    AyarYoneticisi ayar2 = AyarYoneticisi.getInstance();

    ayar1.setTemaKodu(2);

    System.out.println("ayar1 tema kodu: " + ayar1.getTemaKodu());
    System.out.println("ayar2 tema kodu: " + ayar2.getTemaKodu());

    if (ayar1 == ayar2) {
      System.out.println("ayar1 ve ayar2 aynı nesnedir.");
    }
  }
}
```

Kod 5.20: Java Dilinde Statik İç Sınıf ile Singleton

Program çıktısı:

```
AyarYoneticisi oluşturuldu.
ayar1 tema kodu: 2
ayar2 tema kodu: 2
ayar1 ve ayar2 aynı nesnedir.
```

Bu örnekte `AyarYoneticisi` nesnesi, `getInstance()` metodu çağrılmadan oluşturulmaz. `Holder` sınıfı yalnızca ihtiyaç duyulduğunda yüklenir. Java sınıf yükleme mekanizması bu işlemi thread-safe biçimde gerçekleştirir.

Enum ile Singleton

Java'da singleton oluşturmanın en güvenli yollarından biri `enum` kullanmaktır. Bu yöntem özellikle serileştirme ve yansıma (*reflection*) gibi durumlara karşı daha dayanıklıdır.

```
enum LogYoneticisi {
  INSTANCE;

  public void logYaz(String mesaj) {
    System.out.println("[LOG] " + mesaj);
  }
}

public class Main {
  public static void main(String[] args) {
    LogYoneticisi logger1 = LogYoneticisi.INSTANCE;
    LogYoneticisi logger2 = LogYoneticisi.INSTANCE;

    logger1.logYaz("Uygulama başlatıldı.");

    if (logger1 == logger2) {
      System.out.println("logger1 ve logger2 aynı nesnedir.");
    }
  }
}
```

Kod 5.21: Java Dilinde Enum Singleton

Program çıktısı:

```
[LOG] Uygulama başlatıldı.
logger1 ve logger2 aynı nesnedir.
```

`enum` tabanlı singleton kısa, güvenli ve etkilidir. Ancak sınıf kalıtımı gerektiren tasarımlarda kullanılamaz; çünkü Java'da enum türleri başka bir sınıftan kalıtım alamaz.

Tekil Nesne Kalıbının Avantajları

- ◇ Sınıftan yalnızca bir nesne oluşturulmasını garanti eder.
- ◇ Ortak kaynakların merkezi olarak yönetilmesini sağlar.
- ◇ Nesneye global erişim noktası sunar.
- ◇ Gereksiz nesne üretimini engelleyebilir.

Tekil Nesne Kalıbının Dezavantajları

- ◇ Global değişken benzeri davranış oluşturabilir.
- ◇ Test edilebilirliği zorlaştırabilir.
- ◇ Sınıflar arasında gizli bağımlılıklar oluşturabilir.
- ◇ Çok iş parçacıklı ortamlarda hatalı uygulanırsa birden fazla nesne oluşabilir.
- ◇ Aşırı kullanıldığında tasarımı katı hale getirebilir.

Ne Zaman Kullanılmalı?

Tekil nesne kalıbı şu durumlarda tercih edilebilir:

- ◇ Uygulama genelinde tek bir nesneye ihtiyaç varsa.
- ◇ Ortak bir kaynak merkezi olarak yönetilecekse.
- ◇ Nesnenin birden fazla örneğinin oluşması mantıksal hata oluşturuyorsa.
- ◇ Günlükleme, yapılandırma veya merkezi yönetim gibi durumlarda kontrollü kullanım gerekiyorsa.

Dikkat Edilmesi Gerekenler

Singleton, her ortak kullanılan sınıf için otomatik olarak tercih edilmemelidir. Çünkü singleton kullanımı, sınıfların doğrudan global erişim noktalarına bağımlı olmasına neden olabilir. Bu durum özellikle birim testlerde sorun çıkarabilir.

Örneğin aşağıdaki kullanım test edilebilirliği azaltabilir:

```
class SiparisServisi {
  public void siparisOlustur() {
    LogYoneticisi.INSTANCE.logYaz("Sipariş oluşturuldu.");
  }
}
```

Bu sınıf doğrudan singleton nesneye bağımlıdır. Alternatif olarak bağımlılık enjeksiyonu ile daha test edilebilir bir tasarım kurulabilir.

```
interface Logger {
  void logYaz(String mesaj);
}

class SiparisServisi {
  private final Logger logger;

  public SiparisServisi(Logger logger) {
    this.logger = logger;
  }

  public void siparisOlustur() {
    logger.logYaz("Sipariş oluşturuldu.");
  }
}
```

Bu nedenle singleton kalıbı güçlü bir araçtır; ancak dikkatli ve ölçülü kullanılmalıdır.

Özet

Tekil nesne kalıbı, bir sınıftan yalnızca bir nesne oluşturulmasını garanti eden bir tasarım kalıbıdır. Java'da bu kalıp `private constructor`, `static instance` ve `getInstance()` metodu ile uygulanabilir. Daha güvenli yaklaşımlar için statik iç sınıf veya enum tabanlı singleton tercih edilebilir. Ancak kalıbın aşırı kullanımı test edilebilirliği ve esnekliği azaltabileceği için dikkatli olunmalıdır.

§5.2.4 Kurucu

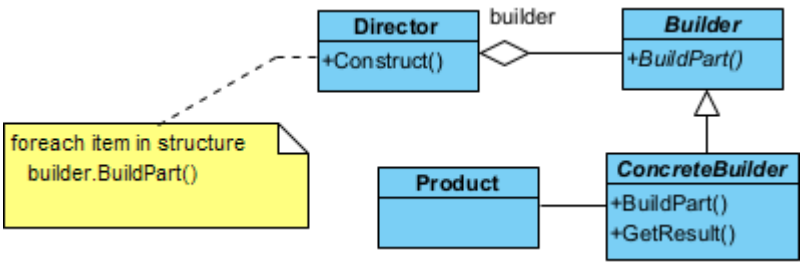
Kurucu Kalıbı (*Builder Pattern*), karmaşık nesnelerin adım adım oluşturulmasını sağlayan yaratıcı (*creational*) bir tasarım kalıbıdır. Bu kalıpta amaç, bir nesnenin oluşturulma süreci ile nesnenin kullanımı arasındaki bağı azaltmaktır.

Bazı nesneler çok sayıda alana, isteğe bağlı özelliğe veya farklı oluşturma varyasyonlarına sahip olabilir. Böyle durumlarda uzun parametre listelerine sahip yapıcı metotlar kodun okunabilirliğini azaltır. Kurucu kalıbı, bu karmaşık oluşturma sürecini ayrı bir kurucu sınıfa veya iç kurucu yapıya taşıyarak daha anlaşılır bir kullanım sağlar.

Örneğin bir fast-food restoranında menü hazırlama sürecini düşünelim. Menüde hamburger, içecek, patates, sos ve tatlı gibi farklı parçalar bulunabilir. Müşteri bazı parçaları ister, bazılarını istemez. En sonunda ise tek bir `Menu` nesnesi ortaya çıkar. Kurucu kalıbı da karmaşık nesneleri buna benzer şekilde adım adım oluşturur.

Kurucu kalıbındaki temel roller şunlardır:

- ◇ **Ürün** (`Product`): İnşa edilmek istenen karmaşık nesnedir. Çok sayıda alan veya parçadan oluşabilir.
- ◇ **Kurucu** (`Builder`): Ürünün parçalarını oluşturmak veya özelliklerini ayarlamak için kullanılan yöntemleri tanımlar.



Şekil 5.4: Kurucu Kalıbı UML Diyagramı

- **Somut Kurucu** (`ConcreteBuilder`): Ürünün belirli bir varyasyonunu adım adım oluşturan sınıftır.
- **Yönetici** (`Director`): İnşa adımlarının hangi sırayla çağrılacağını belirleyen sınıftır. Her kullanımda zorunlu değildir.
- **İstemci** (`Client`): Kurucuyu kullanarak nesneyi elde eden koddur.

Java'da kurucu kalıbı iki farklı biçimde sık kullanılır:

- Klasik GoF yaklaşımı: Ayrı `Builder`, `ConcreteBuilder` ve `Director` sınıflarıyla uygulanır.
- Modern Java yaklaşımı: Ürünün içinde `static Builder` sınıfı tanımlanır ve zincirleme metot çağrılarıyla nesne oluşturulur.

Problem: Uzun Parametreli Yapıcı Metotlar

Kurucu kalıbına neden ihtiyaç duyulduğunu anlamak için önce kötü bir örneğe bakalım. Aşağıdaki `Menu` sınıfı çok sayıda parametreyle oluşturulmaktadır:

```
class Menu {
  private String hamburger;
  private String icecek;
  private String patates;
  private String sos;
  private String tatli;

  public Menu(String hamburger,
  String icecek,
  String patates,
  String sos,
  String tatli) {
    this.hamburger = hamburger;
    this.icecek = icecek;
    this.patates = patates;
    this.sos = sos;
    this.tatli = tatli;
  }
}
```

Kod 5.22: Java Dilinde Uzun Parametreli Yapıcı Problemi

Bu sınıfı kullanırken parametrelerin sırasını karıştırmak kolaydır:

```
Menu menu = new Menu("Double Burger", "Kola", "Buyuk Patates", "Ketcap", "Dondurma");
```

Bu kullanımda hangi parametrenin ne anlama geldiği hemen anlaşılmaz. Ayrıca bazı alanlar isteğe bağlıysa, `null` değerler göndermek gerekebilir:

```
Menu menu = new Menu("Double Burger", "Kola", null, null, null);
```

Bu yapı okunabilirliği azaltır ve hataya açıktır. Kurucu kalıbı bu problemi daha okunabilir bir nesne oluşturma süreciyle çözer.

Modern Java Yaklaşımı: İç Builder Sınıfı

Java'da en yaygın kullanılan kurucu kalıbı biçimlerinden biri, ürün sınıfının içinde statik bir `Builder` sınıfı tanımlamaktır. Bu yaklaşım özellikle çok sayıda isteğe bağlı alan içeren nesnelerde oldukça kullanışlıdır.

```
class Menu {
  private final String hamburger;
  private final String icecek;
  private final String patates;
  private final String sos;
  private final String tatli;

  private Menu(Builder builder) {
    this.hamburger = builder.hamburger;
    this.icecek = builder.icecek;
    this.patates = builder.patates;
    this.sos = builder.sos;
    this.tatli = builder.tatli;
  }

  public void yazdir() {
    System.out.println("Menü İçeriği:");
    System.out.println("Hamburger: " + hamburger);
    System.out.println("İçecek   : " + icecek);
    System.out.println("Patates  : " + patates);
    System.out.println("Sos      : " + sos);
    System.out.println("Tatlı    : " + tatli);
  }

  public static class Builder {
    private String hamburger;
    private String icecek;
    private String patates;
    private String sos;
    private String tatli;

    public Builder hamburger(String hamburger) {
      this.hamburger = hamburger;
      return this;
    }

    public Builder icecek(String icecek) {
      this.icecek = icecek;
      return this;
    }

    public Builder patates(String patates) {
      this.patates = patates;
      return this;
    }

    public Builder sos(String sos) {
      this.sos = sos;
      return this;
    }

    public Builder tatli(String tatli) {
      this.tatli = tatli;
      return this;
    }

    public Menu build() {
      return new Menu(this);
    }
  }
```

```
}
```

Kod 5.23: Java Dilinde Builder Kalıbı

Bu sınıf aşağıdaki gibi kullanılır:

```
public class Main {
  public static void main(String[] args) {
    Menu menu = new Menu.Builder()
    .hamburger("Double Burger")
    .icecek("Kola")
    .patates("Büyük Patates")
    .sos("Ketçap")
    .tatli("Dondurma")
    .build();

    menu.yazdir();
  }
}
```

Kod 5.24: Java Dilinde Builder Kullanımı

Program çıktısı:

```
Menü İçeriği:
Hamburger: Double Burger
İçecek   : Kola
Patates  : Büyük Patates
Sos      : Ketçap
Tatlı    : Dondurma
```

Bu kullanımda nesnenin hangi alanlarının hangi değerlerle doldurulduğu açıkça görülür. Ayrıca isteğe bağlı alanlar kolayca atlanabilir:

```
Menu sadeMenu = new Menu.Builder()
.hamburger("Tavuk Burger")
.icecek("Ayran")
.build();
```

Bu kodda yalnızca hamburger ve içecek ayarlanmıştır. Diğer alanlar varsayılan olarak `null` kalır.

Doğrulama Eklemek

Kurucu kalıbının önemli avantajlarından biri, nesne oluşturulmadan önce doğrulama yapılabilmesidir. Örneğin bir menüde hamburger alanının zorunlu olduğunu düşünelim. Bu durumda `build()` metodu içinde kontrol yapılabilir.

```
public Menu build() {
  if (hamburger == null || hamburger.isBlank()) {
    throw new IllegalStateException("Menüde hamburger bilgisi zorunludur.");
  }

  if (icecek == null || icecek.isBlank()) {
    throw new IllegalStateException("Menüde içecek bilgisi zorunludur.");
  }

  return new Menu(this);
}
```

Kod 5.25: Java Builder İçinde Doğrulama

Bu yaklaşım sayesinde geçersiz nesne oluşturulması engellenir.

Klasik Builder ve Director Kullanımı

GoF yaklaşımında kurucu kalıbı genellikle `Builder`, `ConcreteBuilder` ve `Director` sınıflarıyla açıklanır. Bu yaklaşımda `Director`, nesnenin hangi sırayla oluşturulacağını belirler.

Aşağıdaki örnekte bir kitap nesnesinin bölümleri adım adım oluşturulmaktadır.

```
import java.util.ArrayList;
import java.util.List;

class Kitap {
  private final String kitapAdi;
  private final List<String> bolumler = new ArrayList<>();

  public Kitap(String kitapAdi) {
    this.kitapAdi = kitapAdi;
  }

  public void bolumEkle(String bolum) {
    bolumler.add(bolum);
  }

  public void goster() {
    System.out.println(kitapAdi + " kitabının bölümleri:");

    for (String bolum : bolumler) {
      System.out.println("- " + bolum);
    }
  }
}

interface KitapBuilder {
  void bolum1Olustur();
  void bolum2Olustur();
  void bolum3Olustur();
  Kitap sonucGetir();
}

class JavaKitapBuilder implements KitapBuilder {
  private final Kitap kitap = new Kitap("Java Tasarım Kalıpları");

  @Override
  public void bolum1Olustur() {
    kitap.bolumEkle("Bölüm 1: Nesne Yönelimli Programlama");
  }

  @Override
  public void bolum2Olustur() {
    kitap.bolumEkle("Bölüm 2: SOLID İlkeleri");
  }

  @Override
  public void bolum3Olustur() {
    kitap.bolumEkle("Bölüm 3: Tasarım Kalıpları");
  }

  @Override
  public Kitap sonucGetir() {
    return kitap;
  }
}

class Director {
  public void olustur(KitapBuilder builder) {
    builder.bolum1Olustur();
    builder.bolum2Olustur();
    builder.bolum3Olustur();
```

```
    }
  }

  public class Main {
    public static void main(String[] args) {
      Director director = new Director();
      KitapBuilder builder = new JavaKitapBuilder();

      director.olustur(builder);

      Kitap kitap = builder.sonucGetir();
      kitap.goster();
    }
  }
```

Kod 5.26: Java Dilinde Klasik Builder Yaklaşımı

Program çıktısı:

```
Java Tasarım Kalıpları kitabının bölümleri:
- Bölüm 1: Nesne Yönelimli Programlama
- Bölüm 2: SOLID İlkeleri
- Bölüm 3: Tasarım Kalıpları
```

Bu klasik yaklaşımda:

- `Kitap`, oluşturulan üründür.
- `KitapBuilder`, kurucu arayüzüdür.
- `JavaKitapBuilder`, somut kurucudur.
- `Director`, inşa adımlarının sırasını belirler.

Builder Kalıbının Avantajları

- Uzun parametreli yapıcı metotların oluşturduğu karmaşıklığı azaltır.
- Nesne oluşturma sürecini daha okunabilir hale getirir.
- İsteğe bağlı alanların yönetimini kolaylaştırır.
- Nesne oluşturulmadan önce doğrulama yapılmasına imkân verir.
- Değişmez (*immutable*) nesneler oluşturmaya uygundur.
- Aynı oluşturma süreciyle farklı nesne varyasyonları üretilebilir.

Builder Kalıbının Dezavantajları

- Ek sınıf veya iç sınıf gerektirdiği için kod miktarını artırabilir.
- Basit nesneler için gereksiz karmaşıklık oluşturabilir.
- Her ürün için ayrı builder yazmak gerekebilir.

Ne Zaman Kullanılmalı?

Kurucu kalıbı şu durumlarda tercih edilebilir:

- Nesne çok sayıda parametreyle oluşturuluyorsa.
- Parametrelerin çoğu isteğe bağlıysa.
- Yapıcı metot çağrıları okunamaz hale geliyorsa.
- Nesnenin oluşturulma süreci adım adım yönetilmek isteniyorsa.
- Geçersiz nesne oluşturulmasını engellemek için oluşturma aşamasında doğrulama yapılacaksa.
- Değişmez nesneler daha okunabilir biçimde oluşturulmak isteniyorsa.

Özet

Kurucu kalıbı, karmaşık nesnelerin adım adım ve okunabilir biçimde oluşturulmasını sağlar. Java'da bu kalıp çoğunlukla ürün sınıfının içinde tanımlanan `static Builder` sınıfı ile uygulanır. Bu yaklaşım uzun parametre listelerini ortadan kaldırır, isteğe bağlı alanların yönetimini kolaylaştırır

ve nesne oluşturma kodunu daha anlaşılır hale getirir.

§5.2.5 Prototip

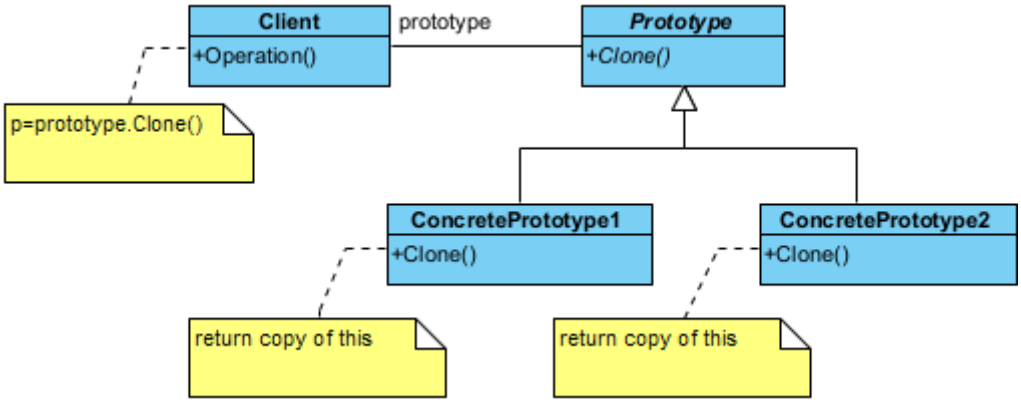
Prototip Kalıbı (*Prototype Pattern*), mevcut bir nesnenin kopyasını veya klonunu oluşturarak yeni nesneler elde etmeyi sağlayan yaratıcı (*creational*) bir tasarım kalıbıdır.

Bu kalıbın temel amacı, bir nesneyi sıfırdan oluşturmanın maliyetli olduğu durumlarda, mevcut bir nesneyi örnek almak ve onun kopyasını üretmektir. Böylece nesne oluşturma işlemi daha hızlı, daha esnek ve daha kontrollü hale getirilebilir.

Bazı nesnelerin oluşturulması karmaşık olabilir. Örneğin bir nesne oluşturulurken:

- Veritabanından veri okunması gerekebilir.
- Dosyadan yapılandırma bilgileri yüklenebilir.
- Ağ üzerinden veri alınabilir.
- Çok sayıda alanın tek tek ayarlanması gerekebilir.
- Hesaplama maliyeti yüksek başlangıç işlemleri yapılabilir.

Bu gibi durumlarda, her seferinde aynı nesneyi baştan oluşturmak yerine, önceden hazırlanmış bir prototip nesneyi klonlamak daha uygun olabilir.



Şekil 5.5: Prototip Kalıbı UML Diyagramı

Prototip kalıbındaki temel roller şunlardır:

- **Prototip** (`Prototype`): Nesnenin kendisini kopyalayabilmesi için gerekli arayüzü tanımlar. Genellikle `clone()` adlı bir yöntem içerir.
- **Somut Prototip** (`ConcretePrototype`): Prototip arayüzünü uygular ve gerçek kopyalama işlemini gerçekleştirir.
- **İstemci** (`Client`): Yeni bir nesneye ihtiyaç duyduğunda, doğrudan `new` ile nesne oluşturmak yerine mevcut prototip nesneden kopya ister.

Java'da Prototip Kalıbı

Java'da prototip kalıbı birkaç farklı şekilde uygulanabilir. Java dilinde `Object` sınıfından gelen `clone()` yöntemi bulunsa da, bu yöntemin dikkatli kullanılması gerekir. Çünkü varsayılan `clone()` davranışı çoğu durumda yüzeysel kopyalama (*shallow copy*) üretir.

Bu nedenle Java'da prototip kalıbını anlatırken çoğu zaman özel bir arayüz tanımlamak daha açık ve öğretici olur.

Önce prototip arayüzünü tanımlayalım:

```
interface Prototype {
  Prototype clone();
  void showData();
}
```

Kod 5.27: Java Dilinde Prototip Arayüzü

Bu arayüzde iki yöntem vardır:

◇ `clone()`: Nesnenin kopyasını oluşturur.
◇ `showData()`: Nesnenin verisini ekrana yazdırır.

Şimdi bu arayüzü gerçekleştiren somut prototip sınıfını yazalım:

```
class ConcretePrototype implements Prototype {
  private String data;

  public ConcretePrototype(String data) {
    this.data = data;
  }

  @Override
  public Prototype clone() {
    return new ConcretePrototype(this.data);
  }

  @Override
  public void showData() {
    System.out.println("Nesne Verisi: " + data);
  }

  public void setData(String data) {
    this.data = data;
  }
}
```

Kod 5.28: Java Dilinde Somut Prototip Sınıfı

Bu örnekte `clone()` yöntemi, mevcut nesnenin verisini kullanarak yeni bir `ConcretePrototype` nesnesi oluşturur.

İstemci kod aşağıdaki gibi yazılabilir:

```
public class Main {
  public static void main(String[] args) {
    Prototype object1 = new ConcretePrototype("40.30");
    object1.showData();

    Prototype clone1 = object1.clone();
    clone1.showData();

    Prototype object2 = new ConcretePrototype("Mehmet");
    object2.showData();

    Prototype clone2 = object2.clone();
    clone2.showData();
  }
}
```

Kod 5.29: Java Dilinde Prototip Kalıbı Kullanımı

Program çıktısı:

```
Nesne Verisi: 40.30
Nesne Verisi: 40.30
Nesne Verisi: Mehmet
Nesne Verisi: Mehmet
```

Bu örnekte istemci, yeni nesne oluşturmak için somut sınıfın ayrıntılarıyla ilgilenmez. Elindeki prototip nesneye `clone()` çağrısı yaparak yeni bir nesne elde eder.

Yüzeysel Kopyalama ve Derin Kopyalama

Nesneleri kopyalarken dikkat edilmesi gereken en önemli konu, kopyalamanın yüzeysel mi yoksa derin mi yapılacağıdır.

Yüzeysel kopyalama (*shallow copy*), nesnenin alanlarını olduğu gibi kopyalar. Eğer nesne içinde başka nesnelere referanslar varsa, bu referansların gösterdiği nesneler yeniden oluşturulmaz. Yani orijinal nesne ve kopya nesne, aynı iç nesneyi paylaşabilir.

Derin kopyalama (*deep copy*) ise nesnenin sahip olduğu iç nesneleri de ayrıca kopyalar. Böylece orijinal nesne ile kopya nesne birbirinden bağımsız hale gelir.

Bu farkı daha net görmek için aşağıdaki örneği inceleyelim.

Önce bir `Adres` sınıfı tanımlayalım:

```
class Adres {
  private String sehir;

  public Adres(String sehir) {
    this.sehir = sehir;
  }

  public String getSehir() {
    return sehir;
  }

  public void setSehir(String sehir) {
    this.sehir = sehir;
  }
}
```

Kod 5.30: Java Dilinde Adres Sınıfı

Şimdi `Kisi` sınıfında yüzeysel kopyalama yapalım:

```
class Kisi {
  private String ad;
  private Adres adres;

  public Kisi(String ad, Adres adres) {
    this.ad = ad;
    this.adres = adres;
  }

  public Kisi shallowCopy() {
    return new Kisi(this.ad, this.adres);
  }

  public void adresDegistir(String yeniSehir) {
    this.adres.setSehir(yeniSehir);
  }

  public void yazdir() {
    System.out.println(ad + " - " + adres.getSehir());
  }
}

public class Main {
  public static void main(String[] args) {
    Adres adres = new Adres("Ankara");

    Kisi kisi1 = new Kisi("Ahmet", adres);
    Kisi kisi2 = kisi1.shallowCopy();

    kisi2.adresDegistir("İstanbul");
```

```
    kisi1.yazdir();
    kisi2.yazdir();
  }
}
```

Kod 5.31: Java Dilinde Yüzeysel Kopyalama

Program çıktısı:

```
Ahmet - İstanbul
Ahmet - İstanbul
```

Burada `kisi1` ve `kisi2` farklı `Kisi` nesneleridir. Ancak ikisi de aynı `Adres` nesnesini paylaşmaktadır. Bu nedenle `kisi2` üzerinden adres değiştirildiğinde, `kisi1` de bu değişiklikten etkilenmiştir.

Bu durum çoğu zaman istenmez. Eğer kopyalanan nesnenin tamamen bağımsız olması gerekiyorsa derin kopyalama yapılmalıdır.

Java Dilinde Derin Kopyalama

Derin kopyalama için iç nesneler de ayrıca kopyalanmalıdır. Bunun için `Adres` sınıfına kopyalama yöntemi ekleyebiliriz:

```
class Adres {
  private String sehir;

  public Adres(String sehir) {
    this.sehir = sehir;
  }

  public Adres copy() {
    return new Adres(this.sehir);
  }

  public String getSehir() {
    return sehir;
  }

  public void setSehir(String sehir) {
    this.sehir = sehir;
  }
}

class Kisi {
  private String ad;
  private Adres adres;

  public Kisi(String ad, Adres adres) {
    this.ad = ad;
    this.adres = adres;
  }

  public Kisi deepCopy() {
    return new Kisi(this.ad, this.adres.copy());
  }

  public void adresDegistir(String yeniSehir) {
    this.adres.setSehir(yeniSehir);
  }

  public void yazdir() {
    System.out.println(ad + " - " + adres.getSehir());
  }
}

public class Main {
```

```
    public static void main(String[] args) {
      Adres adres = new Adres("Ankara");

      Kisi kisi1 = new Kisi("Ahmet", adres);
      Kisi kisi2 = kisi1.deepCopy();

      kisi2.adresDegistir("İstanbul");

      kisi1.yazdir();
      kisi2.yazdir();
    }
  }
```

Kod 5.32: Java Dilinde Derin Kopyalama

Program çıktısı:

```
Ahmet - Ankara
Ahmet - İstanbul
```

Bu kez `kisi1` ve `kisi2` farklı `Adres` nesnelerine sahiptir. Bu nedenle birinin adresini değiştirmek diğerini etkilemez.

Java'daki Cloneable Arayüzü

Java'da nesne kopyalama denince akla gelen yapılardan biri `Cloneable` arayüzüdür. Ancak `Cloneable` arayüzünün kullanımı bazı yönlerden sorunludur:

- ◇ `Cloneable` arayüzü herhangi bir `clone()` yöntemi tanımlamaz.
- ◇ `Object.clone()` yöntemi `protected` olarak tanımlıdır.
- ◇ Varsayılan klonlama yüzeysel kopyalama yapar.
- ◇ Derin kopyalama gerekiyorsa yine elle uygulanmalıdır.

Basit bir `Cloneable` örneği aşağıdaki gibidir:

```
class Rapor implements Cloneable {
  private String baslik;

  public Rapor(String baslik) {
    this.baslik = baslik;
  }

  @Override
  public Rapor clone() {
    try {
      return (Rapor) super.clone();
    } catch (CloneNotSupportedException e) {
      throw new AssertionError("Klonlama desteklenmiyor.", e);
    }
  }

  public void yazdir() {
    System.out.println("Rapor: " + baslik);
  }
}

public class Main {
  public static void main(String[] args) {
    Rapor rapor1 = new Rapor("Satış Raporu");
    Rapor rapor2 = rapor1.clone();

    rapor1.yazdir();
    rapor2.yazdir();
  }
}
```

Kod 5.33: Java Dilinde Cloneable Kullanımı

Bu örnek çalışır; ancak karmaşık nesnelerde `Cloneable` kullanmak yerine açık bir `copy()`, `clone()` veya kopyalama yapıcısı benzeri yöntem yazmak çoğu zaman daha anlaşılırdır.

Kopyalama Yapıcısı Yaklaşımı

Java'da C++ dilindeki gibi özel bir kopya yapıcı zorunluluğu yoktur; ancak benzer mantıkla kopyalama yapıcısı yazılabilir.

```
class Belge {
  private String baslik;
  private String icerik;

  public Belge(String baslik, String icerik) {
    this.baslik = baslik;
    this.icerik = icerik;
  }

  public Belge(Belge diger) {
    this.baslik = diger.baslik;
    this.icerik = diger.icerik;
  }

  public void yazdir() {
    System.out.println(baslik + ": " + icerik);
  }
}

public class Main {
  public static void main(String[] args) {
    Belge belge1 = new Belge("Duyuru", "Toplantı saat 10.00'da.");
    Belge belge2 = new Belge(belge1);

    belge1.yazdir();
    belge2.yazdir();
  }
}
```

Kod 5.34: Java Dilinde Kopyalama Yapıcısı

Bu yaklaşım, `Cloneable` kullanımına göre daha açık ve anlaşılır olabilir. Özellikle nesnenin hangi alanlarının nasıl kopyalandığını kontrol etmek daha kolaydır.

Prototip Deposu

Prototip kalıbı çoğu zaman bir prototip deposu veya kayıt sistemiyle birlikte kullanılır. Bu yapıda önceden hazırlanmış nesneler bir koleksiyonda saklanır. İstemci, ihtiyaç duyduğu nesnenin adını vererek bu nesnenin klonunu alır.

```
import java.util.HashMap;
import java.util.Map;

interface Sekil {
  Sekil clone();
  void ciz();
}

class Daire implements Sekil {
  private String renk;

  public Daire(String renk) {
    this.renk = renk;
  }
```

```
    @Override
    public Sekil clone() {
      return new Daire(this.renk);
    }

    @Override
    public void ciz() {
      System.out.println(renk + " renkli daire çizildi.");
    }
  }

  class Dikdortgen implements Sekil {
    private String renk;

    public Dikdortgen(String renk) {
      this.renk = renk;
    }

    @Override
    public Sekil clone() {
      return new Dikdortgen(this.renk);
    }

    @Override
    public void ciz() {
      System.out.println(renk + " renkli dikdörtgen çizildi.");
    }
  }

  class SekilDeposu {
    private final Map<String, Sekil> prototipler = new HashMap<>();

    public void prototipEkle(String anahtar, Sekil sekil) {
      prototipler.put(anahtar, sekil);
    }

    public Sekil prototiptenUret(String anahtar) {
      Sekil prototip = prototipler.get(anahtar);

      if (prototip == null) {
        throw new IllegalArgumentException("Prototip bulunamadı: " + anahtar);
      }

      return prototip.clone();
    }
  }

  public class Main {
    public static void main(String[] args) {
      SekilDeposu depo = new SekilDeposu();

      depo.prototipEkle("kirmiziDaire", new Daire("Kırmızı"));
      depo.prototipEkle("maviDikdortgen", new Dikdortgen("Mavi"));

      Sekil sekil1 = depo.prototiptenUret("kirmiziDaire");
      Sekil sekil2 = depo.prototiptenUret("maviDikdortgen");

      sekil1.ciz();
      sekil2.ciz();
    }
  }
```

Kod 5.35: Java Dilinde Prototip Deposu

Program çıktısı:

```
Kırmızı renkli daire çizildi.
```

```
Mavi renkli dikdörtgen çizildi.
```

Bu yapı sayesinde istemci kod, nesnelerin nasıl oluşturulduğunu bilmez. Yalnızca prototip deposundan kopya nesne ister.

Prototip Kalıbının Avantajları

- ◇ Nesne oluşturma maliyetini azaltabilir.
- ◇ Karmaşık nesnelerin kolayca kopyalanmasını sağlar.
- ◇ İstemci kodun somut sınıflara bağımlılığını azaltır.
- ◇ Çalışma zamanında farklı prototiplerden nesne üretmeyi kolaylaştırır.
- ◇ Önceden yapılandırılmış nesnelerden hızlıca yeni nesneler elde edilebilir.

Prototip Kalıbının Dezavantajları

- ◇ Derin kopyalama gerekiyorsa uygulama karmaşıklaşabilir.
- ◇ İçinde çok sayıda referans alanı bulunan nesnelerde klonlama hataya açık olabilir.
- ◇ `Cloneable` kullanımı Java'da bazı tasarım sorunları doğurabilir.
- ◇ Kopyalanmaması gereken kaynaklara sahip nesnelerde dikkatli olunmalıdır.

Ne Zaman Kullanılmalı?

Prototip kalıbı şu durumlarda tercih edilebilir:

- ◇ Nesne oluşturma maliyeti yüksekse.
- ◇ Nesneler benzer başlangıç değerleriyle oluşturuluyorsa.
- ◇ Nesnenin oluşturulma süreci karmaşıksa.
- ◇ Çalışma zamanında farklı nesne örneklerinden yeni nesneler üretmek gerekiyorsa.
- ◇ Somut sınıflara doğrudan bağımlılığı azaltmak isteniyorsa.

Özet

Prototip kalıbı, yeni nesneleri mevcut nesneleri klonlayarak oluşturmayı sağlayan bir tasarım kalıbıdır. Bu kalıp özellikle nesne oluşturma maliyetinin yüksek olduğu veya nesnelerin benzer başlangıç durumlarına sahip olduğu durumlarda kullanışlıdır. Java'da prototip kalıbı özel bir `clone()` yöntemi, kopyalama yapıcısı veya prototip deposu kullanılarak uygulanabilir. Kopyalama sırasında yüzeysel ve derin kopyalama farkına dikkat edilmelidir.

5.3 Yapısal Kalıplar

Yapısal kalıplar (*structural pattern*), sınıflar ve nesnelerin çok olduğu daha karmaşık programlarda getirilen yapısal çözüm yöntemleridir.

§5.3.1 Önyüz

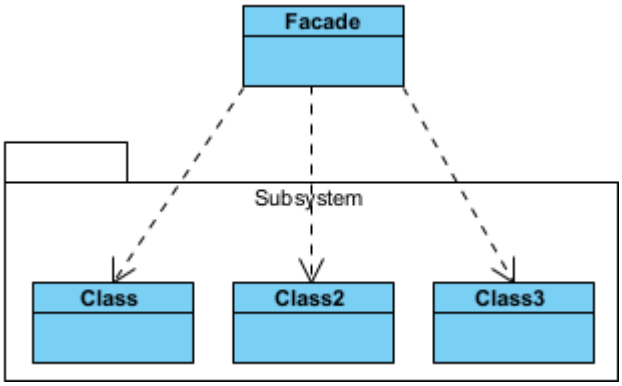
Önyüz Kalıbı (*Facade Pattern*), karmaşık bir alt sistemdeki (*subsystem*) çok sayıda sınıfı ve işlemi, kullanımı kolay tek bir üst düzey arayüz arkasında gizlemek için kullanılan yapısal (*structural*) bir tasarım kalıbıdır.

Bu kalıbın temel amacı, istemci kodun karmaşık alt sistem sınıflarıyla doğrudan uğraşmasını engellemektir. İstemci, alt sistemde hangi sınıfın hangi sırayla çağrılacağını bilmek zorunda kalmaz. Bunun yerine, önyüz sınıfının sunduğu basit metotları çağırır.

Bu kalıp bir restoranın **garsonu** gibi düşünülebilir. Müşteri yalnızca sipariş verir. Garson ise mutfak, kasa ve servis süreçlerini müşterinin yerine koordine eder. Müşteri yemeğin nasıl hazırlandığı, hangi aşamalardan geçtiği veya mutfakta kimlerin çalıştığıyla ilgilenmez.

Benzer şekilde, yazılımda da istemci kod yalnızca basit bir arayüzle çalışır; karmaşık işlemlerin ayrıntıları önyüz sınıfı tarafından yönetilir.

Önyüz kalıbındaki temel roller şunlardır:



Şekil 5.6: Önyüz Kalıbı UML Diyagramı

- **Önyüz** (`Facade`): Karmaşık alt sistemleri bilen ve onları yöneten ana giriş noktasıdır. İstemciden gelen basit isteği, alt sistemdeki doğru sınıflara yönlendirir.
- **Alt Sistem Sınıfları** (`Subsystem Classes`): İşin asıl ayrıntılarını yapan sınıflardır. Örneğin ses sistemi, ışık denetleyici, projeksiyon veya veritabanı bağlantısı gibi sınıflar alt sistem sınıfları olabilir.
- **İstemci** (`Client`): Sistemi kullanan koddur. Alt sistemin karmaşık ayrıntılarıyla uğraşmaz; yalnızca önyüz sınıfının sunduğu basit metotları çağırır.

Java ile Temel Önyüz Örneği

Aşağıdaki örnekte üç farklı alt sistem sınıfı bulunmaktadır. Her alt sistem kendi işini yapar. `Facade` sınıfı ise bu alt sistemleri uygun sırayla çalıştırarak istemciye basit bir kullanım sunar.

```
class SubSystem1 {
  public void operation11() {
    System.out.println("SubSystem1: Yöntem 1 çalıştırıldı.");
  }
}

class SubSystem2 {
  public void operation22() {
    System.out.println("SubSystem2: Yöntem 2 çalıştırıldı.");
  }
}

class SubSystem3 {
  public void operation31() {
    System.out.println("SubSystem3: Yöntem 1 çalıştırıldı.");
  }

  public void operation32() {
    System.out.println("SubSystem3: Yöntem 2 çalıştırıldı.");
  }

  public void operation33() {
    System.out.println("SubSystem3: Yöntem 3 çalıştırıldı.");
  }
}
```

Kod 5.36: Java Dilinde Alt Sistem Sınıfları

Bu sınıflar, alt sistemin farklı parçalarını temsil eder. İstemcinin bu sınıfları tek tek tanıması ve doğru sırayla çağırması istenmez. Bu nedenle bir önyüz sınıfı yazılır.

```
class Facade {
  private final SubSystem1 subSystem1;
  private final SubSystem2 subSystem2;
  private final SubSystem3 subSystem3;
```

```
    public Facade() {
      this.subSystem1 = new SubSystem1();
      this.subSystem2 = new SubSystem2();
      this.subSystem3 = new SubSystem3();
    }

    public void executeComplexOperation() {
      System.out.println("--- Facade karmaşık işlemi başlatıyor ---");

      subSystem1.operation11();
      subSystem2.operation22();
      subSystem3.operation31();
      subSystem3.operation33();
      subSystem3.operation32();

      System.out.println("--- İşlem tamamlandı ---");
    }
  }
```

Kod 5.37: Java Dilinde Önyüz Sınıfı

İstemci kod artık alt sistem sınıflarını doğrudan kullanmak zorunda değildir:

```
public class Main {
  public static void main(String[] args) {
    Facade facade = new Facade();

    facade.executeComplexOperation();
  }
}
```

Kod 5.38: Java Dilinde Önyüz Kullanımı

Program çıktısı:

```
--- Facade karmaşık işlemi başlatıyor ---
SubSystem1: Yöntem 1 çalıştırıldı.
SubSystem2: Yöntem 2 çalıştırıldı.
SubSystem3: Yöntem 1 çalıştırıldı.
SubSystem3: Yöntem 3 çalıştırıldı.
SubSystem3: Yöntem 2 çalıştırıldı.
--- İşlem tamamlandı ---
```

Bu örnekte istemci yalnızca `executeComplexOperation()` metodunu çağırır. Alt sistemlerin hangi sırayla çalışacağına `Facade` sınıfı karar verir.

Gerçek Dünya Örneği: Bilgisayar Başlatma

Önyüz kalıbını daha gerçekçi bir örnekle açıklamak için bilgisayar başlatma sürecini düşünelim. Bir bilgisayar açılırken işlemci durdurulur, diskten veri okunur, bellek yüklenir, işlemci belirli bir adrese yönlendirilir ve komutlar çalıştırılır.

Kullanıcı bu ayrıntıların hiçbirini bilmez. Kullanıcı yalnızca güç düğmesine basar. Geri kalan karmaşık süreç sistem tarafından yürütülür.

Bu örnekte:

- `CPU`, işlemci alt sistemidir.
- `Memory`, bellek alt sistemidir.
- `HardDrive`, disk alt sistemidir.
- `ComputerFacade`, bu alt sistemleri koordine eden önyüz sınıfıdır.

```
class CPU {
  public void freeze() {
    System.out.println("CPU: Durduruldu.");
```

```
  }

  public void jump(long position) {
    System.out.println("CPU: Adrese atlandı: 0x"
    + Long.toHexString(position).toUpperCase());
  }

  public void execute() {
    System.out.println("CPU: Komutlar çalıştırılıyor...");
  }
}

class Memory {
  public void load(long position, String data) {
    System.out.println("Memory: 0x"
    + Long.toHexString(position)
    + " adresine veri yüklendi: [" + data + "]");
  }
}

class HardDrive {
  public String read(long lba, int size) {
    System.out.println("HardDrive: Sector " + lba
    + " üzerinden " + size + " byte veri okundu.");

    return "OS_KERNEL_DATA";
  }
}
```

Kod 5.39: Java Dilinde Bilgisayar Alt Sistemleri

Şimdi bu alt sistemleri yöneten önyüz sınıfı yazılır:

```
class ComputerFacade {
  private static final long BOOT_ADDRESS = 0x7C00;
  private static final int BOOT_SECTOR = 0;
  private static final int SECTOR_SIZE = 512;

  private final CPU cpu;
  private final Memory memory;
  private final HardDrive hardDrive;

  public ComputerFacade() {
    this.cpu = new CPU();
    this.memory = new Memory();
    this.hardDrive = new HardDrive();
  }

  public void start() {
    System.out.println("Bilgisayar başlatılıyor...");
    System.out.println("-----------------------------------");

    cpu.freeze();

    String data = hardDrive.read(BOOT_SECTOR, SECTOR_SIZE);

    memory.load(BOOT_ADDRESS, data);

    cpu.jump(BOOT_ADDRESS);
    cpu.execute();

    System.out.println("-----------------------------------");
    System.out.println("Sistem hazır!");
  }
}
```

Kod 5.40: Java Dilinde ComputerFacade Sınıfı

İstemci kod yalnızca `start()` metodunu çağırır:

```
public class Main {
  public static void main(String[] args) {
    ComputerFacade computer = new ComputerFacade();

    computer.start();
  }
}
```

Kod 5.41: Java Dilinde Bilgisayar Başlatma Örneği

Program çıktısı:

```
Bilgisayar başlatılıyor...
----------------------------------
CPU: Durduruldu.
HardDrive: Sector 0 üzerinden 512 byte veri okundu.
Memory: 0x7c00 adresine veri yüklendi: [OS_KERNEL_DATA]
CPU: Adrese atlandı: 0x7C00
CPU: Komutlar çalıştırılıyor...
----------------------------------
Sistem hazır!
```

Bu örnekte istemci sınıf, `CPU`, `Memory` ve `HardDrive` sınıflarının hangi sırayla çalıştırılması gerektiğini bilmez. Tüm karmaşıklık `ComputerFacade` sınıfı tarafından yönetilir.

Tam Java Örneği

Aşağıda bilgisayar başlatma örneğinin tek dosyada çalıştırılabilecek tam hali verilmiştir:

```
class CPU {
  public void freeze() {
    System.out.println("CPU: Durduruldu.");
  }

  public void jump(long position) {
    System.out.println("CPU: Adrese atlandı: 0x"
    + Long.toHexString(position).toUpperCase());
  }

  public void execute() {
    System.out.println("CPU: Komutlar çalıştırılıyor...");
  }
}

class Memory {
  public void load(long position, String data) {
    System.out.println("Memory: 0x"
    + Long.toHexString(position)
    + " adresine veri yüklendi: [" + data + "]");
  }
}

class HardDrive {
  public String read(long lba, int size) {
    System.out.println("HardDrive: Sector " + lba
    + " üzerinden " + size + " byte veri okundu.");

    return "OS_KERNEL_DATA";
  }
}

class ComputerFacade {
  private static final long BOOT_ADDRESS = 0x7C00;
  private static final int BOOT_SECTOR = 0;
  private static final int SECTOR_SIZE = 512;
```

```
    private final CPU cpu;
    private final Memory memory;
    private final HardDrive hardDrive;

    public ComputerFacade() {
      this.cpu = new CPU();
      this.memory = new Memory();
      this.hardDrive = new HardDrive();
    }

    public void start() {
      System.out.println("Bilgisayar başlatılıyor...");
      System.out.println("-----------------------------------");

      cpu.freeze();

      String data = hardDrive.read(BOOT_SECTOR, SECTOR_SIZE);

      memory.load(BOOT_ADDRESS, data);

      cpu.jump(BOOT_ADDRESS);
      cpu.execute();

      System.out.println("-----------------------------------");
      System.out.println("Sistem hazır!");
    }
  }

  public class Main {
    public static void main(String[] args) {
      ComputerFacade computer = new ComputerFacade();

      computer.start();
    }
  }
```

Kod 5.42: Java Dilinde Önyüz Kalıbı Gerçek Dünya Tam Örnek

Önyüz Kalıbının Sağladığı Avantaj

Önyüz kalıbı sayesinde istemci kod karmaşık alt sistemlerden bağımsız hale gelir. Örneğin bilgisayar başlatma örneğinde istemcinin aşağıdaki gibi bir kod yazmasına gerek kalmaz:

```
CPU cpu = new CPU();
Memory memory = new Memory();
HardDrive hardDrive = new HardDrive();

cpu.freeze();
String data = hardDrive.read(0, 512);
memory.load(0x7C00, data);
cpu.jump(0x7C00);
cpu.execute();
```

Bunun yerine istemci yalnızca şu kodu yazar:

```
ComputerFacade computer = new ComputerFacade();
computer.start();
```

Bu yapı kodun okunabilirliğini artırır ve istemcinin bilmesi gereken ayrıntı miktarını azaltır.

Önyüz Kalıbı ve Kapsülleme

Önyüz kalıbı, alt sistemlerin tamamen gizlenmesi gerektiği anlamına gelmez. İstemci isterse bazı durumlarda alt sistem sınıflarını doğrudan da kullanabilir. Ancak yaygın ve karmaşık işlemler için basit bir giriş noktası sağlanır.

Bu nedenle önyüz kalıbı, alt sistemi ortadan kaldırmaz; yalnızca alt sistemin kullanımını kolaylaştırır.

Önyüz Kalıbının Avantajları

- ◇ Karmaşık alt sistemlere basit bir arayüz sağlar.
- ◇ İstemci kodun alt sistem ayrıntılarına bağımlılığını azaltır.
- ◇ Kodun okunabilirliğini artırır.
- ◇ Alt sistem sınıflarının kullanım sırasını merkezi hale getirir.
- ◇ Büyük sistemlerde giriş noktası sağlar.
- ◇ Katmanlı mimarilerde üst katmanların alt katmanlarla daha düzenli iletişim kurmasına yardımcı olur.

Önyüz Kalıbının Dezavantajları

- ◇ Önyüz sınıfı zamanla çok fazla sorumluluk alırsa "Tanrı Sınıf" haline gelebilir.
- ◇ Yanlış tasarlanırsa alt sistemdeki esnekliği kısıtlayabilir.
- ◇ Tüm işlemler tek bir sınıfa yığılırsa bakım zorlaşabilir.

Ne Zaman Kullanılmalı?

Önyüz kalıbı şu durumlarda tercih edilebilir:

- ◇ Alt sistem çok sayıda sınıftan oluşuyorsa.
- ◇ İstemci kodun alt sistem ayrıntılarını bilmesi istenmiyorsa.
- ◇ Karmaşık bir işlemin belirli bir sırayla çalıştırılması gerekiyorsa.
- ◇ Sisteme basit bir giriş noktası sağlanmak isteniyorsa.
- ◇ Katmanlar arasındaki bağımlılık azaltılmak isteniyorsa.

Özet

Önyüz kalıbı, karmaşık alt sistemleri basit bir arayüz arkasında toplayan yapısal bir tasarım kalıbıdır. İstemci kod alt sistemin ayrıntılarını bilmeden işlem yapabilir. Böylece kod daha okunabilir, daha düzenli ve daha kolay kullanılabilir hale gelir. Ancak önyüz sınıfının gereğinden fazla sorumluluk alarak büyümemesine dikkat edilmelidir.

§5.3.2 Adaptör

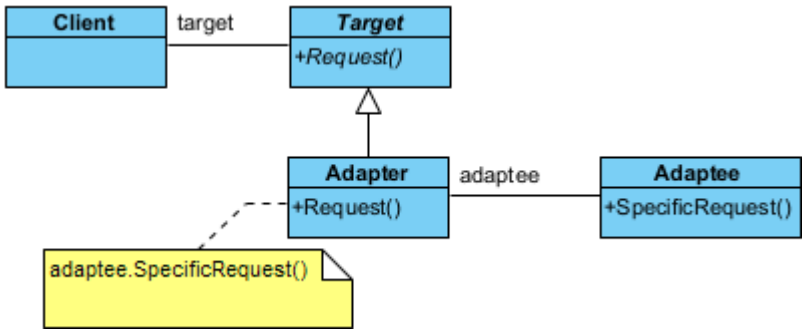
Adaptör Kalıbı (*Adapter Pattern*), uyumsuz bir arayüze sahip olan sınıfı, istemcinin beklediği arayüze dönüştürmek için kullanılan yapısal (*structural*) bir tasarım kalıbıdır.

Bu kalıbın temel amacı, doğrudan birlikte çalışamayan iki yapının bir adaptör sınıfı aracılığıyla birlikte çalışmasını sağlamaktır. Adaptör kalıbı, gerçek hayattaki elektrik prizi dönüştürücülerine benzetilebilir. Cihazın fişi ile priz birbirine uymaz; ancak araya konulan adaptör sayesinde cihaz kullanılabilir hale gelir.

Yazılımda da benzer bir durum vardır. Mevcut bir sınıfın sunduğu metotlar, istemcinin beklediği arayüzle uyumlu olmayabilir. Bu durumda mevcut sınıfı değiştirmek yerine, araya bir adaptör sınıfı eklenir.

Adaptör kalıbındaki temel roller şunlardır:

- ◇ **İstemci** (`Client`): Belirli bir arayüzü kullanarak çalışan ana koddur. İstemci, uyumsuz sınıfın ayrıntılarını bilmez.
- ◇ **Hedef** (`Target`): İstemcinin beklediği arayüzdür. İstemci bu arayüz üzerinden işlem yapar.
- ◇ **Uyumsuz Nesne** (`Adaptee`): Sisteme dahil edilmek istenen, fakat arayüzü istemcinin beklediği biçimde olmayan sınıftır.
- ◇ **Adaptör** (`Adapter`): Uyumsuz nesneyi sarmalayarak hedef arayüze uygun hale getiren sınıftır.



Şekil 5.7: Adaptör Kalıbı UML Diyagramı

Java ile Temel Adaptör Örneği

Aşağıdaki örnekte istemci, `Target` arayüzündeki `request()` metodunu kullanmak istemektedir. Ancak elimizdeki mevcut sınıf olan `Adaptee`, bunun yerine `specificRequest()` metoduna sahiptir. Bu iki yapı doğrudan birlikte çalışamaz.

Önce uyumsuz sınıfı tanımlayalım:

```
class Adaptee {
  public void specificRequest() {
    System.out.println("Adaptee: Özel istek specificRequest() çalıştırıldı.");
  }
}
```

Kod 5.43: Java Dilinde Uyumsuz Sınıf

Şimdi istemcinin beklediği hedef arayüzü tanımlayalım:

```
interface Target {
  void request();
}
```

Kod 5.44: Java Dilinde Hedef Arayüz

Adaptör sınıfı, `Target` arayüzünü gerçekleştirir. İçeride ise `Adaptee` nesnesini kullanır:

```
class Adapter implements Target {
  private final Adaptee adaptee;

  public Adapter(Adaptee adaptee) {
    this.adaptee = adaptee;
  }

  @Override
  public void request() {
    System.out.println("Adapter: Standart istek alındı, Adaptee'ye yönlendiriliyor...
        ");
    adaptee.specificRequest();
  }
}
```

Kod 5.45: Java Dilinde Adaptör Sınıfı

İstemci kod ise yalnızca `Target` arayüzünü bilir:

```
public class Main {
  public static void main(String[] args) {
    Adaptee adaptee = new Adaptee();

    Target target = new Adapter(adaptee);

    target.request();
  }
```

```
}
```

Kod 5.46: Java Dilinde Adaptör Kullanımı

Program çıktısı:

```
Adapter: Standart istek alındı, Adaptee'ye yönlendiriliyor...
Adaptee: Özel istek specificRequest() çalıştırıldı.
```

Bu örnekte istemci, `Adaptee` sınıfının `specificRequest()` metodunu bilmez. İstemci yalnızca `Target` arayüzündeki `request()` metodunu çağırır. Adaptör ise bu çağrıyı uyumsuz sınıfın anlayacağı biçime dönüştürür.

Gerçek Dünya Örneği: Eski Log Sistemini Yeni Sisteme Uyarlama

Adaptör kalıbının en yaygın kullanım alanlarından biri, eski veya dışarıdan gelen bir sınıfı mevcut sistemin beklediği arayüze uyarlamaktır.

Aşağıdaki örnekte eski bir kayıt sistemi bulunmaktadır. Bu eski sistem `logMessage()` metodunu kullanmaktadır. Ancak yeni uygulama `Logger` arayüzündeki `log()` metodunu beklemektedir.

Önce eski kayıt sınıfını yazalım:

```
class OldLogger {
  public void logMessage(String message) {
    System.out.println("[Eski Kayıt Sistemi]: " + message);
  }
}
```

Kod 5.47: Java Dilinde Eski Log Sınıfı

Yeni sistemin beklediği standart arayüz:

```
interface Logger {
  void log(String message);
}
```

Kod 5.48: Java Dilinde Yeni Logger Arayüzü

Şimdi eski sistemi yeni arayüze uyarlayan adaptör sınıfını yazalım:

```
class LoggerAdapter implements Logger {
  private final OldLogger oldLogger;

  public LoggerAdapter(OldLogger oldLogger) {
    this.oldLogger = oldLogger;
  }

  @Override
  public void log(String message) {
    oldLogger.logMessage(message);
  }
}
```

Kod 5.49: Java Dilinde Logger Adaptörü

İstemci kod yalnızca `Logger` arayüzünü kullanır:

```
class Application {
  public void appLog(Logger logger) {
    logger.log("Sistem başarıyla çalıştı.");
  }
}

public class Main {
  public static void main(String[] args) {
    OldLogger oldLogger = new OldLogger();
```

```
    Logger modernLogger = new LoggerAdapter(oldLogger);

    Application app = new Application();

    System.out.println("Modern uygulama log gönderiyor...");
    app.appLog(modernLogger);
  }
}
```

Kod 5.50: Java Dilinde Logger Adaptörü Kullanımı

Program çıktısı:

```
Modern uygulama log gönderiyor...
[Eski Kayıt Sistemi]: Sistem başarıyla çalıştı.
```

Bu örnekte `Application` sınıfı, eski log sisteminden haberdar değildir. Yalnızca `Logger` arayüzünü bilir. Eski sistemin nasıl çalıştığı adaptör sınıfı tarafından gizlenmiştir.

Tam Java Örneği

Aşağıda log sistemi örneğinin tek dosyada çalıştırılabilecek tam hali verilmiştir:

```
class OldLogger {
  public void logMessage(String message) {
    System.out.println("[Eski Kayıt Sistemi]: " + message);
  }
}

interface Logger {
  void log(String message);
}

class LoggerAdapter implements Logger {
  private final OldLogger oldLogger;

  public LoggerAdapter(OldLogger oldLogger) {
    this.oldLogger = oldLogger;
  }

  @Override
  public void log(String message) {
    oldLogger.logMessage(message);
  }
}

class Application {
  public void appLog(Logger logger) {
    logger.log("Sistem başarıyla çalıştı.");
  }
}

public class Main {
  public static void main(String[] args) {
    OldLogger oldLogger = new OldLogger();

    Logger modernLogger = new LoggerAdapter(oldLogger);

    Application app = new Application();

    System.out.println("Modern uygulama log gönderiyor...");
    app.appLog(modernLogger);
  }
}
```

Kod 5.51: Java Dilinde Gerçek Dünya Adaptör Örneği

Sınıf Adaptörü ve Nesne Adaptörü

Adaptör kalıbı genel olarak iki farklı şekilde uygulanabilir:

- **Sınıf Adaptörü** (*Class Adapter*): Kalıtım kullanılarak gerçekleştirilir. Adaptör sınıfı, uyumsuz sınıftan kalıtım alır ve hedef arayüzü uygular.
- **Nesne Adaptörü** (*Object Adapter*): Bileşim kullanılarak gerçekleştirilir. Adaptör sınıfı, uyumsuz sınıfın bir nesnesini içinde tutar.

Java'da çoklu sınıf kalıtımı olmadığı için nesne adaptörü daha sık kullanılır. Yani adaptör sınıfı, uyumsuz sınıfı miras almak yerine içinde bir alan olarak tutar.

Örneğin:

```
class LoggerAdapter implements Logger {
  private final OldLogger oldLogger;

  public LoggerAdapter(OldLogger oldLogger) {
    this.oldLogger = oldLogger;
  }

  @Override
  public void log(String message) {
    oldLogger.logMessage(message);
  }
}
```

Bu yapı bileşim kullandığı için daha esnektir. Adaptör, farklı `OldLogger` nesneleriyle çalışabilir.

Adaptör Kalıbının Avantajları

- Uyumsuz arayüzlere sahip sınıfların birlikte çalışmasını sağlar.
- Mevcut sınıfları değiştirmeden yeni sisteme uyarlama imkânı verir.
- Eski kodların yeni arayüzlerle kullanılmasını kolaylaştırır.
- İstemci kodun uyumsuz sınıfın ayrıntılarını bilmesini engeller.
- Bileşim kullanıldığında esnek ve genişletilebilir bir yapı sunar.

Adaptör Kalıbının Dezavantajları

- Sisteme ek bir sınıf katmanı ekler.
- Çok fazla adaptör kullanılması mimariyi karmaşıklaştırabilir.
- Yanlış kullanılırsa, asıl tasarım problemlerini gizleyen geçici bir çözüm haline gelebilir.

Ne Zaman Kullanılmalı?

Adaptör kalıbı şu durumlarda tercih edilebilir:

- Mevcut bir sınıfın arayüzü, istemcinin beklediği arayüzle uyumlu değilse.
- Eski bir sınıfı yeni bir sistemde kullanmak gerekiyorsa.
- Dış kütüphane veya üçüncü taraf servis mevcut arayüze uymuyorsa.
- Mevcut kodu değiştirmeden uyumluluk sağlanmak isteniyorsa.
- Farklı sistemler arasında arayüz dönüşümü yapılması gerekiyorsa.

Özet

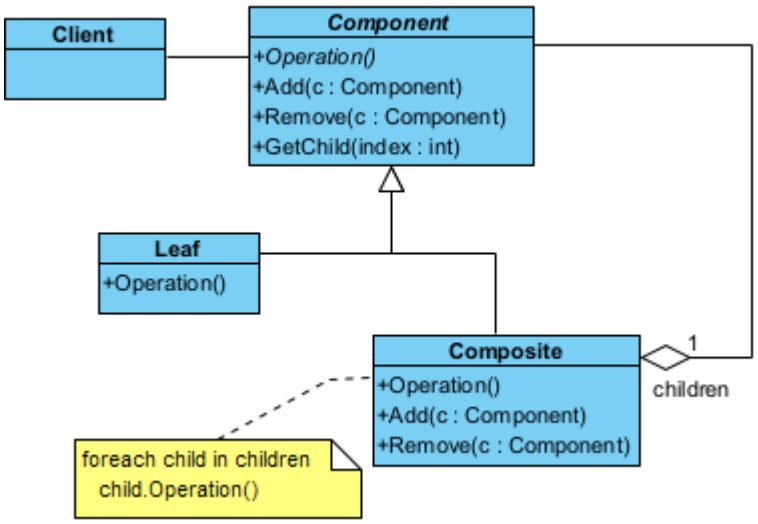
Adaptör kalıbı, uyumsuz arayüzlere sahip sınıfları birlikte çalışabilir hale getiren yapısal bir tasarım kalıbıdır. İstemci kodun beklediği arayüz ile mevcut sınıfın sunduğu arayüz farklı olduğunda, adaptör sınıfı bu iki yapı arasında çeviri görevi görür. Java'da adaptör kalıbı genellikle bileşim kullanılarak uygulanır.

§5.3.3 Bileşik

Bileşik Kalıbı (*Composite Pattern*), nesneleri parça-bütün ilişkisi içinde ağaç yapısına benzer şekilde düzenlemek için kullanılan yapısal (*structural*) bir tasarım kalıbıdır.

Bu kalıp, tekil nesneler ile nesne gruplarının aynı arayüz üzerinden kullanılmasını sağlar. Böylece istemci kod, karşısındaki nesnenin tek bir nesne mi yoksa içinde başka nesneler barındıran bileşik bir yapı mı olduğunu bilmek zorunda kalmaz.

Bileşik kalıbı özellikle özyinelemeli (*recursive*) veya hiyerarşik yapılarda kullanılır. Dosya sistemleri, şirket organizasyon şemaları, grafik arayüz bileşenleri, menü yapıları ve kategori ağaçları bu kalıba örnek olarak verilebilir.



Şekil 5.8: Bileşik Kalıbı UML Diyagramı

Bu kalıbın temel düşüncesi şudur:

- Bir dosya tek başına bir öğedir.
- Bir klasör ise içinde dosyalar veya başka klasörler barındırabilir.
- Buna rağmen hem dosyaya hem klasöre aynı türden bir bileşen gibi davranılabilir.

Bileşik kalıbındaki temel roller şunlardır:

- **Bileşen** (`Component`): Hem yaprak hem de bileşik nesneler için ortak arayüzü tanımlar.
- **Yaprak** (`Leaf`): İçinde başka nesne barındırmayan en küçük birimdir. Dosya örneğinde `File` sınıfı yaprak nesnedir.
- **Bileşik** (`Composite`): İçinde başka bileşenler barındırabilen sınıftır. Dosya sistemi örneğinde `Directory` sınıfı bileşik nesnedir.
- **İstemci** (`Client`): Nesneleri ortak bileşen arayüzü üzerinden kullanır.

Java ile Temel Bileşik Örneği

Aşağıdaki örnekte `Component` arayüzü, hem tekil yaprak nesneler hem de içinde başka bileşenler barındıran bileşik nesneler için ortak davranışı tanımlar.

Önce ortak bileşen arayüzü tanımlanır:

```
interface Component {
  String displayOperation();
}
```

Kod 5.52: Java Dilinde Bileşen Arayüzü

Bu arayüz, hem yaprak hem de bileşik sınıflar tarafından uygulanacaktır.

Şimdi yaprak sınıfı yazalım:

```
class Leaf implements Component {
  @Override
  public String displayOperation() {
    return "Yaprak";
  }
```

```
}
```

Kod 5.53: Java Dilinde Yaprak Sınıfı

`Leaf` sınıfı, içinde başka bileşen barındırmayan tekil nesnedir.

Şimdi bileşik sınıfı yazalım:

```
import java.util.ArrayList;
import java.util.List;

class Composite implements Component {
  private final List<Component> children = new ArrayList<>();

  public void addChild(Component component) {
    children.add(component);
  }

  public void removeChild(Component component) {
    children.remove(component);
  }

  @Override
  public String displayOperation() {
    StringBuilder result = new StringBuilder();

    for (int i = 0; i < children.size(); i++) {
      if (i > 0) {
        result.append("+");
      }

      result.append(children.get(i).displayOperation());
    }

    return "Dal(" + result + ")";
  }
}
```

Kod 5.54: Java Dilinde Bileşik Sınıf

`Composite` sınıfı, içinde `Component` türünden nesneler tutar. Bu nesneler yaprak olabileceği gibi başka bileşik nesneler de olabilir. Bu nedenle yapı özyinelemeli hale gelir.

İstemci kod aşağıdaki gibi yazılabilir:

```
public class Main {
  public static void main(String[] args) {
    Component yaprak = new Leaf();

    System.out.println("Sadece yapraktan oluşan bileşik nesne: "
    + yaprak.displayOperation());

    Composite kok = new Composite();

    Composite dal1 = new Composite();
    dal1.addChild(new Leaf());
    dal1.addChild(new Leaf());

    kok.addChild(dal1);

    System.out.println("Yapraktan ve dallardan oluşan bileşik nesne: "
    + kok.displayOperation());

    Composite dal2 = new Composite();
    dal2.addChild(new Leaf());

    kok.addChild(dal2);
```

```
    System.out.println("Bir dal daha eklenmiş bileşik nesne: "
    + kok.displayOperation());
  }
}
```

Kod 5.55: Java Dilinde Bileşik Kalıbı Kullanımı

Program çıktısı:

```
Sadece yapraktan oluşan bileşik nesne: Yaprak
Yapraktan ve dallardan oluşan bileşik nesne: Dal(Dal(Yaprak+Yaprak))
Bir dal daha eklenmiş bileşik nesne: Dal(Dal(Yaprak+Yaprak)+Dal(Yaprak))
```

Bu örnekte istemci kod, yaprak ve bileşik nesneleri aynı `Component` arayüzü üzerinden kullanır. Bu sayede tekil nesne ile nesne grubu arasında kullanım açısından fark azalır.

Tam Java Örneği

Aşağıda temel bileşik kalıbı örneğinin tek dosyada çalıştırılabilecek tam hali verilmiştir:

```
import java.util.ArrayList;
import java.util.List;

interface Component {
  String displayOperation();
}

class Leaf implements Component {
  @Override
  public String displayOperation() {
    return "Yaprak";
  }
}

class Composite implements Component {
  private final List<Component> children = new ArrayList<>();

  public void addChild(Component component) {
    children.add(component);
  }

  public void removeChild(Component component) {
    children.remove(component);
  }

  @Override
  public String displayOperation() {
    StringBuilder result = new StringBuilder();

    for (int i = 0; i < children.size(); i++) {
      if (i > 0) {
        result.append("+");
      }

      result.append(children.get(i).displayOperation());
    }

    return "Dal(" + result + ")";
  }
}

public class Main {
  public static void main(String[] args) {
    Component yaprak = new Leaf();

    System.out.println("Sadece yapraktan oluşan bileşik nesne: "
```

```
      + yaprak.displayOperation());

      Composite kok = new Composite();

      Composite dal1 = new Composite();
      dal1.addChild(new Leaf());
      dal1.addChild(new Leaf());

      kok.addChild(dal1);

      System.out.println("Yapraktan ve dallardan oluşan bileşik nesne: "
      + kok.displayOperation());

      Composite dal2 = new Composite();
      dal2.addChild(new Leaf());

      kok.addChild(dal2);

      System.out.println("Bir dal daha eklenmiş bileşik nesne: "
      + kok.displayOperation());
    }
  }
```

Kod 5.56: Java Dilinde Bileşik Kalıbı Tam Örnek

Gerçek Dünya Örneği: Dosya Sistemi

Bileşik kalıbının en anlaşılır örneklerinden biri dosya sistemidir. Bir dosya sistemi içinde dosyalar ve klasörler bulunur. Dosya tekil bir nesnedir. Klasör ise içinde dosyalar veya başka klasörler barındırabilir.

Bu örnekte:

- ◇ `FileSystemNode`, ortak bileşen arayüzüdür.
- ◇ `File`, yaprak sınıftır.
- ◇ `Directory`, bileşik sınıftır.

Önce ortak arayüz tanımlanır:

```
interface FileSystemNode {
  void showDetails(int indent);
  long getSize();
}
```

Kod 5.57: Java Dilinde Dosya Sistemi Bileşeni

Bu arayüz hem dosyalar hem klasörler tarafından uygulanacaktır.

Şimdi dosya sınıfını yazalım:

```
class File implements FileSystemNode {
  private final String name;
  private final long size;

  public File(String name, long size) {
    this.name = name;
    this.size = size;
  }

  @Override
  public void showDetails(int indent) {
    String spacing = " ".repeat(indent);

    System.out.println(spacing + "- Dosya: " + name
    + " (" + size + " KB)");
  }
```

```
    @Override
    public long getSize() {
      return size;
    }
  }
```

Kod 5.58: Java Dilinde Dosya Sınıfı

`File` sınıfı yaprak sınıftır. İçinde başka dosya veya klasör barındırmaz.

Şimdi klasör sınıfı yazılır:

```
import java.util.ArrayList;
import java.util.List;

class Directory implements FileSystemNode {
  private final String name;
  private final List<FileSystemNode> children = new ArrayList<>();

  public Directory(String name) {
    this.name = name;
  }

  public void add(FileSystemNode node) {
    children.add(node);
  }

  public void remove(FileSystemNode node) {
    children.remove(node);
  }

  @Override
  public void showDetails(int indent) {
    String spacing = " ".repeat(indent);

    System.out.println(spacing + "+ Klasör: " + name
    + " [Toplam: " + getSize() + " KB]");

    for (FileSystemNode child : children) {
      child.showDetails(indent + 4);
    }
  }

  @Override
  public long getSize() {
    long total = 0;

    for (FileSystemNode child : children) {
      total += child.getSize();
    }

    return total;
  }
}
```

Kod 5.59: Java Dilinde Klasör Sınıfı

`Directory` sınıfı, içinde `FileSystemNode` türünden çocuk nesneler tutar. Bu çocuklar dosya da olabilir, klasör de olabilir. Bu nedenle yapı iç içe geçebilir.

İstemci kod aşağıdaki gibidir:

```
public class Main {
  public static void main(String[] args) {
    Directory root = new Directory("C_Surucusu");
    Directory docs = new Directory("Belgelerim");
```

```
    docs.add(new File("Notlar.txt", 10));

    root.add(docs);
    root.add(new File("Sistem.log", 500));

    root.showDetails(0);
  }
}
```

Kod 5.60: Java Dilinde Dosya Sistemi Örneği

Program çıktısı:

```
+ Klasör: C_Surucusu [Toplam: 510 KB]
+ Klasör: Belgelerim [Toplam: 10 KB]
- Dosya: Notlar.txt (10 KB)
- Dosya: Sistem.log (500 KB)
```

Bu örnekte toplam boyut hesaplama işlemi özyinelemeli olarak yapılır. `root.getSize()` çağrıldığında, kök klasör içindeki tüm dosyaların ve alt klasörlerin boyutları hesaplanır.

Tam Java Örneği

Aşağıda dosya sistemi örneğinin tek dosyada çalıştırılabilecek tam hali verilmiştir:

```
import java.util.ArrayList;
import java.util.List;

interface FileSystemNode {
  void showDetails(int indent);
  long getSize();
}

class File implements FileSystemNode {
  private final String name;
  private final long size;

  public File(String name, long size) {
    this.name = name;
    this.size = size;
  }

  @Override
  public void showDetails(int indent) {
    String spacing = " ".repeat(indent);

    System.out.println(spacing + "- Dosya: " + name
    + " (" + size + " KB)");
  }

  @Override
  public long getSize() {
    return size;
  }
}

class Directory implements FileSystemNode {
  private final String name;
  private final List<FileSystemNode> children = new ArrayList<>();

  public Directory(String name) {
    this.name = name;
  }

  public void add(FileSystemNode node) {
    children.add(node);
  }
```

```
    public void remove(FileSystemNode node) {
      children.remove(node);
    }

    @Override
    public void showDetails(int indent) {
      String spacing = " ".repeat(indent);

      System.out.println(spacing + "+ Klasör: " + name
      + " [Toplam: " + getSize() + " KB]");

      for (FileSystemNode child : children) {
        child.showDetails(indent + 4);
      }
    }

    @Override
    public long getSize() {
      long total = 0;

      for (FileSystemNode child : children) {
        total += child.getSize();
      }

      return total;
    }
  }

  public class Main {
    public static void main(String[] args) {
      Directory root = new Directory("C_Surucusu");
      Directory docs = new Directory("Belgelerim");

      docs.add(new File("Notlar.txt", 10));

      root.add(docs);
      root.add(new File("Sistem.log", 500));

      root.showDetails(0);
    }
  }
```

Kod 5.61: Java Dilinde Dosya Sistemi Bileşik Kalıbı Tam Örnek

Şeffaf ve Güvenli Bileşik Tasarım

Bileşik kalıbı uygulanırken iki farklı yaklaşım tercih edilebilir:

- **Şeffaf Yaklaşım** (*transparent approach*): `add()`, `remove()` gibi çocuk yönetim metotları `Component` arayüzünde tanımlanır. Böylece istemci tüm nesnelere aynı şekilde davranabilir. Ancak yaprak nesneler için bu metotlar anlamsızdır.
- **Güvenli Yaklaşım** (*safe approach*): `add()`, `remove()` gibi metotlar yalnızca `Composite` sınıfında bulunur. Böylece yaprak nesnelerde anlamsız metotlar yer almaz.

Dosya sistemi örneğinde güvenli yaklaşım kullanılmıştır. Çünkü `FileSystemNode` arayüzünde `add()` veya `remove()` metotları yoktur. Bu metotlar yalnızca `Directory` sınıfında yer alır.

Bileşik Kalıbının Avantajları

- Ağaç yapısındaki nesneleri modellemeyi kolaylaştırır.
- Tekil nesneler ile nesne gruplarına aynı arayüz üzerinden davranılmasını sağlar.
- Özyinelemeli işlemleri doğal hale getirir.
- Yeni yaprak veya bileşik türleri eklemeyi kolaylaştırır.
- İstemci kodun hiyerarşik yapının ayrıntılarını bilme ihtiyacını azaltır.

Bileşik Kalıbının Dezavantajları

- ◇ Çok genel bir bileşen arayüzü tasarlanırsa bazı sınıflar için anlamsız metotlar oluşabilir.
- ◇ Büyük ağaç yapılarında performans ve bellek kullanımı dikkatli yönetilmelidir.
- ◇ Hiyerarşik yapı çok karmaşıklaşırsa hata ayıklama zorlaşabilir.

Ne Zaman Kullanılmalı?

Bileşik kalıbı şu durumlarda tercih edilebilir:

- ◇ Nesneler parça-bütün ilişkisi içinde modelleniyorsa.
- ◇ Ağaç benzeri hiyerarşik bir yapı gerekiyorsa.
- ◇ Tekil nesne ile nesne gruplarının aynı şekilde kullanılmasını istiyorsak.
- ◇ Özyinelemeli işlemler yapılacaksa.
- ◇ Dosya sistemi, organizasyon şeması, kategori ağacı veya menü yapısı gibi modeller tasarlanıyorsa.

Özet

Bileşik kalıbı, nesneleri ağaç yapısı içinde düzenlemeyi sağlayan yapısal bir tasarım kalıbıdır. Bu kalıp sayesinde tekil nesneler ve nesne grupları ortak bir arayüz üzerinden kullanılabilir. Özellikle dosya sistemi, klasör yapısı, şirket organizasyonları ve kategori ağaçları gibi hiyerarşik modellerde oldukça kullanışlıdır.

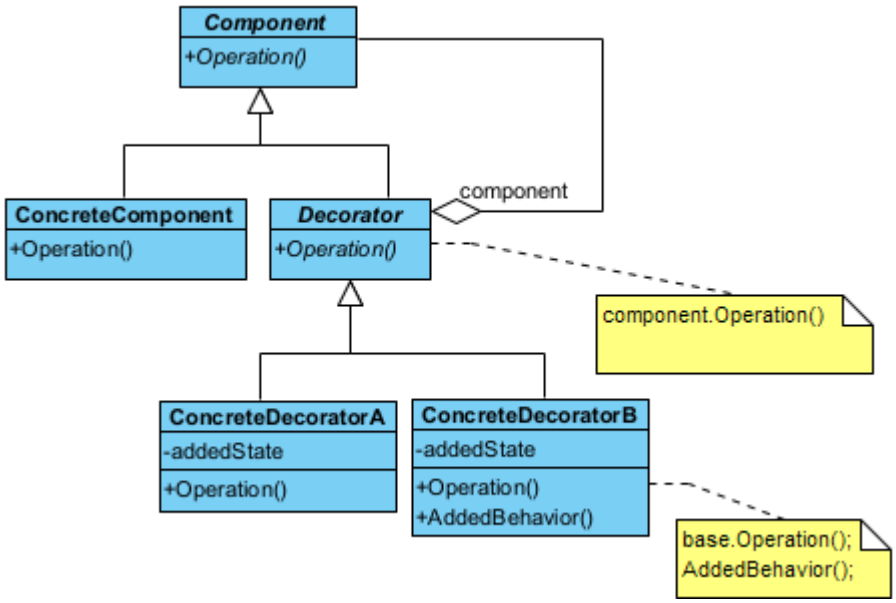
§5.3.4 Süsleyici

Süsleyici Kalıbı (*Decorator Pattern*), bir nesneye çalışma zamanında dinamik olarak yeni sorumluluklar, durumlar veya davranışlar eklemek için kullanılan yapısal (*structural*) bir tasarım kalıbıdır.

Bu kalıpta temel fikir, nesneyi değiştirmek veya ondan yeni alt sınıflar türetmek yerine, nesneyi başka nesnelerle sarmalamaktır (*wrapping*). Her sarmalayıcı nesne, temel nesneye yeni bir özellik ekler. Böylece nesnenin davranışı çalışma zamanında esnek biçimde genişletilebilir.

Süsleyici kalıbı, kalıtıma alternatif olarak düşünülebilir. Kalıtımda davranışlar sınıf seviyesinde belirlenir. Süsleyici kalıbında ise davranışlar nesne seviyesinde ve çalışma zamanında eklenir.

Bu kalıp bir kahve siparişine eklenen ekstralarla açıklanabilir. Önce sade kahve alınır. Daha sonra kahveye süt, şeker, karamel veya krema gibi ek özellikler eklenebilir. Her ek özellik, kahvenin açıklamasını ve fiyatını değiştirir. Ancak sade kahve sınıfını değiştirmeye gerek kalmaz.



Şekil 5.9: Süsleyici Kalıbı UML Diyagramı

Süsleyici kalıbındaki temel roller şunlardır:

- **Bileşen** (`Component`): Hem temel nesnenin hem de süsleyicilerin uyması gereken ortak arayüzü tanımlar.
- **Somut Bileşen** (`ConcreteComponent`): Üzerine ekleme yapılacak temel nesnedir. Davranışın en yalın halini gerçekleştirir.
- **Soyut Süsleyici** (`Decorator`): İçinde bir `Component` referansı tutar. Bu sayede hem temel nesneyi hem de başka bir süsleyiciyi sarmalayabilir.
- **Somut Süsleyici** (`ConcreteDecorator`): Gerçek ek özellikleri veya davranışları ekleyen sınıftır.

Java ile Temel Süsleyici Örneği

Aşağıdaki örnekte önce ortak bileşen arayüzü tanımlanır. Hem temel nesne hem de süsleyiciler bu arayüzü uygular.

```
interface Component {
  void operation();
}
```

Kod 5.62: Java Dilinde Bileşen Arayüzü

Şimdi temel nesne olan somut bileşeni yazalım:

```
class ConcreteComponent implements Component {
  @Override
  public void operation() {
    System.out.println("Temel ürün işlemi");
  }
}
```

Kod 5.63: Java Dilinde Somut Bileşen

Bu sınıf, davranışın en yalın halini temsil eder.

Süsleyiciler için ortak temel sınıf aşağıdaki gibidir:

```
abstract class Decorator implements Component {
  protected final Component component;

  public Decorator(Component component) {
    this.component = component;
  }

  @Override
  public void operation() {
    component.operation();
  }
}
```

Kod 5.64: Java Dilinde Soyut Süsleyici

`Decorator` sınıfı da `Component` arayüzünü uygular. İçinde yine bir `Component` nesnesi tutar. Bu yapı sayesinde süsleyiciler iç içe kullanılabilir.

Şimdi yeni durum ekleyen bir süsleyici yazalım:

```
class ConcreteDecorator1 extends Decorator {
  private final int addedState;

  public ConcreteDecorator1(Component component, int state) {
    super(component);
    this.addedState = state * 100;
  }

  @Override
  public void operation() {
    super.operation();
```

```
    System.out.println("-> Decorator 1: Eklendi. State: " + addedState);
  }
}
```

Kod 5.65: Java Dilinde Durum Ekleyen Süsleyici

Yeni davranış ekleyen başka bir süsleyici:

```
class ConcreteDecorator2 extends Decorator {
  public ConcreteDecorator2(Component component) {
    super(component);
  }

  private String addedBehavior() {
    return "Bu metin yeni davranıştan geliyor.";
  }

  @Override
  public void operation() {
    super.operation();

    System.out.println("-> Decorator 2: Yeni davranış tetiklendi: "
    + addedBehavior());
  }
}
```

Kod 5.66: Java Dilinde Davranış Ekleyen Süsleyici

İstemci kodda nesneler iç içe sarılarak kullanılır:

```
public class Main {
  public static void main(String[] args) {
    Component component = new ConcreteComponent();

    component = new ConcreteDecorator1(component, 2);

    component = new ConcreteDecorator2(component);

    component.operation();
  }
}
```

Kod 5.67: Java Dilinde Süsleyici Kullanımı

Program çıktısı:

```
Temel ürün işlemi
-> Decorator 1: Eklendi. State: 200
-> Decorator 2: Yeni davranış tetiklendi: Bu metin yeni davranıştan geliyor.
```

Bu örnekte önce temel nesne çalışır. Daha sonra sırasıyla birinci ve ikinci süsleyicinin eklediği davranışlar çalışır.

Tam Java Örneği

Aşağıda temel süsleyici kalıbının tek dosyada çalıştırılabilecek tam hali verilmiştir:

```
interface Component {
  void operation();
}

class ConcreteComponent implements Component {
  @Override
  public void operation() {
    System.out.println("Temel ürün işlemi");
  }
}
```

```
abstract class Decorator implements Component {
  protected final Component component;

  public Decorator(Component component) {
    this.component = component;
  }

  @Override
  public void operation() {
    component.operation();
  }
}

class ConcreteDecorator1 extends Decorator {
  private final int addedState;

  public ConcreteDecorator1(Component component, int state) {
    super(component);
    this.addedState = state * 100;
  }

  @Override
  public void operation() {
    super.operation();

    System.out.println("-> Decorator 1: Eklendi. State: " + addedState);
  }
}

class ConcreteDecorator2 extends Decorator {
  public ConcreteDecorator2(Component component) {
    super(component);
  }

  private String addedBehavior() {
    return "Bu metin yeni davranıştan geliyor.";
  }

  @Override
  public void operation() {
    super.operation();

    System.out.println("-> Decorator 2: Yeni davranış tetiklendi: "
    + addedBehavior());
  }
}

public class Main {
  public static void main(String[] args) {
    Component component = new ConcreteComponent();

    component = new ConcreteDecorator1(component, 2);

    component = new ConcreteDecorator2(component);

    component.operation();
  }
}
```

Kod 5.68: Java Dilinde Süsleyici Kalıbı Tam Örnek

Gerçek Dünya Örneği: Kahve Siparişi

Süsleyici kalıbının en bilinen örneklerinden biri kahve siparişidir. Önce sade kahve oluşturulur. Daha sonra bu kahve süt, şeker veya karamel gibi ek özelliklerle süslenir.

Bu örnekte:

- ◇ `Coffee`, ortak bileşen arayüzüdür.
- ◇ `SimpleCoffee`, temel kahvedir.
- ◇ `CoffeeDecorator`, soyut süsleyici sınıftır.
- ◇ `MilkDecorator`, `SugarDecorator` ve `CaramelDecorator` somut süsleyicilerdir.

Önce kahve arayüzünü tanımlayalım:

```
interface Coffee {
  double cost();
  String description();
}
```

Kod 5.69: Java Dilinde Coffee Arayüzü

Temel kahve sınıfı:

```
class SimpleCoffee implements Coffee {
  @Override
  public double cost() {
    return 5.0;
  }

  @Override
  public String description() {
    return "Sade Kahve";
  }
}
```

Kod 5.70: Java Dilinde Sade Kahve

Soyut kahve süsleyicisi:

```
abstract class CoffeeDecorator implements Coffee {
  protected final Coffee coffee;

  public CoffeeDecorator(Coffee coffee) {
    this.coffee = coffee;
  }
}
```

Kod 5.71: Java Dilinde Soyut Kahve Süsleyicisi

Süt ekleyen süsleyici:

```
class MilkDecorator extends CoffeeDecorator {
  public MilkDecorator(Coffee coffee) {
    super(coffee);
  }

  @Override
  public double cost() {
    return coffee.cost() + 1.5;
  }

  @Override
  public String description() {
    return coffee.description() + ", süt";
  }
}
```

Kod 5.72: Java Dilinde Süt Süsleyicisi

Şeker ekleyen süsleyici:

```
class SugarDecorator extends CoffeeDecorator {
  public SugarDecorator(Coffee coffee) {
    super(coffee);
```

```
  }

  @Override
  public double cost() {
    return coffee.cost() + 0.5;
  }

  @Override
  public String description() {
    return coffee.description() + ", şeker";
  }
}
```

Kod 5.73: Java Dilinde Şeker Süsleyicisi

Karamel ekleyen süsleyici:

```
class CaramelDecorator extends CoffeeDecorator {
  public CaramelDecorator(Coffee coffee) {
    super(coffee);
  }

  @Override
  public double cost() {
    return coffee.cost() + 2.0;
  }

  @Override
  public String description() {
    return coffee.description() + ", karamel";
  }
}
```

Kod 5.74: Java Dilinde Karamel Süsleyicisi

İstemci kod kahveyi çalışma zamanında istediği gibi süsleyebilir:

```
public class Main {
  public static void main(String[] args) {
    Coffee kahvem = new SimpleCoffee();

    kahvem = new MilkDecorator(kahvem);

    kahvem = new SugarDecorator(kahvem);

    System.out.println("Sipariş: " + kahvem.description());
    System.out.println("Toplam Fiyat: " + kahvem.cost() + " TL");

    System.out.println();

    Coffee ozelKahve = new CaramelDecorator(
    new MilkDecorator(
    new SimpleCoffee()
    )
    );

    System.out.println("Sipariş: " + ozelKahve.description());
    System.out.println("Toplam Fiyat: " + ozelKahve.cost() + " TL");
  }
}
```

Kod 5.75: Java Dilinde Kahve Süsleme Kullanımı

Program çıktısı:

```
Sipariş: Sade Kahve, süt, şeker
Toplam Fiyat: 7.0 TL
```

```
Sipariş: Sade Kahve, süt, karamel
Toplam Fiyat: 8.5 TL
```

Bu örnekte `SimpleCoffee` sınıfı değiştirilmeden kahveye farklı özellikler eklenmiştir.

Tam Java Örneği

Aşağıda kahve siparişi örneğinin tek dosyada çalıştırılabilecek tam hali verilmiştir:

```
interface Coffee {
  double cost();
  String description();
}

class SimpleCoffee implements Coffee {
  @Override
  public double cost() {
    return 5.0;
  }

  @Override
  public String description() {
    return "Sade Kahve";
  }
}

abstract class CoffeeDecorator implements Coffee {
  protected final Coffee coffee;

  public CoffeeDecorator(Coffee coffee) {
    this.coffee = coffee;
  }
}

class MilkDecorator extends CoffeeDecorator {
  public MilkDecorator(Coffee coffee) {
    super(coffee);
  }

  @Override
  public double cost() {
    return coffee.cost() + 1.5;
  }

  @Override
  public String description() {
    return coffee.description() + ", süt";
  }
}

class SugarDecorator extends CoffeeDecorator {
  public SugarDecorator(Coffee coffee) {
    super(coffee);
  }

  @Override
  public double cost() {
    return coffee.cost() + 0.5;
  }

  @Override
  public String description() {
    return coffee.description() + ", şeker";
  }
}
```

```
class CaramelDecorator extends CoffeeDecorator {
  public CaramelDecorator(Coffee coffee) {
    super(coffee);
  }

  @Override
  public double cost() {
    return coffee.cost() + 2.0;
  }

  @Override
  public String description() {
    return coffee.description() + ", karamel";
  }
}

public class Main {
  public static void main(String[] args) {
    Coffee kahvem = new SimpleCoffee();

    kahvem = new MilkDecorator(kahvem);

    kahvem = new SugarDecorator(kahvem);

    System.out.println("Sipariş: " + kahvem.description());
    System.out.println("Toplam Fiyat: " + kahvem.cost() + " TL");

    System.out.println();

    Coffee ozelKahve = new CaramelDecorator(
    new MilkDecorator(
    new SimpleCoffee()
    )
    );

    System.out.println("Sipariş: " + ozelKahve.description());
    System.out.println("Toplam Fiyat: " + ozelKahve.cost() + " TL");
  }
}
```

Kod 5.76: Java Dilinde Süsleyici Kalıbı Kahve Örneği

Süsleyici Kalıbı ve Kalıtım Farkı

Süsleyici kalıbı, kalıtımla davranış genişletmeye alternatif sunar. Örneğin kahve örneğinde kalıtım kullanılsaydı şu tür sınıflar yazmak gerekebilirdi:

- `SutluKahve`
- `SekerliKahve`
- `SutluSekerliKahve`
- `KaramelliKahve`
- `SutluKaramelliKahve`
- `SutluSekerliKaramelliKahve`

Ek özellik sayısı arttıkça sınıf sayısı hızla artar. Buna sınıf patlaması (*class explosion*) denir. Süsleyici kalıbı, bu sorunu nesneleri çalışma zamanında sarmalayarak çözer.

Kalıtım	Süsleyici Kalıbı
Davranış sınıf seviyesinde belirlenir.	Davranış nesne seviyesinde ve çalışma zamanında eklenir.
Sınıf sayısı artabilir.	Ek özellikler ayrı süsleyici sınıflarıyla esnek biçimde birleştirilir.
Daha statik bir yapıdır.	Daha dinamik bir yapıdır.
Her kombinasyon için sınıf gerekebilir.	Kombinasyonlar nesne sarmalama ile oluşturulur.

Java Kütüphanesinden Örnek

Java standart kütüphanesinde süsleyici kalıbına en bilinen örneklerden biri giriş/çıkış (*I/O*) sınıflarıdır.

Örneğin:

```
BufferedInputStream input = new BufferedInputStream(
new FileInputStream("veri.txt")
);
```

Burada `FileInputStream`, dosyadan veri okuyan temel bileşendir. `BufferedInputStream` ise bu okuma işlemine tamponlama özelliği ekleyen bir süsleyicidir. Temel okuma davranışı değiştirilmeden yeni bir özellik eklenmiştir.

Benzer şekilde:

```
DataInputStream input = new DataInputStream(
new BufferedInputStream(
new FileInputStream("veri.txt")
)
);
```

Bu kullanımda nesneler iç içe sarılarak yeni okuma yetenekleri eklenir.

Süsleyici Kalıbının Avantajları

- Nesnelere çalışma zamanında yeni davranışlar eklenebilir.
- Kalıtım yerine bileşim kullanarak daha esnek bir yapı sağlar.
- Sınıf patlamasını azaltır.
- Açık-kapalı ilkesine uygun bir genişletme sağlar.
- Farklı süsleyiciler farklı sıralarda birleştirilebilir.

Süsleyici Kalıbının Dezavantajları

- Çok sayıda küçük süsleyici sınıf oluşabilir.
- İç içe sarmalanmış nesneleri takip etmek zorlaşabilir.
- Hata ayıklama bazı durumlarda daha karmaşık olabilir.
- Süsleyicilerin sırası davranışı etkileyebilir.

Ne Zaman Kullanılmalı?

Süsleyici kalıbı şu durumlarda tercih edilebilir:

- Bir nesneye çalışma zamanında yeni özellikler eklenmek isteniyorsa.
- Kalıtım kullanımı çok sayıda alt sınıf oluşturuyorsa.
- Farklı özellik kombinasyonları esnek biçimde oluşturulmak isteniyorsa.
- Temel sınıf değiştirilmeden davranış genişletilmek isteniyorsa.
- Nesne davranışları katmanlı olarak artırılacaksa.

Özet

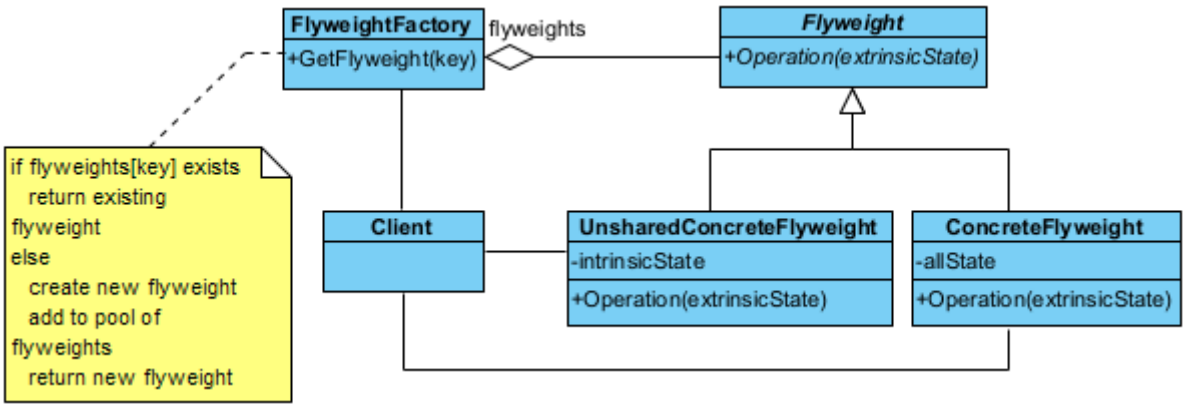
Süsleyici kalıbı, nesneleri sarmalayarak onlara dinamik biçimde yeni özellikler eklemeyi sağlar. Bu kalıp, kalıtım yerine bileşim kullanarak daha esnek bir genişletme modeli sunar. Kahveye süt, şeker veya karamel ekleme örneğinde olduğu gibi, temel nesne değiştirilmeden farklı özellikler çalışma zamanında birleştirilebilir.

§5.3.5 Sinek Sıklet

Sinek Sıklet Kalıbı (*Flyweight Pattern*), çok sayıda benzer nesnenin bulunduğu durumlarda bellek kullanımını azaltmak için kullanılan yapısal (*structural*) bir tasarım kalıbıdır.

Bu kalıbın temel amacı, nesnelerin ortak olan verilerini paylaşmak ve her nesne için tekrar tekrar aynı verileri bellekte tutmamaktır. Böylece özellikle binlerce veya milyonlarca benzer nesnenin bulunduğu sistemlerde bellek kullanımı önemli ölçüde azaltılabilir.

Örneğin bir oyun içinde orman sahnesinde binlerce ağaç olduğunu düşünelim. Her ağacın modeli, dokusu ve türü aynı olabilir. Ancak her ağacın konumu farklıdır. Bu durumda her ağaç için aynı model ve doku bilgisini ayrı ayrı bellekte tutmak gereksizdir. Bunun yerine ortak bilgiler tek bir nesnede tutulur, değişen bilgiler ise dışarıdan verilir.



Şekil 5.10: Sinek Sıklet Kalıbı UML Diyagramı

Sinek sıklet kalıbında iki tür durum bilgisi vardır:

- **İçsel Durum** (*intrinsic state*): Nesneler arasında ortak olan ve paylaşılabilen durumdur. Genellikle değişmezdir. Örneğin araç türü, ağaç modeli veya karakter biçimi.
- **Dışsal Durum** (*extrinsic state*): Her nesneye özgü olan ve paylaşılmayan durumdur. Bu bilgi nesneye dışarıdan verilir. Örneğin konum, hız, renk veya koordinat bilgisi.

Sinek sıklet kalıbındaki temel roller şunlardır:

- **Sinek Sıklet Arayüzü** (`Flyweight`): Paylaşılan nesnelerin uyması gereken arayüzü tanımlar. Dışsal durum genellikle metot parametresi olarak alınır.
- **Somut Sinek Sıklet** (`ConcreteFlyweight`): İçsel durumu kendi içinde saklayan ve paylaşılabilen nesnedir. Bu nesnelerin mümkün olduğunca değişmez (*immutable*) olması tercih edilir.
- **Sinek Sıklet Fabrikası** (`FlyweightFactory`): Paylaşılan nesneleri yönetir. Daha önce oluşturulmuş bir nesne istenirse mevcut nesneyi döndürür; yoksa yeni nesne oluşturur ve saklar.
- **Bağlam** (`Context`): Dışsal durumu tutan veya işlem sırasında dışsal durumu sinek sıklet nesnesine veren yapıdır.
- **İstemci** (`Client`): Fabrika üzerinden sinek sıklet nesnelerini alır ve dışsal durumla birlikte kullanır.

Java ile Sinek Sıklet Örneği

Aşağıdaki örnekte farklı araç türleri için ortak bilgiler paylaşılmaktadır. Araç türü içsel durumdur. Araçların konumu ise dışsal durum olarak metot çağrısı sırasında verilir.

Önce sinek sıklet arayüzü tanımlanır:

```
interface Flyweight {
  void operation(String extrinsicState);
}
```

Kod 5.77: Java Dilinde Flyweight Arayüzü

Bu arayüzdeki `operation()` metodu, dışsal durumu parametre olarak alır.

Şimdi somut sinek sıklet sınıfını yazalım:

```
class ConcreteFlyweight implements Flyweight {
  private final String intrinsicState;

  public ConcreteFlyweight(String intrinsicState) {
    this.intrinsicState = intrinsicState;
  }

  @Override
  public void operation(String extrinsicState) {
    System.out.println("[Paylaşılan] İç Durum: " + intrinsicState
    + " | Dış Durum: " + extrinsicState);
  }
}
```

Kod 5.78: Java Dilinde Somut Flyweight Sınıfı

Burada `intrinsicState` ortak durumdur. `final` olarak tanımlandığı için nesne oluşturulduktan sonra değiştirilemez. Bu, sinek sıklet nesneleri için istenen bir özelliktir.

Şimdi sinek sıklet fabrikasını yazalım:

```
import java.util.HashMap;
import java.util.Map;

class FlyweightFactory {
  private final Map<String, Flyweight> flyweights = new HashMap<>();

  public Flyweight getFlyweight(String key) {
    if (flyweights.containsKey(key)) {
      System.out.println("-> Fabrika: " + key
      + " zaten var, mevcut olan döndürülüyor.");

      return flyweights.get(key);
    }

    System.out.println("-> Fabrika: " + key
    + " bulunamadı, yeni oluşturuluyor.");

    Flyweight flyweight = new ConcreteFlyweight(key);
    flyweights.put(key, flyweight);

    return flyweight;
  }

  public void listFlyweights() {
    System.out.println();
    System.out.println("Fabrikadaki toplam nesne sayısı: "
    + flyweights.size());

    for (String key : flyweights.keySet()) {
      System.out.println("- " + key);
    }
  }
}
```

Kod 5.79: Java Dilinde Flyweight Fabrikası

Bu fabrika sınıfı, aynı anahtara sahip nesnenin tekrar oluşturulmasını engeller. Eğer nesne daha önce oluşturulmuşsa aynı nesneyi döndürür.

İstemci kod aşağıdaki gibi yazılır:

```
public class Main {
  public static void main(String[] args) {
    FlyweightFactory factory = new FlyweightFactory();

    Flyweight v1 = factory.getFlyweight("Binek Araç");
    Flyweight v2 = factory.getFlyweight("Kamyon");
    Flyweight v3 = factory.getFlyweight("Binek Araç");

    v1.operation("Konum: 41.00, 28.97");
    v2.operation("Konum: 39.93, 32.85");
    v3.operation("Konum: 40.71, -74.00");

    factory.listFlyweights();
  }
}
```

Kod 5.80: Java Dilinde Sinek Sıklet Kullanımı

Program çıktısı aşağıdaki gibi olur:

```
-> Fabrika: Binek Araç bulunamadı, yeni oluşturuluyor.
-> Fabrika: Kamyon bulunamadı, yeni oluşturuluyor.
-> Fabrika: Binek Araç zaten var, mevcut olan döndürülüyor.
[Paylaşılan] İç Durum: Binek Araç | Dış Durum: Konum: 41.00, 28.97
[Paylaşılan] İç Durum: Kamyon | Dış Durum: Konum: 39.93, 32.85
[Paylaşılan] İç Durum: Binek Araç | Dış Durum: Konum: 40.71, -74.00

Fabrikadaki toplam nesne sayısı: 2
- Kamyon
- Binek Araç
```

Dikkat edilirse `Binek Araç` iki kez istenmiştir. Ancak fabrika bu nesneyi yalnızca bir kez oluşturmuş, ikinci istekte mevcut nesneyi döndürmüştür.

Gerçek Dünya Örneği: Orman ve Ağaçlar

Sinek sıklet kalıbının klasik örneklerinden biri orman içindeki ağaçlardır. Bir ormanda binlerce ağaç olabilir. Ancak aynı türdeki ağaçların adı, rengi ve dokusu aynı olabilir. Bu bilgiler içsel durumdur ve paylaşılabilir.

Her ağacın konumu ise farklıdır. Bu nedenle konum bilgisi dışsal durum olarak tutulur.

Bu örnekte:

- ◇ `TreeType`, paylaşılan sinek sıklet nesnesidir.
- ◇ `TreeFactory`, aynı ağaç türünün tekrar oluşturulmasını engeller.
- ◇ `Tree`, dışsal durum olan `x` ve `y` koordinatlarını tutar.
- ◇ `Forest`, ağaçları yöneten bağlam sınıfıdır.

Önce paylaşılan ağaç türü sınıfını yazalım:

```
class TreeType {
  private final String name;
  private final String color;
  private final String texture;

  public TreeType(String name, String color, String texture) {
    this.name = name;
    this.color = color;
    this.texture = texture;
  }
```

```
  public void draw(int x, int y) {
    System.out.println(name + " ağacı çizildi. Renk: " + color
    + ", Doku: " + texture
    + ", Konum: (" + x + ", " + y + ")");
  }
}
```

Kod 5.81: Java Dilinde TreeType Sınıfı

Burada `name`, `color` ve `texture` ortak veridir. Bu bilgiler aynı türdeki ağaçlar arasında paylaşılır.

Şimdi fabrika sınıfını yazalım:

```
import java.util.HashMap;
import java.util.Map;

class TreeFactory {
  private static final Map<String, TreeType> treeTypes = new HashMap<>();

  public static TreeType getTreeType(String name, String color, String texture) {
    String key = name + "-" + color + "-" + texture;

    TreeType type = treeTypes.get(key);

    if (type == null) {
      type = new TreeType(name, color, texture);
      treeTypes.put(key, type);

      System.out.println("Yeni ağaç türü oluşturuldu: " + key);
    }

    return type;
  }

  public static int getTreeTypeCount() {
    return treeTypes.size();
  }
}
```

Kod 5.82: Java Dilinde TreeFactory Sınıfı

Şimdi her ağacın dışsal durumunu tutan `Tree` sınıfı yazılır:

```
class Tree {
  private final int x;
  private final int y;
  private final TreeType type;

  public Tree(int x, int y, TreeType type) {
    this.x = x;
    this.y = y;
    this.type = type;
  }

  public void draw() {
    type.draw(x, y);
  }
}
```

Kod 5.83: Java Dilinde Tree Sınıfı

`x` ve `y` koordinatları her ağaca özeldir. Fakat `TreeType` nesnesi ortak kullanılır.

Şimdi ormanı temsil eden sınıfı yazalım:

```
import java.util.ArrayList;
import java.util.List;
```

```
class Forest {
  private final List<Tree> trees = new ArrayList<>();

  public void plantTree(int x, int y, String name, String color, String texture) {
    TreeType type = TreeFactory.getTreeType(name, color, texture);

    Tree tree = new Tree(x, y, type);

    trees.add(tree);
  }

  public void draw() {
    for (Tree tree : trees) {
      tree.draw();
    }
  }

  public int getTreeCount() {
    return trees.size();
  }
}
```

Kod 5.84: Java Dilinde Forest Sınıfı

İstemci kod:

```
public class Main {
  public static void main(String[] args) {
    Forest forest = new Forest();

    forest.plantTree(10, 20, "Çam", "Yeşil", "CamDokusu");
    forest.plantTree(15, 25, "Çam", "Yeşil", "CamDokusu");
    forest.plantTree(30, 40, "Meşe", "Koyu Yeşil", "MeseDokusu");
    forest.plantTree(50, 60, "Çam", "Yeşil", "CamDokusu");

    forest.draw();

    System.out.println();
    System.out.println("Toplam ağaç sayısı: " + forest.getTreeCount());
    System.out.println("Paylaşılan ağaç türü sayısı: "
    + TreeFactory.getTreeTypeCount());
  }
}
```

Kod 5.85: Java Dilinde Orman Örneği Kullanımı

Program çıktısı:

```
Yeni ağaç türü oluşturuldu: Çam-Yeşil-CamDokusu
Yeni ağaç türü oluşturuldu: Meşe-Koyu Yeşil-MeseDokusu
Çam ağacı çizildi. Renk: Yeşil, Doku: CamDokusu, Konum: (10, 20)
Çam ağacı çizildi. Renk: Yeşil, Doku: CamDokusu, Konum: (15, 25)
Meşe ağacı çizildi. Renk: Koyu Yeşil, Doku: MeseDokusu, Konum: (30, 40)
Çam ağacı çizildi. Renk: Yeşil, Doku: CamDokusu, Konum: (50, 60)

Toplam ağaç sayısı: 4
Paylaşılan ağaç türü sayısı: 2
```

Bu örnekte 4 ağaç vardır. Ancak yalnızca 2 tane paylaşılan ağaç türü oluşturulmuştur. Aynı türdeki ağaçlar ortak `TreeType` nesnesini kullanır.

Tam Java Örneği

Aşağıda orman örneğinin tek dosyada çalıştırılabilecek tam hali verilmiştir:

```
import java.util.ArrayList;
import java.util.HashMap;
```

```
import java.util.List;
import java.util.Map;

class TreeType {
  private final String name;
  private final String color;
  private final String texture;

  public TreeType(String name, String color, String texture) {
    this.name = name;
    this.color = color;
    this.texture = texture;
  }

  public void draw(int x, int y) {
    System.out.println(name + " ağacı çizildi. Renk: " + color
    + ", Doku: " + texture
    + ", Konum: (" + x + ", " + y + ")");
  }
}

class TreeFactory {
  private static final Map<String, TreeType> treeTypes = new HashMap<>();

  public static TreeType getTreeType(String name, String color, String texture) {
    String key = name + "-" + color + "-" + texture;

    TreeType type = treeTypes.get(key);

    if (type == null) {
      type = new TreeType(name, color, texture);
      treeTypes.put(key, type);

      System.out.println("Yeni ağaç türü oluşturuldu: " + key);
    }

    return type;
  }

  public static int getTreeTypeCount() {
    return treeTypes.size();
  }
}

class Tree {
  private final int x;
  private final int y;
  private final TreeType type;

  public Tree(int x, int y, TreeType type) {
    this.x = x;
    this.y = y;
    this.type = type;
  }

  public void draw() {
    type.draw(x, y);
  }
}

class Forest {
  private final List<Tree> trees = new ArrayList<>();

  public void plantTree(int x, int y, String name, String color, String texture) {
    TreeType type = TreeFactory.getTreeType(name, color, texture);
```

```
      Tree tree = new Tree(x, y, type);

      trees.add(tree);
    }

    public void draw() {
      for (Tree tree : trees) {
        tree.draw();
      }
    }

    public int getTreeCount() {
      return trees.size();
    }
  }

  public class Main {
    public static void main(String[] args) {
      Forest forest = new Forest();

      forest.plantTree(10, 20, "Çam", "Yeşil", "CamDokusu");
      forest.plantTree(15, 25, "Çam", "Yeşil", "CamDokusu");
      forest.plantTree(30, 40, "Meşe", "Koyu Yeşil", "MeseDokusu");
      forest.plantTree(50, 60, "Çam", "Yeşil", "CamDokusu");

      forest.draw();

      System.out.println();
      System.out.println("Toplam ağaç sayısı: " + forest.getTreeCount());
      System.out.println("Paylaşılan ağaç türü sayısı: "
      + TreeFactory.getTreeTypeCount());
    }
  }
```

Kod 5.86: Java Dilinde Sinek Sıklet Kalıbı Orman Örneği

İçsel ve Dışsal Durum Ayrımı

Sinek sıklet kalıbının doğru uygulanabilmesi için içsel ve dışsal durum ayrımı iyi yapılmalıdır.

Ağaç örneği üzerinden düşünürsek:

İçsel Durum	**Dışsal Durum**
Ağaç türü	X koordinatı
Renk	Y koordinatı
Doku bilgisi	Nesnenin sahnedeki konumu
Model verisi	Anlık hareket veya durum bilgisi

İçsel durum paylaşılır. Dışsal durum ise her nesneye özgüdür ve gerektiğinde dışarıdan verilir.

Java'da Hazır Sinek Sıklet Benzeri Yapılar

Java'da bazı sınıflar ve mekanizmalar sinek sıklet mantığına benzer biçimde çalışır. Örneğin `String` havuzu buna iyi bir örnektir.

```
String a = "Java";
String b = "Java";

System.out.println(a == b);
```

Bu kod çoğu durumda `true` çıktısı verir. Çünkü aynı metin sabiti için Java, *String pool* adı verilen bir havuz kullanır. Aynı içeriğe sahip metin sabitleri bellekte paylaşılabilir.

Benzer şekilde küçük tamsayılar için `Integer` önbelleği de sinek sıklet fikrine benzer:

```
Integer x = 100;
```

```
Integer y = 100;

System.out.println(x == y);
```

Bu örnekte de çoğu Java gerçeklemesinde `true` sonucu alınır. Çünkü belirli aralıktaki `Integer` nesneleri önbellekten paylaşılır.

Sinek Sıklet Kalıbının Avantajları

- ◇ Çok sayıda benzer nesne bulunan sistemlerde bellek kullanımını azaltır.
- ◇ Ortak verilerin tek bir yerde tutulmasını sağlar.
- ◇ Nesne oluşturma maliyetini azaltabilir.
- ◇ Büyük ölçekli grafik, oyun, metin işleme veya simülasyon sistemlerinde performans kazancı sağlayabilir.

Sinek Sıklet Kalıbının Dezavantajları

- ◇ İçsel ve dışsal durum ayrımı tasarımı karmaşıklaştırabilir.
- ◇ Dışsal durumun sürekli dışarıdan verilmesi kodu daha zor takip edilir hale getirebilir.
- ◇ Paylaşılan nesneler değiştirilebilir olursa hatalara neden olabilir.
- ◇ Küçük sistemlerde gereksiz karmaşıklık oluşturabilir.

Ne Zaman Kullanılmalı?

Sinek sıklet kalıbı şu durumlarda tercih edilebilir:

- ◇ Uygulamada çok sayıda benzer nesne varsa.
- ◇ Nesnelerin ortak verileri büyük yer kaplıyorsa.
- ◇ Bellek kullanımı önemli bir problem oluşturuyorsa.
- ◇ Nesneler arasında paylaşılabilecek değişmez veriler bulunuyorsa.
- ◇ Nesneye özgü veriler dışarıdan sağlanabiliyorsa.

Özet

Sinek sıklet kalıbı, çok sayıda benzer nesnenin bulunduğu sistemlerde ortak verileri paylaşarak bellek kullanımını azaltan yapısal bir tasarım kalıbıdır. Bu kalıpta nesnelerin ortak verileri içsel durum olarak saklanır, nesneye özgü değişen veriler ise dışsal durum olarak dışarıdan sağlanır. Özellikle oyunlar, grafik sistemleri, metin işleyiciler ve simülasyonlar gibi çok sayıda benzer nesnenin bulunduğu uygulamalarda oldukça kullanışlıdır.

§5.3.6 Vekil

Vekil Kalıbı (*Proxy Pattern*), bir nesneye doğrudan erişmek yerine, o nesnenin yerine geçen başka bir nesne üzerinden erişim sağlamayı amaçlayan yapısal (*structural*) bir tasarım kalıbıdır.

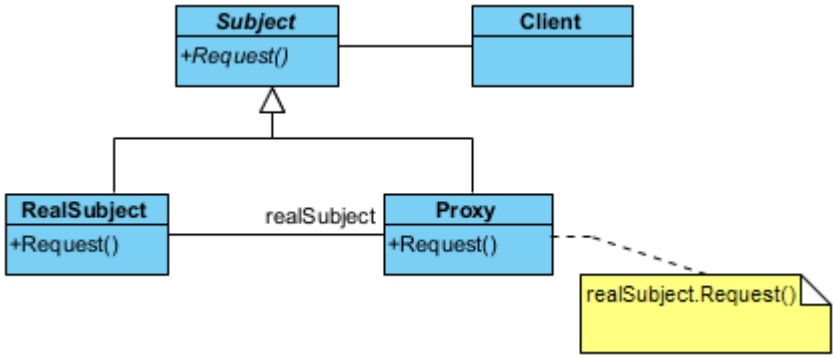
Bu kalıpta vekil nesne, gerçek nesneye erişimi kontrol eder. İstemci, gerçek nesneyle doğrudan iletişim kurmaz; onun yerine aynı arayüzü uygulayan vekil nesneyle çalışır. Vekil nesne gerekirse isteği gerçek nesneye iletir, gerekirse erişimi engeller veya isteği iletmeden önce ek işlemler yapar.

Vekil kalıbı şu amaçlarla kullanılabilir:

- ◇ Gerçek nesneye erişmeden önce güvenlik kontrolü yapmak.
- ◇ Nesnenin oluşturulmasını ihtiyaç duyulana kadar geciktirmek.
- ◇ Ağ üzerinden uzaktaki bir nesneye erişimi gizlemek.
- ◇ İşlem sonuçlarını önbelleğe almak.
- ◇ Gerçek nesneye yapılan erişimleri kayıt altına almak.

Bu kalıp bir şirketteki **yönetici asistanı** örneğine benzetilebilir. Yöneticiyle doğrudan görüşmek yerine önce asistanla görüşürsünüz. Asistan uygun görürse sizi yöneticiye yönlendirir. Burada asistan vekil, yönetici ise gerçek nesnedir.

Vekil kalıbındaki temel roller şunlardır:



Şekil 5.11: Vekil Kalıbı UML Diyagramı

- **Bağlam veya Özne** (`Subject`): Hem gerçek nesnenin hem de vekilin uyması gereken ortak arayüzü tanımlar. İstemci bu arayüz üzerinden çalışır.
- **Gerçek Bağlam** (`RealSubject`): Asıl işi yapan gerçek nesnedir. İş mantığının kendisi burada bulunur.
- **Vekil** (`Proxy`): Gerçek nesneye referans tutar ve erişimi kontrol eder. İstekleri gerçek nesneye iletmeden önce araya girer.
- **İstemci** (`Client`): Gerçek nesneye doğrudan erişmek yerine `Subject` arayüzü üzerinden vekil nesneyle çalışır.

Java ile Temel Vekil Örneği

Aşağıdaki örnekte `Subject` arayüzü, hem gerçek nesne hem de vekil nesne tarafından uygulanır.

Önce ortak arayüzü tanımlayalım:

```
interface Subject {
  void request();
}
```

Kod 5.87: Java Dilinde Subject Arayüzü

Şimdi asıl işi yapan gerçek nesneyi yazalım:

```
class RealSubject implements Subject {
  private final int state;

  public RealSubject(int state) {
    this.state = state;
  }

  @Override
  public void request() {
    System.out.println("RealSubject: İstek işleniyor. Durum: " + state);
  }
}
```

Kod 5.88: Java Dilinde RealSubject Sınıfı

`RealSubject`, gerçek iş mantığını barındıran sınıftır.

Şimdi vekil sınıfını yazalım:

```
class Proxy implements Subject {
  private RealSubject realSubject;

  public Proxy() {
    this.realSubject = new RealSubject(20);
  }
```

```
    public Proxy(RealSubject realSubject) {
      this.realSubject = realSubject;
    }

    private boolean checkAccess() {
      System.out.println("Proxy: Erişim yetkisi kontrol ediliyor...");
      return true;
    }

    private void logAccess() {
      System.out.println("Proxy: İşlem günlüğü kaydedildi.");
    }

    @Override
    public void request() {
      if (checkAccess()) {
        realSubject.request();
        logAccess();
      }
    }
  }
```

Kod 5.89: Java Dilinde Proxy Sınıfı

Bu sınıf, istemciden gelen isteği doğrudan gerçek nesneye göndermek yerine önce erişim kontrolü yapar. Daha sonra gerçek nesnenin `request()` metodunu çağırır ve işlemi kayıt altına alır.

İstemci kod şu şekilde yazılır:

```
public class Main {
  public static void main(String[] args) {
    System.out.println("--- Doğrudan RealSubject Kullanımı ---");
    Subject gercekOzne = new RealSubject(10);
    gercekOzne.request();

    System.out.println();

    System.out.println("--- Proxy Kullanımı ---");
    Subject vekil = new Proxy();
    vekil.request();

    System.out.println();

    System.out.println("--- Mevcut Nesne Üzerinden Proxy Kullanımı ---");
    RealSubject mevcutNesne = new RealSubject(30);
    Subject vekil2 = new Proxy(mevcutNesne);
    vekil2.request();
  }
}
```

Kod 5.90: Java Dilinde Vekil Kullanımı

Program çıktısı:

```
--- Doğrudan RealSubject Kullanımı ---
RealSubject: İstek işleniyor. Durum: 10

--- Proxy Kullanımı ---
Proxy: Erişim yetkisi kontrol ediliyor...
RealSubject: İstek işleniyor. Durum: 20
Proxy: İşlem günlüğü kaydedildi.

--- Mevcut Nesne Üzerinden Proxy Kullanımı ---
Proxy: Erişim yetkisi kontrol ediliyor...
RealSubject: İstek işleniyor. Durum: 30
Proxy: İşlem günlüğü kaydedildi.
```

Bu örnekte istemci, `Subject` arayüzünü kullanır. Nesnenin gerçek nesne mi yoksa vekil mi olduğu istemci açısından önemli değildir.

Tam Java Örneği

Aşağıda temel vekil kalıbının tek dosyada çalıştırılabilecek tam hali verilmiştir:

```
interface Subject {
  void request();
}

class RealSubject implements Subject {
  private final int state;

  public RealSubject(int state) {
    this.state = state;
  }

  @Override
  public void request() {
    System.out.println("RealSubject: İstek işleniyor. Durum: " + state);
  }
}

class Proxy implements Subject {
  private RealSubject realSubject;

  public Proxy() {
    this.realSubject = new RealSubject(20);
  }

  public Proxy(RealSubject realSubject) {
    this.realSubject = realSubject;
  }

  private boolean checkAccess() {
    System.out.println("Proxy: Erişim yetkisi kontrol ediliyor...");
    return true;
  }

  private void logAccess() {
    System.out.println("Proxy: İşlem günlüğü kaydedildi.");
  }

  @Override
  public void request() {
    if (checkAccess()) {
      realSubject.request();
      logAccess();
    }
  }
}

public class Main {
  public static void main(String[] args) {
    System.out.println("--- Doğrudan RealSubject Kullanımı ---");
    Subject gercekOzne = new RealSubject(10);
    gercekOzne.request();

    System.out.println();

    System.out.println("--- Proxy Kullanımı ---");
    Subject vekil = new Proxy();
    vekil.request();

    System.out.println();
```

```
    System.out.println("--- Mevcut Nesne Üzerinden Proxy Kullanımı ---");
    RealSubject mevcutNesne = new RealSubject(30);
    Subject vekil2 = new Proxy(mevcutNesne);
    vekil2.request();
  }
}
```

Kod 5.91: Java Dilinde Vekil Kalıbı Tam Örnek

Gerçek Dünya Örneği: Büyük Resimlerin Geç Yüklenmesi

Vekil kalıbının yaygın kullanım alanlarından biri, oluşturulması maliyetli nesnelerin ihtiyaç duyulana kadar oluşturulmamasıdır. Buna tembel yükleme (*lazy loading*) denir.

Örneğin bir uygulamada büyük boyutlu resimler olduğunu düşünelim. Resim nesnesi oluşturulduğu anda dosyadan veri yüklemek maliyetli olabilir. Bu yüzden resim gerçekten gösterilene kadar yükleme işlemi ertelenebilir.

Bu örnekte:

- ◇ `Image`, ortak arayüzdür.
- ◇ `RealImage`, resmi gerçekten yükleyen sınıftır.
- ◇ `ProxyImage`, resmi yalnızca gerektiğinde yükleyen vekildir.

Önce ortak arayüz tanımlanır:

```
interface Image {
  void display();
}
```

Kod 5.92: Java Dilinde Image Arayüzü

Gerçek resim sınıfı:

```
class RealImage implements Image {
  private final String fileName;

  public RealImage(String fileName) {
    this.fileName = fileName;
    loadFromDisk();
  }

  private void loadFromDisk() {
    System.out.println(fileName + " diskten yükleniyor...");
  }

  @Override
  public void display() {
    System.out.println(fileName + " gösteriliyor.");
  }
}
```

Kod 5.93: Java Dilinde RealImage Sınıfı

Bu sınıf oluşturulduğu anda resmi diskten yükler. Eğer resim hiç gösterilmeyecekse bu yükleme gereksiz olabilir.

Şimdi vekil sınıfı yazılır:

```
class ProxyImage implements Image {
  private final String fileName;
  private RealImage realImage;

  public ProxyImage(String fileName) {
    this.fileName = fileName;
  }
```

```
    @Override
    public void display() {
      if (realImage == null) {
        realImage = new RealImage(fileName);
      }

      realImage.display();
    }
  }
```

Kod 5.94: Java Dilinde ProxyImage Sınıfı

Burada `RealImage` nesnesi hemen oluşturulmaz. Yalnızca `display()` metodu ilk kez çağrıldığında oluşturulur.

İstemci kod:

```
  public class Main {
    public static void main(String[] args) {
      Image image = new ProxyImage("buyuk_resim.jpg");

      System.out.println("Resim nesnesi oluşturuldu, fakat henüz yüklenmedi.");

      System.out.println();

      image.display();

      System.out.println();

      image.display();
    }
  }
```

Kod 5.95: Java Dilinde Lazy Loading Proxy Kullanımı

Program çıktısı:

```
  Resim nesnesi oluşturuldu, fakat henüz yüklenmedi.

  buyuk_resim.jpg diskten yükleniyor...
  buyuk_resim.jpg gösteriliyor.

  buyuk_resim.jpg gösteriliyor.
```

Dikkat edilirse resim yalnızca ilk `display()` çağrısında yüklenmiştir. İkinci çağrıda mevcut `RealImage` nesnesi kullanılmıştır.

Koruma Vekili Örneği

Koruma vekili (*protection proxy*), gerçek nesneye erişmeden önce yetki kontrolü yapar. Örneğin yalnızca yönetici rolüne sahip kullanıcıların hassas bir belgeye erişebileceğini düşünelim.

```
  interface Document {
    void read();
  }

  class SecretDocument implements Document {
    @Override
    public void read() {
      System.out.println("Gizli belge okunuyor.");
    }
  }

  class DocumentProxy implements Document {
    private final String userRole;
    private final SecretDocument secretDocument;
```

```
    public DocumentProxy(String userRole) {
      this.userRole = userRole;
      this.secretDocument = new SecretDocument();
    }

    @Override
    public void read() {
      if ("ADMIN".equalsIgnoreCase(userRole)) {
        secretDocument.read();
      } else {
        System.out.println("Erişim reddedildi. Bu belgeyi okumaya yetkiniz yok.");
      }
    }
  }

  public class Main {
    public static void main(String[] args) {
      Document adminDocument = new DocumentProxy("ADMIN");
      adminDocument.read();

      Document guestDocument = new DocumentProxy("GUEST");
      guestDocument.read();
    }
  }
```

Kod 5.96: Java Dilinde Koruma Vekili Örneği

Program çıktısı:

```
Gizli belge okunuyor.
Erişim reddedildi. Bu belgeyi okumaya yetkiniz yok.
```

Bu örnekte istemci yine `Document` arayüzü üzerinden çalışır. Ancak vekil nesne, gerçek belgeye erişimden önce yetki kontrolü yapar.

Vekil Türleri

Vekil kalıbı farklı amaçlarla uygulanabilir:

- **Uzak Vekil** (*Remote Proxy*): Başka bir makinede, süreçte veya sunucuda bulunan nesneye yerel bir nesne gibi erişilmesini sağlar. Ağ iletişimi ayrıntılarını gizler.
- **Sanal Vekil** (*Virtual Proxy*): Oluşturulması pahalı olan nesneleri ihtiyaç duyulana kadar oluşturmaz. Büyük resimlerin geç yüklenmesi buna örnektir.
- **Koruma Vekili** (*Protection Proxy*): Gerçek nesneye erişimden önce yetkilendirme veya erişim kontrolü yapar.
- **Önbellek Vekili** (*Cache Proxy*): Daha önce hesaplanan veya getirilen sonuçları saklayarak tekrar kullanımda performans sağlar.
- **Akıllı Referans** (*Smart Reference*): Gerçek nesneye erişim sırasında ek işlemler yapar. Örneğin erişim sayısını tutabilir, kilitleme yapabilir veya günlükleme gerçekleştirebilir.

Vekil Kalıbı ve Süsleyici Kalıbı Arasındaki Fark

Vekil ve süsleyici kalıpları yapısal olarak birbirine benzeyebilir. İkisi de gerçek nesneyi sarabilir ve aynı arayüzü uygulayabilir. Ancak amaçları farklıdır.

Vekil Kalıbı	Süsleyici Kalıbı
Amaç erişimi kontrol etmektir.	Amaç nesneye yeni davranış eklemektir.
Gerçek nesneye erişimi kısıtlayabilir veya geciktirebilir.	Temel davranışı genişletir.
Güvenlik, tembel yükleme, uzaktan erişim için kullanılır.	Ek özellikler ve sorumluluklar eklemek için kullanılır.
Vekil çoğu zaman gerçek nesnenin yaşam döngüsünü yönetir.	Süsleyici genellikle mevcut nesneyi zenginleştirir.

Vekil Kalıbının Avantajları

- ◇ Gerçek nesneye erişimi kontrol etmeyi sağlar.
- ◇ Maliyetli nesnelerin ihtiyaç duyulana kadar oluşturulmasını mümkün kılar.
- ◇ Yetkilendirme, günlükleme ve önbellekleme gibi ek işlemleri merkezi hale getirir.
- ◇ İstemci kodun gerçek nesnenin ayrıntılarını bilmesini engeller.
- ◇ Uzaktaki nesnelerle çalışmayı kolaylaştırabilir.

Vekil Kalıbının Dezavantajları

- ◇ Sisteme ek bir sınıf katmanı ekler.
- ◇ Hatalı kullanıldığında kodun akışını takip etmeyi zorlaştırabilir.
- ◇ Gerçek nesneye erişim gecikebilir veya ek kontrol maliyeti oluşabilir.
- ◇ Çok fazla vekil kullanımı tasarımı karmaşıklaştırabilir.

Ne Zaman Kullanılmalı?

Vekil kalıbı şu durumlarda tercih edilebilir:

- ◇ Gerçek nesneye erişim kontrol edilmek isteniyorsa.
- ◇ Nesne oluşturma maliyeti yüksekse ve geç yükleme gerekiyorsa.
- ◇ Uzaktaki bir nesneye yerel bir nesne gibi erişilmek isteniyorsa.
- ◇ Yetkilendirme, günlükleme veya önbellekleme gibi ek işlemler yapılacaksa.
- ◇ İstemci kodun gerçek nesneyle doğrudan çalışması istenmiyorsa.

Özet

Vekil kalıbı, gerçek nesneye erişimi kontrol eden bir aracı nesne kullanmayı sağlar. İstemci, gerçek nesneyle doğrudan çalışmak yerine aynı arayüzü uygulayan vekil nesneyle çalışır. Vekil nesne güvenlik kontrolü yapabilir, nesneyi geç yükleyebilir, erişimleri kaydedebilir veya uzaktaki nesneye erişim ayrıntılarını gizleyebilir.

§5.3.7 Köprü

Köprü Kalıbı (*Bridge Pattern*), bir soyutlamayı (*abstraction*) onun uygulamasından (*implementation*) ayırarak, ikisinin de birbirinden bağımsız olarak geliştirilebilmesini sağlayan yapısal (*structural*) bir tasarım kalıbıdır.

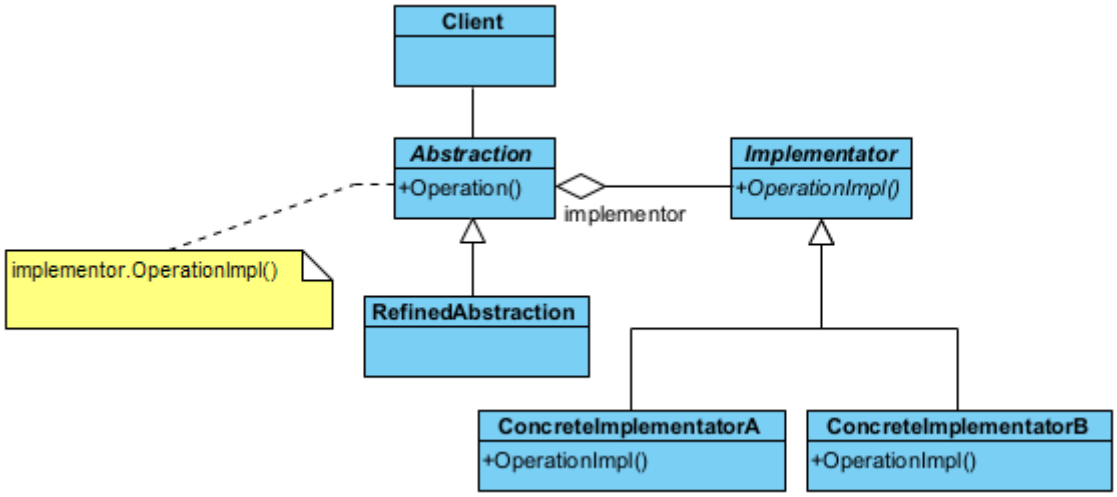
Bu kalıbın temel amacı, işi yaptıran üst düzey sınıf ile işi yapan alt düzey uygulama sınıfları arasındaki sıkı bağımlılığı azaltmaktır. Böylece soyutlama tarafı değişse de uygulama tarafı bundan doğrudan etkilenmez. Aynı şekilde yeni bir uygulama sınıfı eklendiğinde istemci kodun veya soyutlama tarafının değiştirilmesi gerekmez.

Köprü kalıbı, özellikle iki farklı yönde genişleyebilen sınıf hiyerarşileri oluştuğunda kullanışlıdır. Örneğin kumanda ve cihaz ilişkisini düşünelim. Kumanda türleri artabilir: standart kumanda, gelişmiş kumanda, akıllı kumanda. Aynı zamanda cihaz türleri de artabilir: televizyon, radyo, projektör. Eğer bu iki boyut kalıtımla modellenirse sınıf sayısı hızla artar.

Örneğin:

- ◇ `StandartTelevizyonKumandasi`
- ◇ `StandartRadyoKumandasi`
- ◇ `AkilliTelevizyonKumandasi`
- ◇ `AkilliRadyoKumandasi`
- ◇ `GelismisProjektorKumandasi`

Bu durum sınıf patlaması (*class explosion*) olarak adlandırılır. Köprü kalıbı, kalıtım yerine bileşim (*composition*) kullanarak bu problemi azaltır.



Şekil 5.12: Köprü Kalıbı UML Diyagramı

Köprü kalıbındaki temel roller şunlardır:

- ◇ **Soyutlama** (`Abstraction`): İstemcinin kullandığı üst düzey arayüzdür. İçinde bir `Implementor` referansı tutar ve gelen istekleri bu uygulayıcı nesneye aktarır.
- ◇ **Geliştirilmiş Soyutlama** (`RefinedAbstraction`): Soyutlamayı genişleten sınıftır. Daha özel kontrol davranışları ekleyebilir.
- ◇ **Taşeron Arayüzü** (`Implementor`): Somut uygulama sınıflarının uyması gereken arayüzü tanımlar. Genellikle alt seviye işlemleri içerir.
- ◇ **Somut Taşeron** (`ConcreteImplementor`): Gerçek işi yapan sınıftır. Uygulama ayrıntıları burada bulunur.
- ◇ **İstemci** (`Client`): Soyutlama üzerinden çalışır. Somut uygulama sınıflarının ayrıntılarını bilmek zorunda değildir.

Java ile Temel Köprü Örneği

Aşağıdaki örnekte `Abstraction` sınıfı işi yaptıran yüklenici, `Implementor` ise işi yapan taşeron gibi düşünülebilir. Yüklenici, işi doğrudan kendi yapmaz; bağlı olduğu taşerona yaptırır.

Önce taşeron arayüzü tanımlanır:

```
interface Implementor {
  void operationImpl();
}
```

Kod 5.97: Java Dilinde Implementor Arayüzü

Şimdi iki farklı somut taşeron sınıfı yazalım:

```
class ConcreteImplementorA implements Implementor {
  @Override
  public void operationImpl() {
    System.out.println("A taşeronu tarafından yapılan iş...");
  }
}
```

```
class ConcreteImplementorB implements Implementor {
  @Override
  public void operationImpl() {
    System.out.println("B taşeronu tarafından yapılan iş...");
  }
}
```

Kod 5.98: Java Dilinde Somut Implementor Sınıfları

Bu sınıflar aynı işi farklı biçimlerde gerçekleştiren somut uygulamalardır.

Şimdi soyutlama sınıfını yazalım:

```
class Abstraction {
  protected Implementor implementor;

  public Abstraction(Implementor implementor) {
    this.implementor = implementor;
  }

  public void setImplementor(Implementor implementor) {
    this.implementor = implementor;
  }

  public void operation() {
    if (implementor != null) {
      System.out.println("Yüklenici taşerona iş yaptırıyor:");
      implementor.operationImpl();
    }
  }
}
```

Kod 5.99: Java Dilinde Abstraction Sınıfı

`Abstraction`, somut taşeron sınıflarına doğrudan bağımlı değildir. Yalnızca `Implementor` arayüzünü bilir.

Geliştirilmiş soyutlama sınıfı:

```
class RefinedAbstraction extends Abstraction {
  public RefinedAbstraction(Implementor implementor) {
    super(implementor);
  }

  public void refinedOperation() {
    System.out.println("Geliştirilmiş soyutlama ek işlem yapıyor.");
    operation();
  }
}
```

Kod 5.100: Java Dilinde RefinedAbstraction Sınıfı

İstemci kod:

```
public class Main {
  public static void main(String[] args) {
    Abstraction yuklenici =
    new RefinedAbstraction(new ConcreteImplementorA());

    yuklenici.operation();

    System.out.println("--- Taşeron değiştiriliyor ---");

    yuklenici.setImplementor(new ConcreteImplementorB());

    yuklenici.operation();
  }
}
```

Kod 5.101: Java Dilinde Köprü Kalıbı Kullanımı

Program çıktısı:

```
Yüklenici taşerona iş yaptırıyor:
A taşeronu tarafından yapılan iş...
--- Taşeron değiştiriliyor ---
Yüklenici taşerona iş yaptırıyor:
B taşeronu tarafından yapılan iş...
```

Bu örnekte çalışma zamanında taşeron değiştirilebilmiştir. İstemci tarafında işin nasıl yapıldığı bilinmez. İstemci yalnızca soyutlama sınıfını kullanır.

Gerçek Dünya Örneği: Kumanda ve Cihaz

Köprü kalıbını daha gerçekçi bir örnekle inceleyelim. Bir kumanda sistemi olduğunu düşünelim. Kumanda bir cihazı açabilir, kapatabilir ve sesini ayarlayabilir. Ancak kumandanın hangi cihazla çalıştığı değişebilir. Bu cihaz televizyon da olabilir, radyo da olabilir.

Bu örnekte:

- ◇ `Device`, taşeron arayüzüdür.
- ◇ `Television` ve `Radio`, somut taşeronlardır.
- ◇ `RemoteControl`, soyutlama sınıfıdır.
- ◇ `AdvancedRemoteControl`, geliştirilmiş soyutlamadır.

Önce cihaz arayüzünü tanımlayalım:

```
interface Device {
  void turnOn();
  void turnOff();
  void setVolume(int volume);
  int getVolume();
  boolean isEnabled();
}
```

Kod 5.102: Java Dilinde Device Arayüzü

Televizyon sınıfı:

```
class Television implements Device {
  private boolean enabled;
  private int volume;

  @Override
  public void turnOn() {
    enabled = true;
    System.out.println("Televizyon açıldı.");
  }

  @Override
  public void turnOff() {
    enabled = false;
    System.out.println("Televizyon kapatıldı.");
  }

  @Override
  public void setVolume(int volume) {
    this.volume = volume;
    System.out.println("Televizyon ses seviyesi: " + volume);
  }

  @Override
  public int getVolume() {
    return volume;
```

```
    }

    @Override
    public boolean isEnabled() {
      return enabled;
    }
  }
```

Kod 5.103: Java Dilinde Television Sınıfı

Radyo sınıfı:

```
class Radio implements Device {
  private boolean enabled;
  private int volume;

  @Override
  public void turnOn() {
    enabled = true;
    System.out.println("Radyo açıldı.");
  }

  @Override
  public void turnOff() {
    enabled = false;
    System.out.println("Radyo kapatıldı.");
  }

  @Override
  public void setVolume(int volume) {
    this.volume = volume;
    System.out.println("Radyo ses seviyesi: " + volume);
  }

  @Override
  public int getVolume() {
    return volume;
  }

  @Override
  public boolean isEnabled() {
    return enabled;
  }
}
```

Kod 5.104: Java Dilinde Radio Sınıfı

Şimdi soyut kumanda sınıfı yazılır:

```
class RemoteControl {
  protected Device device;

  public RemoteControl(Device device) {
    this.device = device;
  }

  public void power() {
    if (device.isEnabled()) {
      device.turnOff();
    } else {
      device.turnOn();
    }
  }

  public void volumeUp() {
    device.setVolume(device.getVolume() + 10);
  }
```

```
    public void volumeDown() {
      device.setVolume(device.getVolume() - 10);
    }
  }
```

Kod 5.105: Java Dilinde RemoteControl Sınıfı

Bu sınıf `Television` veya `Radio` sınıflarına doğrudan bağlı değildir. Yalnızca `Device` arayüzünü bilir.

Gelişmiş kumanda sınıfı:

```
class AdvancedRemoteControl extends RemoteControl {
  public AdvancedRemoteControl(Device device) {
    super(device);
  }

  public void mute() {
    device.setVolume(0);
    System.out.println("Ses kapatıldı.");
  }
}
```

Kod 5.106: Java Dilinde AdvancedRemoteControl Sınıfı

İstemci kod:

```
public class Main {
  public static void main(String[] args) {
    Device tv = new Television();
    RemoteControl tvRemote = new RemoteControl(tv);

    tvRemote.power();
    tvRemote.volumeUp();

    System.out.println();

    Device radio = new Radio();
    AdvancedRemoteControl radioRemote =
    new AdvancedRemoteControl(radio);

    radioRemote.power();
    radioRemote.volumeUp();
    radioRemote.mute();
  }
}
```

Kod 5.107: Java Dilinde Kumanda-Cihaz Köprü Kullanımı

Program çıktısı:

```
Televizyon açıldı.
Televizyon ses seviyesi: 10

Radyo açıldı.
Radyo ses seviyesi: 10
Radyo ses seviyesi: 0
Ses kapatıldı.
```

Bu örnekte kumanda tarafı ve cihaz tarafı birbirinden bağımsızdır. Yeni bir cihaz türü eklendiğinde kumanda sınıflarını değiştirmeye gerek yoktur. Yeni bir kumanda türü eklendiğinde de cihaz sınıflarını değiştirmeye gerek yoktur.

Tam Java Örneği

Aşağıda kumanda ve cihaz örneğinin tek dosyada çalıştırılabilecek tam hali verilmiştir:

```
interface Device {
  void turnOn();
  void turnOff();
  void setVolume(int volume);
  int getVolume();
  boolean isEnabled();
}

class Television implements Device {
  private boolean enabled;
  private int volume;

  @Override
  public void turnOn() {
    enabled = true;
    System.out.println("Televizyon açıldı.");
  }

  @Override
  public void turnOff() {
    enabled = false;
    System.out.println("Televizyon kapatıldı.");
  }

  @Override
  public void setVolume(int volume) {
    this.volume = volume;
    System.out.println("Televizyon ses seviyesi: " + volume);
  }

  @Override
  public int getVolume() {
    return volume;
  }

  @Override
  public boolean isEnabled() {
    return enabled;
  }
}

class Radio implements Device {
  private boolean enabled;
  private int volume;

  @Override
  public void turnOn() {
    enabled = true;
    System.out.println("Radyo açıldı.");
  }

  @Override
  public void turnOff() {
    enabled = false;
    System.out.println("Radyo kapatıldı.");
  }

  @Override
  public void setVolume(int volume) {
    this.volume = volume;
    System.out.println("Radyo ses seviyesi: " + volume);
  }

  @Override
  public int getVolume() {
```

```
    return volume;
  }

  @Override
  public boolean isEnabled() {
    return enabled;
  }
}

class RemoteControl {
  protected Device device;

  public RemoteControl(Device device) {
    this.device = device;
  }

  public void power() {
    if (device.isEnabled()) {
      device.turnOff();
    } else {
      device.turnOn();
    }
  }

  public void volumeUp() {
    device.setVolume(device.getVolume() + 10);
  }

  public void volumeDown() {
    device.setVolume(device.getVolume() - 10);
  }
}

class AdvancedRemoteControl extends RemoteControl {
  public AdvancedRemoteControl(Device device) {
    super(device);
  }

  public void mute() {
    device.setVolume(0);
    System.out.println("Ses kapatıldı.");
  }
}

public class Main {
  public static void main(String[] args) {
    Device tv = new Television();
    RemoteControl tvRemote = new RemoteControl(tv);

    tvRemote.power();
    tvRemote.volumeUp();

    System.out.println();

    Device radio = new Radio();
    AdvancedRemoteControl radioRemote =
    new AdvancedRemoteControl(radio);

    radioRemote.power();
    radioRemote.volumeUp();
    radioRemote.mute();
  }
}
```

Kod 5.108: Java Dilinde Köprü Kalıbı Kumanda-Cihaz Örneği

Köprü Kalıbı ve Adaptör Kalıbı Arasındaki Fark

Köprü kalıbı, adaptör kalıbı ile karıştırılabilir. İkisi de farklı sınıflar arasında bağlantı kurar. Ancak amaçları farklıdır.

Köprü Kalıbı	Adaptör Kalıbı
Tasarım aşamasında soyutlama ve uygulamayı ayırmak için kullanılır.	Mevcut uyumsuz arayüzleri sonradan uyumlu hale getirmek için kullanılır.
İki bağımsız hiyerarşinin birlikte gelişmesini sağlar.	Var olan bir sınıfı beklenen arayüze dönüştürür.
Önceden planlanan bir tasarım çözümüdür.	Genellikle sonradan entegrasyon amacıyla kullanılır.
Kalıtım yerine bileşimi öne çıkarır.	Sarmalama veya kalıtım ile arayüz dönüşümü yapar.

Köprü Kalıbının Avantajları

- ◇ Soyutlama ile uygulamayı birbirinden ayırır.
- ◇ Sınıf patlamasını azaltır.
- ◇ Yeni soyutlama türleri ve yeni uygulama türleri bağımsız olarak eklenebilir.
- ◇ Kalıtım yerine bileşim kullanarak daha esnek bir yapı sağlar.
- ◇ Çalışma zamanında uygulayıcı nesnenin değiştirilebilmesine olanak tanır.
- ◇ İstemci kodun somut uygulama sınıflarına bağımlılığını azaltır.

Köprü Kalıbının Dezavantajları

- ◇ Basit sistemlerde gereksiz soyutlama oluşturabilir.
- ◇ Sınıf sayısını artırabilir.
- ◇ Tasarımın anlaşılması başlangıçta zor olabilir.
- ◇ Soyutlama ve uygulama ayrımı doğru yapılmazsa karmaşıklık artabilir.

Ne Zaman Kullanılmalı?

Köprü kalıbı şu durumlarda tercih edilebilir:

- ◇ Soyutlama ve uygulama birbirinden bağımsız olarak değişecekse.
- ◇ Kalıtım kullanımı sınıf patlamasına neden oluyorsa.
- ◇ Aynı soyut arayüz farklı uygulama sınıflarıyla çalışacaksa.
- ◇ Çalışma zamanında uygulama nesnesi değiştirilecekse.
- ◇ İstemci kodun somut uygulama sınıflarına bağımlılığı azaltılmak isteniyorsa.
- ◇ Projede birden fazla taşeron veya platform desteği gerekiyorsa.

Özet

Köprü kalıbı, soyutlama ile uygulamayı birbirinden ayırarak her iki tarafın da bağımsız şekilde geliştirilebilmesini sağlar. Kalıtım yerine bileşim kullanır. Bu sayede hem soyutlama tarafında hem uygulama tarafında genişleme yapılabilir ve sınıf patlaması önlenir. Kumanda ve cihaz örneğinde olduğu gibi, kumanda türleri ile cihaz türleri ayrı ayrı geliştirilebilir.

5.4 Davranışla İlgili Kalıplar

Davranışla ilgili kalıplar (*behavioral patterns*) nesnelerin çeşitli olaylar karşısında gösterecekleri davranışlara çözüm getirmektedirler.

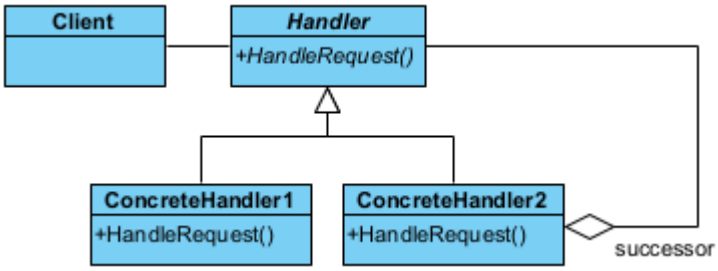
§5.4.1 Sorumluluk Zinciri

Sorumluluk Zinciri Kalıbı (*Chain of Responsibility Pattern*), bir isteğin hangi nesne tarafından işleneceğini gönderen nesneden ayırmak için kullanılan davranışsal (*behavioral*) bir tasarım kalıbıdır.

Bazı durumlarda bir nesne, kendisine gelen isteği doğrudan işleyemeyebilir. Bu durumda is-

teği kendisinden sonra gelen başka bir nesneye devreder. İstek, zincir boyunca ilerler. Zincirdeki her nesne, isteği ya işler ya da bir sonraki nesneye aktarır.

Bu kalıbın temel amacı, isteği gönderen nesne ile isteği işleyen nesne arasındaki bağı gevşetmektir (*decoupling*). İstemci, isteğin kim tarafından çözüleceğini bilmez. Yalnızca isteği zincirin başına gönderir.



Şekil 5.13: Sorumluluk Zinciri Kalıbı UML Diyagramı

Bu kalıp günlük hayatta bir onay sürecine benzetilebilir. Örneğin küçük miktarlı bir harcamayı memur onaylayabilir. Daha büyük harcamalar şefe, daha da büyük olanlar müdüre gönderilir. Başvuruyu yapan kişi, talebin hangi seviyede çözüleceğini bilmek zorunda değildir.

Sorumluluk zinciri kalıbındaki temel roller şunlardır:

- **İşleyici** (`Handler`): İsteklerin nasıl işleneceğine dair ortak arayüzü tanımlar. Genellikle zincirdeki bir sonraki işleyiciyi tutan bir referans içerir.
- **Somut İşleyici** (`ConcreteHandler`): Kendisine gelen isteği kontrol eder. Eğer istek kendi sorumluluk alanındaysa işler. Aksi durumda isteği zincirdeki bir sonraki işleyiciye aktarır.
- **İstemci** (`Client`): Zinciri oluşturur ve isteği zincirin ilk halkasına gönderir. İsteğin hangi işleyici tarafından çözüleceğiyle ilgilenmez.

Java ile Sorumluluk Zinciri Örneği

Aşağıdaki örnekte farklı seviyelerde onay yetkisine sahip işleyiciler vardır. Memur küçük talepleri, şef orta büyüklükteki talepleri, müdür ise daha büyük talepleri onaylar.

Önce soyut işleyici sınıfını tanımlayalım:

```
abstract class Handler {
  protected Handler successor;

  public void setSuccessor(Handler successor) {
    this.successor = successor;
  }

  public abstract void handleRequest(int request);
}
```

Kod 5.109: Java Dilinde Soyut Handler Sınıfı

Bu sınıf, zincirin bir sonraki halkasını `successor` alanında tutar. Her somut işleyici `handleRequest()` metodunu kendine göre uygular.

Şimdi küçük talepleri işleyen memur sınıfını yazalım:

```
class MemurHandler extends Handler {
  @Override
  public void handleRequest(int request) {
    if (request < 100) {
      System.out.println("Memur: " + request
      + " birimlik talebi onayladı.");
    } else if (successor != null) {
      System.out.println("Memur: Talep limitimi aşıyor, şefe devrediliyor...");
      successor.handleRequest(request);
```

```
    } else {
      System.out.println("Memur: Talebi karşılayacak kimse bulunamadı.");
    }
  }
}
```

Kod 5.110: Java Dilinde Memur İşleyicisi

Şef sınıfı:

```
class SefHandler extends Handler {
  @Override
  public void handleRequest(int request) {
    if (request >= 100 && request < 500) {
      System.out.println("Şef: " + request
      + " birimlik talebi onayladı.");
    } else if (successor != null) {
      System.out.println("Şef: Talep limitimi aşıyor, müdüre devrediliyor...");
      successor.handleRequest(request);
    } else {
      System.out.println("Şef: Bu kadar büyük bir yetkim yok ve başka amir tanımlı de
          ğil.");
    }
  }
}
```

Kod 5.111: Java Dilinde Şef İşleyicisi

Daha büyük talepler için müdür sınıfı:

```
class MudurHandler extends Handler {
  @Override
  public void handleRequest(int request) {
    if (request >= 500 && request < 2000) {
      System.out.println("Müdür: " + request
      + " birimlik talebi onayladı.");
    } else if (successor != null) {
      successor.handleRequest(request);
    } else {
      System.out.println("Müdür: " + request
      + " birimlik talep için üst makam gerekiyor.");
    }
  }
}
```

Kod 5.112: Java Dilinde Müdür İşleyicisi

İstemci kod zinciri oluşturur ve talepleri zincirin başına gönderir:

```
public class Main {
  public static void main(String[] args) {
    Handler memur = new MemurHandler();
    Handler sef = new SefHandler();
    Handler mudur = new MudurHandler();

    memur.setSuccessor(sef);
    sef.setSuccessor(mudur);

    System.out.println("--- Talep 1 (50) ---");
    memur.handleRequest(50);

    System.out.println("--- Talep 2 (250) ---");
    memur.handleRequest(250);

    System.out.println("--- Talep 3 (1000) ---");
    memur.handleRequest(1000);

    System.out.println("--- Talep 4 (5000) ---");
```

```
      memur.handleRequest(5000);
    }
  }
```

Kod 5.113: Java Dilinde Sorumluluk Zinciri Kullanımı

Program çıktısı:

```
--- Talep 1 (50) ---
Memur: 50 birimlik talebi onayladı.
--- Talep 2 (250) ---
Memur: Talep limitimi aşıyor, şefe devrediliyor...
Şef: 250 birimlik talebi onayladı.
--- Talep 3 (1000) ---
Memur: Talep limitimi aşıyor, şefe devrediliyor...
Şef: Talep limitimi aşıyor, müdüre devrediliyor...
Müdür: 1000 birimlik talebi onayladı.
--- Talep 4 (5000) ---
Memur: Talep limitimi aşıyor, şefe devrediliyor...
Şef: Talep limitimi aşıyor, müdüre devrediliyor...
Müdür: 5000 birimlik talep için üst makam gerekiyor.
```

Bu örnekte istemci, talebin memur, şef veya müdür tarafından işleneceğini bilmez. Yalnızca talebi zincirin ilk halkasına gönderir.

Gerçek Dünya Örneği: Destek Talebi Sistemi

Sorumluluk zinciri kalıbı teknik destek sistemlerinde sıkça kullanılır. Bir destek talebi önce birinci seviye destek ekibine gider. Eğer sorun basitse çözülür. Daha karmaşıksa ikinci seviye desteğe, gerekirse uzman ekibe aktarılır.

Bu örnekte:

- ◇ `SupportHandler`, ortak işleyici sınıfıdır.
- ◇ `LevelOneSupport`, basit sorunları çözer.
- ◇ `LevelTwoSupport`, orta seviyeli sorunları çözer.
- ◇ `ExpertSupport`, kritik sorunları çözer.

Önce destek talebini temsil eden sınıf yazılır:

```
class SupportRequest {
  private final String description;
  private final int severity;

  public SupportRequest(String description, int severity) {
    this.description = description;
    this.severity = severity;
  }

  public String getDescription() {
    return description;
  }

  public int getSeverity() {
    return severity;
  }
}
```

Kod 5.114: Java Dilinde SupportRequest Sınıfı

Ortak destek işleyicisi:

```
abstract class SupportHandler {
  protected SupportHandler next;

  public void setNext(SupportHandler next) {
    this.next = next;
```

```
    }

    public abstract void handle(SupportRequest request);
}
```

Kod 5.115: Java Dilinde SupportHandler Sınıfı

Birinci seviye destek:

```
class LevelOneSupport extends SupportHandler {
    @Override
    public void handle(SupportRequest request) {
        if (request.getSeverity() <= 1) {
            System.out.println("1. Seviye Destek çözdü: "
            + request.getDescription());
        } else if (next != null) {
            System.out.println("1. Seviye Destek: Sorun daha karmaşık, aktarıldı.");
            next.handle(request);
        }
    }
}
```

Kod 5.116: Java Dilinde Birinci Seviye Destek

İkinci seviye destek:

```
class LevelTwoSupport extends SupportHandler {
    @Override
    public void handle(SupportRequest request) {
        if (request.getSeverity() <= 3) {
            System.out.println("2. Seviye Destek çözdü: "
            + request.getDescription());
        } else if (next != null) {
            System.out.println("2. Seviye Destek: Uzman ekibe aktarıldı.");
            next.handle(request);
        }
    }
}
```

Kod 5.117: Java Dilinde İkinci Seviye Destek

Uzman destek:

```
class ExpertSupport extends SupportHandler {
    @Override
    public void handle(SupportRequest request) {
        if (request.getSeverity() <= 5) {
            System.out.println("Uzman Destek çözdü: "
            + request.getDescription());
        } else {
            System.out.println("Sorun çok kritik, yönetim ekibine aktarılmalı: "
            + request.getDescription());
        }
    }
}
```

Kod 5.118: Java Dilinde Uzman Destek

İstemci kod:

```
public class Main {
    public static void main(String[] args) {
        SupportHandler levelOne = new LevelOneSupport();
        SupportHandler levelTwo = new LevelTwoSupport();
        SupportHandler expert = new ExpertSupport();

        levelOne.setNext(levelTwo);
        levelTwo.setNext(expert);
```

```
    SupportRequest r1 = new SupportRequest("Şifre sıfırlama", 1);
    SupportRequest r2 = new SupportRequest("Uygulama hatası", 3);
    SupportRequest r3 = new SupportRequest("Sunucu çökmesi", 5);
    SupportRequest r4 = new SupportRequest("Veri merkezi arızası", 10);

    levelOne.handle(r1);
    levelOne.handle(r2);
    levelOne.handle(r3);
    levelOne.handle(r4);
  }
}
```

Kod 5.119: Java Dilinde Destek Zinciri Kullanımı

Program çıktısı:

```
1. Seviye Destek çözdü: Şifre sıfırlama
1. Seviye Destek: Sorun daha karmaşık, aktarıldı.
2. Seviye Destek çözdü: Uygulama hatası
1. Seviye Destek: Sorun daha karmaşık, aktarıldı.
2. Seviye Destek: Uzman ekibe aktarıldı.
Uzman Destek çözdü: Sunucu çökmesi
1. Seviye Destek: Sorun daha karmaşık, aktarıldı.
2. Seviye Destek: Uzman ekibe aktarıldı.
Sorun çok kritik, yönetim ekibine aktarılmalı: Veri merkezi arızası
```

Zincirin Kurulumu İçin Alternatif Yaklaşım

Java'da zinciri daha akıcı kurmak için `setNext()` metodunun geriye bir sonraki işleyiciyi döndürmesi sağlanabilir.

```
abstract class Handler {
  protected Handler next;

  public Handler setNext(Handler next) {
    this.next = next;
    return next;
  }

  public abstract void handleRequest(int request);
}
```

Bu durumda zincir şu şekilde kurulabilir:

```
Handler memur = new MemurHandler();
Handler sef = new SefHandler();
Handler mudur = new MudurHandler();

memur.setNext(sef).setNext(mudur);
```

Bu yaklaşım özellikle uzun zincirlerde daha okunabilir olabilir.

Sorumluluk Zinciri Kalıbının Avantajları

- İsteği gönderen ile isteği işleyen arasındaki bağı azaltır.
- Zincire yeni işleyiciler kolayca eklenebilir.
- İşleme sırası çalışma zamanında değiştirilebilir.
- Her işleyici yalnızca kendi sorumluluk alanına odaklanır.
- Tek bir büyük koşul bloğu yerine ayrık sınıflar kullanılmasını sağlar.

Sorumluluk Zinciri Kalıbının Dezavantajları

- İstek zincirde hiç işlenmeden sona erebilir.
- Uzun zincirlerde hata ayıklama zorlaşabilir.
- Zincirdeki sıranın yanlış kurulması beklenmeyen sonuçlara yol açabilir.

◇ Çok basit durumlarda gereksiz sınıf sayısı oluşturabilir.

Ne Zaman Kullanılmalı?

Sorumluluk zinciri kalıbı şu durumlarda tercih edilebilir:

◇ Bir isteği birden fazla nesneden biri işleyebilecekse.
◇ İsteği gönderen nesnenin, isteği kimin işleyeceğini bilmesi istenmiyorsa.
◇ İşleyiciler dinamik olarak eklenip çıkarılacaksa.
◇ Uzun `if-else` veya `switch` blokları yerine daha esnek bir yapı isteniyorsa.
◇ Onay süreçleri, filtreleme, destek talepleri veya hata işleme zincirleri tasarlanıyorsa.

Özet

Sorumluluk zinciri kalıbı, bir isteğin zincirdeki işleyiciler arasında ilerlemesini sağlar. Her işleyici isteği ya işler ya da bir sonraki işleyiciye aktarır. Bu sayede isteği gönderen ile isteği işleyen arasındaki bağımlılık azalır. Özellikle onay mekanizmaları, destek sistemleri, filtre zincirleri ve istek işleme yapılarında kullanışlıdır.

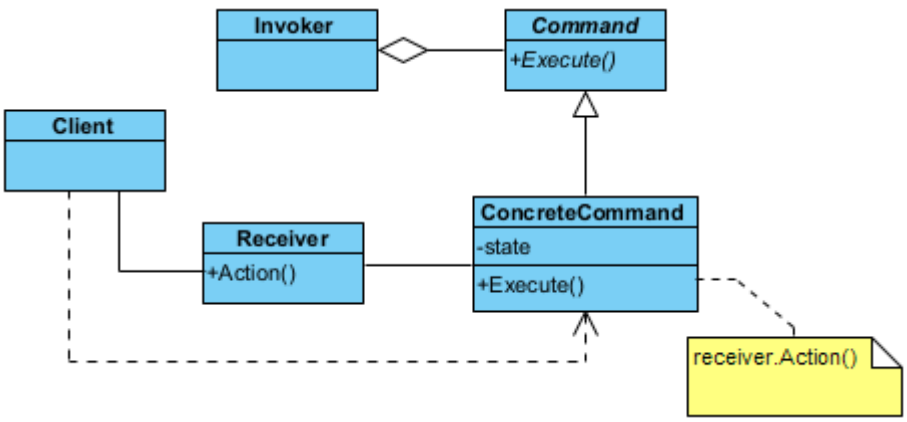
§5.4.2 Komut

Komut Kalıbı (*Command Pattern*), bir isteği veya işlemi nesne haline getiren davranışsal (*behavioral*) bir tasarım kalıbıdır. Bu kalıp sayesinde bir işlem, doğrudan çalıştırılmak yerine bir komut nesnesi içine sarılır.

Komut kalıbının temel amacı, **isteği gönderen nesne** ile **isteği gerçekten yerine getiren nesne** arasındaki bağı azaltmaktır. Böylece bir işlem daha sonra çalıştırılabilir, kuyruğa alınabilir, günlüğe yazılabilir veya geri alınabilir.

Başka bir ifadeyle komut kalıbı, "ne yapılacağı" bilgisini bir nesneye dönüştürür. Bu sayede işlemler tıpkı normal nesneler gibi saklanabilir, taşınabilir ve yönetilebilir.

Örneğin bir uzaktan kumandayı düşünelim. Kumanda üzerindeki düğme, ışığın nasıl açılacağını bilmek zorunda değildir. Düğmeye basıldığında yalnızca kendisine atanmış komutu çalıştırır. Işığı açma veya kapatma işinin nasıl yapılacağını ise alıcı nesne bilir.



Şekil 5.14: Komut Kalıbı UML Diyagramı

Komut kalıbındaki temel roller şunlardır:

◇ **Komut** (`Command`): Tüm komutların uyması gereken ortak arayüzdür. Genellikle `execute()` metodunu içerir. Geri alma desteği gerekiyorsa `undo()` metodu da eklenebilir.
◇ **Somut Komut** (`ConcreteCommand`): Belirli bir işlemi temsil eden komut sınıfıdır. Alıcı nesneyi tanır ve `execute()` çağrıldığında alıcının ilgili metodunu çalıştırır.
◇ **Alıcı** (`Receiver`): İşin asıl yapıldığı sınıftır. Gerçek iş mantığı burada bulunur.
◇ **Çağırıcı** (`Invoker`): Komutu saklayan ve uygun zamanda çalıştıran sınıftır. Komutun ne yaptığını veya hangi alıcıyı kullandığını bilmek zorunda değildir.

◇ **İstemci** (`Client`): Alıcıyı, komutları ve çağırıcıyı oluşturup birbirine bağlayan koddur.

Java ile Temel Komut Örneği

Aşağıdaki örnekte bir `Receiver` sınıfı vardır. Bu sınıf asıl işi yapan sınıftır. Komut nesneleri ise bu alıcının metotlarını çalıştırır.

Önce alıcı sınıfı yazılır:

```
class Receiver {
  public void action(String param) {
    System.out.println("Receiver: [" + param
    + "] komutu için temel işlemler yapılıyor.");
  }

  public void actionA(String param) {
    System.out.println("Receiver: [" + param + "] için A işlemi.");
  }

  public void actionB(String param) {
    System.out.println("Receiver: [" + param + "] için B işlemi.");
  }
}
```

Kod 5.120: Java Dilinde Receiver Sınıfı

Şimdi tüm komutların uyması gereken arayüz tanımlanır:

```
interface Command {
  void execute();
}
```

Kod 5.121: Java Dilinde Command Arayüzü

Basit bir komut sınıfı:

```
class SimpleCommand implements Command {
  private final String state;
  private final Receiver receiver;

  public SimpleCommand(String state, Receiver receiver) {
    this.state = state;
    this.receiver = receiver;
  }

  @Override
  public void execute() {
    receiver.action(state);
  }
}
```

Kod 5.122: Java Dilinde SimpleCommand Sınıfı

Daha karmaşık bir komut sınıfı:

```
class ComplexCommand implements Command {
  private final String state;
  private final Receiver receiver;

  public ComplexCommand(String state, Receiver receiver) {
    this.state = state;
    this.receiver = receiver;
  }

  @Override
  public void execute() {
    receiver.actionA(state);
    receiver.actionB(state);
  }
```

```
}
```

Kod 5.123: Java Dilinde ComplexCommand Sınıfı

Komutları saklayan ve çalıştıran çağırıcı sınıf:

```
import java.util.ArrayList;
import java.util.List;

class Invoker {
  private final List<Command> history = new ArrayList<>();

  public void storeAndExecute(Command command) {
    command.execute();
    history.add(command);
  }
}
```

Kod 5.124: Java Dilinde Invoker Sınıfı

İstemci kod:

```
public class Main {
  public static void main(String[] args) {
    Receiver receiver = new Receiver();
    Invoker invoker = new Invoker();

    System.out.println("--- Komutlar Tetikleniyor ---");

    invoker.storeAndExecute(new SimpleCommand("Yazdır", receiver));
    invoker.storeAndExecute(new ComplexCommand("Logla ve Gönder", receiver));
  }
}
```

Kod 5.125: Java Dilinde Komut Kalıbı Kullanımı

Program çıktısı:

```
--- Komutlar Tetikleniyor ---
Receiver: [Yazdır] komutu için temel işlemler yapılıyor.
Receiver: [Logla ve Gönder] için A işlemi.
Receiver: [Logla ve Gönder] için B işlemi.
```

Bu örnekte `Invoker`, komutun içinde hangi işlemlerin yapıldığını bilmez. Yalnızca `execute()` metodunu çağırır.

Gerçek Dünya Örneği: Işık Açma ve Kapatma

Komut kalıbının en anlaşılır örneklerinden biri uzaktan kumanda ve ışık örneğidir. Kumanda, ışığın nasıl açılıp kapatıldığını bilmez. Yalnızca kendisine verilen komutları çalıştırır.

Bu örnekte:

- `Light`, alıcı sınıftır.
- `Command`, komut arayüzüdür.
- `LightOnCommand`, ışığı açma komutudur.
- `LightOffCommand`, ışığı kapatma komutudur.
- `RemoteControl`, çağırıcı sınıftır.

Önce alıcı sınıf yazılır:

```
class Light {
  public void on() {
    System.out.println("Işık AÇILDI.");
  }

  public void off() {
```

```
    System.out.println("Işık KAPATILDI.");
  }
}
```

Kod 5.126: Java Dilinde Light Alıcı Sınıfı

Geri alma desteği de olan komut arayüzü:

```
interface Command {
  void execute();
  void undo();
}
```

Kod 5.127: Java Dilinde Undo Destekli Command Arayüzü

Işığı açan komut:

```
class LightOnCommand implements Command {
  private final Light light;

  public LightOnCommand(Light light) {
    this.light = light;
  }

  @Override
  public void execute() {
    light.on();
  }

  @Override
  public void undo() {
    light.off();
  }
}
```

Kod 5.128: Java Dilinde LightOnCommand Sınıfı

Işığı kapatan komut:

```
class LightOffCommand implements Command {
  private final Light light;

  public LightOffCommand(Light light) {
    this.light = light;
  }

  @Override
  public void execute() {
    light.off();
  }

  @Override
  public void undo() {
    light.on();
  }
}
```

Kod 5.129: Java Dilinde LightOffCommand Sınıfı

Komutları çalıştıran ve geçmişi tutan kumanda sınıfı:

```
import java.util.Stack;

class RemoteControl {
  private final Stack<Command> history = new Stack<>();

  public void executeCommand(Command command) {
    command.execute();
    history.push(command);
```

```
  }

  public void undoLast() {
    if (!history.isEmpty()) {
      System.out.print("[UNDO] ");

      Command lastCommand = history.pop();
      lastCommand.undo();
    } else {
      System.out.println("[UYARI] Geri alınacak işlem yok!");
    }
  }
}
```

Kod 5.130: Java Dilinde RemoteControl Sınıfı

İstemci kod:

```
public class Main {
  public static void main(String[] args) {
    Light oturmaOdasiIsigi = new Light();
    RemoteControl kumanda = new RemoteControl();

    System.out.println("--- İşlemler Başlıyor ---");

    kumanda.executeCommand(new LightOnCommand(oturmaOdasiIsigi));
    kumanda.executeCommand(new LightOffCommand(oturmaOdasiIsigi));

    System.out.println();
    System.out.println("--- Geri Alma İşlemleri ---");

    kumanda.undoLast();
    kumanda.undoLast();
    kumanda.undoLast();
  }
}
```

Kod 5.131: Java Dilinde Işık Komutları Kullanımı

Program çıktısı:

```
--- İşlemler Başlıyor ---
Işık AÇILDI.
Işık KAPATILDI.

--- Geri Alma İşlemleri ---
[UNDO] Işık AÇILDI.
[UNDO] Işık KAPATILDI.
[UYARI] Geri alınacak işlem yok!
```

Bu örnekte `LightOffCommand` komutunun geri alma işlemi ışığı açmaktır. Benzer şekilde `LightOnCommand` komutunun geri alma işlemi ışığı kapatmaktır.

Tam Java Örneği

Aşağıda ışık açma-kapatma örneğinin tek dosyada çalıştırılabilecek tam hali verilmiştir:

```
import java.util.Stack;

class Light {
  public void on() {
    System.out.println("Işık AÇILDI.");
  }

  public void off() {
    System.out.println("Işık KAPATILDI.");
  }
```

```
}

interface Command {
  void execute();
  void undo();
}

class LightOnCommand implements Command {
  private final Light light;

  public LightOnCommand(Light light) {
    this.light = light;
  }

  @Override
  public void execute() {
    light.on();
  }

  @Override
  public void undo() {
    light.off();
  }
}

class LightOffCommand implements Command {
  private final Light light;

  public LightOffCommand(Light light) {
    this.light = light;
  }

  @Override
  public void execute() {
    light.off();
  }

  @Override
  public void undo() {
    light.on();
  }
}

class RemoteControl {
  private final Stack<Command> history = new Stack<>();

  public void executeCommand(Command command) {
    command.execute();
    history.push(command);
  }

  public void undoLast() {
    if (!history.isEmpty()) {
      System.out.print("[UNDO] ");

      Command lastCommand = history.pop();
      lastCommand.undo();
    } else {
      System.out.println("[UYARI] Geri alınacak işlem yok!");
    }
  }
}

public class Main {
  public static void main(String[] args) {
    Light oturmaOdasiIsigi = new Light();
```

```
    RemoteControl kumanda = new RemoteControl();

    System.out.println("--- İşlemler Başlıyor ---");

    kumanda.executeCommand(new LightOnCommand(oturmaOdasiIsigi));
    kumanda.executeCommand(new LightOffCommand(oturmaOdasiIsigi));

    System.out.println();
    System.out.println("--- Geri Alma İşlemleri ---");

    kumanda.undoLast();
    kumanda.undoLast();
    kumanda.undoLast();
  }
}
```

Kod 5.132: Java Dilinde Komut Kalıbı Işık Örneği

Komut Kuyruğu

Komut kalıbı, komutların hemen çalıştırılması yerine daha sonra çalıştırılmasını da sağlar. Bunun için komutlar bir kuyruğa alınabilir.

```
import java.util.LinkedList;
import java.util.Queue;

class CommandQueue {
  private final Queue<Command> queue = new LinkedList<>();

  public void addCommand(Command command) {
    queue.add(command);
  }

  public void executeAll() {
    while (!queue.isEmpty()) {
      Command command = queue.poll();
      command.execute();
    }
  }
}
```

Kod 5.133: Java Dilinde Komut Kuyruğu

Kullanımı:

```
Light light = new Light();

CommandQueue queue = new CommandQueue();

queue.addCommand(new LightOnCommand(light));
queue.addCommand(new LightOffCommand(light));

queue.executeAll();
```

Bu yaklaşım, görev planlama, toplu işlem yürütme veya arka plan işleri için kullanışlıdır.

Komut Kalıbının Avantajları

- ◇ İsteği gönderen ile isteği işleyen nesne arasındaki bağı azaltır.
- ◇ İşlemler nesne haline getirildiği için saklanabilir, taşınabilir ve kuyruğa alınabilir.
- ◇ Geri alma (*undo*) ve tekrar yapma (*redo*) mekanizmaları kurulabilir.
- ◇ Komut geçmişi tutulabilir.
- ◇ Yeni komutlar eklemek kolaydır.
- ◇ Açık-kapalı ilkesine uygun bir yapı sağlar.

Komut Kalıbının Dezavantajları

- ◇ Her işlem için ayrı komut sınıfı gerekebileceğinden sınıf sayısı artabilir.
- ◇ Basit işlemler için gereksiz karmaşıklık oluşturabilir.
- ◇ Geri alma işlemleri karmaşık nesne durumlarında dikkatli tasarlanmalıdır.

Ne Zaman Kullanılmalı?

Komut kalıbı şu durumlarda tercih edilebilir:

- ◇ Bir işlem nesne olarak temsil edilmek isteniyorsa.
- ◇ İşlemler kuyruğa alınacaksa.
- ◇ İşlem geçmişi tutulacaksa.
- ◇ Geri alma ve tekrar yapma desteği gerekiyorsa.
- ◇ İsteği veren ile isteği işleyen nesne arasındaki bağı azaltmak isteniyorsa.
- ◇ Menü seçenekleri, düğmeler, makrolar veya görev planlayıcılar tasarlanıyorsa.

Özet

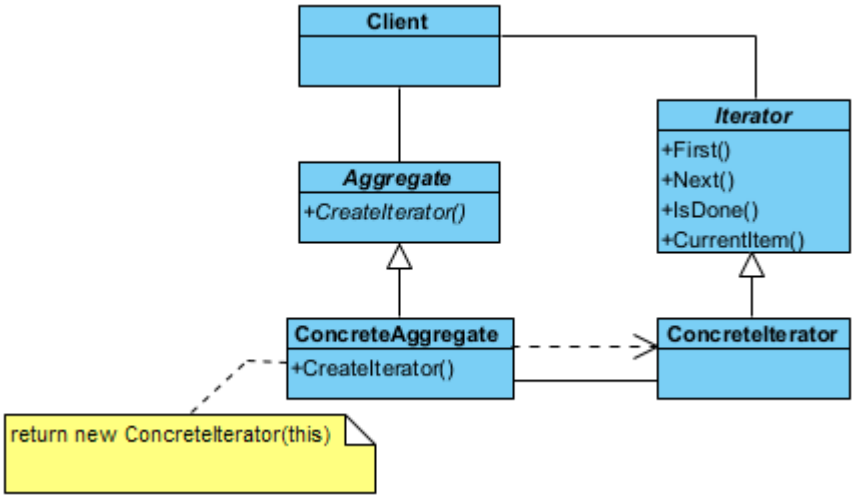
Komut kalıbı, bir isteği veya işlemi nesne haline getirir. Böylece işlemler saklanabilir, kuyruğa alınabilir, daha sonra çalıştırılabilir veya geri alınabilir. Bu kalıp, isteği gönderen nesne ile işi gerçekten yapan nesne arasındaki bağı azaltır. Özellikle uzaktan kumanda, görev kuyruğu, menü komutları, işlem geçmişi ve geri alma mekanizmalarında oldukça kullanışlıdır.

§5.4.3 Yineleyici

Yineleyici Kalıbı (*Iterator Pattern*), bir koleksiyonun veya veri grubunun iç yapısını dışarıya açmadan, elemanlarına sırayla erişmeyi sağlayan davranışsal (*behavioral*) bir tasarım kalıbıdır.

Bu kalıbın temel amacı, istemci kodun koleksiyonun nasıl saklandığını bilmeden elemanlar üzerinde gezinebilmesini sağlamaktır. Koleksiyonun arka planda dizi, liste, ağaç, yığın veya başka bir veri yapısı kullanması istemci açısından önemli değildir. İstemci yalnızca yineleyici arayüzü üzerinden elemanlara erişir.

Başka bir ifadeyle yineleyici kalıbı, veri yapısı ile veri yapısı üzerinde gezinme işlemini birbirinden ayırır.



Şekil 5.15: Yineleyici Kalıbı UML Diyagramı

Yineleyici kalıbındaki temel roller şunlardır:

- ◇ **Yineleyici** (`Iterator`): Elemanlar üzerinde gezinmek için gerekli ortak arayüzü tanımlar. Genellikle `hasNext()`, `next()` ve bazen `currentItem()` gibi metotlar içerir.
- ◇ **Somut Yineleyici** (`ConcreteIterator`): Belirli bir koleksiyon türü için gezinme mantığını uygular. O anda hangi elemanda olduğunu takip eder.

- ◇ **Soyut Koleksiyon** (`Aggregate`): Koleksiyonların ortak arayüzünü tanımlar. Genellikle kendi elemanları üzerinde gezinmek için bir yineleyici döndüren `createIterator()` metoduna sahiptir.
- ◇ **Somut Koleksiyon** (`ConcreteAggregate`): Verileri asıl saklayan sınıftır. Kendi iç yapısına uygun bir somut yineleyici oluşturur.

Java ile Temel Yineleyici Örneği

Java dilinde `Iterator` arayüzü zaten standart kütüphanede bulunur. Ancak tasarım kalıbını daha iyi anlamak için önce kendi basit yineleyici arayüzümüzü tanımlayalım.

```
interface Iterator {
  double first();
  double next();
  boolean hasNext();
  double currentItem();
}
```

Kod 5.134: Java Dilinde Iterator Arayüzü

Bu arayüzdeki metotların görevleri şöyledir:

- ◇ `first()`: Yineleyiciyi ilk elemana götürür.
- ◇ `next()`: Sonraki elemanı döndürür.
- ◇ `hasNext()`: Gezilecek başka eleman olup olmadığını kontrol eder.
- ◇ `currentItem()`: O anki elemanı döndürür.

Şimdi koleksiyon arayüzünü tanımlayalım:

```
interface Aggregate {
  void addItem(double item);
  int size();
  double getItem(int index);
  Iterator createIterator();
}
```

Kod 5.135: Java Dilinde Aggregate Arayüzü

Bu arayüz, koleksiyonun eleman ekleyebilmesini, eleman sayısını döndürebilmesini ve kendi yineleyicisini oluşturabilmesini sağlar.

Şimdi somut yineleyici sınıfını yazalım:

```
class ConcreteIterator implements Iterator {
  private final Aggregate aggregate;
  private int current = 0;

  public ConcreteIterator(Aggregate aggregate) {
    this.aggregate = aggregate;
  }

  @Override
  public double first() {
    current = 0;
    return currentItem();
  }

  @Override
  public double next() {
    if (hasNext()) {
      return aggregate.getItem(current++);
    }

    throw new java.util.NoSuchElementException("Gezilecek eleman kalmadı.");
  }
```

```
    @Override
    public boolean hasNext() {
      return current < aggregate.size();
    }

    @Override
    public double currentItem() {
      if (current < aggregate.size()) {
        return aggregate.getItem(current);
      }

      throw new java.util.NoSuchElementException("Geçerli eleman yok.");
    }
  }
```

Kod 5.136: Java Dilinde ConcreteIterator Sınıfı

Bu sınıf, hangi elemanda olduğunu `current` değişkeniyle takip eder. Koleksiyonun iç yapısını bilmek zorunda değildir; yalnızca `Aggregate` arayüzündeki metotları kullanır.

Şimdi somut koleksiyon sınıfını yazalım:

```
import java.util.ArrayList;
import java.util.List;

class ConcreteAggregate implements Aggregate {
  private final List<Double> items = new ArrayList<>();

  @Override
  public void addItem(double item) {
    items.add(item);
  }

  @Override
  public int size() {
    return items.size();
  }

  @Override
  public double getItem(int index) {
    return items.get(index);
  }

  @Override
  public Iterator createIterator() {
    return new ConcreteIterator(this);
  }
}
```

Kod 5.137: Java Dilinde ConcreteAggregate Sınıfı

İstemci kod:

```
public class Main {
  public static void main(String[] args) {
    Aggregate aggregate = new ConcreteAggregate();

    aggregate.addItem(10.5);
    aggregate.addItem(20.7);
    aggregate.addItem(30.9);

    Iterator iterator = aggregate.createIterator();

    System.out.println("Koleksiyon elemanları geziliyor:");

    for (iterator.first(); iterator.hasNext(); ) {
      System.out.println("Değer: " + iterator.next());
```

```
    }
  }
}
```

Kod 5.138: Java Dilinde Yineleyici Kullanımı

Program çıktısı:

```
Koleksiyon elemanları geziliyor:
Değer: 10.5
Değer: 20.7
Değer: 30.9
```

Bu örnekte istemci, koleksiyonun arka planda `ArrayList` kullandığını bilmek zorunda değildir. Yalnızca `Iterator` arayüzü üzerinden elemanları dolaşır.

Java'nın Hazır Iterator Arayüzü

Java'da yineleyici kalıbı standart kütüphanede yaygın olarak kullanılır. `java.util.Iterator` arayüzü bunun en bilinen örneğidir.

Örneğin bir `ArrayList` üzerinde şu şekilde gezilebilir:

```
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

public class Main {
  public static void main(String[] args) {
    List<String> isimler = new ArrayList<>();

    isimler.add("Ahmet");
    isimler.add("Mehmet");
    isimler.add("Ayşe");

    Iterator<String> iterator = isimler.iterator();

    while (iterator.hasNext()) {
      String isim = iterator.next();
      System.out.println(isim);
    }
  }
}
```

Kod 5.139: Java Dilinde Hazır Iterator Kullanımı

Program çıktısı:

```
Ahmet
Mehmet
Ayşe
```

Burada `ArrayList`, kendi iç yapısını dışarı açmadan bir `Iterator` nesnesi döndürür. İstemci yalnızca `hasNext()` ve `next()` metotlarını kullanır.

Gerçek Dünya Örneği: Kitaplık

Yineleyici kalıbını daha anlaşılır hale getirmek için bir kitaplık örneği düşünelim. Kitaplık içinde kitaplar bulunur. İstemci, kitapların hangi veri yapısında tutulduğunu bilmeden tüm kitapları gezebilmelidir.

Bu örnekte:

- `Book`, kitap nesnesidir.
- `BookCollection`, kitapları saklayan koleksiyondur.
- `BookIterator`, kitaplar üzerinde gezinmeyi sağlayan yineleyicidir.

Önce kitap sınıfı yazılır:

```
class Book {
  private final String title;

  public Book(String title) {
    this.title = title;
  }

  public String getTitle() {
    return title;
  }
}
```

Kod 5.140: Java Dilinde Book Sınıfı

Kitap yineleyici arayüzü:

```
interface BookIterator {
  boolean hasNext();
  Book next();
}
```

Kod 5.141: Java Dilinde BookIterator Arayüzü

Kitap koleksiyonu:

```
import java.util.ArrayList;
import java.util.List;

class BookCollection {
  private final List<Book> books = new ArrayList<>();

  public void addBook(Book book) {
    books.add(book);
  }

  public Book getBookAt(int index) {
    return books.get(index);
  }

  public int size() {
    return books.size();
  }

  public BookIterator iterator() {
    return new BookCollectionIterator(this);
  }
}
```

Kod 5.142: Java Dilinde BookCollection Sınıfı

Somut kitap yineleyicisi:

```
class BookCollectionIterator implements BookIterator {
  private final BookCollection collection;
  private int currentIndex = 0;

  public BookCollectionIterator(BookCollection collection) {
    this.collection = collection;
  }

  @Override
  public boolean hasNext() {
    return currentIndex < collection.size();
  }

  @Override
```

```
    public Book next() {
      if (!hasNext()) {
        throw new java.util.NoSuchElementException("Kitap kalmadı.");
      }

      return collection.getBookAt(currentIndex++);
    }
  }
```

Kod 5.143: Java Dilinde BookCollectionIterator Sınıfı

İstemci kod:

```
  public class Main {
    public static void main(String[] args) {
      BookCollection library = new BookCollection();

      library.addBook(new Book("Tasarım Kalıpları"));
      library.addBook(new Book("Temiz Kod"));
      library.addBook(new Book("Java Programlama"));

      BookIterator iterator = library.iterator();

      while (iterator.hasNext()) {
        Book book = iterator.next();

        System.out.println("Kitap: " + book.getTitle());
      }
    }
  }
```

Kod 5.144: Java Dilinde Kitaplık Yineleyici Kullanımı

Program çıktısı:

```
Kitap: Tasarım Kalıpları
Kitap: Temiz Kod
Kitap: Java Programlama
```

Bu örnekte istemci, kitapların `ArrayList` içinde tutulduğunu bilmez. Yalnızca `BookIterator` arayüzü üzerinden kitapları dolaşır.

Tam Java Örneği

Aşağıda kitaplık örneğinin tek dosyada çalıştırılabilecek tam hali verilmiştir:

```
  import java.util.ArrayList;
  import java.util.List;
  import java.util.NoSuchElementException;

  class Book {
    private final String title;

    public Book(String title) {
      this.title = title;
    }

    public String getTitle() {
      return title;
    }
  }

  interface BookIterator {
    boolean hasNext();
    Book next();
  }
```

```
class BookCollection {
  private final List<Book> books = new ArrayList<>();

  public void addBook(Book book) {
    books.add(book);
  }

  public Book getBookAt(int index) {
    return books.get(index);
  }

  public int size() {
    return books.size();
  }

  public BookIterator iterator() {
    return new BookCollectionIterator(this);
  }
}

class BookCollectionIterator implements BookIterator {
  private final BookCollection collection;
  private int currentIndex = 0;

  public BookCollectionIterator(BookCollection collection) {
    this.collection = collection;
  }

  @Override
  public boolean hasNext() {
    return currentIndex < collection.size();
  }

  @Override
  public Book next() {
    if (!hasNext()) {
      throw new NoSuchElementException("Kitap kalmadı.");
    }

    return collection.getBookAt(currentIndex++);
  }
}

public class Main {
  public static void main(String[] args) {
    BookCollection library = new BookCollection();

    library.addBook(new Book("Tasarım Kalıpları"));
    library.addBook(new Book("Temiz Kod"));
    library.addBook(new Book("Java Programlama"));

    BookIterator iterator = library.iterator();

    while (iterator.hasNext()) {
      Book book = iterator.next();

      System.out.println("Kitap: " + book.getTitle());
    }
  }
}
```

Kod 5.145: Java Dilinde Kitaplık Yineleyici Tam Örnek

Farklı Gezinme Biçimleri

Yineleyici kalıbı yalnızca baştan sona gezinmek için kullanılmaz. Aynı koleksiyon için farklı yineleyiciler de tanımlanabilir.

Örneğin:

- ◇ Baştan sona yineleyici
- ◇ Sondan başa yineleyici
- ◇ Sadece belirli koşulu sağlayan elemanları gezen yineleyici
- ◇ Ağaç yapısı için derinlik öncelikli veya genişlik öncelikli yineleyici

Bu sayede koleksiyon sınıfının kendisini değiştirmeden farklı gezinme stratejileri eklenebilir.

Yineleyici Kalıbının Avantajları

- ◇ Koleksiyonun iç yapısını gizler.
- ◇ Farklı veri yapıları üzerinde ortak gezinme arayüzü sağlar.
- ◇ Gezinme sorumluluğunu koleksiyon sınıfından ayırır.
- ◇ Aynı koleksiyon için birden fazla gezinme biçimi tanımlanabilir.
- ◇ İstemci kodun koleksiyonun nasıl saklandığını bilme ihtiyacını azaltır.

Yineleyici Kalıbının Dezavantajları

- ◇ Basit koleksiyonlarda gereksiz sınıf ve arayüz sayısı oluşturabilir.
- ◇ Koleksiyon üzerinde değişiklik yapılırken yineleyici kullanımı dikkat gerektirir.
- ◇ Çok özel gezinme ihtiyaçlarında yineleyici sınıfları karmaşıklaşabilir.

Ne Zaman Kullanılmalı?

Yineleyici kalıbı şu durumlarda tercih edilebilir:

- ◇ Bir koleksiyonun iç yapısı gizlenmek isteniyorsa.
- ◇ Farklı koleksiyon türleri üzerinde aynı gezinme arayüzü kullanılacaksa.
- ◇ Koleksiyon üzerinde birden fazla gezinme yöntemi tanımlanacaksa.
- ◇ İstemci kodun veri yapısına bağımlılığı azaltılmak isteniyorsa.
- ◇ Liste, ağaç, yığın, kuyruk veya özel koleksiyon sınıfları tasarlanıyorsa.

Özet

Yineleyici kalıbı, bir koleksiyonun iç yapısını açığa çıkarmadan elemanları üzerinde sırayla gezmeyi sağlar. İstemci, koleksiyonun verileri nasıl sakladığını bilmeden standart bir arayüz üzerinden elemanlara erişir. Java'daki `Iterator` arayüzü bu kalıbın doğrudan bir örneğidir.

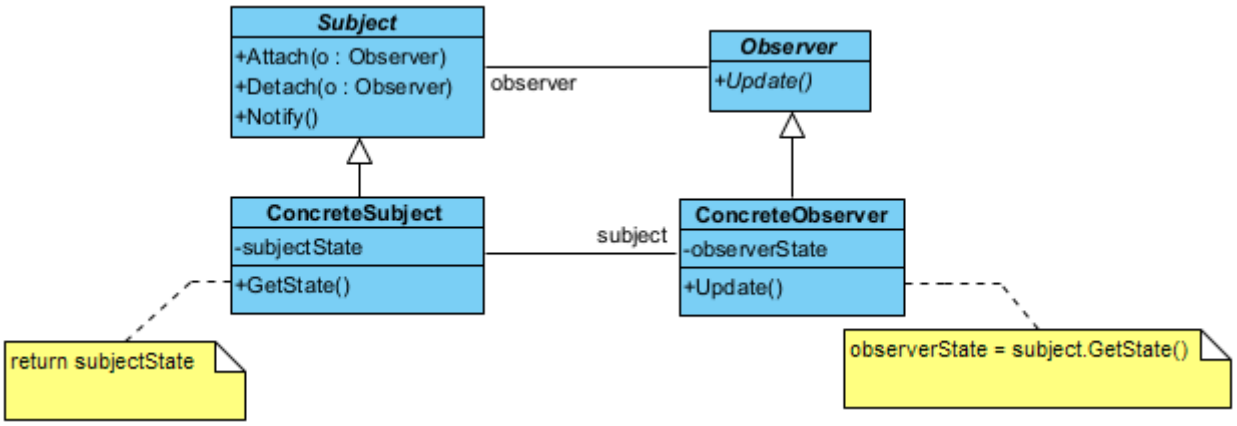
§5.4.4 Gözlemci

Gözlemci Kalıbı (*Observer Pattern*), bir nesnenin durumunda meydana gelen değişikliklerden, ona bağlı olan diğer nesnelerin otomatik olarak haberdar edilmesini sağlayan davranışsal (*behavioral*) bir tasarım kalıbıdır.

Bu kalıp, nesneler arasında bire-çok (*one-to-many*) ilişki kurar. Bir nesnenin durumu değiştiğinde, bu nesneye abone olan tüm gözlemciler bilgilendirilir. Böylece gözlemciler, yayıncı nesnenin durum değişikliğine göre kendilerini güncelleyebilir.

Gözlemci kalıbı özellikle olay güdümlü (*event-driven*) sistemlerde, kullanıcı arayüzlerinde, bildirim sistemlerinde, haber aboneliklerinde ve MVC (*Model-View-Controller*) mimarisinde yaygın olarak kullanılır.

Günlük hayattan bir örnek vermek gerekirse, bir haber ajansına abone olan haber kanallarını düşünebiliriz. Haber ajansı yeni bir haber yayınladığında, ona abone olan tüm kanallar bu haberden haberdar olur. Haber ajansı her kanalı tek tek özel olarak yönetmek zorunda değildir; yalnızca abonelerine bildirim gönderir.



Şekil 5.16: Gözlemci Kalıbı UML Diyagramı

Gözlemci kalıbındaki temel roller şunlardır:

- **Bağlam veya Yayıncı** (`Subject`): Takip edilen asıl nesnedir. İçinde gözlemcilerin listesini tutar. Gözlemci ekleme, çıkarma ve gözlemcilere bildirim gönderme işlemlerini yönetir.
- **Gözlemci** (`Observer`): Yayıncıdan gelen güncelleme mesajlarını alabilmek için gerekli ortak arayüzü tanımlar. Genellikle `update()` adlı bir metot içerir.
- **Somut Yayıncı** (`ConcreteSubject`): Gerçek durumu veya veriyi saklayan sınıftır. Durumu değiştiğinde gözlemcilere haber verir.
- **Somut Gözlemci** (`ConcreteObserver`): Yayıncıdan gelen bildirimi alan ve buna göre işlem yapan sınıftır. Örneğin ekrana yazabilir, veritabanına kaydedebilir veya başka bir işlem başlatabilir.

Java ile Temel Gözlemci Örneği

Aşağıdaki örnekte bir haber ajansı ve bu ajansa abone olan haber kanalları bulunmaktadır. Haber ajansı yeni bir haber yayınladığında tüm abonelerine bildirim gönderir.

Önce gözlemci arayüzünü tanımlayalım:

```
interface Observer {
  void update(int state);
}
```

Kod 5.146: Java Dilinde Observer Arayüzü

Bu arayüz, gözlemcilerin yayıncıdan gelen güncellemeleri alabilmesini sağlar.

Şimdi yayıncı sınıfını yazalım:

```
import java.util.ArrayList;
import java.util.List;

class Subject {
  private final List<Observer> observers = new ArrayList<>();

  public void attach(Observer observer) {
    observers.add(observer);
  }

  public void detach(Observer observer) {
    observers.remove(observer);
  }

  protected void notifyObservers(int state) {
    for (Observer observer : observers) {
      observer.update(state);
    }
```

```
    }
}
```

Kod 5.147: Java Dilinde Subject Sınıfı

`Subject` sınıfı, gözlemcileri liste içinde tutar. Yeni gözlemci eklemek için `attach()`, gözlemci çıkarmak için `detach()` ve tüm gözlemcilere haber vermek için `notifyObservers()` metodunu kullanır.

Şimdi somut yayıncı sınıfı yazılır:

```
class NewsAgency extends Subject {
  private int newsId;

  public void setNews(int id) {
    this.newsId = id;

    System.out.println("Haber Ajansı: Yeni haber yayınlandı. ID: " + id);

    notifyObservers(newsId);
  }
}
```

Kod 5.148: Java Dilinde NewsAgency Sınıfı

`NewsAgency` sınıfı, yeni bir haber geldiğinde kendi durumunu değiştirir ve ardından tüm gözlemcilere bildirim gönderir.

Şimdi somut gözlemci sınıfı yazalım:

```
class NewsChannel implements Observer {
  private final String name;

  public NewsChannel(String name) {
    this.name = name;
  }

  @Override
  public void update(int state) {
    System.out.println(" -> " + name
    + " kanalı bilgiyi aldı. Yeni haber ID: " + state);
  }
}
```

Kod 5.149: Java Dilinde NewsChannel Sınıfı

İstemci kod:

```
public class Main {
  public static void main(String[] args) {
    NewsAgency agency = new NewsAgency();

    Observer channel1 = new NewsChannel("TRT Haber");
    Observer channel2 = new NewsChannel("NTV");
    Observer channel3 = new NewsChannel("CNN Türk");

    agency.attach(channel1);
    agency.attach(channel2);
    agency.attach(channel3);

    agency.setNews(101);

    System.out.println("-------------------------------------");

    agency.detach(channel2);

    agency.setNews(102);
  }
```

```
}
```

Kod 5.150: Java Dilinde Gözlemci Kullanımı

Program çıktısı:

```
Haber Ajansı: Yeni haber yayınlandı. ID: 101
-> TRT Haber kanalı bilgiyi aldı. Yeni haber ID: 101
-> NTV kanalı bilgiyi aldı. Yeni haber ID: 101
-> CNN Türk kanalı bilgiyi aldı. Yeni haber ID: 101
-----------------------------------
Haber Ajansı: Yeni haber yayınlandı. ID: 102
-> TRT Haber kanalı bilgiyi aldı. Yeni haber ID: 102
-> CNN Türk kanalı bilgiyi aldı. Yeni haber ID: 102
```

Bu örnekte `NTV` kanalı ikinci haberden önce abonelikten çıkarılmıştır. Bu nedenle `102` numaralı haber yalnızca `TRT Haber` ve `CNN Türk` kanallarına gönderilir.

Gerçek Dünya Örneği: Hava Durumu İstasyonu

Gözlemci kalıbının yaygın bir örneği hava durumu istasyonudur. Hava durumu istasyonu sıcaklık, nem veya basınç gibi değerleri ölçer. Bu değerler değiştiğinde, istasyona bağlı ekranlar veya uygulamalar otomatik olarak güncellenir.

Bu örnekte:

- `WeatherStation`, yayıncı sınıftır.
- `WeatherObserver`, gözlemci arayüzüdür.
- `PhoneDisplay` ve `WebDisplay`, somut gözlemcilerdir.

Önce gözlemci arayüzü tanımlanır:

```
interface WeatherObserver {
  void update(double temperature, double humidity);
}
```

Kod 5.151: Java Dilinde WeatherObserver Arayüzü

Şimdi hava durumu istasyonu yazılır:

```
import java.util.ArrayList;
import java.util.List;

class WeatherStation {
  private final List<WeatherObserver> observers = new ArrayList<>();

  private double temperature;
  private double humidity;

  public void attach(WeatherObserver observer) {
    observers.add(observer);
  }

  public void detach(WeatherObserver observer) {
    observers.remove(observer);
  }

  public void setMeasurements(double temperature, double humidity) {
    this.temperature = temperature;
    this.humidity = humidity;

    System.out.println("Hava durumu güncellendi.");

    notifyObservers();
  }

  private void notifyObservers() {
```

```
    for (WeatherObserver observer : observers) {
      observer.update(temperature, humidity);
    }
  }
}
```

Kod 5.152: Java Dilinde WeatherStation Sınıfı

Telefon ekranı:

```
class PhoneDisplay implements WeatherObserver {
  @Override
  public void update(double temperature, double humidity) {
    System.out.println("Telefon Ekranı -> Sıcaklık: " + temperature
    + "Santigrat C, Nem: %" + humidity);
  }
}
```

Kod 5.153: Java Dilinde PhoneDisplay Sınıfı

Web ekranı:

```
class WebDisplay implements WeatherObserver {
  @Override
  public void update(double temperature, double humidity) {
    System.out.println("Web Ekranı -> Sıcaklık: " + temperature
    + "Santigrat C, Nem: %" + humidity);
  }
}
```

Kod 5.154: Java Dilinde WebDisplay Sınıfı

İstemci kod:

```
public class Main {
  public static void main(String[] args) {
    WeatherStation station = new WeatherStation();

    WeatherObserver phoneDisplay = new PhoneDisplay();
    WeatherObserver webDisplay = new WebDisplay();

    station.attach(phoneDisplay);
    station.attach(webDisplay);

    station.setMeasurements(24.5, 60);

    System.out.println();

    station.detach(phoneDisplay);

    station.setMeasurements(26.0, 55);
  }
}
```

Kod 5.155: Java Dilinde Hava Durumu Gözlemci Kullanımı

Program çıktısı:

```
Hava durumu güncellendi.
Telefon Ekranı -> Sıcaklık: 24.5 Santigrat C, Nem: %60.0
Web Ekranı -> Sıcaklık: 24.5 Santigrat C, Nem: %60.0

Hava durumu güncellendi.
Web Ekranı -> Sıcaklık: 26.0  Santigrat C, Nem: %55.0
```

Bu örnekte ikinci güncellemeden önce telefon ekranı abonelikten çıkarılmıştır. Bu nedenle ikinci bildirim yalnızca web ekranına gönderilir.

Push ve Pull Yaklaşımı

Gözlemci kalıbında gözlemcilere bilgi göndermenin iki yaygın yolu vardır:

- **Push Modeli**: Yayıncı, değişen veriyi doğrudan gözlemcilere gönderir. Örneğin `update(int state)` veya `update(double temperature, double humidity)` gibi.
- **Pull Modeli**: Yayıncı, gözlemcilere yalnızca "değişiklik oldu" mesajı gönderir. Gözlemci ihtiyaç duyduğu veriyi yayıncıdan kendisi çeker.

Örneğin push modelinde:

```
observer.update(temperature, humidity);
```

Pull modelinde ise gözlemciye yayıncı referansı verilebilir:

```
observer.update(this);
```

Bu durumda gözlemci, yayıncıdan gerekli veriyi kendi alır.

Pull Modeline Kısa Örnek

Aşağıda gözlemcinin veriyi yayıncıdan çektiği basit bir yapı verilmiştir:

```
interface StockObserver {
  void update(StockMarket market);
}

class StockMarket {
  private final java.util.List<StockObserver> observers =
  new java.util.ArrayList<>();

  private double price;

  public void attach(StockObserver observer) {
    observers.add(observer);
  }

  public double getPrice() {
    return price;
  }

  public void setPrice(double price) {
    this.price = price;

    for (StockObserver observer : observers) {
      observer.update(this);
    }
  }
}

class Investor implements StockObserver {
  private final String name;

  public Investor(String name) {
    this.name = name;
  }

  @Override
  public void update(StockMarket market) {
    System.out.println(name + " yeni fiyatı gördü: "
    + market.getPrice());
  }
}
```

Kod 5.156: Java Dilinde Pull Modeli Kısa Örnek

Bu yaklaşımda yayıncı tüm verileri parametre olarak göndermek zorunda kalmaz. Gözlemci

ihtiyaç duyduğu veriyi yayıncı üzerinden alır.

Gözlemci Kalıbı ve MVC

Gözlemci kalıbı, MVC mimarisinde önemli bir role sahiptir.

- **Model**: Veriyi ve iş mantığını tutar.
- **View**: Modeldeki değişiklikleri kullanıcıya gösterir.
- **Controller**: Kullanıcı etkileşimlerini alır ve modeli günceller.

Model değiştiğinde, ona bağlı olan görünümler (*views*) otomatik olarak güncellenebilir. Bu yapı gözlemci kalıbının doğal bir uygulamasıdır.

Gözlemci Kalıbının Avantajları

- Yayıncı ve gözlemciler arasındaki bağı azaltır.
- Bir nesnedeki değişikliğin birçok nesneye otomatik bildirilmesini sağlar.
- Çalışma zamanında yeni gözlemciler eklenip çıkarılabilir.
- Olay güdümlü sistemlerin kurulmasını kolaylaştırır.
- MVC gibi mimarilerde model ve görünüm ayrımını destekler.

Gözlemci Kalıbının Dezavantajları

- Çok fazla gözlemci varsa bildirim maliyeti artabilir.
- Bildirim sırası her zaman açık olmayabilir.
- Gözlemciler arasında dolaylı bağımlılıklar oluşursa hata ayıklama zorlaşabilir.
- Gözlemciler abonelikten çıkarılmazsa bazı dillerde bellek sızıntısı veya gereksiz referans problemleri oluşabilir.

Ne Zaman Kullanılmalı?

Gözlemci kalıbı şu durumlarda tercih edilebilir:

- Bir nesnedeki değişiklik başka nesneleri de ilgilendiriyorsa.
- Yayıncı nesnenin, kendisini takip eden nesneleri sıkı biçimde tanıması istenmiyorsa.
- Çalışma zamanında aboneler eklenip çıkarılacaksa.
- Bildirim, olay veya abonelik sistemi tasarlanıyorsa.
- MVC benzeri bir yapı kuruluyorsa.

Özet

Gözlemci kalıbı, bir nesnenin durumundaki değişiklikleri ona bağlı olan diğer nesnelere otomatik olarak bildirmeyi sağlar. Yayıncı nesne gözlemcilerin listesini tutar ve durumu değiştiğinde onları günceller. Bu kalıp, olay güdümlü sistemlerde, bildirim mekanizmalarında, kullanıcı arayüzlerinde ve MVC mimarisinde yaygın olarak kullanılır.

§5.4.5 Strateji

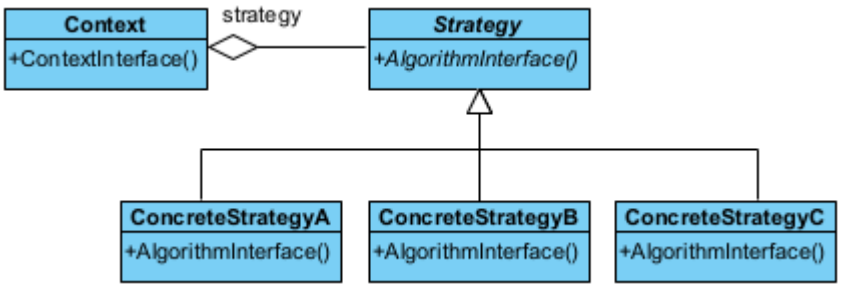
Strateji Kalıbı (*Strategy Pattern*), belirli bir işi yapmanın birden fazla yolu olduğunda, bu yolları ayrı sınıflar halinde kapsülleyerek birbirinin yerine kullanılabilir hale getiren davranışsal (*behavioral*) bir tasarım kalıbıdır.

Bu kalıpta her algoritma ayrı bir strateji sınıfı olarak tanımlanır. Bağlam sınıfı (`Context`) işi kendisi doğrudan yapmaz; bunun yerine kendisine verilen strateji nesnesine devreder. Böylece çalışma zamanında farklı algoritmalar arasında geçiş yapılabilir.

Strateji kalıbı, özellikle bir sınıf içinde çok sayıda `if-else` veya `switch` bloğu ile algoritma seçimi yapılıyorsa kullanışlıdır. Bu koşullu yapılar yerine her algoritma ayrı sınıfa taşınır ve sistem daha esnek hale gelir.

Örneğin bir ödeme sisteminde ödeme işlemi kredi kartı, banka havalesi veya kapıda ödeme ile yapılabilir. Bu ödeme yöntemlerinin her biri ayrı bir strateji olarak modellenebilir. Bağlam sınıfı

sadece ödeme stratejisini kullanır; ödemenin nasıl yapıldığını bilmek zorunda kalmaz.



Şekil 5.17: Strateji Kalıbı UML Diyagramı

Strateji kalıbındaki temel roller şunlardır:

- **Bağlam** (`Context`): İşi yapacak olan sınıftır. Ancak işi nasıl yapacağını kendisi belirlemek yerine bir strateji nesnesine devreder.
- **Strateji** (`Strategy`): Tüm algoritmaların uyması gereken ortak arayüzdür.
- **Somut Strateji** (`ConcreteStrategy`): Belirli bir algoritmayı uygulayan sınıftır. Her somut strateji aynı işi farklı bir yöntemle gerçekleştirir.

Java ile Temel Strateji Örneği

Aşağıdaki örnekte üç farklı algoritma vardır. Bağlam nesnesi çalışma zamanında bu algoritmalardan birini kullanır.

Önce strateji arayüzü tanımlanır:

```
interface Strategy {
  void execute();
}
```

Kod 5.157: Java Dilinde Strategy Arayüzü

Şimdi somut strateji sınıflarını yazalım:

```
class ConcreteStrategyA implements Strategy {
  @Override
  public void execute() {
    System.out.println("Algoritma A: Hızlı ama daha fazla bellek kullanıyor.");
  }
}

class ConcreteStrategyB implements Strategy {
  @Override
  public void execute() {
    System.out.println("Algoritma B: Yavaş ama bellek dostu.");
  }
}

class ConcreteStrategyC implements Strategy {
  @Override
  public void execute() {
    System.out.println("Algoritma C: Dengeli bir yaklaşım.");
  }
}
```

Kod 5.158: Java Dilinde Somut Strateji Sınıfları

Bağlam sınıfı:

```
class Context {
  private Strategy strategy;

  public Context(Strategy strategy) {
```

```
    this.strategy = strategy;
  }

  public void setStrategy(Strategy strategy) {
    this.strategy = strategy;
  }

  public void run() {
    if (strategy != null) {
      strategy.execute();
    } else {
      System.out.println("Hata: Henüz bir strateji belirlenmedi.");
    }
  }
}
```

Kod 5.159: Java Dilinde Context Sınıfı

`Context` sınıfı somut algoritmaları bilmez. Sadece `Strategy` arayüzünü kullanır. Bu sayede algoritma çalışma zamanında değiştirilebilir.

İstemci kod:

```
public class Main {
  public static void main(String[] args) {
    Strategy s1 = new ConcreteStrategyA();
    Strategy s2 = new ConcreteStrategyB();
    Strategy s3 = new ConcreteStrategyC();

    Context context = new Context(s1);

    System.out.println("--- Durum 1 ---");
    context.run();

    context.setStrategy(s2);

    System.out.println("--- Durum 2 ---");
    context.run();

    context.setStrategy(s3);

    System.out.println("--- Durum 3 ---");
    context.run();
  }
}
```

Kod 5.160: Java Dilinde Strateji Kalıbı Kullanımı

Program çıktısı:

```
--- Durum 1 ---
Algoritma A: Hızlı ama daha fazla bellek kullanıyor.
--- Durum 2 ---
Algoritma B: Yavaş ama bellek dostu.
--- Durum 3 ---
Algoritma C: Dengeli bir yaklaşım.
```

Bu örnekte `Context` nesnesi aynı kalmış, yalnızca kullanılan strateji değiştirilmiştir.

Gerçek Dünya Örneği: Sıralama Stratejileri

Strateji kalıbının yaygın örneklerinden biri sıralama algoritmalarıdır. Aynı veri kümesi farklı algoritmalarla sıralanabilir. Örneğin hızlı sıralama, birleştirmeli sıralama veya kabarcık sıralama kullanılabilir.

Bu örnekte:

- ◇ `SortStrategy`, strateji arayüzüdür.
- ◇ `QuickSort`, hızlı sıralama stratejisidir.
- ◇ `MergeSort`, birleştirmeli sıralama stratejisidir.
- ◇ `Sorter`, bağlam sınıfıdır.

Önce sıralama stratejisi arayüzünü yazalım:

```
import java.util.List;

interface SortStrategy {
  void sort(List<Integer> data);
}
```

Kod 5.161: Java Dilinde SortStrategy Arayüzü

Hızlı sıralama stratejisi:

```
import java.util.Collections;
import java.util.List;

class QuickSort implements SortStrategy {
  @Override
  public void sort(List<Integer> data) {
    System.out.println("QuickSort algoritması kullanılıyor...");

    Collections.sort(data);
  }
}
```

Kod 5.162: Java Dilinde QuickSort Stratejisi

Birleştirmeli sıralama stratejisi:

```
import java.util.Collections;
import java.util.List;

class MergeSort implements SortStrategy {
  @Override
  public void sort(List<Integer> data) {
    System.out.println("MergeSort algoritması kullanılıyor...");

    Collections.sort(data);
  }
}
```

Kod 5.163: Java Dilinde MergeSort Stratejisi

Burada örnek olması için iki stratejide de Java'nın hazır `Collections.sort()` metodu kullanılmıştır. Gerçek uygulamalarda bu sınıfların içine farklı algoritma kodları yazılabilir.

Bağlam sınıfı:

```
import java.util.List;

class Sorter {
  private SortStrategy strategy;

  public void setStrategy(SortStrategy strategy) {
    this.strategy = strategy;
  }

  public void sort(List<Integer> data) {
    if (strategy == null) {
      System.out.println("Hata: Önce bir strateji belirlenmelidir.");
      return;
    }

    strategy.sort(data);
```

```
    }
}
```

Kod 5.164: Java Dilinde Sorter Bağlam Sınıfı

İstemci kod:

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class Main {
  private static void printList(List<Integer> data) {
    for (int number : data) {
      System.out.print(number + " ");
    }

    System.out.println();
  }

  public static void main(String[] args) {
    List<Integer> sayilar =
    new ArrayList<>(Arrays.asList(34, 12, 5, 78, 1, 45));

    Sorter siralayici = new Sorter();

    System.out.print("Orijinal liste: ");
    printList(sayilar);

    siralayici.setStrategy(new QuickSort());
    siralayici.sort(sayilar);

    printList(sayilar);

    sayilar = new ArrayList<>(Arrays.asList(99, 2, 56, 11, 0, 7));

    System.out.println();
    System.out.print("Yeni liste: ");
    printList(sayilar);

    siralayici.setStrategy(new MergeSort());
    siralayici.sort(sayilar);

    printList(sayilar);
  }
}
```

Kod 5.165: Java Dilinde Sıralama Stratejisi Kullanımı

Program çıktısı:

```
Orijinal liste: 34 12 5 78 1 45
QuickSort algoritması kullanılıyor...
1 5 12 34 45 78

Yeni liste: 99 2 56 11 0 7
MergeSort algoritması kullanılıyor...
0 2 7 11 56 99
```

Bu örnekte sıralayıcı nesne değişmemiştir. Yalnızca kullanılan sıralama stratejisi çalışma zamanında değiştirilmiştir.

Tam Java Örneği

Aşağıda sıralama stratejisi örneğinin tek dosyada çalıştırılabilecek tam hali verilmiştir:

```
import java.util.ArrayList;
```

```
import java.util.Arrays;
import java.util.Collections;
import java.util.List;

interface SortStrategy {
  void sort(List<Integer> data);
}

class QuickSort implements SortStrategy {
  @Override
  public void sort(List<Integer> data) {
    System.out.println("QuickSort algoritması kullanılıyor...");

    Collections.sort(data);
  }
}

class MergeSort implements SortStrategy {
  @Override
  public void sort(List<Integer> data) {
    System.out.println("MergeSort algoritması kullanılıyor...");

    Collections.sort(data);
  }
}

class Sorter {
  private SortStrategy strategy;

  public void setStrategy(SortStrategy strategy) {
    this.strategy = strategy;
  }

  public void sort(List<Integer> data) {
    if (strategy == null) {
      System.out.println("Hata: Önce bir strateji belirlenmelidir.");
      return;
    }

    strategy.sort(data);
  }
}

public class Main {
  private static void printList(List<Integer> data) {
    for (int number : data) {
      System.out.print(number + " ");
    }

    System.out.println();
  }

  public static void main(String[] args) {
    List<Integer> sayilar =
    new ArrayList<>(Arrays.asList(34, 12, 5, 78, 1, 45));

    Sorter siralayici = new Sorter();

    System.out.print("Orijinal liste: ");
    printList(sayilar);

    siralayici.setStrategy(new QuickSort());
    siralayici.sort(sayilar);

    printList(sayilar);
```

```
        sayilar = new ArrayList<>(Arrays.asList(99, 2, 56, 11, 0, 7));

        System.out.println();
        System.out.print("Yeni liste: ");
        printList(sayilar);

        siralayici.setStrategy(new MergeSort());
        siralayici.sort(sayilar);

        printList(sayilar);
    }
}
```

Kod 5.166: Java Dilinde Strateji Kalıbı Sıralama Örneği

Gerçek Dünya Örneği: Ödeme Stratejileri

Strateji kalıbı ödeme sistemlerinde de sık kullanılır. Bir alışveriş sistemi farklı ödeme yöntemlerini destekleyebilir. Kredi kartı, nakit ödeme veya banka havalesi gibi yöntemler ayrı stratejiler olarak tanımlanabilir.

Ödeme stratejisi arayüzü:

```
interface PaymentStrategy {
    void pay(double amount);
}
```

Kod 5.167: Java Dilinde PaymentStrategy Arayüzü

Kredi kartı stratejisi:

```
class CreditCardPayment implements PaymentStrategy {
    private final String cardNumber;

    public CreditCardPayment(String cardNumber) {
        this.cardNumber = cardNumber;
    }

    @Override
    public void pay(double amount) {
        System.out.println(amount + " TL kredi kartı ile ödendi. Kart: "
        + cardNumber);
    }
}
```

Kod 5.168: Java Dilinde Kredi Kartı Stratejisi

Nakit ödeme stratejisi:

```
class CashPayment implements PaymentStrategy {
    @Override
    public void pay(double amount) {
        System.out.println(amount + " TL nakit olarak ödendi.");
    }
}
```

Kod 5.169: Java Dilinde Nakit Ödeme Stratejisi

Bağlam sınıfı:

```
class ShoppingCart {
    private PaymentStrategy paymentStrategy;

    public void setPaymentStrategy(PaymentStrategy paymentStrategy) {
        this.paymentStrategy = paymentStrategy;
    }

    public void checkout(double amount) {
```

```
    if (paymentStrategy == null) {
      System.out.println("Ödeme yöntemi seçilmedi.");
      return;
    }

    paymentStrategy.pay(amount);
  }
}
```

Kod 5.170: Java Dilinde ShoppingCart Bağlam Sınıfı

Kullanım:

```
public class Main {
  public static void main(String[] args) {
    ShoppingCart cart = new ShoppingCart();

    cart.setPaymentStrategy(new CreditCardPayment("**** **** **** 1234"));
    cart.checkout(250.0);

    cart.setPaymentStrategy(new CashPayment());
    cart.checkout(100.0);
  }
}
```

Kod 5.171: Java Dilinde Ödeme Stratejisi Kullanımı

Program çıktısı:

```
250.0 TL kredi kartı ile ödendi. Kart: **** **** **** 1234
100.0 TL nakit olarak ödendi.
```

Strateji Kalıbı ve Durum Kalıbı Arasındaki Fark

Strateji kalıbı, yapı olarak durum (*state*) kalıbına benzeyebilir. Ancak amaçları farklıdır.

Strateji Kalıbı	Durum Kalıbı
Amaç, algoritmaları birbirinin yerine kullanılabilir hale getirmektir.	Amaç, nesnenin durumuna göre davranış değiştirmesini sağlamaktır.
Strateji genellikle istemci veya bağlam tarafından seçilir.	Durum geçişleri çoğunlukla nesnenin iç mantığına göre gerçekleşir.
Algoritmalar birbirinden bağımsızdır.	Durumlar arasında geçiş ilişkileri bulunabilir.
Örneğin farklı sıralama veya ödeme yöntemleri.	Örneğin kapı: açık, kapalı, kilitli durumları.

Strateji Kalıbının Avantajları

- ◇ Algoritmaları ayrı sınıflarda kapsüller.
- ◇ Çalışma zamanında algoritma değiştirmeyi sağlar.
- ◇ Uzun `if-else` veya `switch` bloklarını azaltır.
- ◇ Yeni strateji eklemek kolaydır.
- ◇ Açık-kapalı ilkesine uygun bir yapı sağlar.
- ◇ Bağlam sınıfının algoritma ayrıntılarını bilmesini engeller.

Strateji Kalıbının Dezavantajları

- ◇ Her algoritma için ayrı sınıf gerektiğinden sınıf sayısı artabilir.
- ◇ İstemcinin doğru stratejiyi seçmesi gerekebilir.
- ◇ Çok basit algoritmalar için gereksiz soyutlama oluşturabilir.

Ne Zaman Kullanılmalı?

Strateji kalıbı şu durumlarda tercih edilebilir:

- Bir işi yapmanın birden fazla yolu varsa.
- Algoritmalar çalışma zamanında değiştirilecekse.
- Bir sınıf içinde çok sayıda koşullu algoritma seçimi varsa.
- Algoritmanın iç ayrıntıları istemciden gizlenmek isteniyorsa.
- Ödeme, sıralama, sıkıştırma, doğrulama veya rota hesaplama gibi değişebilir işlemler modelleniyorsa.

Özet

Strateji kalıbı, farklı algoritmaları ayrı sınıflarda kapsülleyerek birbirinin yerine kullanılabilir hale getirir. Bağlam sınıfı işi doğrudan yapmak yerine seçilen stratejiye devreder. Bu sayede çalışma zamanında algoritma değiştirilebilir, koşullu kodlar azaltılır ve sistem daha esnek hale gelir.

§5.4.6 Şablon Yöntem

Şablon Yöntem Kalıbı (*Template Method Pattern*), bir algoritmanın genel iskeletini üst sınıfta tanımlayıp, algoritmanın bazı adımlarının uygulanmasını alt sınıflara bırakan davranışsal (*behavioral*) bir tasarım kalıbıdır.

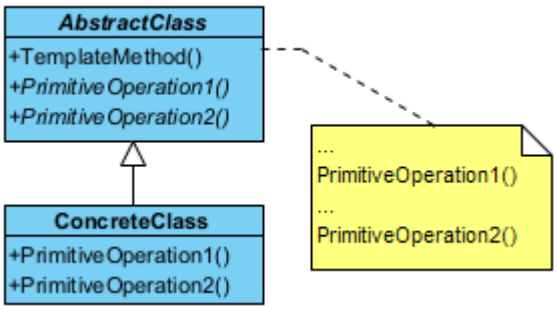
Bu kalıpta algoritmanın adım sırası üst sınıfta belirlenir. Alt sınıflar bu sırayı değiştiremez; yalnızca kendilerine bırakılan adımları kendi ihtiyaçlarına göre uygular. Böylece hem kod tekrarı azaltılır hem de algoritmanın genel akışı korunur.

Şablon yöntem kalıbı, nesne yönelimli programlamada sık kullanılan bir kalıptır. Özellikle aynı işlem akışına sahip fakat bazı adımları farklı olan sınıflarda kullanılır.

Örneğin çay ve kahve hazırlama işlemlerini düşünelim. Her ikisinde de genel süreç benzerdir:

- Su kaynatılır.
- İçecek hazırlanır.
- Bardağa dökülür.
- İsteğe bağlı ek malzeme eklenir.

Ancak kahve demleme ve çay demleme adımları farklıdır. Şablon yöntem kalıbı, ortak akışı üst sınıfta tutar; değişen adımları alt sınıflara bırakır.



Şekil 5.18: Şablon Yöntem Kalıbı UML Diyagramı

Şablon yöntem kalıbındaki temel roller şunlardır:

- **Soyut Sınıf** (`AbstractClass`): Algoritmanın iskeletini içeren `templateMethod()` metodunu tanımlar. Ortak adımları kendi içinde gerçekleştirir, değişken adımları ise alt sınıflara bırakır.
- **Somut Sınıf** (`ConcreteClass`): Soyut sınıfta boş bırakılan veya soyut tanımlanan adımları uygular. Algoritmanın genel sırasını değiştirmez.
- **Kanca Metot** (*Hook Method*): Alt sınıfların isterse ezebileceği, varsayılan olarak boş veya basit davranış içeren metotlardır. Algoritmaya isteğe bağlı genişleme noktası sağlar.

Java ile Temel Şablon Yöntem Örneği

Aşağıdaki örnekte `AbstractClass`, algoritmanın genel iskeletini tanımlar. Alt sınıf ise gerekli adımları uygular.

Önce soyut sınıfı yazalım:

```
abstract class AbstractClass {
  public final void templateMethod() {
    baseOperation();
    requiredOperation1();
    hook();
    requiredOperation2();
  }

  private void baseOperation() {
    System.out.println("Sistem: Algoritmanın ortak temel adımı çalıştırılıyor.");
  }

  protected abstract void requiredOperation1();

  protected abstract void requiredOperation2();

  protected void hook() {
    // İsteğe bağlı olarak alt sınıflar tarafından ezilebilir.
  }
}
```

Kod 5.172: Java Dilinde AbstractClass

Burada `templateMethod()` metodu `final` olarak tanımlanmıştır. Böylece alt sınıflar algoritmanın sırasını değiştiremez.

`baseOperation()` tüm alt sınıflar için ortak olan adımdır. `requiredOperation1()` ve `requiredOperation2()` ise alt sınıfların mutlaka uygulaması gereken adımlardır. `hook()` ise isteğe bağlı olarak ezilebilecek bir kanca metottur.

Şimdi somut sınıfı yazalım:

```
class ConcreteClass extends AbstractClass {
  @Override
  protected void requiredOperation1() {
    System.out.println("ConcreteClass: Adım 1 özel olarak uygulandı.");
  }

  @Override
  protected void requiredOperation2() {
    System.out.println("ConcreteClass: Adım 2 özel olarak uygulandı.");
  }

  @Override
  protected void hook() {
    System.out.println("ConcreteClass: Hook metodu devreye girdi.");
  }
}
```

Kod 5.173: Java Dilinde ConcreteClass

İstemci kod:

```
public class Main {
  public static void main(String[] args) {
    AbstractClass algoritma = new ConcreteClass();

    System.out.println("--- Algoritma Başlatılıyor ---");

    algoritma.templateMethod();
  }
```

```
}
```

Kod 5.174: Java Dilinde Şablon Yöntem Kullanımı

Program çıktısı:

```
--- Algoritma Başlatılıyor ---
Sistem: Algoritmanın ortak temel adımı çalıştırılıyor.
ConcreteClass: Adım 1 özel olarak uygulandı.
ConcreteClass: Hook metodu devreye girdi.
ConcreteClass: Adım 2 özel olarak uygulandı.
```

Bu örnekte algoritmanın sırası üst sınıf tarafından belirlenmiştir. Alt sınıf sadece kendisine bırakılan adımları uygulamıştır.

Gerçek Dünya Örneği: İçecek Hazırlama

Şablon yöntem kalıbının klasik örneklerinden biri içecek hazırlama sürecidir. Çay ve kahve hazırlama süreçleri benzer adımlardan oluşur. Ancak demleme ve ek malzeme ekleme adımları farklıdır.

Bu örnekte:

◇ `Beverage`, soyut sınıftır.
◇ `Tea`, çay hazırlama sınıfıdır.
◇ `Coffee`, kahve hazırlama sınıfıdır.

Önce soyut içecek sınıfı yazılır:

```
abstract class Beverage {
  public final void prepareRecipe() {
    boilWater();
    brew();
    pourInCup();

    if (customerWantsCondiments()) {
      addCondiments();
    }
  }

  private void boilWater() {
    System.out.println("Su kaynatılıyor.");
  }

  protected abstract void brew();

  private void pourInCup() {
    System.out.println("İçecek bardağa dökülüyor.");
  }

  protected abstract void addCondiments();

  protected boolean customerWantsCondiments() {
    return true;
  }
}
```

Kod 5.175: Java Dilinde Beverage Soyut Sınıfı

Burada `prepareRecipe()` şablon yöntemdir. Adımların sırası sabittir. `brew()` ve `addCondiments()` metotları alt sınıflar tarafından uygulanır.

Ayrıca `customerWantsCondiments()` bir kanca metottur. Alt sınıflar isterse bu metodu ezerek ek malzeme eklenip eklenmeyeceğine karar verebilir.

Çay sınıfı:

```
class Tea extends Beverage {
  @Override
  protected void brew() {
    System.out.println("Çay demleniyor.");
  }

  @Override
  protected void addCondiments() {
    System.out.println("Limon ekleniyor.");
  }
}
```

Kod 5.176: Java Dilinde Tea Sınıfı

Kahve sınıfı:

```
class Coffee extends Beverage {
  @Override
  protected void brew() {
    System.out.println("Kahve demleniyor.");
  }

  @Override
  protected void addCondiments() {
    System.out.println("Şeker ve süt ekleniyor.");
  }

  @Override
  protected boolean customerWantsCondiments() {
    return true;
  }
}
```

Kod 5.177: Java Dilinde Coffee Sınıfı

İstemci kod:

```
public class Main {
  public static void main(String[] args) {
    Beverage tea = new Tea();
    Beverage coffee = new Coffee();

    System.out.println("--- Çay Hazırlanıyor ---");
    tea.prepareRecipe();

    System.out.println();

    System.out.println("--- Kahve Hazırlanıyor ---");
    coffee.prepareRecipe();
  }
}
```

Kod 5.178: Java Dilinde İçecek Hazırlama Kullanımı

Program çıktısı:

```
--- Çay Hazırlanıyor ---
Su kaynatılıyor.
Çay demleniyor.
İçecek bardağa dökülüyor.
Limon ekleniyor.

--- Kahve Hazırlanıyor ---
Su kaynatılıyor.
Kahve demleniyor.
İçecek bardağa dökülüyor.
Şeker ve süt ekleniyor.
```

Bu örnekte su kaynatma ve bardağa dökme işlemleri ortak olarak üst sınıfta tanımlanmıştır. Çay ve kahveye özgü adımlar ise alt sınıflarda uygulanmıştır.

Tam Java Örneği

Aşağıda içecek hazırlama örneğinin tek dosyada çalıştırılabilecek tam hali verilmiştir:

```
abstract class Beverage {
  public final void prepareRecipe() {
    boilWater();
    brew();
    pourInCup();

    if (customerWantsCondiments()) {
      addCondiments();
    }
  }

  private void boilWater() {
    System.out.println("Su kaynatılıyor.");
  }

  protected abstract void brew();

  private void pourInCup() {
    System.out.println("İçecek bardağa dökülüyor.");
  }

  protected abstract void addCondiments();

  protected boolean customerWantsCondiments() {
    return true;
  }
}

class Tea extends Beverage {
  @Override
  protected void brew() {
    System.out.println("Çay demleniyor.");
  }

  @Override
  protected void addCondiments() {
    System.out.println("Limon ekleniyor.");
  }
}

class Coffee extends Beverage {
  @Override
  protected void brew() {
    System.out.println("Kahve demleniyor.");
  }

  @Override
  protected void addCondiments() {
    System.out.println("Şeker ve süt ekleniyor.");
  }

  @Override
  protected boolean customerWantsCondiments() {
    return true;
  }
}

public class Main {
  public static void main(String[] args) {
```

```
        Beverage tea = new Tea();
        Beverage coffee = new Coffee();

        System.out.println("--- Çay Hazırlanıyor ---");
        tea.prepareRecipe();

        System.out.println();

        System.out.println("--- Kahve Hazırlanıyor ---");
        coffee.prepareRecipe();
    }
}
```

Kod 5.179: Java Dilinde Şablon Yöntem İçecek Örneği

Kanca Metotlar

Şablon yöntem kalıbında kanca metotlar (*hook methods*), alt sınıflara algoritmanın belirli noktalarında isteğe bağlı davranış ekleme imkânı verir.

Kanca metotlar genellikle şu amaçlarla kullanılır:

- ◇ Bir adımın çalışıp çalışmayacağına karar vermek.
- ◇ Algoritmanın belirli noktasında isteğe bağlı ek işlem yapmak.
- ◇ Alt sınıfa sınırlı bir genişletme noktası vermek.

Örneğin içecek örneğinde şu metot bir kanca metottur:

```
protected boolean customerWantsCondiments() {
    return true;
}
```

Alt sınıf bu metodu ezerek ek malzeme eklenmesini engelleyebilir:

```
@Override
protected boolean customerWantsCondiments() {
    return false;
}
```

Bu durumda `addCondiments()` adımı çalıştırılmaz.

Hollywood İlkesi

Şablon yöntem kalıbı, **Hollywood İlkesi** (*Hollywood Principle*) ile yakından ilişkilidir:

"Bizi aramayın, biz sizi ararız."

Bu ilkeye göre alt sınıflar üst sınıfın akışını doğrudan kontrol etmez. Bunun yerine üst sınıf, algoritmanın akışı içinde gerekli noktalarda alt sınıfların metotlarını çağırır.

Yani kontrol akışı üst sınıftadır. Alt sınıf yalnızca kendisine bırakılan adımları doldurur.

Şablon Yöntem Kalıbı ve Strateji Kalıbı Arasındaki Fark

Şablon yöntem ve strateji kalıpları bazen benzer amaçlarla kullanılabilir. İkisi de değişen davranışları ayırır. Ancak kullandıkları mekanizma farklıdır.

Şablon Yöntem Kalıbı	Strateji Kalıbı
Kalıtım kullanır.	Bileşim kullanır.
Algoritmanın iskeleti üst sınıfta sabittir.	Algoritmanın tamamı değiştirilebilir.
Alt sınıflar belirli adımları özelleştirir.	Farklı strateji nesneleri farklı algoritmalar sunar.
Çalışma zamanında algoritma değiştirmek daha sınırlıdır.	Çalışma zamanında strateji değiştirmek kolaydır.

Şablon Yöntem Kalıbının Avantajları

- Ortak algoritma akışını tek bir yerde toplar.
- Kod tekrarını azaltır.
- Algoritmanın adım sırasını korur.
- Alt sınıflara yalnızca gerekli adımları özelleştirme imkânı verir.
- Büyük işlemleri standart bir iskelet içinde yönetmeyi kolaylaştırır.

Şablon Yöntem Kalıbının Dezavantajları

- Kalıtıma dayalı olduğu için esneklik bazı durumlarda sınırlı olabilir.
- Üst sınıftaki algoritma akışı değişirse tüm alt sınıflar etkilenebilir.
- Çok fazla soyut adım varsa alt sınıfların uygulanması zorlaşabilir.
- Yanlış tasarlanırsa alt sınıflar gereksiz metotları uygulamak zorunda kalabilir.

Ne Zaman Kullanılmalı?

Şablon yöntem kalıbı şu durumlarda tercih edilebilir:

- Bir algoritmanın genel akışı sabitse, bazı adımları değişiyorsa.
- Birden fazla sınıfta benzer işlem sırası tekrar ediyorsa.
- Ortak kod üst sınıfta toplanmak isteniyorsa.
- Alt sınıfların algoritmanın tamamını değil, yalnızca belirli adımlarını değiştirmesi isteniyorsa.
- Çerçeve veya kütüphane tasarımında kullanıcıya belirli genişletme noktaları sunulacaksa.

Özet

Şablon yöntem kalıbı, bir algoritmanın iskeletini üst sınıfta tanımlar ve değişken adımların uygulanmasını alt sınıflara bırakır. Böylece ortak işlem akışı korunur, kod tekrarı azalır ve alt sınıflar yalnızca ihtiyaç duyulan adımları özelleştirir. Bu kalıp özellikle işlem sırası sabit fakat bazı adımları değişken olan durumlarda kullanışlıdır.

§5.4.7 Ziyaretçi

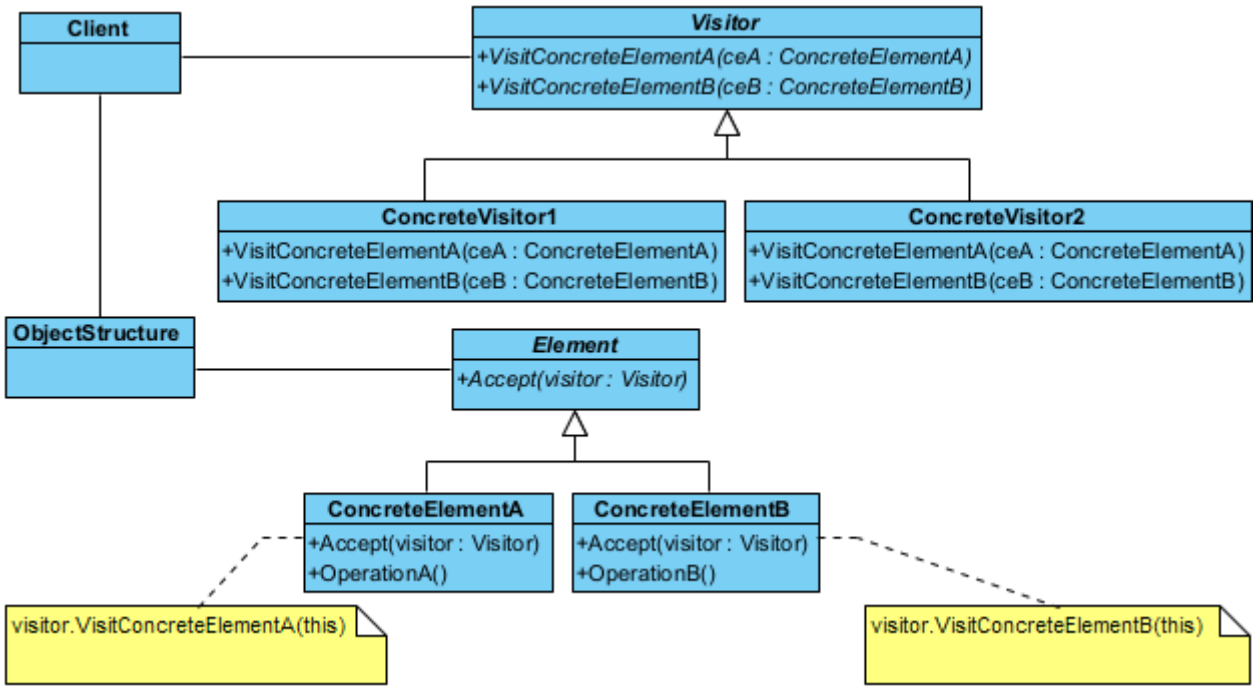
Ziyaretçi Kalıbı (*Visitor Pattern*), bir nesne yapısını oluşturan sınıfları değiştirmeden, bu sınıflar üzerinde yeni işlemler tanımlamayı sağlayan davranışsal (*behavioral*) bir tasarım kalıbıdır.

Bu kalıbın temel amacı, veri yapısı ile bu veri yapısı üzerinde çalışacak algoritmaları birbirinden ayırmaktır. Böylece var olan sınıfların içine yeni metotlar eklemek yerine, yeni işlemler ayrı ziyaretçi sınıfları olarak tanımlanabilir.

Ziyaretçi kalıbı özellikle sınıf hiyerarşisinin çok sık değişmediği, ancak bu sınıflar üzerinde yapılacak işlemlerin sık sık arttığı durumlarda kullanışlıdır. Örneğin bir doküman yapısında paragraf, resim ve tablo gibi elemanlar olabilir. Bu elemanların yapısı sabit kalırken, zamanla PDF'e dönüştürme, HTML'e dönüştürme, metin sayma veya doğrulama gibi yeni işlemler eklenebilir.

Bu durumda her yeni işlem için `Paragraph`, `Image` ve `Table` sınıflarının içine yeni metotlar eklemek yerine, her işlem ayrı bir ziyaretçi sınıfı olarak tanımlanır.

Ziyaretçi kalıbındaki temel roller şunlardır:



Şekil 5.19: Ziyaretçi Kalıbı UML Diyagramı

- ◇ **Ziyaretçi** (`Visitor`): Her somut eleman türü için ayrı bir `visit()` metodu tanımlar. Örneğin `visit(ConcreteElementA)` ve `visit(ConcreteElementB)` gibi.
- ◇ **Somut Ziyaretçi** (`ConcreteVisitor`): Gerçek işlemi gerçekleştiren sınıftır. Her eleman türü için yapılacak işlemi bilir.
- ◇ **Eleman** (`Element`): Ziyaretçiyi kabul eden ortak arayüzdür. Genellikle `accept(Visitor visitor)` metodunu içerir.
- ◇ **Somut Eleman** (`ConcreteElement`): Nesne yapısının gerçek parçalarıdır. `accept()` metodu çağrıldığında, ziyaretçinin kendi türüne uygun `visit()` metodunu çalıştırır.
- ◇ **Nesne Yapısı** (`ObjectStructure`): Elemanları bir koleksiyon içinde tutar ve ziyaretçinin tüm elemanlar üzerinde dolaşmasını sağlar.

Çifte Sevkiyat

Ziyaretçi kalıbının en önemli kavramlarından biri **çifte sevkiyat** (*double dispatch*) mekanizmasıdır.

Normalde bir metot çağrısında hangi metodun çalışacağı, çağrının yapıldığı nesnenin gerçek türüne göre belirlenir. Ziyaretçi kalıbında ise hem ziyaretçinin türü hem de ziyaret edilen elemanın türü önemlidir.

Somut eleman içindeki `accept()` metodu genellikle şu şekilde çalışır:

```
visitor.visit(this);
```

Buradaki `this`, somut elemanın gerçek türünü temsil eder. Böylece ziyaretçi içinde doğru `visit()` metodu çağrılır.

Java ile Temel Ziyaretçi Örneği

Aşağıdaki örnekte iki farklı eleman türü ve iki farklı ziyaretçi vardır. Ziyaretçiler aynı elemanlar üzerinde farklı işlemler yapar.

Önce ziyaretçi arayüzünü tanımlayalım:

```
interface Visitor {
  void visit(ConcreteElementA element);
  void visit(ConcreteElementB element);
```

```
}
```

Kod 5.180: Java Dilinde Visitor Arayüzü

Şimdi eleman arayüzü yazılır:

```
interface Element {
  void accept(Visitor visitor);
}
```

Kod 5.181: Java Dilinde Element Arayüzü

Somut eleman sınıfları:

```
class ConcreteElementA implements Element {
  @Override
  public void accept(Visitor visitor) {
    visitor.visit(this);
  }

  public void operationA() {
    System.out.println("ConcreteElementA: Özel işlem çalıştırılıyor.");
  }
}

class ConcreteElementB implements Element {
  @Override
  public void accept(Visitor visitor) {
    visitor.visit(this);
  }

  public void operationB() {
    System.out.println("ConcreteElementB: Özel işlem çalıştırılıyor.");
  }
}
```

Kod 5.182: Java Dilinde Somut Eleman Sınıfları

Burada `accept()` metodu, ziyaretçiye kendi nesnesini gönderir. Bu sayede ziyaretçinin doğru `visit()` metodu çalışır.

Şimdi somut ziyaretçi sınıflarını yazalım:

```
class ConcreteVisitor1 implements Visitor {
  @Override
  public void visit(ConcreteElementA element) {
    System.out.println("Visitor 1: ElementA ziyaret edildi.");
    element.operationA();
  }

  @Override
  public void visit(ConcreteElementB element) {
    System.out.println("Visitor 1: ElementB ziyaret edildi.");
    element.operationB();
  }
}

class ConcreteVisitor2 implements Visitor {
  @Override
  public void visit(ConcreteElementA element) {
    System.out.println("Visitor 2: ElementA üzerinde farklı bir işlem yapılıyor.");
  }

  @Override
  public void visit(ConcreteElementB element) {
    System.out.println("Visitor 2: ElementB üzerinde farklı bir işlem yapılıyor.");
  }
}
```

Kod 5.183: Java Dilinde Somut Ziyaretçiler

Nesne yapısı sınıfı:

```
import java.util.ArrayList;
import java.util.List;

class ObjectStructure {
  private final List<Element> elements = new ArrayList<>();

  public void add(Element element) {
    elements.add(element);
  }

  public void accept(Visitor visitor) {
    for (Element element : elements) {
      element.accept(visitor);
    }
  }
}
```

Kod 5.184: Java Dilinde ObjectStructure Sınıfı

İstemci kod:

```
public class Main {
  public static void main(String[] args) {
    ObjectStructure structure = new ObjectStructure();

    structure.add(new ConcreteElementA());
    structure.add(new ConcreteElementB());

    Visitor visitor1 = new ConcreteVisitor1();
    Visitor visitor2 = new ConcreteVisitor2();

    System.out.println("--- Ziyaretçi 1 Başlıyor ---");
    structure.accept(visitor1);

    System.out.println("--- Ziyaretçi 2 Başlıyor ---");
    structure.accept(visitor2);
  }
}
```

Kod 5.185: Java Dilinde Ziyaretçi Kalıbı Kullanımı

Program çıktısı:

```
--- Ziyaretçi 1 Başlıyor ---
Visitor 1: ElementA ziyaret edildi.
ConcreteElementA: Özel işlem çalıştırılıyor.
Visitor 1: ElementB ziyaret edildi.
ConcreteElementB: Özel işlem çalıştırılıyor.
--- Ziyaretçi 2 Başlıyor ---
Visitor 2: ElementA üzerinde farklı bir işlem yapılıyor.
Visitor 2: ElementB üzerinde farklı bir işlem yapılıyor.
```

Bu örnekte eleman sınıfları değiştirilmeden yeni işlemler ziyaretçi sınıfları aracılığıyla eklenmiştir.

Gerçek Dünya Örneği: Doküman Elemanları

Bir doküman sisteminde paragraf, resim ve tablo gibi farklı elemanlar olduğunu düşünelim. Bu elemanlar üzerinde farklı işlemler yapılabilir:

- ◇ HTML'e dönüştürme

- ◇ Metin istatistiği çıkarma
- ◇ PDF'e dönüştürme

Bu işlemleri her elemanın içine ayrı ayrı eklemek yerine ziyaretçi kalıbı kullanılabilir.

Bu örnekte:

- ◇ `DocumentElement`, eleman arayüzüdür.
- ◇ `Paragraph`, `ImageElement` ve `TableElement`, somut elemanlardır.
- ◇ `DocumentVisitor`, ziyaretçi arayüzüdür.
- ◇ `HtmlExportVisitor`, HTML çıktısı üreten ziyaretçidir.
- ◇ `StatisticsVisitor`, istatistik çıkaran ziyaretçidir.

Önce ziyaretçi arayüzü yazılır:

```
interface DocumentVisitor {
  void visit(Paragraph paragraph);
  void visit(ImageElement image);
  void visit(TableElement table);
}
```

Kod 5.186: Java Dilinde DocumentVisitor Arayüzü

Eleman arayüzü:

```
interface DocumentElement {
  void accept(DocumentVisitor visitor);
}
```

Kod 5.187: Java Dilinde DocumentElement Arayüzü

Paragraf sınıfı:

```
class Paragraph implements DocumentElement {
  private final String text;

  public Paragraph(String text) {
    this.text = text;
  }

  public String getText() {
    return text;
  }

  @Override
  public void accept(DocumentVisitor visitor) {
    visitor.visit(this);
  }
}
```

Kod 5.188: Java Dilinde Paragraph Sınıfı

Resim sınıfı:

```
class ImageElement implements DocumentElement {
  private final String fileName;

  public ImageElement(String fileName) {
    this.fileName = fileName;
  }

  public String getFileName() {
    return fileName;
  }

  @Override
  public void accept(DocumentVisitor visitor) {
    visitor.visit(this);
```

```
    }
  }
```

Kod 5.189: Java Dilinde ImageElement Sınıfı

Tablo sınıfı:

```
class TableElement implements DocumentElement {
  private final int rowCount;
  private final int columnCount;

  public TableElement(int rowCount, int columnCount) {
    this.rowCount = rowCount;
    this.columnCount = columnCount;
  }

  public int getRowCount() {
    return rowCount;
  }

  public int getColumnCount() {
    return columnCount;
  }

  @Override
  public void accept(DocumentVisitor visitor) {
    visitor.visit(this);
  }
}
```

Kod 5.190: Java Dilinde TableElement Sınıfı

HTML'e dönüştüren ziyaretçi:

```
class HtmlExportVisitor implements DocumentVisitor {
  @Override
  public void visit(Paragraph paragraph) {
    System.out.println("<p>" + paragraph.getText() + "</p>");
  }

  @Override
  public void visit(ImageElement image) {
    System.out.println("<img src=\"" + image.getFileName() + "\" />");
  }

  @Override
  public void visit(TableElement table) {
    System.out.println("<table rows=\"" + table.getRowCount()
    + "\" cols=\"" + table.getColumnCount() + "\"></table>");
  }
}
```

Kod 5.191: Java Dilinde HtmlExportVisitor Sınıfı

İstatistik çıkaran ziyaretçi:

```
class StatisticsVisitor implements DocumentVisitor {
  private int paragraphCount;
  private int imageCount;
  private int tableCount;

  @Override
  public void visit(Paragraph paragraph) {
    paragraphCount++;
  }

  @Override
  public void visit(ImageElement image) {
```

```
    imageCount++;
  }

  @Override
  public void visit(TableElement table) {
    tableCount++;
  }

  public void printStatistics() {
    System.out.println("Paragraf sayısı: " + paragraphCount);
    System.out.println("Resim sayısı: " + imageCount);
    System.out.println("Tablo sayısı: " + tableCount);
  }
}
```

Kod 5.192: Java Dilinde StatisticsVisitor Sınıfı

Doküman sınıfı:

```
import java.util.ArrayList;
import java.util.List;

class Document {
  private final List<DocumentElement> elements = new ArrayList<>();

  public void addElement(DocumentElement element) {
    elements.add(element);
  }

  public void accept(DocumentVisitor visitor) {
    for (DocumentElement element : elements) {
      element.accept(visitor);
    }
  }
}
```

Kod 5.193: Java Dilinde Document Sınıfı

İstemci kod:

```
public class Main {
  public static void main(String[] args) {
    Document document = new Document();

    document.addElement(new Paragraph("Ziyaretçi kalıbı örneği."));
    document.addElement(new ImageElement("diagram.png"));
    document.addElement(new TableElement(3, 4));

    System.out.println("--- HTML Çıktısı ---");

    HtmlExportVisitor htmlVisitor = new HtmlExportVisitor();
    document.accept(htmlVisitor);

    System.out.println();

    System.out.println("--- İstatistik ---");

    StatisticsVisitor statisticsVisitor = new StatisticsVisitor();
    document.accept(statisticsVisitor);
    statisticsVisitor.printStatistics();
  }
}
```

Kod 5.194: Java Dilinde Doküman Ziyaretçi Kullanımı

Program çıktısı:

```
--- HTML Çıktısı ---
```

```
<p>Ziyaretçi kalıbı örneği.</p>
<img src="diagram.png" />
<table rows="3" cols="4"></table>

--- İstatistik ---
Paragraf sayısı: 1
Resim sayısı: 1
Tablo sayısı: 1
```

Bu örnekte doküman elemanları değiştirilmeden iki farklı işlem eklenmiştir: HTML'e dönüştürme ve istatistik çıkarma.

Ziyaretçi Kalıbı Ne Sağlar?

Ziyaretçi kalıbı, özellikle yeni işlemler eklemeyi kolaylaştırır. Örneğin doküman örneğine daha sonra `PdfExportVisitor` veya `SpellCheckVisitor` eklenebilir. Bu durumda `Paragraph`, `ImageElement` ve `TableElement` sınıflarının değiştirilmesi gerekmez.

Yeni işlem ekleme örneği:

```
class PlainTextVisitor implements DocumentVisitor {
  @Override
  public void visit(Paragraph paragraph) {
    System.out.println(paragraph.getText());
  }

  @Override
  public void visit(ImageElement image) {
    System.out.println("[Resim: " + image.getFileName() + "]");
  }

  @Override
  public void visit(TableElement table) {
    System.out.println("[Tablo: " + table.getRowCount()
    + "x" + table.getColumnCount() + "]");
  }
}
```

Bu sınıf eklendiğinde mevcut eleman sınıflarına dokunulmaz.

Ziyaretçi Kalıbının Avantajları

- ◇ Var olan sınıfları değiştirmeden yeni işlemler eklemeyi kolaylaştırır.
- ◇ İlgili işlemleri tek bir ziyaretçi sınıfında toplar.
- ◇ Veri yapısı ile algoritmaları birbirinden ayırır.
- ◇ Nesne yapısındaki tüm elemanlar üzerinde dolaşmayı kolaylaştırır.
- ◇ Farklı eleman türlerine özel işlemler merkezi biçimde tanımlanabilir.

Ziyaretçi Kalıbının Dezavantajları

- ◇ Yeni eleman türü eklemek zordur. Çünkü `Visitor` arayüzüne yeni bir `visit()` metodu eklemek gerekir.
- ◇ Tüm somut ziyaretçilerin yeni eleman türünü desteklemesi gerekir.
- ◇ Bazı durumlarda elemanların iç bilgilerine erişim gerektiği için kapsülleme (*encapsulation*) zayıflayabilir.
- ◇ Sınıf hiyerarşisi sık değişiyorsa bakım maliyeti artabilir.

Ne Zaman Kullanılmalı?

Ziyaretçi kalıbı şu durumlarda tercih edilebilir:

- ◇ Sınıf hiyerarşisi büyük ölçüde sabitse.
- ◇ Bu sınıflar üzerinde sık sık yeni işlemler tanımlanıyorsa.

- ◇ İşlemler sınıfların içine eklenmek istenmiyorsa.
- ◇ Bir nesne yapısındaki farklı eleman türleri üzerinde türlerine özel işlemler yapılacaksa.
- ◇ Veri yapısı ile algoritmalar birbirinden ayrılmak isteniyorsa.

Ne Zaman Kaçınılmalı?

Ziyaretçi kalıbı şu durumlarda uygun olmayabilir:

- ◇ Sık sık yeni eleman sınıfları ekleniyorsa.
- ◇ Eleman hiyerarşisi sürekli değişiyorsa.
- ◇ İşlemler çok azsa ve ayrı ziyaretçi sınıfları gereksiz karmaşıklık oluşturuyorsa.
- ◇ Elemanların iç durumunu dışarı açmak kapsüllemeyi ciddi biçimde bozuyorsa.

Özet

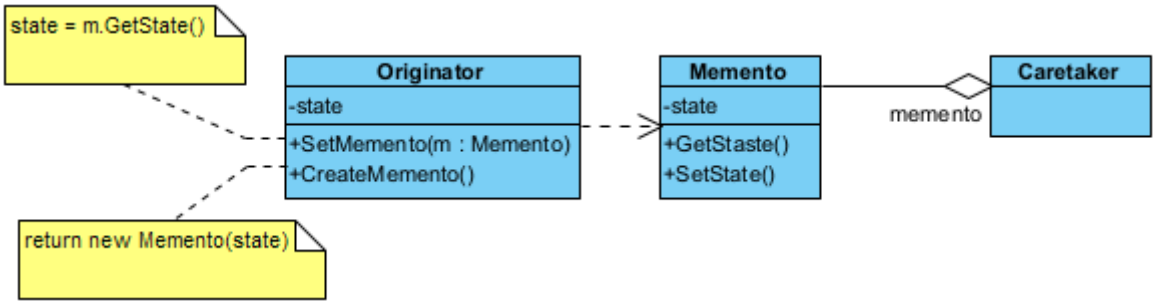
Ziyaretçi kalıbı, sabit bir nesne yapısı üzerinde yeni işlemler eklemeyi kolaylaştırır. İşlemler, eleman sınıflarının içine yazılmak yerine ziyaretçi sınıflarına taşınır. Böylece veri yapısı ile algoritma birbirinden ayrılır. Ancak yeni eleman türleri sık ekleniyorsa bu kalıp bakım açısından zorlayıcı olabilir.

§5.4.8 Hatıra

Hatıra Kalıbı (*Memento Pattern*), bir nesnenin belirli bir andaki iç durumunu saklayıp, daha sonra nesneyi bu eski duruma geri döndürmeyi sağlayan davranışsal (*behavioral*) bir tasarım kalıbıdır.

Bu kalıbın temel amacı, nesnenin iç durumunu dışarıya açmadan kaydetmek ve gerektiğinde geri yüklemektir. Böylece geri alma (*undo*), kayıt noktası (*checkpoint*), oyun kaydı veya editör geçmişi gibi mekanizmalar kurulabilir.

Hatıra kalıbı, nesnenin durumunu saklarken **kapsüllemeyi** (*encapsulation*) korumaya çalışır. Yani nesnenin iç bilgileri dış dünyaya doğrudan açılmaz. Durumu bilen ve bu durumu geri yükleyebilen asıl nesne yine durumun sahibi olan nesnedir.



Şekil 5.20: Hatıra Kalıbı UML Diyagramı

Hatıra kalıbındaki temel roller şunlardır:

- ◇ **Yaratıcı** (`Originator`): Durumu değişen asıl nesnedir. Kendi anlık görüntüsünü oluşturur ve gerektiğinde eski bir hatıradan durumunu geri yükler.
- ◇ **Hatıra** (`Memento`): Yaratıcının belirli bir andaki durumunu saklayan nesnedir. Genellikle pasif bir veri nesnesidir. Dışarıdan değiştirilmemesi tercih edilir.
- ◇ **Bakıcı** (`Caretaker`): Hatıraları saklayan sınıftır. Hatıraların içeriğini değiştirmez veya yorumlamaz. Sadece saklar ve gerektiğinde geri verir.

Bu kalıp bir oyun kaydetme sistemine benzetilebilir. Oyuncunun canı, puanı ve bölümü belirli bir anda kaydedilir. Oyuncu daha sonra başarısız olursa, oyun eski kayıt noktasına geri yüklenebilir.

Java ile Temel Hatıra Örneği

Aşağıdaki örnekte bir `Originator` nesnesinin sayısal durumu kaydedilmekte ve daha sonra geri yüklenmektedir.

Önce hatıra sınıfını yazalım:

```
class Memento {
  private final int state;

  public Memento(int state) {
    this.state = state;
  }

  int getState() {
    return state;
  }
}
```

Kod 5.195: Java Dilinde Memento Sınıfı

Bu sınıf, kaydedilen durumu tutar. `state` alanı `final` olduğu için hatıra oluşturulduktan sonra değiştirilemez.

Şimdi durumu değişen asıl sınıfı yazalım:

```
class Originator {
  private int state;

  public void setState(int state) {
    this.state = state;

    System.out.println("Originator: Durum değiştirildi -> " + state);
  }

  public Memento save() {
    System.out.println("Originator: Durum yedekleniyor... (" + state + ")");

    return new Memento(state);
  }

  public void restore(Memento memento) {
    if (memento != null) {
      this.state = memento.getState();

      System.out.println("Originator: Durum geri yüklendi -> " + state);
    }
  }
}
```

Kod 5.196: Java Dilinde Originator Sınıfı

`Originator`, kendi durumunu `save()` metodu ile `Memento` nesnesine kaydeder. Daha sonra `restore()` metodu ile eski durumuna dönebilir.

Şimdi hatıraları saklayan bakıcı sınıfı yazılır:

```
import java.util.Stack;

class Caretaker {
  private final Stack<Memento> history = new Stack<>();
  private final Originator originator;

  public Caretaker(Originator originator) {
    this.originator = originator;
  }

  public void backup() {
    history.push(originator.save());
  }

  public void undo() {
    if (history.isEmpty()) {
```

```
        System.out.println("Geri yüklenecek kayıt bulunamadı.");
        return;
      }

      Memento memento = history.pop();

      originator.restore(memento);
    }
  }
```

Kod 5.197: Java Dilinde Caretaker Sınıfı

`Caretaker`, hatıraların içeriğini bilmez. Sadece onları bir yığın içinde saklar.

İstemci kod:

```
public class Main {
  public static void main(String[] args) {
    Originator player = new Originator();
    Caretaker saveManager = new Caretaker(player);

    player.setState(100);
    saveManager.backup();

    player.setState(50);
    player.setState(0);

    System.out.println("--- Geri Yükleniyor ---");

    saveManager.undo();
  }
}
```

Kod 5.198: Java Dilinde Hatıra Kalıbı Kullanımı

Program çıktısı:

```
Originator: Durum değiştirildi -> 100
Originator: Durum yedekleniyor... (100)
Originator: Durum değiştirildi -> 50
Originator: Durum değiştirildi -> 0
--- Geri Yükleniyor ---
Originator: Durum geri yüklendi -> 100
```

Bu örnekte `100` durumu kaydedilmiş, daha sonra nesne farklı durumlara geçmiş ve sonunda eski kayıt noktasına geri dönmüştür.

Gerçek Dünya Örneği: Metin Editörü Geçmişi

Hatıra kalıbının en yaygın örneklerinden biri metin editörlerinde geri alma işlemidir. Editörün metni belirli anlarda kaydedilir. Kullanıcı hata yaptığında editör eski metin durumuna geri döner.

Bu örnekte:

- `TextEditor`, durumu değişen asıl nesnedir.
- `EditorMemento`, metnin bir andaki durumunu saklar.
- `History`, hatıraları saklayan bakıcı sınıftır.

Önce hatıra sınıfı yazılır:

```
class EditorMemento {
  private final String content;

  public EditorMemento(String content) {
    this.content = content;
  }
```

```
  String getSavedContent() {
    return content;
  }
}
```

Kod 5.199: Java Dilinde EditorMemento Sınıfı

Metin editörü sınıfı:

```
class TextEditor {
  private String text = "";

  public void type(String newText) {
    text += newText;
  }

  public String getText() {
    return text;
  }

  public EditorMemento save() {
    return new EditorMemento(text);
  }

  public void restore(EditorMemento memento) {
    if (memento != null) {
      text = memento.getSavedContent();
    }
  }
}
```

Kod 5.200: Java Dilinde TextEditor Sınıfı

Geçmişi yöneten sınıf:

```
import java.util.Stack;

class History {
  private final Stack<EditorMemento> mementos = new Stack<>();

  public void push(EditorMemento memento) {
    mementos.push(memento);
  }

  public EditorMemento pop() {
    if (mementos.isEmpty()) {
      return null;
    }

    return mementos.pop();
  }

  public boolean isEmpty() {
    return mementos.isEmpty();
  }
}
```

Kod 5.201: Java Dilinde History Sınıfı

İstemci kod:

```
public class Main {
  public static void main(String[] args) {
    TextEditor editor = new TextEditor();
    History history = new History();

    editor.type("Merhaba ");
    history.push(editor.save());
```

```
    System.out.println("Güncel Metin: " + editor.getText());

    editor.type("Dünya!");
    history.push(editor.save());

    System.out.println("Güncel Metin: " + editor.getText());

    editor.type(" Yanlış bir ekleme.");

    System.out.println("Hata Sonrası: " + editor.getText());

    EditorMemento lastSavedState = history.pop();

    if (lastSavedState != null) {
      editor.restore(lastSavedState);

      System.out.println("Geri Alındı: " + editor.getText());
    }
  }
}
```

Kod 5.202: Java Dilinde Metin Editörü Hatıra Kullanımı

Program çıktısı:

```
Güncel Metin: Merhaba
Güncel Metin: Merhaba Dünya!
Hata Sonrası: Merhaba Dünya! Yanlış bir ekleme.
Geri Alındı: Merhaba Dünya!
```

Bu örnekte hata yapılmadan önceki son kayıt `Merhaba Dünya!` olduğu için geri alma işlemi bu duruma dönmüştür.

Undo Mantığında Dikkat Edilmesi Gereken Nokta

Geri alma mantığı tasarlanırken, kayıtların ne zaman alındığı önemlidir. Genellikle bir değişiklik yapmadan önce mevcut durum kaydedilir. Böylece hata yapıldığında önceki duruma dönülebilir.

Örneğin:

```
history.push(editor.save());
editor.type("Yeni metin");
```

Bu yaklaşımda `undo()` çağrıldığında değişiklikten önceki duruma dönülür.

Buna karşılık değişiklikten sonra kayıt alınırsa, geri alma işlemi son kaydedilen duruma döner. Bu durum bazen beklenen davranışla karışabilir. Bu nedenle hatıra kalıbında kayıt stratejisi açık şekilde belirlenmelidir.

Hatıra Kalıbının Avantajları

- ◇ Nesnenin eski durumuna geri dönmesini sağlar.
- ◇ Geri alma ve kayıt noktası mekanizmalarını kolaylaştırır.
- ◇ Nesnenin iç durumunu doğrudan dışarı açmadan saklamaya yardımcı olur.
- ◇ Bakıcı sınıf, hatıraların içeriğini bilmeden geçmişi yönetebilir.
- ◇ Oyun kaydı, editör geçmişi ve işlem geri alma yapıları için uygundur.

Hatıra Kalıbının Dezavantajları

- ◇ Çok büyük nesne durumları saklanıyorsa bellek kullanımı artabilir.
- ◇ Çok sık kayıt almak performans maliyeti oluşturabilir.
- ◇ Derin kopyalama gereken durumlarda uygulama karmaşıklaşabilir.
- ◇ Hangi durumda kayıt alınacağı doğru tasarlanmazsa geri alma davranışı beklenenden farklı

olabilir.

Ne Zaman Kullanılmalı?

Hatıra kalıbı şu durumlarda tercih edilebilir:

- ◇ Bir nesnenin eski durumuna dönmesi gerekiyorsa.
- ◇ Geri alma veya tekrar yapma mekanizması kurulacaksa.
- ◇ Oyunlarda kayıt noktası sistemi geliştirilecekse.
- ◇ Metin editörü, çizim programı veya form düzenleme gibi geçmiş takibi gerekiyorsa.
- ◇ Nesnenin iç durumunu dış dünyaya açmadan saklamak isteniyorsa.

Özet

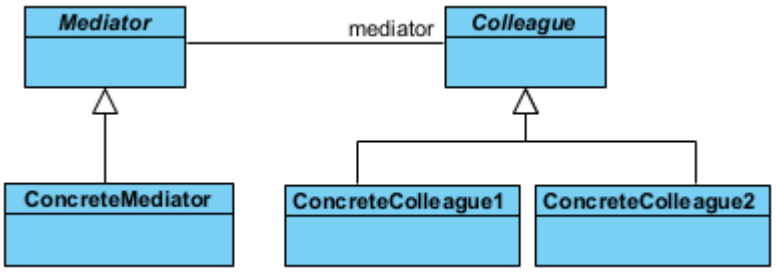
Hatıra kalıbı, bir nesnenin belirli bir andaki durumunu saklayıp daha sonra bu duruma geri dönmeyi sağlar. Durumu kaydeden nesne `Originator`, kayıt nesnesi `Memento`, kayıtları saklayan nesne ise `Caretaker` olarak adlandırılır. Bu kalıp özellikle geri alma, kayıt noktası ve geçmiş yönetimi gereken sistemlerde kullanışlıdır.

§5.4.9 Ara Bulucu

Ara Bulucu Kalıbı (*Mediator Pattern*), nesneler arasındaki karmaşık iletişim ağını basitleştiren ve merkezi bir kontrol sağlayan davranışsal (*behavioral*) bir tasarım kalıbıdır.

Bu kalıbın temel amacı, nesnelerin birbirleriyle doğrudan iletişim kurmasını engelleyerek aralarındaki bağımlılığı azaltmaktır (*loose coupling*). Nesneler birbirini doğrudan tanımak yerine, tüm haberleşmeyi tek bir merkezi "aracı" nesne üzerinden gerçekleştirir.

Kompleks sistemlerde nesne sayısı arttıkça aralarındaki bağlantılar bir "örümcek ağı" halini alabilir. Ara bulucu kalıbı, bu karmaşık bağıntıları merkezi bir noktada toplayarak kodun bakımını ve okunabilirliğini kolaylaştırır.



Şekil 5.21: Ara Bulucu Kalıbı UML Diyagramı

Ara bulucu kalıbındaki temel roller şunlardır:

- ◇ **Ara Bulucu** (`Mediator`): Nesneler arasındaki iletişim protokolünü (arayüzünü) tanımlar. Meslektaşlar arasında mesaj iletim kurallarını belirler.
- ◇ **Somut Ara Bulucu** (`ConcreteMediator`): İletişim trafiğini yöneten asıl merkezdir. Tüm meslektaşları tanır ve aralarındaki koordinasyonu sağlar. Bir bileşenden mesaj geldiğinde uygun diğer bileşenlere yönlendirir.
- ◇ **Meslektaş** (`Colleague`): Kendi işlevini yerine getiren bağımsız sınıflardır. Diğer meslektaşların varlığından haberdar değildirler; sadece ara bulucu ile haberleşirler.

Bu kalıp günlük hayatta bir havaalanı kontrol kulesine benzetilebilir. Uçaklar pistte doğrudan birbirleriyle konuşmazlar; bunun yerine tüm pilotlar kule ile iletişim kurar ve kule uçaklar arası trafiği koordine eder.

Java ile Temel Ara Bulucu Örneği

Aşağıdaki örnekte, kullanıcıların birbirleriyle doğrudan mesajlaşmak yerine ortak bir sohbet odası (`ChatRoom`) üzerinden haberleştiği temel bir yapı kurulmuştur.

Önce ara bulucu arayüzünü yazalım:

```
interface Mediator {
  void registerColleague(Colleague colleague);
  void relayMessage(String message, Colleague sender);
}
```

Kod 5.203: Java Dilinde Mediator Arayüzü

Şimdi meslektaşların türeyeceği soyut sınıfı yazalım:

```
abstract class Colleague {
  protected String name;
  protected Mediator mediator;

  public Colleague(String name, Mediator mediator) {
    this.name = name;
    this.mediator = mediator;
  }

  public String getName() {
    return name;
  }

  public void send(String message) {
    mediator.relayMessage(message, this);
  }

  public void receive(String message, String senderName) {
    System.out.println("[" + name + "] " + senderName + " kişisinden mesaj aldı: " +
        message);
  }
}
```

Kod 5.204: Java Dilinde Colleague Sınıfı

Şimdi somut ara bulucu sınıfımızı (Sohbet Odası) yazalım:

```
import java.util.ArrayList;
import java.util.List;

class ChatRoom implements Mediator {
  private final List<Colleague> colleagues = new ArrayList<>();

  @Override
  public void registerColleague(Colleague colleague) {
    colleagues.add(colleague);
  }

  @Override
  public void relayMessage(String message, Colleague sender) {
    System.out.println("--- LOG: " + sender.getName() + " bir mesaj yayınladı ---");

    for (Colleague colleague : colleagues) {
      // Mesajı gönderen kişiye geri göndermiyoruz
      if (colleague != sender) {
        colleague.receive(message, sender.getName());
      }
    }
  }
}
```

Kod 5.205: Java Dilinde ConcreteMediator (ChatRoom) Sınıfı

Somut meslektaş sınıfını (Kullanıcı) tanımlayalım:

```
class User extends Colleague {
  public User(String name, Mediator mediator) {
```

```
    super(name, mediator);
  }
}
```

Kod 5.206: Java Dilinde Somut Meslektaş (User) Sınıfı

İstemci kod:

```
public class Main {
  public static void main(String[] args) {
    ChatRoom oda = new ChatRoom();

    User ilhan = new User("Ilhan", oda);
    User mehmet = new User("Mehmet", oda);
    User ayse = new User("Ayse", oda);

    oda.registerColleague(ilhan);
    oda.registerColleague(mehmet);
    oda.registerColleague(ayse);

    ilhan.send("Selam millet!");
    System.out.println();
    mehmet.send("Merhabalar Ilhan.");
    System.out.println();
    ayse.send("Nasılsınız?");
  }
}
```

Kod 5.207: Java Dilinde Ara Bulucu Kalıbı Kullanımı

Program çıktısı:

```
--- LOG: Ilhan bir mesaj yayınladı ---
[Mehmet] Ilhan kişisinden mesaj aldı: Selam millet!
[Ayse] Ilhan kişisinden mesaj aldı: Selam millet!

--- LOG: Mehmet bir mesaj yayınladı ---
[Ilhan] Mehmet kişisinden mesaj aldı: Merhabalar Ilhan.
[Ayse] Mehmet kişisinden mesaj aldı: Merhabalar Ilhan.

--- LOG: Ayse bir mesaj yayınladı ---
[Ilhan] Ayse kişisinden mesaj aldı: Nasılsınız?
[Mehmet] Ayse kişisinden mesaj aldı: Nasılsınız?
```

Bu örnekte kullanıcılar birbirlerinin bellek adreslerini veya nesne referanslarını bilmeden, tamamen `ChatRoom` üzerinden haberleşmiştir.

Gerçek Dünya Örneği: Akıllı Ev Otomasyon Sistemi

Ara bulucu kalıbının bir diğer yaygın kullanım alanı akıllı ev sistemleridir. Klima açıldığında pencerelerin otomatik kapanması, yangın sensörü tetiklendiğinde alarmın çalması ve ışıkların yanması gibi bileşenler arası mantıksal kararlar merkezi bir `SmartHomeHub` tarafından yönetilir.

Önce ara bulucu arayüzünü tanımlayalım:

```
interface SmartHomeMediator {
  void notify(Component sender, String event);
  void registerComponent(Component component);
}
```

Kod 5.208: Java Dilinde Akıllı Ev Mediator Arayüzü

Temel bileşen (Component) sınıfı:

```
abstract class Component {
  protected SmartHomeMediator mediator;

  public void setMediator(SmartHomeMediator mediator) {
```

```
    this.mediator = mediator;
  }

  public abstract String getName();
}
```

Kod 5.209: Java Dilinde Temel Bileşen Sınıfı

Somut Bileşenler (Sensörler ve Eyleyiciler):

```
class SmokeDetector extends Component {
  @Override
  public String getName() { return "Duman Sensörü"; }

  public void detectSmoke() {
    System.out.println(getName() + ": Duman algılandı!");
    mediator.notify(this, "SMOKE_DETECTED");
  }
}

class SprinklerSystem extends Component {
  @Override
  public String getName() { return "Fıskiye Sistemi"; }

  public void activate() {
    System.out.println(getName() + ": Su püskürtme sistemi aktif edildi.");
  }
}

class AlarmSystem extends Component {
  @Override
  public String getName() { return "Alarm"; }

  public void soundAlarm() {
    System.out.println(getName() + ": Sirenler çalmaya başladı! DİKKAT!");
  }
}
```

Kod 5.210: Java Dilinde Somut Bileşen Sınıfları

Merkezi Ev Otomasyonu (Somut Ara Bulucu):

```
class SmartHomeHub implements SmartHomeMediator {
  private SmokeDetector smokeDetector;
  private SprinklerSystem sprinklerSystem;
  private AlarmSystem alarmSystem;

  public void setSmokeDetector(SmokeDetector smokeDetector) {
    this.smokeDetector = smokeDetector;
    smokeDetector.setMediator(this);
  }

  public void setSprinklerSystem(SprinklerSystem sprinklerSystem) {
    this.sprinklerSystem = sprinklerSystem;
    sprinklerSystem.setMediator(this);
  }

  public void setAlarmSystem(AlarmSystem alarmSystem) {
    this.alarmSystem = alarmSystem;
    alarmSystem.setMediator(this);
  }

  @Override
  public void registerComponent(Component component) {
    // Hub bileşenleri doğrudan alan veya liste tutan esnek yapı kurabilir
  }
```

```
    @Override
    public void notify(Component sender, String event) {
      if (sender == smokeDetector && event.equals("SMOKE_DETECTED")) {
        System.out.println("Hub: Acil durum senaryosu çalıştırılıyor...");
        sprinklerSystem.activate();
        alarmSystem.soundAlarm();
      }
    }
  }
```

Kod 5.211: Java Dilinde Somut Ara Bulucu (Hub) Sınıfı

İstemci Kod:

```
  public class Main {
    public static void main(String[] args) {
      SmartHomeHub hub = new SmartHomeHub();

      SmokeDetector smokeSensor = new SmokeDetector();
      SprinklerSystem sprinkler = new SprinklerSystem();
      AlarmSystem alarm = new AlarmSystem();

      hub.setSmokeDetector(smokeSensor);
      hub.setSprinklerSystem(sprinkler);
      hub.setAlarmSystem(alarm);

      // Duman algılanıyor
      smokeSensor.detectSmoke();
    }
  }
```

Kod 5.212: Java Dilinde Akıllı Ev Otomasyonu Kullanımı

Program Çıktısı:

```
Duman Sensörü: Duman algılandı!
Hub: Acil durum senaryosu çalıştırılıyor...
Fıskiye Sistemi: Su püskürtme sistemi aktif edildi.
Alarm: Sirenler çalmaya başladı! DİKKAT!
```

Ara Bulucu Kalıbının Avantajları

- Nesneler arası bağımlılıkları azaltır (*decoupling*).
- Sınıflar arası doğrudan iletişim karmaşasını önler.
- Bileşenlerin yeniden kullanılabilirliğini (*reusability*) artırır.
- İletişim mantığını tek bir yerde toplayarak yönetimi kolaylaştırır.

Ara Bulucu Kalıbının Dezavantajları

- Ara bulucu sınıfı zamanla aşırı büyüyebilir ve bir "Tanrı Nesne"ye (*God Object*) dönüşebilir.
- Çok fazla işlevi yönettiği için bakım zorlaşabilir.

Ne Zaman Kullanılmalı?

- Bir grup nesne arasındaki iletişim standart dışı ve karmaşık bir yapıya sahipse.
- Bir nesne diğer birçok nesneyle bağımlı olduğu için değiştirilmesi veya yeniden kullanılması güçse.
- Çeşitli alt sınıflar arasında çok fazla ara bağlantı bulunuyorsa ve iletişim tek bir merkezden yönetilmek isteniyorsa.

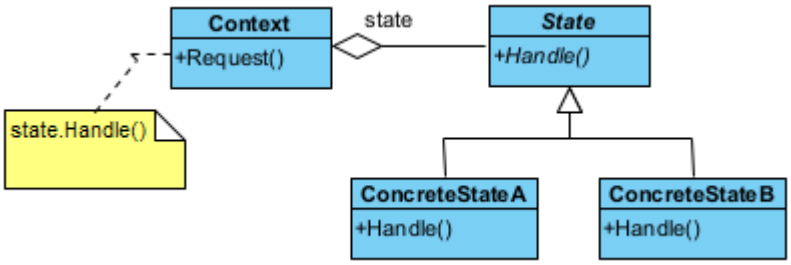
Özet

Ara bulucu kalıbı, nesnelerin doğrudan haberleşmesi yerine araya merkezi bir "aracı" koyarak iletişimi sadeleştirir. Kodun modülerliğini artırır ancak merkezi sınıfın çok şişmemesine dikkat edilmelidir.

§5.4.10 Durum

Durum Kalıbı (*State Pattern*), bir nesnenin içsel durumuna göre davranışını değiştirmesine olanak tanıyan davranışsal (*behavioral*) bir tasarım kalıbıdır.

Bu kalıbın temel amacı, nesnenin durumu değiştiğinde sanki sınıfını değiştiriyormuş gibi görünmesini sağlamaktır. Duruma bağlı olarak değişen koşullu ifadeler (`if-else` veya `switch-case` blokları) yerine, her durumu ayrı bir sınıfa (*class*) hapsederek kodun bakımını ve genişletilebilirliğini kolaylaştırır.



Şekil 5.22: Durum Kalıbı UML Diyagramı

Durum kalıbındaki temel roller şunlardır:

- **Bağlam** (`Context`): İstemcinin (*Client*) etkileşime girdiği ana sınıftır. Mevcut durum nesnesini bir referans olarak tutar ve gelen istekleri bu durum nesnesine devreder.
- **Soyut Durum** (`State`): Tüm somut durumların uyması gereken arayüzü (*Interface*) veya soyut sınıfı tanımlar.
- **Somut Durumlar** (`ConcreteState`): Her sınıf, belirli bir durumdaki davranışı temsil eder (Örn: `OynatiliyorState`, `DuraklatildiState`). Bir durumdan diğerine geçiş mantığını genellikle bu sınıflar veya bağlam yönetir.

Bu kalıbın bağlam (`Context`) nesnesi, diğer tasarım kalıpları ile iç içe kullanıldığında özel varyasyonlar oluşturabilir. Örneğin **Null Nesne Kalıbı** (*Null Object Pattern*) ile birleştirildiğinde, geçersiz veya eksik durumlar için hata fırlatmak yerine varsayılan sessiz davranışlar sergileyen durum sınıfları yazılabilir.

Java ile Temel Durum Örneği

Aşağıdaki örnekte, iki farklı durum arasında geçiş yapan ve durumuna göre davranış sergileyen temel bir yapı (`ConcreteStateA` ve `ConcreteStateB`) kurulmuştur.

Önce soyut durum arayüzünü ve bağlam sınıfını yazalım:

```
interface State {
  void handle(Context context);
}

class Context {
  private State currentState;

  public Context(State state) {
    this.currentState = state;
  }

  public void setState(State state) {
    System.out.println("Context: Durum degistiriliyor...");
    this.currentState = state;
  }

  public void request() {
    currentState.handle(this);
  }
}
```

Kod 5.213: Java Dilinde State Arayüzü ve Context Sınıfı

Şimdi somut durum sınıflarını yazalım:

```
class ConcreteStateA implements State {
  @Override
  public void handle(Context context) {
    System.out.println("ConcreteStateA: Talep isleniyor. B durumuna gecis
        tetikleniyor.");
    context.setState(new ConcreteStateB());
  }
}

class ConcreteStateB implements State {
  @Override
  public void handle(Context context) {
    System.out.println("ConcreteStateB: Talep isleniyor. A durumuna geri donuluyor.")
        ;
    context.setState(new ConcreteStateA());
  }
}
```

Kod 5.214: Java Dilinde Somut Durum Sınıfları

İstemci kod:

```
public class Main {
  public static void main(String[] args) {
    // Baslangic durumu A ile Context olusturuluyor
    Context context = new Context(new ConcreteStateA());

    // Istekler gonderildikce durumlar kendi iclerinde birbirini tetikler
    context.request(); // A -> B gecisi
    context.request(); // B -> A gecisi
    context.request(); // A -> B gecisi
  }
}
```

Kod 5.215: Java Dilinde Durum Kalıbı Kullanımı

Program çıktısı:

```
ConcreteStateA: Talep isleniyor. B durumuna gecis tetikleniyor.
Context: Durum degistiriliyor...
ConcreteStateB: Talep isleniyor. A durumuna geri donuluyor.
Context: Durum degistiriliyor...
ConcreteStateA: Talep isleniyor. B durumuna gecis tetikleniyor.
Context: Durum degistiriliyor...
```

Gerçek Dünya Örneği: Otomat Makinesi (*Vending Machine*)

Durum kalıbının en klasik ve gerçekçi örneklerinden biri otomat makineleridir. Makinenin davranışı; içinde para olup olmamasına, ürünün kalıp kalmamasına veya ürün seçilip seçilmediğine göre tamamen değişir. Örneğin para yoksa ürün seçme isteği reddedilir; para varsa kabul edilir.

Önce ortak durum arayüzünü tanımlayalım:

```
interface VendingState {
  void insertMoney();
  void ejectMoney();
  void selectProduct();
  void dispense();
}
```

Kod 5.216: Java Dilinde Otomat State Arayüzü

Bağlam sınıfımız olan VendingMachine:

```
class VendingMachine {
  private VendingState noMoneyState;
  private VendingState hasMoneyState;
  private VendingState soldState;
  private VendingState soldOutState;

  private VendingState currentState;
  private int count = 0;

  public VendingMachine(int productCount) {
    noMoneyState = new NoMoneyState(this);
    hasMoneyState = new HasMoneyState(this);
    soldState = new SoldState(this);
    soldOutState = new SoldOutState(this);

    this.count = productCount;
    if (count > 0) {
      currentState = noMoneyState;
    } else {
      currentState = soldOutState;
    }
  }

  public void setState(VendingState state) {
    this.currentState = state;
  }

  public VendingState getNoMoneyState() { return noMoneyState; }
  public VendingState getHasMoneyState() { return hasMoneyState; }
  public VendingState getSoldState() { return soldState; }
  public VendingState getSoldOutState() { return soldOutState; }

  public int getCount() { return count; }
  public void releaseProduct() {
    if (count > 0) {
      System.out.println(">>> URUN VERILDI. Afiyet olsun!");
      count--;
    }
  }

  // Proxy metotlar (İstekler geçerli duruma yönlendirilir)
  public void insertMoney() { currentState.insertMoney(); }
  public void ejectMoney() { currentState.ejectMoney(); }
  public void selectProduct() { currentState.selectProduct(); }
  public void dispense() { currentState.dispense(); }
}
```

Kod 5.217: Java Dilinde VendingMachine Sınıfı

Somut Durum Sınıfları:

```
class NoMoneyState implements VendingState {
  private final VendingMachine machine;
  public NoMoneyState(VendingMachine machine) { this.machine = machine; }

  @Override
  public void insertMoney() {
    System.out.println("Para kabul edildi.");
    machine.setState(machine.getHasMoneyState());
  }
  @Override
  public void ejectMoney() { System.out.println("Hata: Zaten para yok."); }
  @Override
  public void selectProduct() { System.out.println("Hata: Once para yuklemelisiniz.")
      ; }
```

```
    @Override
    public void dispense() { System.out.println("Hata: Odeme yapilmadan urun verilmez."
        ); }
}

class HasMoneyState implements VendingState {
    private final VendingMachine machine;
    public HasMoneyState(VendingMachine machine) { this.machine = machine; }

    @Override
    public void insertMoney() { System.out.println("Bilgi: Zaten para yuklu."); }
    @Override
    public void ejectMoney() {
        System.out.println("Para iade ediliyor...");
        machine.setState(machine.getNoMoneyState());
    }
    @Override
    public void selectProduct() {
        System.out.println("Urun secildi, hazirlaniyor...");
        machine.setState(machine.getSoldState());
        machine.dispense();
    }
    @Override
    public void dispense() { System.out.println("Bilgi: Once urun secimi yapmalisiniz."
        ); }
}

class SoldState implements VendingState {
    private final VendingMachine machine;
    public SoldState(VendingMachine machine) { this.machine = machine; }

    @Override
    public void insertMoney() { System.out.println("Lutfen bekleyin: Urununuz
        hazirlaniyor."); }
    @Override
    public void ejectMoney() { System.out.println("Hata: Islem geri alinamaz, urun
        veriliyor."); }
    @Override
    public void selectProduct() { System.out.println("Hata: Islem zaten devam ediyor.")
        ; }
    @Override
    public void dispense() {
        machine.releaseProduct();
        if (machine.getCount() > 0) {
            machine.setState(machine.getNoMoneyState());
        } else {
            System.out.println("Dikkat: Son urun satildi!");
            machine.setState(machine.getSoldOutState());
        }
    }
}

class SoldOutState implements VendingState {
    @Override
    public void insertMoney() { System.out.println("Hata: Stok bitti, para
        yukleyemezsiniz."); }
    @Override
    public void ejectMoney() { System.out.println("Hata: Para yok."); }
    @Override
    public void selectProduct() { System.out.println("Hata: Stok bitti."); }
    @Override
    public void dispense() { System.out.println("Hata: Stok yok."); }
}
```

Kod 5.218: Java Dilinde Otomat Somut Durum Sınıfları

İstemci Kod:

```
public class Main {
  public static void main(String[] args) {
    // 2 ürünlük bir otomat oluşturalım
    VendingMachine machine = new VendingMachine(2);

    System.out.println("--- Senaryo 1: Basarili Alim ---");
    machine.insertMoney();
    machine.selectProduct();

    System.out.println("\n--- Senaryo 2: Para Iadesi ---");
    machine.insertMoney();
    machine.ejectMoney();
  }
}
```

Kod 5.219: Java Dilinde Otomat Örneği Kullanımı

Program Çıktısı:

```
--- Senaryo 1: Basarili Alim ---
Para kabul edildi.
Urun secildi, hazirlaniyor...
>>> URUN VERILDI. Afiyet olsun!

--- Senaryo 2: Para Iadesi ---
Para kabul edildi.
Para iade ediliyor...
```

Tasarım Sürecinde Dikkat Edilmesi Gereken Hususlar

- **Durum geçişleri kim tarafından tanımlanmalıdır?** Eğer geçiş koşulları sabit ve basitse, yönetim `Context` sınıfı içinde yapılabilir. Şartlar karmaşık ve dinamik ise geçiş mantığı `State` nesnelerinin içine devredilir.
- **Tablo tabanlı alternatif:** Durum geçişlerinin kurallarla bir matris üzerinden yürütüldüğü yaklaşımdır. Yeni durum eklemek kolaylaşır ancak çalışma zamanı hızı düşebilir.
- **State nesnelerini yaratma ve yok etme:** Durum nesnelerinin her geçişte anlık olarak yaratılıp silinmesi (*dynamic instantiation*) veya baştan bir kere yaratılıp bellekte tutulması (*flyweight-like approach*) performans açısından değerlendirilmelidir. Genellikle ihtiyaç anında yaratılıp yok edilmesi tercih edilir.

Durum Kalıbının Avantajları

- Karmaşık ve şişkin `if-else` veya `switch-case` bloklarını ortadan kaldırır.
- Duruma özel davranışları ayrı sınıflara toplayarak tekil sorumluluk ilkesini (*Single Responsibility Principle*) destekler.
- Yeni durumlar eklenirken mevcut sınıfların değiştirilmesini önleyerek açık/kapalı prensibini (*Open/Closed Principle*) güçlendirir.

Durum Kalıbının Dezavantajları

- Sınıf sayısını (her durum için ayrı sınıf) artırdığı için kod tabanını karmaşıklaştırabilir.
- Durum geçişleri mantığı dağınık hale gelirse takibi zorlaşabilir.

Ne Zaman Kullanılmalı?

- Bir nesnenin davranışı iç durumuna bağlı olarak değişiyorsa ve bu durumlar çalışma zamanında dinamik olarak değişebiliyorsa.
- Kod içerisinde nesnenin mevcut durumunu kontrol eden devasa koşullu ifadeler (`if-else` yığınları) yer alıyorsa.

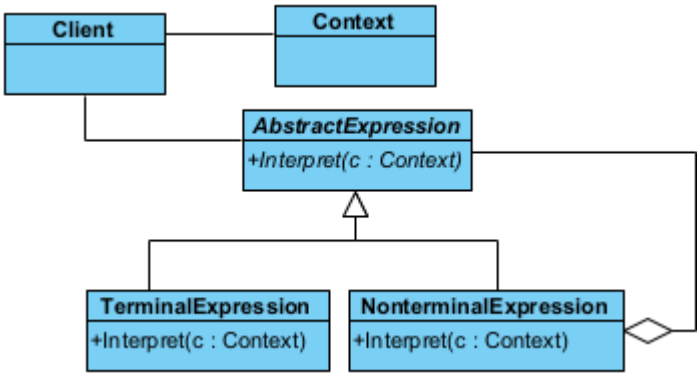
Özet

Durum kalıbı, nesne davranışlarının içsel duruma göre biçimlenmesini sağlayan güçlü bir mimari sunar. Koşullu ifadeleri sınıflara bölerek kodu daha modüler ve sürdürülebilir hale getirir.

§5.4.11 Yorumlayıcı

Yorumlayıcı Kalıbı (*Interpreter Pattern*), belirli bir gramer yapısına sahip ifadeleri (matematiksel işlemler, SQL komutları, arama filtreleri vb.) çözümlemek ve yorumlamak için kullanılan davranışsal (*behavioral*) bir tasarım kalıbıdır.

Bu kalıbın temel amacı, bir dil bilgisi için nesne yönelimli bir temsil oluşturmak ve bu temsil üzerinden gelen girdileri yorumlayıp sonuca dönüştürecek mekanizmayı sağlamaktır.



Şekil 5.23: Yorumlayıcı Kalıbı UML Diyagramı

Yorumlayıcı kalıbındaki temel roller şunlardır:

- **Soyut İfade** (`AbstractExpression`): Tüm ağaç düğümlerinin sahip olduğu `interpret()` yöntemini tanımlayan ortak arayüzdür.
- **Uç İfade** (`TerminalExpression`): Gramerin en küçük, bölünemez birimleridir (Sayılar, sabitler veya değişkenler).
- **Uç Olmayan İfade** (`NonTerminalExpression`): Gramerin kurallarını (Toplama, Çarpma, Koşul vb.) temsil eder. Genellikle içinde diğer uç veya uç olmayan ifadeleri barındırarak bir ağaç yapısı (*Composite*) oluşturur.
- **Bağlam** (`Context`): Yorumlama sırasında ihtiyaç duyulan küresel bilgileri (değişken değerleri, sembol tabloları vb.) taşır.

Java ile Temel Yorumlayıcı Örneği

Aşağıdaki örnekte, `x` ve `y` değişkenleri ile sabit sayıların kullanıldığı basit bir aritmetik ifade ağacı (`(x + y) + 20`) Java dilinde yorumlanmıştır.

Önce bağlam sınıfımızı ve soyut ifade arayüzünü yazalım:

```
import java.util.HashMap;
import java.util.Map;

class Context {
  private final Map<String, Integer> variables = new HashMap<>();

  public void setVariable(String name, int value) {
    variables.put(name, value);
  }

  public int getVariable(String name) {
    if (!variables.containsKey(name)) {
      throw new IllegalArgumentException("Degisken bulunamadi: " + name);
    }
    return variables.get(name);
  }
```

```
}

interface AbstractExpression {
  int interpret(Context context);
}
```

Kod 5.220: Java Dilinde Context ve AbstractExpression

Şimdi uç ve uç olmayan ifade sınıflarını yazalım:

```
class NumberExpression implements AbstractExpression {
  private final int number;

  public NumberExpression(int number) {
    this.number = number;
  }

  @Override
  public int interpret(Context context) {
    return number;
  }
}

class VariableExpression implements AbstractExpression {
  private final String name;

  public VariableExpression(String name) {
    this.name = name;
  }

  @Override
  public int interpret(Context context) {
    return context.getVariable(name);
  }
}

class AddExpression implements AbstractExpression {
  private final AbstractExpression left;
  private final AbstractExpression right;

  public AddExpression(AbstractExpression left, AbstractExpression right) {
    this.left = left;
    this.right = right;
  }

  @Override
  public int interpret(Context context) {
    return left.interpret(context) + right.interpret(context);
  }
}
```

Kod 5.221: Java Dilinde Terminal ve NonTerminal Sınıfları

İstemci kod:

```
public class Main {
  public static void main(String[] args) {
    Context context = new Context();
    context.setVariable("x", 10);
    context.setVariable("y", 5);

    // Ifade: (x + y) + 20
    // Agac yapisi olusturuluyor
    AbstractExpression expression = new AddExpression(
    new AddExpression(
    new VariableExpression("x"),
    new VariableExpression("y")
```

```
      ),
      new NumberExpression(20)
      );

      System.out.println("Sonuc: " + expression.interpret(context));
    }
  }
```

Kod 5.222: Java Dilinde Yorumlayıcı Kalıbı Kullanımı

Program çıktısı:

```
Sonuc: 35
```

Gerçek Dünya Örneği: Dinamik Aritmetik İfade Çözümleyici

Sabit ağaç yapısı kurmak yerine, düz metin halinde gelen matematiksel ifadeleri (*string*) dinamik olarak çözümleyen bir yorumlayıcı tasarlayabiliriz.

```
import java.util.*;

interface Expression {
  int interpret();
}

class NumExpr implements Expression {
  private final int number;
  public NumExpr(int number) { this.number = number; }
  @Override public int interpret() { return number; }
}

class AddExpr implements Expression {
  private final Expression left, right;
  public AddExpr(Expression left, Expression right) { this.left = left; this.right =
      right; }
  @Override public int interpret() { return left.interpret() + right.interpret(); }
}

class MulExpr implements Expression {
  private final Expression left, right;
  public MulExpr(Expression left, Expression right) { this.left = left; this.right =
      right; }
  @Override public int interpret() { return left.interpret() * right.interpret(); }
}

class InterpreterEngine {
  public int interpret(String rawExpression) {
    Expression expressionTree = buildExpressionTree(rawExpression);
    return expressionTree.interpret();
  }

  private List<String> tokenize(String str) {
    List<String> tokens = new ArrayList<>();
    Scanner scanner = new Scanner(str);
    while (scanner.hasNext()) {
      tokens.add(scanner.next());
    }
    return tokens;
  }

  private Expression buildExpressionTree(String expression) {
    List<String> tokens = tokenize(expression);
    Stack<Expression> nodes = new Stack<>();
    Stack<Character> operators = new Stack<>();

    Runnable applyOp = () -> {
```

```
        Expression right = nodes.pop();
        Expression left = nodes.pop();
        char op = operators.pop();
        if (op == '+') {
          nodes.push(new AddExpr(left, right));
        } else if (op == '*') {
          nodes.push(new MulExpr(left, right));
        }
      };

      for (String token : tokens) {
        if (Character.isDigit(token.charAt(0))) {
          nodes.push(new NumExpr(Integer.parseInt(token)));
        } else {
          char currentOp = token.charAt(0);
          while (!operators.empty() &&
          ((currentOp == '+' && (operators.peek() == '+' || operators.peek() == '*'))
              ||
          (currentOp == '*' && operators.peek() == '*'))) {
            applyOp.run();
          }
          operators.push(currentOp);
        }
      }
      while (!operators.empty()) {
        applyOp.run();
      }
      return nodes.peek();
    }
  }
```

Kod 5.223: Java Dilinde Dinamik Interpreter Sınıfı

İstemci Kod ve Test Senaryoları:

```
public class MainParser {
  public static void main(String[] args) {
    InterpreterEngine parser = new InterpreterEngine();

    String expr1 = "2 + 3 * 4";       // Beklenen: 14
    String expr2 = "10 * 2 + 5 * 3"; // Beklenen: 35
    String expr3 = "1 + 2 + 3 + 4";  // Beklenen: 10

    System.out.println(expr1 + " = " + parser.interpret(expr1));
    System.out.println(expr2 + " = " + parser.interpret(expr2));
    System.out.println(expr3 + " = " + parser.interpret(expr3));
  }
}
```

Kod 5.224: Java Dilinde Dinamik Interpreter Kullanımı

Program Çıktısı:

```
2 + 3 * 4 = 14
10 * 2 + 5 * 3 = 35
1 + 2 + 3 + 4 = 10
```

Tasarım Sürecinde Dikkat Edilmesi Gereken Hususlar

- **Soyut sözdizim ağacının (AST) oluşturulması:** Kalıp, metinlerin ağaca nasıl dönüştürüleceğini dikte etmez; bu işlem manuel, tablo tabanlı veya ayrıştırıcılar (*parsers*) aracılığıyla yapılabilir.
- **Yorumlama mantığının ayrıştırılması:** Çok karmaşık gramer yapılarında her sınıfa `interpret()` metodu yazmak yerine **Ziyaretçi Kalıbı** (*Visitor Pattern*) kullanılarak mantık dışarıya taşınabilir.

- ◇ **Sineksıklet Kalıbı** (*Flyweight Pattern*): Cümle içindeki tekrar eden uç birimler (örneğin sık kullanılan aynı sayılar veya operatörler) ortak paylaşılabilir nesneler olarak tasarlanabilir.

Yorumlayıcı Kalıbının Avantajları

- ◇ Dil ve gramer kurallarını değiştirmek ve yeni kurallar eklemek son derece kolaydır.
- ◇ Basit gramer yapılarını uygulamaya entegre etmek için harici araçlara olan bağımlılığı ortadan kaldırır.

Yorumlayıcı Kalıbının Dezavantajları

- ◇ Çok karmaşık dil yapılarında sınıf sayısı (her kural için bir sınıf) hızla artar ve ağaç yapısı yönetilemez hale gelebilir.
- ◇ Performans maliyeti yüksektir; derin ağaç yapıları özyinelemeli (*recursive*) olarak gezildiği için yavaştır.

Ne Zaman Kullanılmalı?

- ◇ Çözümlenecek dil veya ifade yapısı basit ve değişime açık olduğunda.
- ◇ Bir ifadenin soyut sözdizim ağacı olarak temsil edilip tekrar tekrar çalıştırılması gerektiğinde.

Özet

Yorumlayıcı kalıbı, belirli bir dil kuralını nesne yönelimli sınıflarla modelleyerek uygulama içinde yorumlamaya olanak tanır. Basit gramerler için etkili bir çözüm sunarken, karmaşık dillerde parser jeneratörleri tercih edilmelidir.

§5.4.12 MVC Mimari Kalıbı

MVC (*Model-View-Controller*), bir yazılım projesinde kodları sorumluluklarına göre üç ana parçaya ayıran bir mimari kalıptır. Temel amacı, veriyi (iş mantığı), kullanıcı arayüzünü ve bu ikisi arasındaki iletişimi kontrol eden yapıyı birbirinden bağımsız hale getirerek kodun bakımını ve ölçeklenebilirliğini kolaylaştırmaktır.

- ◇ **Veri ve Mantık** (`Model`): Uygulamanın beynidir. Veritabanı işlemleri, iş kuralları ve hesaplamalar burada yürütülür. "Veri nedir?" sorusuna cevap verir.
- ◇ **Arayüz / Görünüm** (`View`): Kullanıcının gördüğü ekrandır (HTML, grafik arayüz pencereleri vb.). Sadece veriyi görselleştirir; arka plandaki iş mantığıyla ilgilenmez. "Veri nasıl görünmeli?" sorusuna cevap verir.
- ◇ **Aracı / Kontrolcü** (`Controller`): Kullanıcıdan gelen istekleri karşılar. `Model`'den veriyi alır ve hangi `View`'da gösterileceğine karar verir. "Hangi veri nereye gitmeli?" sorusuna cevap verir.

Basit bir restoran senaryosunda MVC şu şekilde özetlenebilir:

- ◇ **Model (Aşçı):** Mutfağın içindedir. Yemeği hazırlar, malzemeleri kontrol eder. Müşteriyi görmez, sadece siparişe göre yemeği (veriyi) üretir.
- ◇ **View (Yemek Masası / Sunum):** Müşterinin önüne gelen tabak ve sunumdur. Müşteri yemeğin nasıl piştiğini görmez, sadece sunulan tabağı görür.
- ◇ **Controller (Garson):** Müşteriden siparişi alır, aşçıya iletir. Yemek piştiğinde ise tabağı alır ve müşteriye servis eder.

MVC kullanmanın temel avantajları şunlardır:

- ◇ **Kod Karmaşasını Önler:** Tasarımcı sadece `View` ile, yazılımcı ise `Model` ve `Controller` ile ilgilenir.
- ◇ **Bakım Kolaylığı:** Arayüzü değiştirmek istediğinizde veritabanı veya iş mantığı kodlarına dokunmanız gerekmez.
- ◇ **Yeniden Kullanılabilirlik:** Aynı `Model` verisini hem bir web arayüzünde hem de bir mobil uygulamada farklı `View` sınıflarıyla gösterebilirsiniz.

Java ile MVC Mimarisi Örneği

Aşağıdaki örnekte, bileşenler arasındaki bağımlılığı minimize edecek şekilde bir "Öğrenci Bilgi Sistemi" kurgulanmıştır. `Model` veriyi tutar, `View` ekrana basar, `Controller` ise aradaki akışı yönetir.

1. **Model (Veri ve İş Mantığı):** Sadece veriyi ve bu veriye erişim yöntemlerini içerir. Görselleştirme ile ilgilenmez.

```
class OgrenciModel {
  private String isim;
  private String numara;

  public OgrenciModel(String isim, String numara) {
    this.isim = isim;
    this.numara = numara;
  }

  public void setIsim(String isim) {
    this.isim = isim;
  }

  public String getIsim() {
    return isim;
  }

  public void setNumara(String numara) {
    this.numara = numara;
  }

  public String getNumara() {
    return numara;
  }
}
```

Kod 5.225: Java Dilinde OgrenciModel Sınıfı

2. **View (Görünüm):** Verinin kullanıcıya nasıl sunulacağını belirler. Genellikle yazdırma veya çizdirme yöntemlerine sahiptir.

```
class OgrenciView {
  public void ogrenciDetaylariniBas(String isim, String numara) {
    System.out.println("--- Ogrenci Bilgileri ---");
    System.out.println("Ad: " + isim);
    System.out.println("No: " + numara);
    System.out.println("-------------------------");
  }
}
```

Kod 5.226: Java Dilinde OgrenciView Sınıfı

3. **Controller (Denetleyici):** Model ve View arasındaki köprüdür. Kullanıcıdan gelen güncellemeleri Model'e yansıtır ve View'u güncellenmeye zorlar.

```
class OgrenciController {
  private final OgrenciModel model;
  private final OgrenciView view;

  public OgrenciController(OgrenciModel model, OgrenciView view) {
    this.model = model;
    this.view = view;
  }

  public void setOgrenciIsim(String isim) {
    model.setIsim(isim);
  }

  public String getOgrenciIsim() {
    return model.getIsim();
```

```
    }

    public void setOgrenciNumara(String numara) {
      model.setNumara(numara);
    }

    public String getOgrenciNumara() {
      return model.getNumara();
    }

    // View'u güncelleme komutu
    public void gorunumuGuncelle() {
      view.ogrenciDetaylariniBas(model.getIsim(), model.getNumara());
    }
  }
```

Kod 5.227: Java Dilinde OgrenciController Sınıfı

4. **Sistemin Çalıştırılması (İstemci):**

```
  public class Main {
    public static void main(String[] args) {
      // 1. Model'i (Veriyi) olustur
      OgrenciModel model = new OgrenciModel("Ahmet Yilmaz", "2026001");

      // 2. View'u (Arayuzu) olustur
      OgrenciView view = new OgrenciView();

      // 3. Controller'i bagla
      OgrenciController controller = new OgrenciController(model, view);

      // Ilk hali ekrana bas
      controller.gorunumuGuncelle();

      // Veriyi Controller uzerinden guncelle
      controller.setOgrenciIsim("Mehmet Demir");

      // Guncel hali tekrar bas
      System.out.println("\nVeri guncellendikten sonra:");
      controller.gorunumuGuncelle();
    }
  }
```

Kod 5.228: Java Dilinde MVC Kullanımı

Program Çıktısı:

```
--- Ogrenci Bilgileri ---
Ad: Ahmet Yilmaz
No: 2026001
-------------------------

Veri guncellendikten sonra:
--- Ogrenci Bilgileri ---
Ad: Mehmet Demir
No: 2026001
-------------------------
```

Mimari ile İlgili Değerlendirmeler

- **Bağımsızlık (*Decoupling*):** `OgrenciModel` sınıfının içinde ne konsol çıktı komutları (`System.out.println`) ne de doğrudan `OgrenciView` referansı bulunur. Bu sayede veriyi ekrana basmak yerine bir dosyaya kaydetmek isterseniz (yeni bir `View` ekleyerek), Model koduna hiç dokunmanız gerekmez.
- **Bellek Yönetimi:** Java, çöp toplayıcı (*Garbage Collector*) mekanizmasına sahip olduğu için nesne ömürleri otomatik olarak yönetilir.

- ◇ **Gözlemci Tasarım Kalıbı İlişkisi:** Gelişmiş MVC mimarilerinde, `Model` değiştiğinde `Controller`'a veya `View`'a otomatik haber verebilmek için genellikle **Gözlemci Kalıbı** (*Observer Pattern*) ile birlikte entegre edilir.

6. Bölüm
Temiz Kod ve Mimari Standartlar

Temiz kod, yalnızca derlenen ve çalışan kod anlamına gelmez. Temiz kod; okunabilir, anlaşılabilir, değiştirilebilir, test edilebilir ve sürdürülebilir koddur. Yazılım projelerinde asıl maliyet çoğu zaman ilk geliştirme aşamasında değil, bakım ve değişiklik aşamalarında ortaya çıkar. Bu nedenle iyi bir yazılımcı yalnızca problemi çözen kodu değil, gelecekte değiştirilebilecek ve başkaları tarafından anlaşılabilecek kodu yazmayı hedeflemelidir.

Önceki bölümlerde nesne yönelimli programlama, kalıtım, çok biçimlilik, arayüzler, istisna yönetimi, UML ve SOLID ilkeleri ele alınmıştır. Bu bölümde ise bu kavramların gerçek yazılım projelerinde nasıl temiz, düzenli ve mimari açıdan sağlıklı bir yapıya dönüştürülebileceği açıklanacaktır.

6.1 Temiz Kod Nedir?

Temiz kod, okuyan kişiye ne yaptığını açıkça anlatan koddur. Temiz kodda gereksiz karmaşıklık yoktur; sınıflar, fonksiyonlar ve değişkenler anlamlı isimlere sahiptir; hata yönetimi açık ve kontrollüdür; bağımlılıklar yönetilebilir durumdadır.

Temiz kodun temel özellikleri şunlardır:

- ◇ Okunabilir olmalıdır.
- ◇ Amacını açıkça ifade etmelidir.
- ◇ Gereksiz tekrarlardan kaçınmalıdır.
- ◇ Kolay test edilebilir olmalıdır.
- ◇ Değişikliklere karşı dayanıklı olmalıdır.
- ◇ Hata durumlarını açık biçimde yönetmelidir.

§6.1.1 Kodun Okunabilir Olması

Kod, yalnızca bilgisayar tarafından değil, insanlar tarafından da okunur. Hatta kaynak kodun ömrü boyunca insanlar tarafından okunma sayısı, bilgisayar tarafından derlenme sayısından çok daha fazladır. Bu nedenle kodun okunabilir olması, yazılım kalitesinin temel ölçütlerinden biridir.

Aşağıdaki Java kodu çalışabilir ancak okunabilir değildir:

```
public int h(int a, int b) {
  if (a > b) return a - b;
  else return b - a;
}
```

Kod 6.1: Okunabilir Olmayan Java Kodu

Aynı kod daha okunabilir şekilde şöyle yazılabilir:

```
public int mutlakFarkHesapla(int birinciSayi, int ikinciSayi) {
  if (birinciSayi > ikinciSayi) {
    return birinciSayi - ikinciSayi;
  }

  return ikinciSayi - birinciSayi;
```

```
}
```

Kod 6.2: Okunabilir Java Kodu

Bu örnekte fonksiyon adı, parametre adları ve kod biçimi daha açıktır. Kodun ne yaptığı, yorum satırına ihtiyaç duyulmadan anlaşılmaktadır.

§6.1.2 Kodun Niyetini Açık Etmesi

Temiz kod, yalnızca işlemi değil, niyeti de açık eder. Bir kod parçasını okuyan kişi, yazılımcının ne yapmak istediğini anlayabilmelidir.

Aşağıdaki Java kodunda değişkenlerin amacı belirsizdir:

```
double x = 1000;
double y = x * 0.18;
double z = x + y;
```

Kod 6.3: Niyeti Açık Olmayan Java Kodu

Daha açık hali şöyledir:

```
double urunFiyati = 1000;
double kdvTutari = urunFiyati * 0.18;
double toplamFiyat = urunFiyati + kdvTutari;
```

Kod 6.4: Niyeti Açık Java Kodu

Burada amaç yalnızca hesaplama yapmak değil, hesaplamanın ne anlama geldiğini de göstermektir.

§6.1.3 Yorum Satırları Ne Zaman Kullanılmalı?

Yorum satırları, kötü yazılmış kodu açıklamak için değil, kodun kendisinin anlatamadığı bağlamı açıklamak için kullanılmalıdır. Eğer bir kod parçası ancak yorumla anlaşılabiliyorsa, öncelikle kodun kendisi iyileştirilmelidir.

Kötü yorum örneği:

```
int yas = 20; // yas degiskenine 20 degeri atandi
```

Kod 6.5: Gereksiz Yorum Kullanımı

Bu yorum gereksizdir. Kod zaten bunu açıkça söylemektedir.

Faydalı yorum örneği:

```
// Banka kurali geregi 18 yasindan kucuk kullanicilar hesap acamaz.
if (yas < 18) {
  throw new IllegalArgumentException("Kullanici yasi 18'den kucuk olamaz.");
}
```

Kod 6.6: Java Dilinde Faydalı Yorum Kullanımı

Burada yorum, kodun ne yaptığını değil, neden yaptığını açıklar.

6.2 Anlamlı İsimlendirme

İsimlendirme, temiz kodun en önemli parçalarından biridir. Değişken, fonksiyon, sınıf, arayüz ve sabit isimleri, kodun niyetini açıkça göstermelidir.

§6.2.1 Değişken İsimlendirme

Değişken isimleri, değişkenin neyi temsil ettiğini açıkça anlatmalıdır.

Kötü Java örneği:

```
int x = 45;
int y = 60;
```

```
int z = x + y;
```

İyi Java örneği:

```
int vizeNotu = 45;
int finalNotu = 60;
int toplamNot = vizeNotu + finalNotu;
```

Tek harfli değişkenler yalnızca kısa döngülerde veya matematiksel bağlamlarda tercih edilmelidir.

§6.2.2 Fonksiyon ve Yöntem İsimlendirme

Fonksiyon ve yöntem isimleri genellikle fiil içermelidir. Çünkü fonksiyonlar bir eylem gerçekleştirir.

Kötü örnek:

```
void islem() { }
```

İyi örnek:

```
void faturaYazdir() { }
void kullaniciDogrula() { }
void toplamTutarHesapla() { }
```

Fonksiyon adı, fonksiyonun içinde ne yapıldığını mümkün olduğunca açık göstermelidir.

§6.2.3 Sınıf İsimlendirme

Sınıflar genellikle isim veya isim tamlaması ile adlandırılır. Çünkü sınıflar bir kavramı, varlığı veya sorumluluğu temsil eder.

Kötü sınıf isimleri:

```
class Manager { }
class Helper { }
class Processor { }
```

Bu isimler çok geneldir. Sınıfın ne yaptığı anlaşılamaz.

Daha iyi isimler:

```
class FaturaYazici { }
class KullaniciDogrulayici { }
class OdemeHesaplayici { }
```

§6.2.4 Sabit, Enum ve Arayüz İsimlendirme

Sabitler, değişmeyen değerleri temsil eder. Java dilinde sabitler genellikle `public static final` ile tanımlanır:

```
public static final double KDV_ORANI = 0.20;
public static final int MAKSIMUM_DENEME_SAYISI = 3;
```

Kod 6.7: Java Dilinde Sabit Tanımlama

Enum isimleri de anlamlı olmalıdır.

```
enum SiparisDurumu {
  HAZIRLANIYOR,
  KARGODA,
  TESLIM_EDILDI,
  IPTAL_EDILDI
}
```

Kod 6.8: Java Enum Örneği

6.3 Küçük Fonksiyonlar ve Yöntemler

Fonksiyonlar küçük, anlaşılır ve tek bir iş yapan yapılar olmalıdır. Bir fonksiyon ne kadar uzarsa, anlaşılması ve test edilmesi o kadar zorlaşır.

§6.3.1 Fonksiyonların Tek İş Yapması

Bir fonksiyon yalnızca tek bir soyutlama düzeyinde tek bir işi yapmalıdır.

Kötü Java örneği:

```
void siparisIsle() {
  // Siparis dogrula
  // Stok kontrol et
  // Odeme al
  // Fatura olustur
  // E-posta gonder
}
```

Kod 6.9: Birden Fazla İş Yapan Fonksiyon

Daha iyi tasarım:

```
void siparisIsle() {
  siparisDogrula();
  stokKontrolEt();
  odemeAl();
  faturaOlustur();
  bildirimGonder();
}
```

Kod 6.10: Tek İş Yapan Fonksiyonlara Ayrılmış Kod

Bu yapıda ana yöntem süreci yönetir; detaylar ayrı yöntemlere aktarılır.

§6.3.2 Parametre Sayısını Azaltma

Çok fazla parametre alan fonksiyonlar zor okunur ve hata yapmaya açıktır.

Kötü örnek:

```
void kullaniciOlustur(String ad, String soyad, String email,
String telefon, int yas, String adres) { }
```

Daha iyi yaklaşım, parametre nesnesi kullanmaktır:

```
class KullaniciBilgisi {
  String ad;
  String soyad;
  String email;
  String telefon;
  int yas;
  String adres;
}

void kullaniciOlustur(KullaniciBilgisi bilgi) {
  // Kullanici olusturma islemleri
}
```

§6.3.3 Yan Etkilerden Kaçınma

Yan etki, bir fonksiyonun beklenmeyen şekilde dış durumu değiştirmesidir. Yan etkiler kodu anlamayı ve test etmeyi zorlaştırır.

Kötü Java örneği:

```
int toplam = 0;

void ekle(int sayi) {
```

```
    toplam += sayi;
}
```

Daha iyi yaklaşım:

```
int topla(int mevcutToplam, int eklenecekSayi) {
  return mevcutToplam + eklenecekSayi;
}
```

§6.3.4 Erken Çıkış ve Guard Clause Kullanımı

İç içe geçmiş `if` blokları kodu karmaşıklaştırır. Geçersiz durumları fonksiyonun başında eleyerek kod daha okunabilir hale getirilebilir.

Kötü örnek:

```
void paraCek(double miktar) {
  if (miktar > 0) {
    if (miktar <= bakiye) {
      bakiye -= miktar;
    }
  }
}
```

İyi örnek:

```
void paraCek(double miktar) {
  if (miktar <= 0) {
    throw new IllegalArgumentException("Miktar pozitif olmalidir.");
  }

  if (miktar > bakiye) {
    throw new IllegalStateException("Yetersiz bakiye.");
  }

  bakiye -= miktar;
}
```

6.4 Küçük Sınıflar ve Sorumluluk Ayrımı

Sınıflar küçük ve tek sorumluluğa sahip olmalıdır. Bir sınıfın büyüklüğü yalnızca satır sayısıyla değil, üstlendiği sorumluluk sayısıyla ölçülmelidir.

§6.4.1 Sınıfın Boyutu Ne ile Ölçülür?

Bir sınıfın çok sayıda farklı nedenle değişmesi gerekiyorsa, o sınıf büyük ihtimalle birden fazla sorumluluk taşıyordur.

Kötü örnek:

```
class Fatura {
  public void toplamHesapla() { }
  public void veritabaninaKaydet() { }
  public void pdfOlustur() { }
  public void emailGonder() { }
}
```

Bu sınıf hem hesaplama, hem veri kaydetme, hem belge üretme, hem bildirim gönderme görevlerini üstlenmiştir.

Daha iyi tasarım:

```
class Fatura { }
class FaturaHesaplayici { }
class FaturaDeposu { }
class FaturaPdfOlusturucu { }
```

```
class FaturaBildirimServisi { }
```

§6.4.2 Tanrı Sınıf Problemi

Tanrı sınıf, sistemdeki hemen hemen her işi yapan büyük sınıftır. Genellikle `Manager`, `SystemController`, `ApplicationService` gibi isimlerle ortaya çıkar.

Kötü örnek:

```
class KutuphaneManager {
  public void kitapEkle() { }
  public void uyeEkle() { }
  public void oduncVer() { }
  public void cezaHesapla() { }
  public void raporOlustur() { }
  public void emailGonder() { }
  public void veritabaninaKaydet() { }
}
```

Daha iyi yaklaşım:

```
class KitapServisi { }
class UyeServisi { }
class OduncServisi { }
class CezaHesaplayici { }
class RaporServisi { }
class BildirimServisi { }
```

§6.4.3 Sınıfların Birlikte Değişen Davranışlara Göre Ayrılması

Birlikte değişen kodlar aynı sınıfta, farklı nedenlerle değişen kodlar farklı sınıflarda olmalıdır. Örneğin ceza hesaplama kuralları değiştiğinde yalnızca `CezaHesaplayici` sınıfı etkilenmelidir.

```
class CezaHesaplayici {
  double cezaHesapla(int gecikmeGunu) {
    return gecikmeGunu * 5.0;
  }
}
```

§6.4.4 Bağlaşım ve Uyum

Bağlaşım (coupling), sınıfların birbirine ne kadar bağımlı olduğunu gösterir. Uyum (cohesion) ise bir sınıfın içindeki üyelerin aynı amaca ne kadar hizmet ettiğini gösterir.

İyi tasarımda:

- ◇ Düşük bağlaşım olmalıdır.
- ◇ Yüksek uyum olmalıdır.

Düşük bağlaşım, değişikliklerin sistemin her yerine yayılmasını engeller. Yüksek uyum ise sınıfın tek ve açık bir amaca hizmet etmesini sağlar.

6.5 Kod Tekrarı ve Yeniden Kullanılabilirlik

Kod tekrarı, yazılım bakım maliyetini artıran en önemli problemlerden biridir. Aynı mantığın birden fazla yerde tekrar edilmesi, değişiklik gerektiğinde tüm kopyaların güncellenmesini zorunlu kılar.

§6.5.1 DRY İlkesi

DRY, Don't Repeat Yourself" ifadesinin kısaltmasıdır. Türkçeye Kendini Tekrar Etme" olarak çevrilebilir.

Kötü Java örneği:

```
double ogrenciIndirimi(double tutar) {
  return tutar * 0.10;
}

double kampanyaIndirimi(double tutar) {
  return tutar * 0.10;
}
```

Eğer aynı indirim oranı iş kuralını temsil ediyorsa, tekrar azaltılmalıdır:

```
class IndirimHesaplayici {
  private static final double INDIRIM_ORANI = 0.10;

  double indirimHesapla(double tutar) {
    return tutar * INDIRIM_ORANI;
  }
}
```

§6.5.2 Gereksiz Soyutlama Tehlikesi

Her tekrar hemen soyutlanmamalıdır. İki kod parçası şu an benziyor olabilir ancak farklı nedenlerle değişecekse aynı soyutlama altında birleştirilmemelidir.

Yanlış soyutlama, kod tekrarından daha tehlikeli olabilir. Çünkü yanlış soyutlama, farklı davranışları aynı yapıya zorlar.

§6.5.3 Kalıtım Yerine Bileşim

Kod tekrarını azaltmak için her zaman kalıtım kullanılmamalıdır. Kalıtım, sınıflar arasında güçlü bir bağ kurar. Eğer ilişki bir türüdür" değil de sahiptir" biçimindeyse bileşim tercih edilmelidir.

Kötü yaklaşım:

```
class Motor { }
class Araba extends Motor { }
```

Araba bir motor değildir; motoru vardır. Doğru yaklaşım:

```
class Motor { }

class Araba {
  private Motor motor = new Motor();
}
```

6.6 Hata Yönetimi ve İstisna Mimarisi

Hata yönetimi, temiz kodun önemli bir parçasıdır. Hata durumları iş kodunun arasına dağılmamalı, anlamlı istisna sınıfları ve tutarlı bir mimari ile yönetilmelidir.

§6.6.1 Hata Kodları Yerine İstisnalar

Hata kodları, fonksiyonun normal dönüş değeri ile hata bilgisini aynı kanal üzerinden taşır.

Kötü Java örneği:

```
int paraCek(double miktar) {
  if (miktar <= 0) return -1;
  if (miktar > bakiye) return -2;

  bakiye -= miktar;
  return 0;
}
```

Daha iyi yaklaşım:

```
void paraCek(double miktar) {
  if (miktar <= 0) {
    throw new IllegalArgumentException("Miktar pozitif olmalidir.");
  }

  if (miktar > bakiye) {
    throw new IllegalStateException("Yetersiz bakiye.");
  }

  bakiye -= miktar;
}
```

§6.6.2 Anlamlı İstisna Sınıfları

Genel istisnalar yerine alan odaklı istisnalar tercih edilmelidir.

Java örneği:

```
class YetersizBakiyeException extends RuntimeException {
  public YetersizBakiyeException() {
    super("Yetersiz bakiye.");
  }
}
```

§6.6.3 İstisnayı Yutmamak

Boş `catch` blokları hatanın kaybolmasına neden olur.

Kötü örnek:

```
try {
  dosyaOku();
} catch (Exception e) {
  // Hata yutuldu
}
```

Daha iyi yaklaşım:

```
try {
  dosyaOku();
} catch (IOException e) {
  System.err.println("Dosya okuma hatasi: " + e.getMessage());
  throw new RuntimeException(e);
}
```

§6.6.4 Kaynak Yönetimi

Java dilinde kaynak yönetimi için `try-with-resources` kullanılır:

```
try (BufferedReader reader = Files.newBufferedReader(Path.of("veri.txt"))) {
  String satir = reader.readLine();
}
```

§6.6.5 Katmanlara Göre Hata Yönetimi

Alt katmanda oluşan teknik hatalar, üst katmanda iş anlamı taşıyan hatalara dönüştürülebilir.

Java örneği:

```
try {
  repository.kaydet(kullanici);
} catch (SQLException e) {
  throw new KullaniciKaydedilemediException("Kullanici kaydedilemedi.", e);
}
```

6.7 Boş Değerler, Optional ve Geçersiz Durum Yönetimi

Boş değerler, yazılım hatalarının önemli kaynaklarından biridir. Null referanslar, özellikle nesne yönelimli dillerde çalışma zamanı hatalarına neden olabilir.

§6.7.1 Null Referans Problemi

Java'da null referans üzerinden yöntem çağırmak `NullPointerException` hatasına neden olur.

```
Kullanici kullanici = null;
System.out.println(kullanici.getAd()); // NullPointerException
```

§6.7.2 Optional Kullanımı

Bir değerin olmayabileceği açıkça belirtilmek isteniyorsa Optional kullanılabilir.

Java örneği:

```
Optional kullaniciBul(int id) {
  if (id == 1) {
    return Optional.of(new Kullanici("Ilhan"));
  }

  return Optional.empty();
}
```

§6.7.3 Null Object Pattern

Null Object Pattern, null yerine zararsız davranış gösteren nesne kullanma yaklaşımıdır.

Java örneği:

```
interface Bildirim {
  void gonder(String mesaj);
}

class BosBildirim implements Bildirim {
  public void gonder(String mesaj) {
    // Bilerek bos bırakıldı
  }
}
```

§6.7.4 Geçersiz Nesne Oluşturmamak

Bir nesne, oluşturulduğu anda geçerli durumda olmalıdır. Yapıcılar bu nedenle doğrulama yapmalıdır.

Java örneği:

```
class Kullanici {
  private String ad;

  public Kullanici(String ad) {
    if (ad == null || ad.isBlank()) {
      throw new IllegalArgumentException("Ad bos olamaz.");
    }

    this.ad = ad;
  }
}
```

6.8 Kapsülleme ve Değişmez Nesneler

Kapsülleme, nesnenin iç durumunu dış dünyadan koruyarak bu duruma yalnızca kontrollü yöntemler üzerinden erişilmesini sağlar. Nesne yönelimli programlamada kapsülleme yalnızca alanları `private` yapmak değildir; asıl amaç, nesnenin her zaman geçerli ve tutarlı durumda kalmasını

sağlamaktır.

Değişmez nesneler ise oluşturulduktan sonra iç durumu değiştirilemeyen nesnelerdir. Bu tür nesneler daha güvenli, daha öngörülebilir ve çok iş parçacıklı ortamlarda daha kolay yönetilebilir yapılar sunar.

§6.8.1 Alanları Doğrudan Açmamak

Bir sınıfın alanlarının doğrudan dışarıya açılması, nesnenin iç durumunun kontrolsüz biçimde değiştirilmesine neden olur. Bu durum, nesnenin geçersiz durumlara düşmesine yol açabilir.

Java dilinde kontrollü erişim örneği:

```
class BankaHesabi {
  private double bakiye;

  public BankaHesabi() {
    this.bakiye = 0.0;
  }

  public void paraYatir(double miktar) {
    if (miktar <= 0) {
      throw new IllegalArgumentException("Yatırılacak miktar pozitif olmalıdır.");
    }

    this.bakiye += miktar;
  }

  public void paraCek(double miktar) {
    if (miktar <= 0) {
      throw new IllegalArgumentException("Çekilecek miktar pozitif olmalıdır.");
    }

    if (miktar > this.bakiye) {
      throw new IllegalStateException("Yetersiz bakiye.");
    }

    this.bakiye -= miktar;
  }

  public double bakiyeOgren() {
    return this.bakiye;
  }
}
```

Kod 6.11: Java Dilinde Kontrollü Erişim

Burada `bakiye` alanı dışarıdan doğrudan değiştirilemez. Bakiye ancak iş kurallarını uygulayan `paraYatir()` ve `paraCek()` yöntemleriyle değiştirilebilir.

§6.8.2 Değişmez Nesneler

Değişmez nesne, oluşturulduktan sonra durumu değiştirilemeyen nesnedir. Özellikle değer nesneleri (value object) için bu yaklaşım oldukça uygundur.

Java dilinde değişmez nesne tasarımı için alanlar `private final` yapılır:

```
class Para {
  private final int kurus;
  private final String paraBirimi;

  public Para(int kurus, String paraBirimi) {
    if (kurus < 0) {
      throw new IllegalArgumentException("Para miktarı negatif olamaz.");
    }

    if (paraBirimi == null || paraBirimi.isBlank()) {
```

```
        throw new IllegalArgumentException("Para birimi boş olamaz.");
      }

      this.kurus = kurus;
      this.paraBirimi = paraBirimi;
    }

    public int getKurus() {
      return kurus;
    }

    public String getParaBirimi() {
      return paraBirimi;
    }

    public Para ekle(Para diger) {
      if (!this.paraBirimi.equals(diger.paraBirimi)) {
        throw new IllegalArgumentException("Farklı para birimleri toplanamaz.");
      }

      return new Para(this.kurus + diger.kurus, this.paraBirimi);
    }
  }
```

Kod 6.12: Java Dilinde Değişmez Nesne

Değişmez nesnelerin en önemli üstünlükleri şunlardır:

- ◇ Yan etki oluşturmazlar.
- ◇ Test edilmeleri kolaydır.
- ◇ Çok iş parçacıklı ortamlarda daha güvenlidirler.
- ◇ Nesnenin beklenmedik biçimde değişmesi engellenir.

§6.8.3 Nesne Durumunun Tutarlılığı

Bir nesne, yaşamı boyunca geçerli durumda kalmalıdır. Nesnenin geçerli kalmasını sağlayan kurallara **invariant** adı verilir.

Java örneği:

```
  class Ogrenci {
    private double notOrtalamasi;

    public Ogrenci(double notOrtalamasi) {
      notOrtalamasiAyarla(notOrtalamasi);
    }

    public void notOrtalamasiAyarla(double deger) {
      if (deger < 0 || deger > 100) {
        throw new IllegalArgumentException("Not ortalaması 0 ile 100 arasında olmalıdır
            .");
      }

      this.notOrtalamasi = deger;
    }

    public double notOrtalamasiOgren() {
      return notOrtalamasi;
    }
  }
```

Kod 6.13: Java Dilinde Nesne Tutarlılığı

6.9 Katmanlı Mimari Temelleri

Katmanlı mimari, yazılım sistemini farklı sorumluluklara sahip katmanlara ayıran bir mimari yaklaşımdır. Amaç, kullanıcı arayüzü, iş kuralları, veri erişimi ve altyapı kodlarının birbirine karışmasını engellemektir.

Genel olarak bir uygulama şu katmanlardan oluşabilir:

- ◇ Sunum Katmanı
- ◇ İş Katmanı
- ◇ Veri Erişim Katmanı
- ◇ Altyapı Katmanı

§6.9.1 Katmanlı Mimari Nedir?

Katmanlı mimaride her katman belirli bir sorumluluğa sahiptir. Örneğin kullanıcıdan veri almak sunum katmanının, iş kurallarını uygulamak iş katmanının, veritabanı işlemleri ise veri erişim katmanının görevidir.

Java örneği:

```java
class Siparis {
  private final String urunAdi;

  public Siparis(String urunAdi) {
    this.urunAdi = urunAdi;
  }

  public String getUrunAdi() {
    return urunAdi;
  }
}

class SiparisDeposu {
  public void kaydet(Siparis siparis) {
    System.out.println(siparis.getUrunAdi() + " veritabanına kaydedildi.");
  }
}

class SiparisServisi {
  private final SiparisDeposu depo;

  public SiparisServisi(SiparisDeposu depo) {
    this.depo = depo;
  }

  public void siparisOlustur(String urunAdi) {
    if (urunAdi == null || urunAdi.isBlank()) {
      throw new IllegalArgumentException("Ürün adı boş olamaz.");
    }

    Siparis siparis = new Siparis(urunAdi);
    depo.kaydet(siparis);
  }
}
```

Kod 6.14: Java Dilinde Basit Katmanlı Mimari

§6.9.2 Katmanlar Arası Bağımlılık

Katmanlar arası bağımlılık dikkatli yönetilmelidir. Sunum katmanı iş katmanını kullanabilir; iş katmanı veri erişim katmanını kullanabilir. Ancak veri erişim katmanı sunum katmanına bağımlı olmamalıdır.

§6.9.3 DTO, Entity ve Model Ayrımı

DTO, Entity ve Model kavramları çoğu projede karıştırılır.

- **DTO**: Katmanlar arasında veri taşımak için kullanılır.
- **Entity**: Veritabanındaki bir tabloyu veya kalıcı varlığı temsil eder.
- **Model**: İş alanındaki kavramı temsil eder.

Java dilinde basit bir DTO örneği:

```
class KullaniciDto {
  public String ad;
  public String email;
}
```

Kod 6.15: Java Dilinde DTO Örneği

Entity örneği:

```
class KullaniciEntity {
  private Long id;
  private String ad;
  private String email;

  // Getter ve setter yöntemleri
}
```

Kod 6.16: Java Dilinde Entity Örneği

§6.9.4 Repository ve Service Kavramları

Repository, veri erişimini soyutlayan yapıdır. Service ise iş kurallarını yürüten sınıftır.

Java örneği:

```
interface KullaniciDeposu {
  void kaydet(Kullanici kullanici);
}

class KullaniciServisi {
  private final KullaniciDeposu depo;

  public KullaniciServisi(KullaniciDeposu depo) {
    this.depo = depo;
  }

  public void kullaniciKaydet(Kullanici kullanici) {
    depo.kaydet(kullanici);
  }
}
```

Kod 6.17: Java Dilinde Repository ve Service

6.10 Bağımlılık Yönetimi ve Bağımlılık Enjeksiyonu

Bağımlılık yönetimi, bir sınıfın ihtiyaç duyduğu diğer sınıflarla ilişkisini düzenleme işidir. Kötü yönetilen bağımlılıklar, kodun test edilmesini ve değiştirilmesini zorlaştırır.

§6.10.1 Bağımlılık Enjeksiyonu (Dependency Injection)

Bağımlılık enjeksiyonu, bir sınıfın ihtiyaç duyduğu bağımlılıkların sınıf içinde oluşturulması yerine dışarıdan verilmesidir.

Java örneği:

```
interface BildirimGonderici {
  void gonder(String mesaj);
}
```

```java
class EpostaGonderici implements BildirimGonderici {
  public void gonder(String mesaj) {
    System.out.println("[E-posta] " + mesaj);
  }
}

class SiparisServisi {
  private final BildirimGonderici gonderici;

  public SiparisServisi(BildirimGonderici gonderici) {
    this.gonderici = gonderici;
  }

  public void siparisVer() {
    gonderici.gonder("Sipariş alındı.");
  }
}
```

Kod 6.18: Java Dilinde Bağımlılık Enjeksiyonu

§6.10.2 Denetimin Tersine Çevrilmesi (IoC) Kavramı

Denetimin Tersine Çevrilmesi (Inversion of Control - IoC), nesnelerin oluşturulması ve birbirine bağlanması sorumluluğunun uygulama kodundan alınarak dış bir yapıya verilmesidir.

Java örneği:

```java
public class Main {
  public static void main(String[] args) {
    BildirimGonderici gonderici = new EpostaGonderici();
    SiparisServisi servis = new SiparisServisi(gonderici);

    servis.siparisVer();
  }
}
```

6.11 Test Edilebilir Kod Yazmak

Test edilebilir kod, doğruluğu otomatik testlerle kolayca kontrol edilebilen koddur. Test edilebilir kod genellikle düşük bağımlılığa, küçük fonksiyonlara ve açık sorumluluklara sahiptir.

§6.11.1 Birim Test Mantığı

Birim test, bir sınıfın veya yöntemin küçük ve izole bir parçasını test etmektir. Birim testlerde genellikle Arrange-Act-Assert düzeni kullanılır.

Java örneği:

```java
class Hesaplayici {
  int topla(int a, int b) {
    return a + b;
  }
}

class HesaplayiciTest {
  public static void main(String[] args) {
    // Arrange
    Hesaplayici hesaplayici = new Hesaplayici();

    // Act
    int sonuc = hesaplayici.topla(2, 3);

    // Assert
    if (sonuc != 5) {
      throw new RuntimeException("Test başarısız.");
    }
```

```
    System.out.println("Test başarılı.");
  }
}
```

Kod 6.19: Java Dilinde Basit Birim Test Mantığı

§6.11.2 Mock ve Fake Nesneler

Testlerde gerçek bağımlılık yerine sahte nesneler kullanılabilir.

Java örneği:

```
class FakeBildirimGonderici implements BildirimGonderici {
  String sonMesaj;

  public void gonder(String mesaj) {
    this.sonMesaj = mesaj;
  }
}
```

Kod 6.20: Java Dilinde Fake Nesne

6.12 Kod Biçimlendirme Standartları

Kod biçimlendirme, kodun görsel düzenini belirler. Aynı ekipte çalışan geliştiricilerin ortak stil kullanması, kodun okunabilirliğini artırır.

Java projelerinde standart paket yapısı kullanılır:

```
src/main/java/
com/ornek/uygulama/domain/
com/ornek/uygulama/service/
com/ornek/uygulama/repository/
```

6.13 Yeniden Düzenleme (Refactoring) ve Kod Kokuları

Yeniden düzenleme, kodun dış davranışını değiştirmeden iç yapısını iyileştirme işlemidir. Amaç, kodu daha okunabilir, daha test edilebilir ve daha sürdürülebilir hale getirmektir.

6.14 Performans ve Temiz Kod Dengesi

Temiz kod yazmak, performansı önemsememek anlamına gelmez. Benzer şekilde performanslı kod yazmak da kodu okunamaz hale getirmek anlamına gelmemelidir.

Java dilinde algoritmik verimlilik için uygun veri yapıları (`HashSet` vb.) seçilmelidir:

```
import java.util.Set;

public class KumeArama {
  public static void main(String[] args) {
    Set<Integer> sayilar = Set.of(1, 2, 3, 4, 5);

    if (sayilar.contains(5)) {
      System.out.println("Sayi bulundu.");
    }
  }
}
```

Kod 6.21: Java Dilinde HashSet ile Arama

6.15 Güvenli Kodlama Standartları

Güvenli kodlama, yazılımın kötü niyetli veya hatalı girdiler karşısında güvenilir biçimde çalışmasını sağlamayı amaçlar. Java dilinde taşmayı denetlemek için `Math.addExact()` kullanılabilir:

```
class GuvenliAritmetik {
  int guvenliTopla(int a, int b) {
    return Math.addExact(a, b);
  }
}
```

Kod 6.22: Java Dilinde Güvenli Toplama

6.16 Dokümantasyon ve API Tasarımı

Dokümantasyon, yazılımın nasıl kullanılacağını, nasıl kurulacağını ve hangi sorumluluklara sahip olduğunu açıklar. Java tarafında Javadoc kullanılır:

```
/
* İki sayının toplamını hesaplar.
*
* @param a birinci sayı
* @param b ikinci sayı
* @return iki sayının toplamı
*/
int topla(int a, int b) {
  return a + b;
}
```

Kod 6.23: Java Dilinde Javadoc Örneği

6.17 Sürüm Kontrolü ve Kod Kalitesi Süreci

Sürüm kontrolü, yazılım geliştirme sürecinde kodun geçmişini izlemeyi, farklı geliştiricilerin birlikte çalışmasını ve hatalı değişikliklerden geri dönmeyi sağlar. Java projelerinde Maven veya Gradle, JUnit, SpotBugs ve Checkstyle gibi araçlar yaygın olarak kullanılır.

6.18 Temiz Mimari İlkelerine Giriş

Temiz mimari, yazılım sistemlerinde iş kurallarını teknik ayrıntılardan ayırmayı amaçlayan bir mimari yaklaşımdır.

§6.18.1 İş Kurallarını Merkeze Almak

Temiz mimaride iş kuralları sistemin merkezinde yer alır. Java dilinde iş kuralı sınıfı aşağıdaki gibi yazılabilir:

```
class OduncKurali {
  public void oduncAlabilirMi(int mevcutOduncSayisi,
  boolean kitapMevcutMu,
  boolean uyeAktifMi) {
    if (!uyeAktifMi) {
      throw new IllegalStateException("Pasif üyeye kitap ödünç verilemez.");
    }

    if (!kitapMevcutMu) {
      throw new IllegalStateException("Mevcut olmayan kitap ödünç verilemez.");
    }

    if (mevcutOduncSayisi >= 5) {
      throw new IllegalStateException("Üye en fazla beş kitap ödünç alabilir.");
    }
  }
}
```

Kod 6.24: Java Dilinde İş Kuralı Sınıfı

6.19 Bölüm Sonu Uygulaması: Kütüphane Ödünç Verme Sistemi

Bu bölümde, temiz kod ve mimari standartlar başlığı altında anlatılan kavramları pekiştirmek için küçük bir kütüphane ödünç verme sistemi Java dili ile tasarlanacaktır.

Java ile Temiz Tasarım

```
class Odunc {
  private final String uyeNo;
  private final String kitapNo;

  public Odunc(String uyeNo, String kitapNo) {
    if (uyeNo == null || uyeNo.isBlank()) {
      throw new IllegalArgumentException("Üye no boş olamaz.");
    }

    if (kitapNo == null || kitapNo.isBlank()) {
      throw new IllegalArgumentException("Kitap no boş olamaz.");
    }

    this.uyeNo = uyeNo;
    this.kitapNo = kitapNo;
  }

  public String getUyeNo() {
    return uyeNo;
  }

  public String getKitapNo() {
    return kitapNo;
  }
}
```

Kod 6.25: Java Dilinde Ödünç Kaydı Sınıfı

```
class OduncKurali {
  public void kontrolEt(boolean uyeAktifMi,
  boolean kitapMevcutMu,
  int mevcutOduncSayisi) {
    if (!uyeAktifMi) {
      throw new IllegalStateException("Pasif üyeye kitap ödünç verilemez.");
    }

    if (!kitapMevcutMu) {
      throw new IllegalStateException("Mevcut olmayan kitap ödünç verilemez.");
    }

    if (mevcutOduncSayisi >= 5) {
      throw new IllegalStateException("Üye en fazla beş kitap ödünç alabilir.");
    }
  }
}
```

Kod 6.26: Java Dilinde Ödünç Verme İş Kuralı

```
interface OduncDeposu {
  void kaydet(Odunc odunc);
}

interface BildirimGonderici {
  void gonder(String uyeNo, String mesaj);
}
```

Kod 6.27: Java Dilinde Depo ve Bildirim Arayüzleri

```
class BellekOduncDeposu implements OduncDeposu {
  @Override
  public void kaydet(Odunc odunc) {
```

```
    System.out.println("Ödünç kaydı oluşturuldu: "
    + odunc.getUyeNo() + " - " + odunc.getKitapNo());
  }
}

class EpostaBildirimGonderici implements BildirimGonderici {
  @Override
  public void gonder(String uyeNo, String mesaj) {
    System.out.println(uyeNo + " üyesine e-posta gönderildi: " + mesaj);
  }
}
```

Kod 6.28: Java Dilinde Somut Depo ve Bildirim Sınıfları

```
class OduncServisi {
  private final OduncDeposu oduncDeposu;
  private final BildirimGonderici bildirimGonderici;
  private final OduncKurali oduncKurali;

  public OduncServisi(OduncDeposu oduncDeposu,
  BildirimGonderici bildirimGonderici,
  OduncKurali oduncKurali) {
    this.oduncDeposu = oduncDeposu;
    this.bildirimGonderici = bildirimGonderici;
    this.oduncKurali = oduncKurali;
  }

  public void kitapOduncVer(String uyeNo,
  String kitapNo,
  boolean uyeAktifMi,
  boolean kitapMevcutMu,
  int mevcutOduncSayisi) {
    oduncKurali.kontrolEt(uyeAktifMi, kitapMevcutMu, mevcutOduncSayisi);

    Odunc odunc = new Odunc(uyeNo, kitapNo);

    oduncDeposu.kaydet(odunc);

    bildirimGonderici.gonder(uyeNo, "Kitap ödünç verme işlemi başarılı.");
  }
}
```

Kod 6.29: Java Dilinde Ödünç Verme Servisi

```
public class KutuphaneUygulamasi {
  public static void main(String[] args) {
    OduncDeposu depo = new BellekOduncDeposu();
    BildirimGonderici bildirim = new EpostaBildirimGonderici();
    OduncKurali kural = new OduncKurali();

    OduncServisi servis = new OduncServisi(depo, bildirim, kural);

    servis.kitapOduncVer("UYE-1", "KITAP-1", true, true, 2);
  }
}
```

Kod 6.30: Java Dilinde Uygulamanın Çalıştırılması

Bölüm Özeti

Bu uygulamada, her şeyi yapan büyük bir sınıf yerine, sorumlulukları ayrılmış, arayüzlere bağımlı, test edilebilir ve genişletilebilir bir tasarım oluşturulmuştur. Böylece temiz kodun yalnızca satırların düzenli yazılması değil, aynı zamanda doğru sorumluluk ayrımı ve doğru bağımlılık yönetimi olduğu görülmüştür.